

Università di Pisa

Dipartimento di Informatica

Generative and Deep Learning

Dispensa del Corso

Laurea Magistrale in Informatica — A.A. 2025/2026

Prof. Davide Bacciu `davide.bacciu@unipi.it`

Dr. Riccardo Massidda `riccardo.massidda@di.unipi.it`

*Questa dispensa è basata sulle slides del corso e sugli appunti delle lezioni.
Non sostituisce lo studio diretto dei materiali originali.*

August 25, 2026

Contents

Prefazione	11
I Fondamenti di Modelli Probabilistici e Causalità	13
1 Introduzione al Corso	14
1.1 Organizzazione e Filosofia	14
1.2 Modalità di Esame	15
1.3 Approfondimenti	15
1.3.1 Domande per l'Esame	15
2 Ripasso di Probabilità e Modelli Grafici Probabilistici	16
2.1 Variabili Casuali	16
2.2 Distribuzioni Congiunte, Marginali e Condizionali	16
2.2.1 Regola della Catena (Chain Rule)	17
2.2.2 Teorema di Bayes	17
2.3 Indipendenza e Indipendenza Condizionale	18
2.4 Valore Atteso	19
2.5 Distribuzioni Notevoli	19
2.5.1 Bernoulli e Categoricala	19
2.5.2 Gaussiana Univariata e Multivariata	19
2.5.3 Beta e Dirichlet	19
2.6 Framework dei Modelli Grafici	19
2.6.1 Tipi di Inferenza	20
2.7 Approfondimenti	20
2.7.1 Entropia e Divergenza KL	20
2.7.2 Domande per l'Esame	20
3 Reti Bayesiane I: Rappresentazione e Fattorizzazione	21
3.1 Sfida Dimensionale	21
3.2 Definizione di Rete Bayesiana	21
3.2.1 Fattorizzazione Congiunta	21
3.3 Local Markov Property	22
3.4 Notazione Plate	22
3.5 Ancestral Sampling	22
3.6 Approfondimenti	23
3.6.1 Risparmio di Parametri	23
3.6.2 Markov Equivalence	23
3.6.3 Domande per l'Esame	24

4	Reti Bayesiane II: d-Separazione e Markov Blanket	25
4.1	d-Separazione	25
4.2	Le Tre Strutture Fondamentali	26
4.2.1	Collider ed Explaining Away	26
4.3	Markov Blanket	27
4.4	Faithfulness Assumption	27
4.5	Approfondimenti	28
4.5.1	Algoritmo per Verificare d-Separazione	28
4.5.2	Domande per l'Esame	28
5	Causalità e Inferenza Causale	29
5.1	Correlazione vs. Causalità	29
5.2	Reti Bayesiane Causali e l'Operatore $do(\cdot)$	29
5.2.1	Osservazione vs. Intervento	29
5.2.2	Formula di Aggiustamento (Adjustment Formula)	30
5.3	Modelli Causali Strutturali (SCM)	31
5.4	Gerarchia Causale di Pearl	31
5.5	Do-Calculus	32
5.6	Approfondimenti	32
5.6.1	Backdoor Criterion	32
5.6.2	Domande per l'Esame	32
6	Apprendimento di Strutture (Structure Learning)	33
6.1	Il Problema	33
6.2	Metodi Basati su Vincoli: PC Algorithm	33
6.2.1	Fase 1: Skeleton Learning	33
6.2.2	Fase 2: Orientamento dei Collider	34
6.2.3	Fase 3: Regole di Meek	34
6.3	Metodi Score-based: GES	34
6.3.1	Score BIC	34
6.3.2	Algoritmo GES	35
6.4	Assunzioni Parametriche: Additive Noise Models (ANM)	35
6.5	FCI per Variabili Latenti	35
6.6	Approfondimenti	35
6.6.1	Domande per l'Esame	35
II	Learning nei Modelli Probabilistici	36
7	Apprendimento con Variabili Osservate	37
7.1	Learning come Inferenza	37
7.1.1	Tre Flavori dell'Apprendimento	37
7.2	MLE per Reti Bayesiane Completamente Osservate	38
7.2.1	Stability Numerica	38
7.3	Classificatori Naive Bayes	39
7.3.1	Inferenza nel Naive Bayes	39
7.3.2	MAP Estimate e Laplace Smoothing	39
7.4	Generativo vs Discriminativo	40
7.4.1	Inferenza Bayesiana	40
7.5	Approfondimenti	41

8	Apprendimento con Variabili Nascoste: l'Algoritmo EM	42
8.1	Il Problema delle Variabili Latenti	42
8.2	L'Algoritmo Expectation-Maximization	43
8.2.1	Proprietà di Convergenza	43
8.3	Mixture of Gaussians	43
8.3.1	E-step di EM per GMM	44
8.3.2	M-step di EM per GMM	44
8.3.3	Hard vs Soft Assignment	45
8.4	EM per Reti Bayesiane con Variabili Nascoste	45
8.5	Approfondimenti	45
9	Modelli Markov Nascosti	46
9.1	Modellazione di Sequenze	46
9.2	Tre Problemi Fondamentali	46
9.2.1	Problema 1: Valutazione (Forward Algorithm)	46
9.2.2	Problema 2: Decodifica (Viterbi Algorithm)	47
9.2.3	Problema 3: Learning (Baum-Welch Algorithm)	47
9.3	Algoritmo Backward	48
9.4	Baum-Welch: EM per HMM	48
9.4.1	E-step	48
9.4.2	M-step	49
9.5	Reti Bayesiane Dinamiche	49
9.6	Approfondimenti	49
10	Inferenza Variazionale	50
10.1	Il Problema dell'Inferenza Intractable	50
10.2	Evidence Lower Bound (ELBO)	50
10.3	Approssimazione Mean-Field	52
10.4	Divergenza KL	52
10.5	EM Generalizzato	52
10.6	Disuguaglianza di Jensen	52
10.7	Approfondimenti	53
11	Latent Dirichlet Allocation	54
11.1	Topic Modeling	54
11.2	Processo Generativo di LDA	54
11.3	Inferenza in LDA	56
11.4	Learning di LDA	56
11.5	Plate Notation	56
11.6	Applicazioni e Differenze da pLSA	57
11.7	Approfondimenti	57
12	Metodi di Campionamento	58
12.1	Motivazione del Campionamento	58
12.2	Campionamento Univariato	58
12.2.1	Inverse Transform Sampling	58
12.2.2	Box-Muller	58
12.3	Rejection Sampling	59
12.4	Importance Sampling	59
12.5	Gibbs Sampling	60
12.6	MCMC Generali	60
12.7	Metropolis-Hastings	60

12.8 Sampling per LDA	61
12.9 Approfondimenti	61
13 Campi Casuali Markoviani	62
13.1 Modelli Grafici Indiretti	62
13.2 Fattorizzazione di Gibbs	62
13.3 Proprietà di Markov	63
13.4 CRF e Modelli Discriminativi	63
13.5 Macchina di Boltzmann	63
13.6 Restricted Boltzmann Machine	64
13.7 Contrastive Divergence	65
13.8 Approfondimenti	66
III Deep Learning Fondamentale	68
14 Convolutional Neural Networks (Lezioni 15–16)	69
14.1 Contesto storico e motivazioni biologiche	69
14.2 Inductive biases nelle CNN	69
14.3 Operazione di convoluzione	70
14.4 Pooling e Feature Maps	72
14.4.1 Parametri per layer	72
14.5 Architetture CNN fondamentali	72
14.5.1 LeNet-5 (1998)	72
14.5.2 AlexNet (2012)	72
14.5.3 VGG Networks (2014)	73
14.5.4 ResNet (2015): Residual Connections	73
14.5.5 Inception e convoluzioni 1×1	73
14.6 Convoluzioni dilate e separabili	74
14.6.1 Dilated Convolutions	74
14.6.2 Depthwise Separable Convolutions	74
14.7 Receptive field analysis	74
14.8 Approfondimenti	74
14.8.1 Domande d'esame	74
15 Propagazione dell'informazione in reti profonde (Lezione 17)	76
15.1 RNN: basic architecture	76
15.2 Backpropagation Through Time (BPTT)	76
15.3 Gradient flow e Jacobian matrices	77
15.4 Vanishing e exploding gradients	77
15.4.1 Vanishing Gradient Problem	77
15.4.2 Exploding Gradient Problem	78
15.5 Soluzioni ai problemi di gradiente	79
15.5.1 Gradient Clipping	79
15.5.2 Proper Weight Initialization	79
15.5.3 Alternative Activation Functions	79
15.5.4 Architetture specializzate	79
15.6 Echo State Network perspective	79
15.7 Approfondimenti	80
15.7.1 Domande d'esame	80

16 Modelli ricorrenti con porte (Lezione 18)	81
16.1 LSTM: long-short term memory	81
16.2 Gradient flow in LSTM	82
16.3 GRU: gated recurrent unit	83
16.4 Confronto LSTM vs GRU	83
16.5 Bidirectional RNNs	83
16.6 Autoregressive modeling	83
16.6.1 Teacher forcing	84
16.7 Approfondimenti	84
16.7.1 Domande d'esame	84
17 Encoder-Decoder e Cross-Attention (Lezione 19)	85
17.1 Sequence-to-sequence tasks	85
17.2 Encoder-decoder con bottleneck	85
17.3 Information bottleneck problem	86
17.4 Attention mechanism	86
17.5 Cross-attention vs self-attention	87
17.6 Attention come soft alignment	88
17.7 Encoder-decoder con attention	88
17.8 Applicazioni dell'attention	88
17.9 Approfondimenti	89
17.9.1 Domande d'esame	89
18 Transformers (Lezione 20)	90
18.1 Self-attention	90
18.2 Multi-head attention	91
18.3 Positional encoding	91
18.4 Transformer block	91
18.5 Full Transformer architecture	92
18.6 Complexity analysis	92
18.6.1 Self-attention complexity	92
18.6.2 RNN complexity	92
18.7 Gradient flow advantage	93
18.8 Self-supervised training	93
18.8.1 BERT (Bidirectional Encoder)	93
18.8.2 GPT (Generative Pre-training)	93
18.8.3 Pre-training e fine-tuning	94
18.9 Inductive biases	94
18.10 Appendix: Efficient Transformers	94
18.11 Approfondimenti	94
18.11.1 Domande d'esame	94
IV Generative Deep Learning	96
19 Autoencoders e Manifold Learning	98
19.1 Motivazione e posizionamento	98
19.2 Autoencoder neurale	98
19.3 Varianti regolarizzate	99
19.3.1 Sparse autoencoder (SAE)	99
19.3.2 Denoising autoencoder (DAE)	99
19.3.3 Contractive autoencoder (CAE)	100

19.4	Deep autoencoders e pretraining	100
19.5	Applicazioni	100
20	Variational Autoencoders	102
20.1	Dalla DAE al modello a variabile latente	102
20.2	ELBO	102
20.3	Architettura VAE	103
20.3.1	Reparametrization trick	103
20.3.2	KL in forma chiusa	104
20.3.3	Loss finale	104
20.4	Generazione e CVAE	105
20.5	Interpretazione info-teorica	105
20.6	Patologie e varianti	105
20.6.1	β -VAE	105
20.6.2	Posterior collapse	105
20.6.3	Confronto con AE classico e DAE	105
21	Generative Adversarial Networks	107
21.1	Idea: imparare a campionare	107
21.2	Formulazione minimax	107
21.2.1	Discriminatore ottimale	107
21.2.2	Connessione con Jensen-Shannon	108
21.3	Problemi pratici	108
21.3.1	Gradient saturation	108
21.3.2	Mode collapse	108
21.3.3	Instabilità del training	109
21.4	WGAN	109
21.5	Varianti notevoli	110
21.6	Adversarial Autoencoder (AAE)	110
22	Normalizing Flows	112
22.1	Idea: cambiare variabile in modo invertibile	112
22.2	Formula del cambio di variabile	112
22.3	Composizione di flussi	112
22.4	Layer notevoli	113
22.4.1	Affine flow	113
22.4.2	Coupling layer (NICE, RealNVP)	113
22.4.3	Autoregressive flow (MAF/IAF)	114
22.4.4	Residual flow	114
22.4.5	Continuous Normalizing Flow (Neural ODE)	115
22.5	Dequantizzazione	115
22.6	Universalità	116
23	Diffusion Models	118
23.1	Idea: corrompere e ricostruire	118
23.2	Forward process	118
23.3	Reverse process	119
23.4	Training: ELBO e parametrizzazione del rumore	119
23.4.1	ELBO per la catena di diffusione	119
23.4.2	Parametrizzazione ϵ	120
23.5	Sampling: DDPM e DDIM	121
23.5.1	DDPM (stocastico)	121

23.5.2 DDIM (deterministico)	121
23.6 Architettura: U-Net	121
23.7 Conditional generation	121
23.7.1 Classifier guidance	121
23.7.2 Classifier-free guidance	121
23.8 Latent diffusion	122
24 Score Matching e Flow Matching	124
24.1 Score function e diffusione	124
24.1.1 Vantaggio della score	124
24.2 Formula di Tweedie: il ponte fra denoising e score	125
24.3 Score matching	126
24.3.1 Score matching esplicito	126
24.3.2 Implicit Score Matching (Hyvärinen 2005)	126
24.3.3 Denoising Score Matching (Vincent 2011)	126
24.4 Score-based SDE	126
24.4.1 Forward SDE	126
24.4.2 Reverse SDE (Anderson 1982)	126
24.4.3 Probability flow ODE	127
24.5 Flow Matching	127
24.5.1 Probabilità path e velocità	127
24.5.2 Conditional Flow Matching (CFM)	127
24.5.3 Path notevoli	127
24.6 Sampling	128
24.7 Riepilogo unificante	129
25 Causal Representation Learning	131
25.1 Disentanglement: definizione e motivazione	131
25.2 Approcci unsupervised	131
25.2.1 β -VAE (Higgins et al. 2017)	131
25.2.2 FactorVAE (Kim & Mnih 2018)	132
25.2.3 β -TCVAE (Chen et al. 2018)	132
25.3 Impossibilità (Locatello et al. 2019)	132
25.4 Disentanglement supervisionato debolmente	133
25.4.1 Time-Contrastive Learning (TCL, Hyvärinen & Morioka 2016)	133
25.4.2 iVAE (Khemakhem et al. 2020)	133
25.5 Causal Representation Learning	133
25.5.1 Setup	133
25.5.2 Identifiability via interventi	133
25.5.3 CITRIS (Lippe et al. 2022)	133
25.5.4 Varianti	134
25.6 Triplet evaluation	134
Confronto finale Parte IV	136
V Graph Neural Networks	138
26 Fondamenti di Deep Learning per Grafi	140
26.1 Perché i grafi?	140
26.2 Formalizzazione del task	140
26.2.1 Tassonomia dei task	140

26.3	Prospettiva storica	141
26.3.1	Modelli contractive (Scarselli et al. 2009)	141
26.3.2	Modelli contextual (Micheli 2009, NN4G)	141
26.3.3	Deep Graph Network — la sintesi moderna	141
27	Message Passing e GNN Convoluzionali	143
27.1	Schema generale: message passing	143
27.2	Convolutional Graph Networks	144
27.2.1	Approccio spettrale	144
27.2.2	Graph Convolutional Network (GCN, Kipf & Welling 2017)	144
27.2.3	GraphSAGE (Hamilton et al. 2017)	145
27.2.4	Graph Isomorphism Network (GIN, Xu et al. 2019)	145
27.3	Edge features	147
28	Graph Attention e Graph Transformer	148
28.1	Graph Attention Network (GAT, Veličković et al. 2018)	148
28.2	Graph Transformer	149
28.2.1	Positional encoding per grafi	149
28.2.2	GPS (Rampášek et al. 2022)	149
28.3	Pooling e gerarchia	150
28.4	Patologie	150
28.4.1	Over-smoothing	150
28.4.2	Over-squashing	150
28.4.3	Under-reaching	150
28.5	Modelli randomized: Reservoir per grafi	151
VI	Reinforcement Learning	153
29	Fondamenti di Reinforcement Learning	154
29.1	Posizionamento	154
29.2	Markov Decision Process	154
29.2.1	Ritorno e value functions	155
29.2.2	Equazioni di Bellman	155
30	Algoritmi RL	157
30.1	Tassonomia	157
30.2	Model-based: Policy Iteration	157
30.3	Model-based: Value Iteration	158
30.4	Model-free: Temporal Difference e Q-learning	158
30.4.1	TD(0) per value	158
30.4.2	Q-learning (off-policy)	158
30.4.3	SARSA (on-policy)	159
30.5	Function approximation	159
30.5.1	Deep Q-Network (DQN, Mnih et al. 2015)	159
30.6	Policy gradient	159
30.6.1	REINFORCE	160
30.6.2	Riduzione di varianza: baseline e advantage	161
30.6.3	Vantaggi/svantaggi policy-based	161
30.6.4	Policy gradient deterministico (DPG)	161
30.7	Actor-Critic	161
30.7.1	Algoritmi notevoli	162

VII Il paper del quarto assignment: GrIDDD	164
31 GrIDDD: Graph Diffusion that can Insert and Delete	166
31.1 Il problema: i diffusion model su grafi hanno taglia fissa	166
31.2 Che cosa facevano gli altri, e perché non basta	167
31.3 Lo schema degli edit: quanti, e quando	167
31.4 Forward process: tre casi	168
31.4.1 Caso base ($\Delta^T = 0$): il forward ereditato	168
31.4.2 Cancellazioni monotone ($\Delta^T < 0$): il trucco DEL/DEL*	168
31.4.3 Inserimenti monotoni ($\Delta^T > 0$)	169
31.5 Il posterior generalizzato	169
31.6 Rete ausiliaria, loss, guidance	169
31.7 Risultati	170
31.7.1 Property targeting	170
31.7.2 Property optimization (ZINC-250k)	170
31.7.3 Out-of-distribution e ablation	170
31.8 Limiti dichiarati dagli autori	171
31.9 Dove si colloca nel programma	171
31.10 Domande d'esame — GrIDDD	172
A Raccolta domande d'esame — preparazione orale	174
A.1 Domande trasversali (le più difficili)	175
A.2 Parte I — PGM e Causalità	176
A.3 Parte II — Learning Probabilistico	177
A.4 Parte III — Deep Learning Fondamentale	178
A.5 Parte IV — Generative Deep Learning	179
A.6 Parte V — Graph Neural Networks	180
A.7 Parte VI — Reinforcement Learning	180
B Cheat sheet delle formule	181
B.1 Probabilità e PGM	181
B.2 Learning probabilistico	181
B.3 Deep Learning fondamentale	183
B.4 Generative Deep Learning	184
B.5 Graph Neural Networks	186
B.6 Reinforcement Learning	186
C Che cosa è stato chiesto davvero	188
C.1 Le regole del gioco	188
C.2 Gli orali riportati	190
C.3 Distribuzione degli argomenti (rivista)	191
C.4 Che cosa è escluso	192
C.5 Domande verbatim	192
Riferimenti Bibliografici	195

Prefazione

Questa dispensa raccoglie i contenuti del corso *Generative and Deep Learning* (GDL) tenuto dal Prof. Davide Bacciu e dal Dr. Riccardo Massidda presso l'Università di Pisa. Il materiale è organizzato in sei parti più due appendici:

Parte I — Fondamenti Probabilistici e Causalità (Lezioni 1–6): probabilità, modelli grafici, reti Bayesiane, causalità, structure learning.

Parte II — Learning nei Modelli Probabilistici (Lezioni 7–14): apprendimento MLE/MAP/EM, HMM, inferenza variazionale, LDA, metodi di sampling, Markov Random Fields.

Parte III — Deep Learning Fondamentale (Lezioni 15–20): CNN, propagazione dell'informazione, LSTM/GRU, meccanismo di attenzione, Transformers.

Parte IV — Generative Deep Learning (Lezioni 23–31): autoencoders, VAE, GAN, normalizing flows, diffusion, flow matching, causal representation learning.

Parte V — Graph Neural Networks (Lezione 32): message passing, GCN, GraphSAGE, GIN, GAT, graph transformer.

Parte VI — Reinforcement Learning (Lezione 34): MDP, Bellman, Q-learning, policy gradient, DQN, actor-critic, PPO.

Parte VII — Il paper del quarto assignment GrIDDD (Ninniri, Podda, Bacciu, NeurIPS 2025): diffusion su grafi con inserimento e cancellazione di nodi. Non è una lezione del corso, ma è il punto da cui l'orale tipicamente parte.

Appendice B raccolta domande d'esame organizzate per parte e domande trasversali.

Appendice C cheat sheet di tutte le formule chiave.

Appendice D resoconto di un appello reale (fine luglio 2026): che cosa è stato chiesto, dove i candidati si sono fermati, e che cosa se ne ricava sulla strategia di preparazione.

Nota sul materiale d'esame: come indicato dal prof nella lezione finale (GDL35), la lezione *Deep Learning for Graphs II — Advanced Models* (GDL33) **non** è parte del programma d'esame.

Come usare questa dispensa: ogni capitolo contiene definizioni formali, derivazioni complete, tabelle di confronto, sezioni “Quando usare questo modello”, “Connessioni” e “Domande d'esame”. Si consiglia di studiare insieme alle slides originali e ai capitoli del libro di testo ([BRML] Barber per Parti I–II, [SD] Prince per Parti III–VI).

I box colorati. Dalla revisione di agosto 2026 ogni capitolo apre con un box *Risorse video* e alterna, accanto alla trattazione formale, due tipi di box pensati per la preparazione orale:

Intuizione la spiegazione in linguaggio piano, prima delle formule: che cosa sta succedendo davvero e perché il modello è fatto così. È la risposta che serve quando all’orale la domanda è “me lo spieghi a parole?”.

Esempio svolto un caso minimo con numeri piccoli, verificabile a mano in pochi minuti. Serve a trasformare una formula in un gesto: è quello che si riesce a rifare alla lavagna sotto esame.

Risorse video la videolezione del corso più una selezione di lezioni universitarie e video divulgativi esterni sull’argomento, di durate diverse.

Trappola un punto in cui la risposta intuitiva è sbagliata. Sono pochi e scelti: se una domanda tocca uno di questi, la differenza fra la risposta giusta e quella plausibile è tutta la differenza di voto.

Gli esempi svolti seguono la stessa impostazione dei *worked examples* presenti negli handout del corso: nessun conto lungo, ma il passaggio che di solito viene saltato.

Registrazioni delle lezioni. Le videolezioni del corso stanno nella cartella *Recordings* del team dell’insegnamento:

https://unipiit.sharepoint.com/sites/a__td_69202/SharedDocuments/General/Recordings

Richiede l’autenticazione di ateneo. I file sono nominati per data della riunione e non per numero di lezione: il riferimento puntuale per capitolo va quindi verificato sul calendario del corso.

Riferimenti principali:

- D. Barber, *Bayesian Reasoning and Machine Learning*, Cambridge UP, 2012
- S.J.D. Prince, *Understanding Deep Learning*, MIT Press, 2023

Part I

Fondamenti di Modelli Probabilistici e Causalità

Chapter 1

Introduzione al Corso

Risorse video — GDL1 — Inquadramento del corso

Videolezione del corso: registrazione della lezione GDL1 nella cartella *Recordings* del team del corso (v. Prefazione).

Stanford CS236 — Lecture 1: Introduction to Deep Generative Models
(*Stanford Online*, 1h15m)

La stessa mappa concettuale del corso, vista da Stanford: perché si parte dai modelli probabilistici e quali famiglie generative esistono. Utile prima ancora di aprire le slide, per avere il quadro d'insieme.

1.1 Organizzazione e Filosofia

Il corso *Generative and Deep Learning* (GDL) è tenuto dal Prof. Davide Bacciu e dal Dr. Riccardo Massidda presso il Dipartimento di Informatica dell'Università di Pisa. Il corso è strutturato in cinque moduli che costruiscono progressivamente la conoscenza:

1. **Fondamenti di Modelli Probabilistici e Causalità:** probabilità, modelli grafici, reti Bayesiane, causalità, structure learning
2. **(Generative) Learning in modelli probabilistici:** apprendimento ML/MAP/Bayesiano, EM, HMM, inferenza variazionale, LDA, sampling, MRF
3. **Deep Learning fondamentale:** CNN, RNN, attention, Transformers, problemi di propagazione
4. **Generative Deep Learning:** autoencoder, GAN, diffusion models, flow models
5. **Modelli avanzati e applicazioni**

La **filosofia del corso** è quella di partire dai modelli probabilistici classici (dove la probabilità è esplicita) per arrivare al deep learning generativo moderno, mostrando come i due approcci si connettano e si complementino.

Riferimenti Bibliografici

- **Barber, D.** (2012). *Bayesian Reasoning and Machine Learning*. Cambridge University Press. (Gratuito online, con codice)
- **Prince, S.J.D.** (2023). *Understanding Deep Learning*. MIT Press. (Gratuito online)

1.2 Modalità di Esame

L'esame per gli studenti che frequentano le lezioni è composto da:

1. **4 midterm assignments:** notebook Python individuali (3 individuali + 1 di gruppo), da consegnare ogni 3-4 settimane
2. **Esame orale finale:** discussione degli argomenti del programma

1.3 Approfondimenti

1.3.1 Domande per l'Esame

1. Qual è la progressione logica dei cinque moduli del corso? Perché si inizia da modelli probabilistici e non direttamente dal deep learning?
2. Come si relazionano i modelli probabilistici grafici e le reti neurali profonde nell'ambito del generative deep learning?

Chapter 2

Ripasso di Probabilità e Modelli Grafici Probabilistici

Risorse video — GDL2 — Probabilità e modelli grafici

Videolezione del corso: registrazione della lezione GDL2 nella cartella *Recordings* del team del corso (v. Prefazione).

Introduction to Probabilistic Graphical Models (*Kayhan Batmanghelich, 1h*)

Versione estesa: parte dalle basi di probabilità e arriva al formalismo grafico. È il video più vicino al taglio di questa lezione.

Graphical Models 1 — Christopher Bishop, MLSS 2013 (*MPI-IS, 1h30m*)

Più lungo e più elegante: se hai tempo, è la trattazione che rende ovvio il passaggio da congiunta a fattorizzazione.

2.1 Variabili Casuali

Una **variabile casuale** (RV) X è una funzione che mappa gli esiti di un esperimento casuale a valori numerici. Si distinguono:

- **Variabili discrete:** assumono valori in un insieme finito o numerabile $\mathcal{X}$. La *PMF* (*Probability Mass Function*) è $P(X=x) \in [0, 1]$ con $\sum_x P(X=x) = 1$.
- **Variabili continue:** la *PDF* (*Probability Density Function*) $p(x) \geq 0$ con $\int p(x) dx = 1$. La probabilità di un evento è $P(X \in A) = \int_A p(x) dx$.

2.2 Distribuzioni Congiunte, Marginali e Condizionali

Per un insieme di N RV $\{X_1, \dots, X_N\}$:

- **Distribuzione congiunta:** $P(x_1, \dots, x_N)$ — richiede $k^N - 1$ parametri per N RV discrete con k valori
- **Marginalizzazione:** $P(x_i) = \sum_{x_{-i}} P(x_1, \dots, x_N)$
- **Condizionale:** $P(x_i | x_j) = P(x_i, x_j) / P(x_j)$

2.2.1 Regola della Catena (Chain Rule)

La distribuzione congiunta può sempre essere fattorizzata come:

$$P(x_1, \dots, x_N) = \prod_{i=1}^N P(x_i \mid x_1, \dots, x_{i-1})$$

Questa decomposizione è valida per qualsiasi ordinamento delle variabili. Il vantaggio si ottiene quando si sfruttano le *indipendenze condizionali* per ridurre il numero di parametri.

Intuizione — la chain rule non “scopre” niente, serve a fare spazio

La regola della catena vale sempre, per qualunque ordinamento: da sola non riduce di un solo parametro il costo della congiunta. Riscrivere $P(x_1, \dots, x_N)$ come prodotto di condizionate è solo un cambio di notazione.

Il guadagno arriva al passo successivo, quando si *cancellano* dei condizionanti perché sono irrilevanti: $P(x_i \mid x_1, \dots, x_{i-1}) = P(x_i \mid \text{pochi genitori})$. La chain rule apre gli slot; le indipendenze condizionali li svuotano. Tutta la teoria delle reti Bayesiane vive in questo secondo passo — ed è per questo che l'ordinamento delle variabili conta: un ordinamento sbagliato apre slot che nessuna indipendenza riesce poi a svuotare.

2.2.2 Teorema di Bayes

Dati un'ipotesi h e osservazioni $\mathbf{d}$:

$$P(h \mid \mathbf{d}) = \frac{P(\mathbf{d} \mid h) P(h)}{P(\mathbf{d})}$$

dove:

- $P(h)$: **prior** — probabilità dell'ipotesi prima di osservare i dati
- $P(\mathbf{d} \mid h)$: **likelihood** — probabilità dei dati data l'ipotesi
- $P(\mathbf{d}) = \sum_{h'} P(\mathbf{d} \mid h') P(h')$: **evidence** (costante di normalizzazione)
- $P(h \mid \mathbf{d})$: **posterior** — probabilità dell'ipotesi dopo i dati

Esempio svolto — il prior domina: test medico con bassa prevalenza

Una malattia ha prevalenza $P(D=1) = 0.01$. Il test ha sensibilità $P(+ \mid D=1) = 0.90$ e specificità $P(- \mid D=0) = 0.95$, quindi $P(+ \mid D=0) = 0.05$.

Il test è positivo. Qual è la probabilità di essere malati?

$$P(D=1 \mid +) = \frac{0.90 \cdot 0.01}{\underbrace{0.90 \cdot 0.01}_{0.0090} + \underbrace{0.05 \cdot 0.99}_{0.0495}} = \frac{0.0090}{0.0585} \approx 0.154.$$

Il 15,4%, non il 90%. La ragione è tutta nell'evidence a denominatore: i sani sono 99 volte più numerosi dei malati, quindi anche un 5% di falsi positivi produce più positivi (0.0495) di quanti ne producano i veri malati (0.0090).

Perché ricordarselo: il posterior è il prodotto $\text{prior} \times \text{likelihood}$, non la sola likelihood. Ogni volta che nel corso comparirà un MAP invece di un ML, il termine in più è esattamente il prior che qui vale 0.01 e ribalta la conclusione.

2.3 Indipendenza e Indipendenza Condizionale

Definizione 2.1. X e Y sono **indipendenti**, $X \perp Y$, se e solo se:

$$P(X, Y) = P(X)P(Y) \Leftrightarrow P(X | Y) = P(X)$$

Definizione 2.2. X e Y sono **condizionalmente indipendenti** dato Z , $X \perp Y | Z$, se e solo se:

$$P(X, Y | Z) = P(X | Z)P(Y | Z)$$

Nota Importante

$X \perp Y$ **non implica** $X \perp Y | Z$ e viceversa. Questo è fondamentale per capire d-separazione e il comportamento dei collider.

Intuizione — condizionare non è “restringere l’attenzione”, è cambiare mondo

L’errore tipico è pensare che condizionare su Z possa solo *ridurre* la dipendenza, perché “si sa qualcosa in più”. Non è così: condizionare cambia la popolazione su cui si guarda, e in una popolazione selezionata possono nascere correlazioni che nella popolazione intera non esistevano.

L’immagine giusta: due variabili indipendenti che concorrono a produrre un effetto comune. Finché guardi tutti, non c’è legame. Ma se guardi solo i casi in cui l’effetto si è verificato, allora sapere che una delle due cause non c’era ti dice che l’altra probabilmente c’era. La selezione ha creato la dipendenza. È esattamente il collider del capitolo successivo, ed è il motivo per cui la d-separazione ha una regola diversa per le strutture a V .

Esempio svolto — le due direzioni, con numeri minimi

(a) $X \perp Y$ **ma** $X \not\perp Y | Z$. Siano X, Y due lanci di moneta equa indipendenti e $Z = X \oplus Y$ (XOR). Marginalmente $P(X=1, Y=1) = 0.25 = P(X=1)P(Y=1)$: indipendenti.

Condizioniamo su $Z=0$. Allora $X = Y$ necessariamente, quindi $P(X=1 | Y=1, Z=0) = 1 \neq P(X=1 | Z=0) = 0.5$. Da indipendenza totale a dipendenza *deterministica*, solo condizionando.

(b) $X \not\perp Y$ **ma** $X \perp Y | Z$. Sia $Z \sim \text{Bernoulli}(0.5)$ una causa comune, e

$$P(X=1 | Z=0) = 0.1, \quad P(X=1 | Z=1) = 0.9, \quad P(Y=1 | Z=0) = 0.2, \quad P(Y=1 | Z=1) = 0.8,$$

con $X \perp Y | Z$ per costruzione. Marginalizzando:

$$P(X=1) = 0.5, \quad P(Y=1) = 0.5, \quad P(X=1, Y=1) = 0.5(0.1)(0.2) + 0.5(0.9)(0.8) = 0.37.$$

Ma $P(X=1)P(Y=1) = 0.25 \neq 0.37$: marginalmente dipendenti. La dipendenza osservata è tutta prodotta da Z , e sparisce appena Z è noto.

Morale: (a) è il collider, (b) è la causa comune (confounder). Sono i due casi opposti, e la d-separazione serve esattamente a distinguerli guardando solo il grafo.

2.4 Valore Atteso

$$\mathbb{E}_{x \sim p(X)}[f(x)] = \sum_x p(x) f(x) \quad (\text{discreto}), \quad \mathbb{E}[f(x)] = \int p(x) f(x) dx \quad (\text{continuo})$$

L'aspettazione è un operatore lineare: $\mathbb{E}[af(x) + bg(x)] = a \mathbb{E}[f(x)] + b \mathbb{E}[g(x)]$.

Se $X \perp Y$: $\mathbb{E}_{x,y}[f(x)g(y)] = \mathbb{E}_x[f(x)] \cdot \mathbb{E}_y[g(y)]$.

2.5 Distribuzioni Notevoli

2.5.1 Bernoulli e Categoricala

$$\text{Bernoulli}(x \mid \lambda) = \lambda^x (1 - \lambda)^{1-x}, \quad x \in \{0, 1\}$$

$$\text{Cat}(x \mid \boldsymbol{\lambda}) = \prod_{k=1}^K \lambda_k^{\mathbb{I}[x=k]}, \quad \sum_k \lambda_k = 1$$

2.5.2 Gaussiana Univariata e Multivariata

$$\mathcal{N}(x \mid \mu, \sigma^2) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(x - \mu)^2}{2\sigma^2}\right)$$

$$\mathcal{N}(\mathbf{x} \mid \boldsymbol{\mu}, \boldsymbol{\Sigma}) = \frac{1}{(2\pi)^{D/2} |\boldsymbol{\Sigma}|^{1/2}} \exp\left(-\frac{1}{2}(\mathbf{x} - \boldsymbol{\mu})^\top \boldsymbol{\Sigma}^{-1}(\mathbf{x} - \boldsymbol{\mu})\right)$$

2.5.3 Beta e Dirichlet

La distribuzione **Beta**(α, β) con $\alpha, \beta > 0$ è definita su $[0, 1]$:

$$p(x \mid \alpha, \beta) = \frac{\Gamma(\alpha + \beta)}{\Gamma(\alpha)\Gamma(\beta)} x^{\alpha-1} (1 - x)^{\beta-1}$$

È la distribuzione coniugata della Bernoulli/Binomiale.

La distribuzione **Dirichlet**($\boldsymbol{\alpha}$) è la generalizzazione multivariata sul semplice:

$$p(\boldsymbol{\mu} \mid \boldsymbol{\alpha}) = \frac{\Gamma(\sum_k \alpha_k)}{\prod_k \Gamma(\alpha_k)} \prod_{k=1}^K \mu_k^{\alpha_k-1}, \quad \sum_k \mu_k = 1$$

È la distribuzione coniugata della Categoricala.

2.6 Framework dei Modelli Grafici

I modelli probabilistici grafici sono una coppia $(G, \mathcal{P})$ dove il grafo G codifica le assunzioni di indipendenza condizionale. Affrontano tre problemi:

1. **Rappresentazione**: codificare distribuzioni congiunte di alta dimensionalità in modo compatto
2. **Inferenza**: calcolare $P(X \mid \mathbf{d})$ per variabili di interesse X dato evidenza $\mathbf{d}$
3. **Apprendimento**: stimare parametri θ (e struttura G) dai dati

2.6.1 Tipi di Inferenza

Dato un insieme di ipotesi $h \in H$ e osservazioni $\mathbf{d}$:

- **Bayesiana:** $P(X | \mathbf{d}) = \sum_h P(X | h) P(h | \mathbf{d})$ — somma su tutte le ipotesi
- **MAP:** $\hat{h}_{\text{MAP}} = \arg \max_h P(h | \mathbf{d}) = \arg \max_h P(\mathbf{d} | h) P(h)$
- **ML:** $\hat{h}_{\text{ML}} = \arg \max_h P(\mathbf{d} | h)$ — equivalente a MAP con prior uniforme

2.7 Approfondimenti

2.7.1 Entropia e Divergenza KL

Entropia e Informazione

L'**entropia** di una distribuzione discreta misura l'incertezza media:

$$H(X) = - \sum_x P(x) \log P(x) \geq 0$$

La **divergenza di Kullback-Leibler** tra P e Q è:

$$D_{\text{KL}}(P \| Q) = \sum_x P(x) \log \frac{P(x)}{Q(x)} \geq 0$$

con uguaglianza sse $P = Q$. Non è simmetrica in generale.

2.7.2 Domande per l'Esame

1. Dimostrare la regola della catena. Quando è utile sfruttare le indipendenze condizionali?
2. Enunciare il Teorema di Bayes e spiegare prior, likelihood, posterior e evidence.
3. Dare un esempio dove $X \perp Y$ ma $X \not\perp Y | Z$ e viceversa.
4. Scrivere la PDF della Gaussiana multivariata. Cosa rappresenta la matrice Σ ?
5. Confrontare stima ML, MAP e Bayesiana. Pro e contro di ciascun approccio.
6. Qual è il ruolo della distribuzione Dirichlet come prior per distribuzioni categoriche?

Chapter 3

Reti Bayesiane I: Rappresentazione e Fattorizzazione

Risorse video — GDL3 — Reti Bayesiane: rappresentazione

Videolezione del corso: registrazione della lezione GDL3 nella cartella *Recordings* del team del corso (v. Prefazione).

Graphical Models 1 — Christopher Bishop, MLSS 2013 (*MPI-IS, 1h30m*)
Fattorizzazione, plate notation e ancestral sampling nella presentazione originale di Bishop: la fonte da cui derivano quasi tutte le slide su questo argomento.

Introduction to Probabilistic Graphical Models (*Batmanghelich, 1h*)
Alternativa più breve e più diretta se ti serve solo il meccanismo della fattorizzazione.

3.1 Sfida Dimensionale

Rappresentare la distribuzione congiunta di N variabili discrete con k valori richiede $k^N - 1$ parametri. Con $N = 100$ variabili binarie: $2^{100} \approx 10^{30}$ parametri — computazionalmente intrattabile.

La soluzione è sfruttare le *indipendenze condizionali* per fattorizzare la distribuzione in termini più semplici.

3.2 Definizione di Rete Bayesiana

Definizione 3.1. Una **rete Bayesiana** è una coppia $(G, \mathcal{P})$ dove:

- $G = (\mathcal{V}, \mathcal{E})$ è un DAG (*Directed Acyclic Graph*)
- I nodi $v \in \mathcal{V}$ rappresentano variabili casuali
- Gli archi $e \in \mathcal{E}$ codificano relazioni di dipendenza condizionale
- Ad ogni nodo i è associata una CPT (*Conditional Probability Table*): $P(x_i \mid \mathbf{x}_{pa(i)})$

3.2.1 Fattorizzazione Congiunta

La distribuzione congiunta si fattorizza secondo la struttura del grafo:

$$P(x_1, \dots, x_N) = \prod_{i \in \mathcal{V}} P(x_i \mid \mathbf{x}_{pa(i)})$$

Per nodi senza genitori: $P(x_i | \mathbf{x}_{pa(i)}) = P(x_i)$.

Esempio: catena $X_1 \rightarrow X_2 \rightarrow X_3$

$$P(x_1, x_2, x_3) = P(x_1) P(x_2 | x_1) P(x_3 | x_2)$$

Risparmio di parametri: invece di $k^3 - 1$, servono solo $k - 1 + k(k - 1) + k(k - 1)$ parametri.

Esempio svolto — quanto si risparmia davvero: contiamo i parametri

Cinque variabili binarie A, B, C, D, E . Senza struttura, la congiunta costa $2^5 - 1 = 31$ parametri liberi.

Ora imponiamo il DAG $A \rightarrow C, B \rightarrow C, C \rightarrow D, C \rightarrow E$. Ogni nodo costa $(k - 1) \cdot k^{|pa|}$ parametri, con $k = 2$:

$$\underbrace{A : 1}_{\text{no genitori}} + \underbrace{B : 1}_{\text{no genitori}} + \underbrace{C : 1 \cdot 2^2 = 4}_{\text{due genitori}} + \underbrace{D : 1 \cdot 2 = 2}_{\text{un genitore}} + \underbrace{E : 1 \cdot 2 = 2}_{\text{un genitore}} = 10.$$

Da 31 a 10: circa un terzo. E il risparmio esplode con N : per la catena $X_1 \rightarrow X_2 \rightarrow \dots \rightarrow X_{100}$ su variabili binarie servono $1 + 99 \cdot 2 = 199$ parametri contro $2^{100} - 1 \approx 10^{30}$.

Il punto da dire all'orale: il costo è esponenziale nel *numero di genitori*, non nel numero di variabili. Un DAG con 100 nodi ma un solo genitore ciascuno è banale; un DAG con 10 nodi di cui uno ha 9 genitori ha già 512 parametri in quel singolo nodo. La sparsità che conta è quella locale.

3.3 Local Markov Property

Teorema 3.1 (Local Markov Property). In una rete Bayesiana, ogni nodo è condizionalmente indipendente da tutti i suoi non-discendenti, dati i suoi genitori:

$$X_i \perp \text{NonDesc}(X_i) \mid \mathbf{X}_{pa(i)}$$

Questa proprietà è equivalente alla fattorizzazione congiunta ed è la base di tutte le indipendenze nella rete.

3.4 Notazione Plate

La **notazione plate** è una convenzione grafica per rappresentare in modo compatto variabili ripetute in un modello.

Un rettangolo (plate) con etichetta N che racchiude un nodo X rappresenta N copie i.i.d. di X : $X_1, X_2, \dots, X_N$.

Esempio: in un modello generativo per N documenti con K topic, si usano plates annidate per documenti e parole.

3.5 Ancestral Sampling

L'**ancestral sampling** genera campioni dalla distribuzione congiunta in modo efficiente:

1. Ordinare i nodi in ordine topologico (ogni genitore precede i figli)
2. Per ogni nodo i (in questo ordine):

- Se $pa(i) = \emptyset$: campionare $x_i \sim P(x_i)$

- Altrimenti: campionare $x_i \sim P(x_i \mid \mathbf{x}_{pa(i)})$ usando i valori già campionati

L'algoritmo produce campioni corretti da $P(\mathbf{x})$ per qualsiasi DAG.

Esempio svolto — ancestral sampling passo per passo

Rete $A \rightarrow C \leftarrow B$, tutte binarie, con

$$P(A=1) = 0.3, \quad P(B=1) = 0.6, \quad P(C=1 \mid A, B) = \begin{cases} 0.9 & A=1, B=1 \\ 0.5 & A=1, B=0 \\ 0.4 & A=0, B=1 \\ 0.1 & A=0, B=0 \end{cases}$$

Un ordinamento topologico valido è (A, B, C) . Supponiamo di estrarre tre uniformi $u_1 = 0.21$, $u_2 = 0.77$, $u_3 = 0.35$ e di usare la convenzione “ $X=1$ se $u < P(X=1)$ ”.

1. A : $u_1 = 0.21 < 0.3 \Rightarrow A = 1$.
2. B : $u_2 = 0.77 \not< 0.6 \Rightarrow B = 0$.
3. C : i genitori sono ora *fissati* a $(A=1, B=0)$, quindi si legge la riga corrispondente della CPT: $P(C=1 \mid 1, 0) = 0.5$. Poiché $u_3 = 0.35 < 0.5 \Rightarrow C = 1$.

Il campione è $(A, B, C) = (1, 0, 1)$.

Due cose da notare. Primo: non si è mai toccata la congiunta a 8 celle — si sono lette tre tabelline locali. Secondo: l'ordinamento topologico non è un dettaglio implementativo ma la condizione di correttezza, perché quando arriva il turno di un nodo i suoi genitori devono già avere un valore. Se avessimo campionato C per primo non avremmo saputo quale riga della CPT leggere.

Limite da conoscere: l'ancestral sampling genera dalla congiunta, ma *non* sa condizionare su evidenza sui discendenti. Per $P(A \mid C=1)$ questo schema costringe a generare e buttare via i campioni con $C=0$ — che è rejection sampling, con tutta la sua inefficienza (Parte II, capitolo sui metodi di campionamento).

3.6 Approfondimenti

3.6.1 Risparmio di Parametri

Analisi del risparmio

Consideriamo N variabili binarie. Una BN con struttura a catena $X_1 \rightarrow X_2 \rightarrow \dots \rightarrow X_N$ richiede:

$$1 + (N - 1) \cdot 2 = 2N - 1 \text{ parametri}$$

invece di $2^N - 1$. Per $N = 20$: 39 vs 1.048.575 parametri.

3.6.2 Markov Equivalence

Due DAG sono **Markov equivalenti** se codificano esattamente le stesse indipendenze condizionali. Due DAG sono equivalenti sse hanno lo stesso scheletro (archi non orientati) e le stesse v-structure (collider $X \rightarrow Z \leftarrow Y$ con X, Y non connessi). Questo implica che dai soli dati osservazionali si può identificare solo una classe di equivalenza.

3.6.3 Domande per l'Esame

1. Illustrare come una BN permette di ridurre il numero di parametri. Esempio numerico.
2. Enunciare la Local Markov Property e mostrare come segue dalla fattorizzazione.
3. Descrivere l'Ancestral Sampling. Perché produce campioni corretti dalla congiunta?
4. Cosa è la notazione plate? Esempio di utilizzo.
5. Due DAG con strutture diverse possono rappresentare le stesse distribuzioni? Quando?

Chapter 4

Reti Bayesiane II: d-Separazione e Markov Blanket

Risorse video — GDL4 — d-separazione e Markov blanket

Videolezione del corso: registrazione della lezione GDL4 nella cartella *Recordings* del team del corso (v. Prefazione).

Bayesian Networks: Independence and d-Separation (*UCLA Automated Reasoning Group, 50m*)

La spiegazione migliore in circolazione su questo specifico argomento: costruisce le tre strutture una alla volta e mostra perché il collider si comporta al contrario. Se guardi un solo video di tutta la Parte I, guarda questo.

Graphical Models 1 — Christopher Bishop (*MPI-IS, 1h30m*)

Contiene anche d-separazione e Markov blanket, inseriti nel discorso più ampio sulla fattorizzazione.

4.1 d-Separazione

La **d-separazione** è il criterio grafico per determinare le indipendenze condizionali in una BN.

Definizione 4.1. X e Y sono **d-separati** da Z in un DAG ($X \perp_d Y \mid Z$) se ogni percorso non orientato tra X e Y è *bloccato* da Z .

Un percorso è *bloccato* se esiste un nodo W sul percorso tale che:

1. W è una **chain** ($\cdot \rightarrow W \rightarrow \cdot$) o **fork** ($\cdot \leftarrow W \rightarrow \cdot$) e $W \in Z$, oppure
2. W è un **collider** ($\cdot \rightarrow W \leftarrow \cdot$) e né W né alcun suo discendente è in Z

Soundness: se $X \perp_d Y \mid Z$ nel grafo, allora $X \perp Y \mid Z$ nella distribuzione (sotto l'assunzione di fedeltà).

Intuizione — la d-separazione è un problema di tubi, non di frecce

Il modo più rapido per non sbagliare: immagina che l'informazione scorra lungo i percorsi come acqua in un sistema di tubi, e che condizionare su un nodo significhi *chiudere una valvola* in quel punto. Due variabili sono indipendenti se non esiste nessun tubo aperto che le colleghi.

Per chain e fork la regola è quella che ti aspetti: il nodo intermedio è una valvola

normale, osservarlo chiude il passaggio. Sono i due casi in cui il nodo intermedio è la ragione per cui le due variabili si somigliano, quindi conoscerlo esaurisce l'informazione. Il collider è una valvola *montata al contrario*: da chiusa si apre quando la osservi. E c'è di più: si apre anche osservando un qualunque suo discendente, perché un discendente è una prova indiretta sul collider stesso.

Attenzione a un punto che confonde spesso: la direzione delle frecce non dice se il percorso è aperto, dice solo *che tipo di valvola* è ciascun nodo. Si guardano sempre i percorsi *non orientati*: si può risalire una freccia contromano senza problemi.

4.2 Le Tre Strutture Fondamentali

Struttura	Senza cond.	Condiz. su Z
Chain: $X \rightarrow Z \rightarrow Y$	$X \not\perp Y$	$X \perp Y \mid Z$
Fork: $X \leftarrow Z \rightarrow Y$	$X \not\perp Y$	$X \perp Y \mid Z$
Collider: $X \rightarrow Z \leftarrow Y$	$X \perp Y$	$X \not\perp Y \mid Z$

4.2.1 Collider ed Explaining Away

Il comportamento del collider è controintuitivo e importante:

Explaining Away

Se X e Y sono cause indipendenti di Z , marginalmente $X \perp Y$. Ma *condizionando su* Z , X e Y diventano dipendenti: conoscere che Z è avvenuto e che X non è avvenuto aumenta la probabilità di Y .

Esempio: Z = “la strada è bagnata”, X = “è piovuto”, Y = “l'irrigatore è acceso”. Marginalmente $X \perp Y$. Ma se sappiamo Z = vero e X = falso, allora $P(Y = \text{vero} \mid Z, \neg X)$ è alta.

Esempio svolto — la rete dell'allarme: d-separazione e explaining away con i numeri

Rete classica: Furto $\rightarrow$ Allarme $\leftarrow$ Terremoto, e Allarme $\rightarrow$ John, Allarme $\rightarrow$ Mary.

Parte 1 — leggere il grafo.

- $F \perp T$? L'unico percorso è $F \rightarrow A \leftarrow T$, un collider non osservato: *bloccato*. Sì, indipendenti.
- $F \perp T \mid A$? Lo stesso collider è ora osservato: il percorso si *apre*. No, dipendenti.
- $F \perp T \mid J$? J è un discendente di A : apre comunque il collider. No, dipendenti — è l'errore che si fa più spesso.
- $J \perp M \mid A$? L'unico percorso è $J \leftarrow A \rightarrow M$, un fork con il centro osservato: *bloccato*. Sì.
- $J \perp M$? Stesso fork, ma senza osservare A : aperto. No.

Parte 2 — vedere l'explaining away nei numeri. Con $P(F=1) = 0.01$, $P(T=1) = 0.02$ e

$$P(A=1 \mid F, T) = 0.95 \ (1, 1), \quad 0.94 \ (1, 0), \quad 0.29 \ (0, 1), \quad 0.001 \ (0, 0),$$

si ottiene

$$P(F=1) = 0.010 \rightarrow P(F=1 \mid A=1) \approx 0.584 \rightarrow P(F=1 \mid A=1, T=1) \approx 0.032.$$

Il primo salto è l'evidenza che sale lungo la freccia: l'allarme suona, il furto diventa molto più probabile. Il secondo è l'explaining away: scoprire che c'è stato un terremoto *spiega già* l'allarme, e la probabilità del furto crolla da 58% a 3%, quasi tornando al valore a priori.

Il punto concettuale: F e T non si sono mai influenzati causalmente. Quello che è cambiato è che, avendo fissato il loro effetto comune, le due cause si contendono la stessa spiegazione. Questa competizione è tutta la dipendenza introdotta dal condizionamento — e vale la pena dirlo con queste parole all'orale, perché mostra che hai capito che si tratta di un effetto della *selezione*, non della causalità.

4.3 Markov Blanket

Definizione 4.2. Il **Markov Blanket** di un nodo X_i è il minimo insieme che rende X_i condizionalmente indipendente da tutti gli altri nodi:

$$MB(X_i) = \underbrace{\text{Genitori}(X_i)}_{\text{cause dirette}} \cup \underbrace{\text{Figli}(X_i)}_{\text{effetti diretti}} \cup \underbrace{\text{Co-genitori}(X_i)}_{\text{altri genitori dei figli}}$$

$$X_i \perp (\mathcal{V} \setminus \{X_i\} \setminus MB(X_i)) \mid MB(X_i)$$

Il Markov Blanket è fondamentale per:

- Inferenza locale: $P(X_i \mid \text{resto}) = P(X_i \mid MB(X_i))$
- Feature selection: le variabili in $MB(X_i)$ sono le uniche rilevanti per predire X_i

Intuizione — il Markov blanket è “tutto ciò che serve sapere”

Se dovessi predire X_i e potessi osservare qualunque cosa nella rete, quali variabili chiederesti? I genitori, ovviamente: sono le cause dirette. I figli, altrettanto ovviamente: sono le prove. Ma anche i *co-genitori*, cioè gli altri genitori dei tuoi figli — e questa è la parte che sorprende.

Il motivo è l'explaining away appena visto: se osservi un figlio, quel figlio è un collider, e osservarlo mette in competizione X_i con gli altri genitori. Per neutralizzare quella competizione devi sapere anche come sono andati loro. Il co-genitore non è nel blanket perché influenza X_i , ma perché senza di lui l'informazione che arriva dal figlio è ambigua. Da qui la formula: genitori, figli, co-genitori. Nulla di più — tutto il resto della rete diventa irrilevante — e nulla di meno.

4.4 Faithfulness Assumption

Definizione 4.3. Una distribuzione P è **fedele** al DAG G se tutte e sole le indipendenze condizionali in P sono spiegabili dalla d-separazione in G :

$$X \perp Y \mid Z \text{ in } P \Leftrightarrow X \perp_d Y \mid Z \text{ in } G$$

Senza faithfulness, non si può recuperare la struttura dai soli dati.

4.5 Approfondimenti

4.5.1 Algoritmo per Verificare d-Separazione

1. Marcare come osservati tutti i nodi in Z
2. Per ogni percorso non diretto tra X e Y , verificare se è bloccato:
 - Chain/Fork con nodo osservato: bloccato
 - Collider con nodo non osservato (e nessun discendente osservato): bloccato
3. Se tutti i percorsi sono bloccati: $X \perp_d Y \mid Z$

4.5.2 Domande per l'Esame

1. Definire d-separazione e descrivere il criterio per catene, fork e collider.
2. Dato il DAG $A \rightarrow B \leftarrow C \rightarrow D$, verificare se $A \perp D$ e $A \perp D \mid C$.
3. Spiegare il fenomeno di explaining away con un esempio concreto.
4. Definire il Markov Blanket. Perché è importante per feature selection?
5. Cosa afferma la faithfulness assumption? Perché è necessaria per structure learning?

Chapter 5

Causalità e Inferenza Causale

Risorse video — GDL5 — Causalità, do-operator, SCM

Videolezione del corso: registrazione della lezione GDL5 nella cartella *Recordings* del team del corso (v. Prefazione).

Causal Models — lezione 4 del corso Causal Inference (*Brady Neal, 1h*)
SCM, do-operator e modularità con lo stesso formalismo delle slide. Il corso di Neal è la fonte più vicina al taglio di Bacciu su questa parte.

Causal Discovery from Observational Data — lezione 10 (*Brady Neal, 1h*)
Anticipa il capitolo successivo, ma vale la pena vederlo di seguito: rende chiaro perché la causalità non si legge dai dati osservativi senza assunzioni.

5.1 Correlazione vs. Causalità

Principio Fondamentale

Correlazione non implica causalità. Strutture causali radicalmente diverse possono produrre le stesse indipendenze condizionali osservabili. È impossibile distinguerle con soli dati osservazionali.

Esempio: se osserviamo $X \perp Y \mid Z$, questo è compatibile con:

- $X \rightarrow Z \rightarrow Y$ (catena causale)
- $X \leftarrow Z \rightarrow Y$ (causa comune)
- $X \rightarrow Z \leftarrow Y$ (collider, se non si condiziona)

5.2 Reti Bayesiane Causali e l'Operatore $do(\cdot)$

In una **rete Bayesiana causale**, gli archi hanno significato causale: $X \rightarrow Y$ significa che X è causa diretta di Y .

5.2.1 Osservazione vs. Intervento

Definizione 5.1. L'intervento $do(X = x)$ imposta deterministicamente $X = x$, rimuovendo tutti gli archi entranti in X nel grafo causale ("mutilated graph").

$$P(Y | X = x) \neq P(Y | do(X = x)) \quad \text{in generale}$$

La differenza è cruciale: $P(Y | X = x)$ è confusa dalle cause comuni di X e Y ; $P(Y | do(X = x))$ è l'effetto causale puro.

Intuizione — $do(x)$ è chirurgia sul grafo, non condizionamento

$P(y | x)$ e $P(y | do(x))$ rispondono a due domande diverse, e la differenza si vede meglio sul grafo che nelle formule.

$P(y | x)$ significa: *ho osservato* che X vale x ; restringo l'attenzione a quella sottopopolazione. Ma le persone in cui $X = x$ ci sono finite per delle ragioni, e quelle ragioni influenzano anche Y . L'osservazione porta con sé tutto il bagaglio delle cause di X .

$P(y | do(x))$ significa: *ho imposto* $X = x$ dall'esterno. Nessuno ha più scelto X in base alle proprie caratteristiche, quindi tutte le frecce che entravano in X vanno tagliate. Il grafo mutilato $G_{\bar{X}}$ è il modello del mondo dopo l'intervento, e la formula di aggiustamento non è altro che il calcolo di $P(y)$ in quel grafo.

Da qui la regola pratica: la differenza fra i due è tutta nei percorsi *backdoor*, quelli che entrano in X da dietro. Se non ce ne sono, osservare e intervenire coincidono e $P(y | do(x)) = P(y | x)$. È il motivo per cui un esperimento randomizzato funziona: la randomizzazione taglia le frecce entranti *fisicamente*, invece che sulla lavagna.

5.2.2 Formula di Aggiustamento (Adjustment Formula)

Se $\mathbf{Z}$ è un insieme di variabili che blocca tutti i percorsi confondenti tra X e Y (backdoor criterion):

$$P(Y | do(X = x)) = \sum_{\mathbf{z}} P(Y | X = x, \mathbf{Z} = \mathbf{z}) P(\mathbf{Z} = \mathbf{z})$$

Esempio svolto — paradosso di Simpson: aggiustare ribalta la conclusione

Due trattamenti per i calcoli renali, A (chirurgia aperta) e B (percutanea), su 700 pazienti. Il confondente Z è la gravità del caso (calcoli piccoli o grandi), che influenza sia il trattamento scelto sia la guarigione: $Z \rightarrow T$ e $Z \rightarrow R$, oltre a $T \rightarrow R$.

	calcoli piccoli	calcoli grandi	totale
Trattamento A	81/87 = 0.93	192/263 = 0.73	273/350 = 0.78
Trattamento B	234/270 = 0.87	55/80 = 0.69	289/350 = 0.83

Guardando i totali, B sembra migliore (0.83 contro 0.78). Ma A è migliore *in entrambi i sottogruppi*. Questo è il paradosso.

La spiegazione sta nell'assegnazione: ai casi gravi (calcoli grandi, prognosi peggiore) è stato dato quasi sempre A , ai casi lievi quasi sempre B . Il confronto grezzo sta confrontando anche le gravità, non solo i trattamenti.

Applichiamo la formula di aggiustamento su Z , con $P(\text{piccoli}) = 357/700 = 0.51$ e $P(\text{grandi}) = 343/700 = 0.49$:

$$P(R=1 | do(T=A)) = 0.51 \cdot 0.931 + 0.49 \cdot 0.730 = \mathbf{0.833},$$

$$P(R=1 | do(T=B)) = 0.51 \cdot 0.867 + 0.49 \cdot 0.688 = \mathbf{0.779}.$$

Aggiustando, l'ordine si ribalta e coincide con quello dei sottogruppi: A è il trattamento migliore.

Che cosa ha fatto l'aggiustamento: ha ricalcolato le medie usando la *stessa* distribuzione di gravità per entrambi i trattamenti, cioè quella della popolazione complessiva. È esattamente ciò che farebbe un esperimento randomizzato, dove la gravità si distribuisce ugualmente nei due bracci.

Attenzione al rovescio della medaglia: aggiustare non è sempre giusto. Se Z fosse stato un *collider* o una variabile a valle del trattamento (un mediatore), condizionare avrebbe *introdotto* bias invece di rimuoverlo. È il backdoor criterion a dire su cosa si aggiusta, e la risposta dipende dal grafo, non dai dati.

5.3 Modelli Causali Strutturali (SCM)

Definizione 5.2. Un **Modello Causale Strutturale** (SCM) è un sistema di equazioni strutturali:

$$X_i = f_i(PA_i, U_i), \quad i = 1, \dots, N$$

dove PA_i sono i genitori causali di X_i e $U_i \sim p(U_i)$ è un termine di rumore esogeno (indipendente per variabili diverse).

Un intervento $do(X_j = c)$ modifica l'equazione: $X_j = c$ (fisso), rimuovendo l'influenza di PA_j e U_j .

5.4 Gerarchia Causale di Pearl

1. **Livello 1 — Associazione:** $P(Y \mid X = x)$ — da dati osservazionali
2. **Livello 2 — Intervento:** $P(Y \mid do(X = x))$ — richiede esperimenti o do-calculus
3. **Livello 3 — Controfattuali:** “Se avessi fatto $X = x'$ invece di $X = x$, cosa sarebbe successo a Y ?” — richiede SCM

Ogni livello è strettamente più informativo del precedente e non è derivabile da livelli inferiori in generale.

Trappola — “nel dubbio condiziono su tutto”

È l'errore più costoso di tutta la Parte I, perché suona prudente. L'intuizione statistica standard dice: più variabili controllo, meno confondimento resta. Sui grafi causali è falso.

Condizionare su un **collider**, o su un suo discendente, *apre* un percorso che era chiuso e **introduce** una dipendenza spuria che prima non c'era. Condizionare su un **mediatore**, cioè una variabile a valle del trattamento sul cammino causale, blocca parte dell'effetto che si voleva misurare e produce una stima distorta verso il basso.

Quindi non esiste un insieme di aggiustamento “sicuro per costruzione”: l'insieme giusto va scelto, e a dire quale sia è il grafo, tramite il *backdoor criterion*. La regola in due punti: Z non deve contenere discendenti di X , e Z deve bloccare tutti i percorsi backdoor da X a Y .

Come dirlo all'orale: “aggiustare non è gratis: ogni variabile aggiunta può chiudere un percorso spurio ma anche aprirne uno”. È la frase che distingue chi ha capito la d-separazione da chi ha memorizzato la formula di aggiustamento.

5.5 Do-Calculus

Pearl ha formulato tre regole per derivare $P(Y \mid do(X))$ da distribuzioni osservabili, quando possibile. Queste regole determinano la **identificabilità** dell'effetto causale.

Definizione 5.3. Un effetto causale $P(Y \mid do(X = x))$ è **identificabile** se può essere espresso come funzione deterministica della distribuzione osservazionale $P(\mathbf{X})$.

5.6 Approfondimenti

5.6.1 Backdoor Criterion

Un insieme $\mathbf{Z}$ soddisfa il **backdoor criterion** rispetto a (X, Y) se:

1. Nessun nodo in $\mathbf{Z}$ è discendente di X
2. $\mathbf{Z}$ blocca tutti i percorsi “backdoor” tra X e Y (percorsi con archi entranti in X)

Se il backdoor criterion è soddisfatto, la formula di aggiustamento è valida.

5.6.2 Domande per l'Esame

1. Perché correlazione non implica causalità? Esempio con tre strutture causali diverse compatibili.
2. Definire l'operatore $do(\cdot)$. Come differisce da condizionare su un'osservazione?
3. Data una struttura con confondimento $C \rightarrow X, C \rightarrow Y, X \rightarrow Y$, derivare $P(Y \mid do(X))$.
4. Cos'è uno SCM? Come si rappresenta un intervento in uno SCM?
5. Descrivere la gerarchia causale. Perché i controfattuali sono al livello più alto?
6. Cos'è il backdoor criterion? Darne un'applicazione pratica.

Chapter 6

Apprendimento di Strutture (Structure Learning)

Risorse video — GDL6 — Structure learning e causal discovery

Videolezione del corso: registrazione della lezione GDL6 nella cartella *Recordings* del team del corso (v. Prefazione).

Causal Discovery from Observational Data (*Brady Neal, 1h*)

Copre PC, classi di equivalenza di Markov e il ruolo delle assunzioni parametriche: è la mappa completa di questo capitolo.

Causal Models (*Brady Neal, 1h*)

Da rivedere se le nozioni di faithfulness e di equivalenza di Markov non sono ancora solide: qui vengono introdotte con calma.

6.1 Il Problema

Dati N campioni i.i.d. $D = \{\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(N)}\}$ di d variabili, trovare il DAG G^* che ha generato i dati. Le principali difficoltà sono:

- **Markov equivalence:** solo la classe di equivalenza è identificabile dai dati osservazionali
- **Complessità:** il numero di DAG cresce super-esponenzialmente con d
- **Errori statistici:** test di indipendenza con dati finiti sono soggetti a falsi positivi/negativi

6.2 Metodi Basati su Vincoli: PC Algorithm

Usano test di indipendenza condizionale (CI test) per rimuovere archi e orientarli.

6.2.1 Fase 1: Skeleton Learning

1. Iniziare con grafo non diretto completo su d nodi
2. Per ogni coppia (i, j) , testare $X_i \perp X_j \mid \mathbf{S}$ per sottoinsiemi $\mathbf{S}$ crescenti:
 - Se CI trovata: rimuovere arco $i-j$, salvare il separating set $sep(i, j) = \mathbf{S}$

6.2.2 Fase 2: Orientamento dei Collider

1. Per ogni tripletta $i-k-j$ con arco $i-j$ assente: se $k \notin \text{sep}(i, j)$ allora $i \rightarrow k \leftarrow j$

6.2.3 Fase 3: Regole di Meek

Orientare gli archi rimanenti per evitare cicli e nuove v-structure non giustificate.

Proprietà del PC

Sotto faithfulness e con test CI perfetti, PC recupera la CPDAG (Completed Partially DAG) corretta, che rappresenta la Markov equivalence class.

Esempio svolto — PC su tre nodi, a mano

Variabili X, Y, Z con la vera struttura $X \rightarrow Z \leftarrow Y$ (e $X \perp Y$).

Fase 1 — scheletro. Si parte dal grafo completo non orientato: $X - Y$, $X - Z$, $Y - Z$.

- Ordine 0 (nessun condizionante): si testa $X \perp Y$. Il test passa $\Rightarrow$ si rimuove l'arco $X - Y$ e si registra $\text{sepset}(X, Y) = \emptyset$. Si testano anche $X \perp Z$ e $Y \perp Z$: entrambi falliscono, gli archi restano.
- Ordine 1: per l'arco $X - Z$ l'unico condizionante possibile è Y ; il test $X \perp Z \mid Y$ fallisce. Idem per $Y - Z$ con X . Nessuna rimozione.

Scheletro finale: $X - Z - Y$.

Fase 2 — orientamento dei collider. Per ogni tripla non adiacente $X - Z - Y$ (cioè X e Y non collegati) si guarda il sepset : $Z \notin \text{sepset}(X, Y) = \emptyset$, quindi la tripla è un *v-structure* e si orienta $X \rightarrow Z \leftarrow Y$.

Fase 3 — regole di Meek. Non restano archi da orientare. Risultato: la struttura vera, recuperata esattamente.

Ora il caso in cui non funziona. Se la vera struttura fosse stata la catena $X \rightarrow Z \rightarrow Y$, la Fase 1 avrebbe prodotto lo stesso scheletro $X - Z - Y$, ma il test che rimuove $X - Y$ sarebbe passato *condizionando su Z* , cioè con $\text{sepset}(X, Y) = \{Z\}$. Poiché ora $Z \in \text{sepset}$, non è un collider e non si orienta nulla.

Il risultato sarebbe la CPDAG $X - Z - Y$, che rappresenta tre DAG indistinguibili: $X \rightarrow Z \rightarrow Y$, $X \leftarrow Z \leftarrow Y$ e $X \leftarrow Z \rightarrow Y$. Tutti e tre implicano le stesse indipendenze, quindi *nessun* algoritmo basato su dati osservativi può sceglierne uno.

Il punto da portare all'orale: il collider è l'unica struttura con una firma osservazionale propria, ed è per questo che è l'unica che PC riesce a orientare senza assunzioni aggiuntive. Per andare oltre servono interventi (Cap. 5) oppure ipotesi parametriche come gli ANM.

6.3 Metodi Score-based: GES

GES (Greedy Equivalence Search) massimizza uno score di qualità del modello.

6.3.1 Score BIC

$$\text{BIC}(G \mid D) = \log P(D \mid G, \hat{\theta}_G) - \frac{\dim(G)}{2} \log N$$

Il termine di penalità contrasta l'overfitting di grafi troppo complessi.

6.3.2 Algoritmo GES

1. **Fase Forward:** aggiungere archi uno alla volta, scegliendo ogni volta quello che aumenta di più il BIC, finché non si può migliorare
2. **Fase Backward:** rimuovere archi uno alla volta, scegliendo quello che aumenta di più il BIC, finché non si può migliorare

6.4 Assunzioni Parametriche: Additive Noise Models (ANM)

Definizione 6.1. Un modello $Y = f(X) + N$, con $N \perp X$, è un **Additive Noise Model**. Se f è non-lineare o il rumore è non-Gaussiano, la direzione causale $X \rightarrow Y$ è identificabile.

Il modello **LiNGAM** (Linear Non-Gaussian Acyclic Model) assume:

$$\mathbf{x} = B\mathbf{x} + \mathbf{u}$$

dove B è la matrice degli effetti diretti (triangolare inferiore per un DAG) e $\mathbf{u}$ ha componenti indipendenti non-Gaussiane. In questo caso il DAG è identificabile.

6.5 FCI per Variabili Latenti

Quando ci sono variabili latenti confondenti, il PC assume incorrettamente l'assenza di confondimento. L'algoritmo **FCI** (Fast Causal Inference) gestisce variabili latenti usando archi bidirezionali $X \leftrightarrow Y$ per rappresentare possibile confondimento nascosto.

6.6 Approfondimenti

6.6.1 Domande per l'Esame

1. Descrivere le tre fasi dell'algoritmo PC. Perché produce una CPDAG e non un DAG unico?
2. Definire il BIC score. Perché include un termine di penalità?
3. Descrivere GES (fasi forward e backward). Quale proprietà di convergenza ha?
4. Cos'è un Additive Noise Model? Perché consente di identificare la direzione causale?
5. Qual è la differenza tra PC e FCI? Quando si usa FCI?
6. Implementare lo pseudocodice per la skeleton learning del PC.

Part II

Learning nei Modelli Probabilistici

Chapter 7

Apprendimento con Variabili Osservate

Risorse video — GDL7 — ML, MAP, Bayesiano, Naive Bayes

Videolezione del corso: registrazione della lezione GDL7 nella cartella *Recordings* del team del corso (v. Prefazione).

Cornell CS4780 — Lecture 7: Estimating Probabilities from Data (MLE, MAP, Bayesian) (*Kilian Weinberger, 50m*)

Copre esattamente i tre “flavori” di questa lezione e, alla fine, anche la posterior predictive. Weinberger insiste sul punto che qui conta: che cosa cambia davvero fra massimizzare e integrare.

Cornell CS4780 — Lecture 8: Naive Bayes (*Kilian Weinberger, 50m*)

Il seguito naturale, tutto su Naive Bayes e Laplace smoothing.

7.1 Learning come Inferenza

L'apprendimento nel contesto dei modelli probabilistici consiste nel trovare i parametri θ che spieghino meglio i dati osservati D . Data una collezione di osservazioni $D = \{x_1, x_2, \dots, x_N\}$, vogliamo stimare i parametri θ della distribuzione di probabilità sottostante.

Definizione 7.1. L'*apprendimento probabilistico* è il processo di stima dei parametri θ di un modello probabilistico $P(x|\theta)$ dato un dataset osservato D .

7.1.1 Tre Flavori dell'Apprendimento

Esistono tre approcci principali:

Maximum Likelihood (ML): Massimizziamo la verosimiglianza:

$$\theta_{\text{ML}} = \arg \max_{\theta} P(D|\theta) = \arg \max_{\theta} \prod_{n=1}^N P(x_n|\theta)$$

Maximum A Posteriori (MAP): Massimizziamo il posteriore:

$$\theta_{\text{MAP}} = \arg \max_{\theta} P(\theta|D) = \arg \max_{\theta} P(D|\theta)P(\theta)$$

dove $P(\theta)$ è una distribuzione a priori.

Bayesiano: Calcoliamo la distribuzione posteriore:

$$P(\theta|D) = \frac{P(D|\theta)P(\theta)}{P(D)}$$

che rappresenta l'incertezza sui parametri.

Intuizione — tre risposte alla stessa domanda, non tre algoritmi diversi

ML, MAP e Bayesiano non sono tre tecniche in competizione: sono tre livelli di risposta alla domanda “che cosa so di θ dopo aver visto i dati?”.

L'approccio *bayesiano* risponde per intero: restituisce la distribuzione $P(\theta | D)$, cioè tutta l'incertezza residua. È la risposta completa, e anche la più costosa.

MAP risponde con un solo numero: il picco di quella distribuzione. Butta via la larghezza e tiene la posizione. *ML* fa la stessa cosa ma ignorando anche il prior, cioè assumendo implicitamente che tutti i valori di θ fossero ugualmente plausibili prima di vedere i dati.

Da qui due conseguenze che vale la pena avere pronte all'orale. Primo: con molti dati la likelihood domina il prior e le tre risposte convergono — la differenza si vede solo quando i dati sono pochi. Secondo, e più sottile: ML e MAP collassano una distribuzione in un punto, e un punto non può rappresentare “non lo so”. È esattamente questa perdita che la posterior predittiva evita, integrando su θ invece che scegliere.

7.2 MLE per Reti Bayesiane Completamente Osservate

Per una rete Bayesiana con tutti i nodi osservati, la log-verosimiglianza è:

$$\ell(\theta) = \log P(D|\theta) = \sum_{n=1}^N \log P(x_n|\theta) = \sum_{n=1}^N \sum_i \log P(x_i^{(n)} | \text{pa}(x_i), \theta_i)$$

Poiché i termini riguardanti nodi diversi si separano, possiamo massimizzare i parametri per ciascuna famiglia condizionale indipendentemente.

Teorema 7.1. Per distribuzioni categoriche, lo stimatore di massima verosimiglianza è:

$$\theta_{\text{ML},ik} = \frac{\#(x_i = k, \text{pa}(x_i) = j)}{\#(\text{pa}(x_i) = j)}$$

cioè il rapporto tra la frequenza congiunta osservata e la frequenza marginale dei genitori.

7.2.1 Stability Numerica

La log-verosimiglianza è più stabile numericamente della verosimiglianza:

$$\ell(\theta) = \sum_{n=1}^N \log P(x_n|\theta) \quad \text{vs} \quad L(\theta) = \prod_{n=1}^N P(x_n|\theta)$$

Lavorare direttamente con le probabilità produce underflow in problemi realistici.

Trappola — MAP non è invariante per riparametrizzazione, ML sì

Punto sottile e perfettamente chiedibile, perché smonta l'idea che MAP sia “ML con un prior e basta”.

La stima ML è invariante: se g è una trasformazione biettiva dei parametri, allora

$\hat{\theta}_{\text{ML}}$ trasformato è la stima ML nel nuovo sistema di coordinate, $g(\hat{\theta}_{\text{ML}}) = \widehat{g(\theta)}_{\text{ML}}$. Il motivo è che la likelihood è una *funzione* di θ , non una densità su θ : riparametrizzare rietichetta l'asse, e il punto di massimo si sposta insieme all'etichetta.

La stima MAP non lo è. Il posterior è una densità su θ , quindi sotto un cambio di variabile acquista un fattore Jacobiano:

$$p(\eta \mid D) = p(\theta \mid D) \left| \frac{d\theta}{d\eta} \right|.$$

Se la trasformazione è non lineare quel fattore non è costante, quindi *sposta la moda*. Massimizzare in θ e massimizzare in $\eta = g(\theta)$ danno risposte diverse, che non sono l'una la trasformata dell'altra.

La conseguenza concettuale: la moda di una distribuzione non è una quantità geometricamente robusta, dipende dalla parametrizzazione scelta. La *media* a posteriori e l'intera predittiva bayesiana non hanno questo problema, perché integrano invece di massimizzare — e l'integrale si trasforma correttamente con il Jacobiano. È un altro argomento, e più profondo del solito “il bayesiano tiene conto dell'incertezza”, a favore dell'integrare.

7.3 Classificatori Naive Bayes

Il classificatore Naive Bayes assume indipendenza condizionale dei features dato la classe:

Definizione 7.2. Un modello Naive Bayes per la classificazione è definito da:

$$P(y, x_1, \dots, x_D) = P(y) \prod_{d=1}^D P(x_d|y)$$

Questa assume che $P(x_d|y, x_1, \dots, x_{d-1}) = P(x_d|y)$ per ogni feature d .

7.3.1 Inferenza nel Naive Bayes

Per classificare un nuovo esempio, calcoliamo:

$$\hat{c} = \arg \max_c P(c|x) = \arg \max_c \frac{P(c) \prod_{d=1}^D P(x_d|c)}{P(x)}$$

Poiché $P(x)$ è costante rispetto a c , possiamo omettere il denominatore:

$$\hat{c} = \arg \max_c P(c) \prod_{d=1}^D P(x_d|c)$$

7.3.2 MAP Estimate e Laplace Smoothing

Lo stimatore MAP per un Naive Bayes con prior Dirichlet evita il problema delle probabilità zero:

$$\theta_{\text{MAP},ik} = \frac{\#(x_i = k, y = c) + \alpha}{\#(y = c) + K\alpha}$$

dove K è il numero di categorie e $\alpha > 0$ è il parametro di smoothing. Con $\alpha = 1$ si parla di *add-one smoothing* o *Laplace smoothing*.

Nota

Il Laplace smoothing previene le probabilità zero dovute a dati limitati, cruciale quando i dati di training non coprono tutte le possibili combinazioni.

7.4 Generativo vs Discriminativo

I modelli generativi (come Naive Bayes) modellano $P(x, y)$, mentre i discriminativi modellano direttamente $P(y|x)$. I modelli discriminativi spesso richiedono meno dati per le stesse prestazioni su compiti di classificazione, ma i generativi possono gestire dati mancanti e trasferimento di conoscenza più facilmente.

7.4.1 Inferenza Bayesiana

L'approccio bayesiano mantiene la distribuzione posteriore completa su θ :

$$P(\theta|D) = \frac{P(D|\theta)P(\theta)}{P(D)}$$

Per la predizione, marginalizziamo:

$$P(x_{\text{new}}|D) = \int P(x_{\text{new}}|\theta)P(\theta|D)d\theta$$

Questo è più robusto di ML/MAP poiché incorpora l'incertezza su θ .

Esempio svolto — tre monete lanciate, tre teste: ML, MAP e predittiva a confronto

Lanciamo una moneta 3 volte e otteniamo 3 teste. Sia $\lambda = P(\text{testa})$.

ML. La likelihood è λ^3 , massimizzata in

$$\hat{\lambda}_{\text{ML}} = 1.$$

Il modello dichiara che la moneta *non può* dare croce. Con tre soli lanci è una conclusione assurda, ed è il classico problema della probabilità zero.

MAP con prior Beta(2, 2). Il prior $\text{Beta}(\alpha, \beta)$ è coniugato: il posterior è $\text{Beta}(\alpha + h, \beta + n - h) = \text{Beta}(5, 2)$. La moda di una Beta è $\frac{\alpha' - 1}{\alpha' + \beta' - 2}$, quindi

$$\hat{\lambda}_{\text{MAP}} = \frac{5 - 1}{5 + 2 - 2} = \frac{4}{5} = 0.8.$$

Il prior $\text{Beta}(2, 2)$ vale “come” aver visto una testa e una croce fittizie: basta a impedire lo 0 e l'1.

Predittiva bayesiana. Non scegliamo nessun λ : integriamo.

$$P(x' = \text{testa} | D) = \int_0^1 \underbrace{P(x' = \text{testa} | \lambda)}_{=\lambda} P(\lambda | D) d\lambda = \mathbb{E}[\lambda | D] = \frac{\alpha + h}{\alpha + \beta + n} = \frac{5}{7} \approx 0.714.$$

Che cosa distingue i tre numeri. 1, 0.8, 0.714: la stessa evidenza, tre risposte. La predittiva è la più prudente perché è l'unica che tiene conto del fatto che λ resta incerto — è una *media pesata* delle predizioni di tutti i valori di λ , pesati dalla loro plausibilità, non la predizione del λ più plausibile.

Nota sulla notazione, che è fonte di confusione ricorrente. Nella formula della predittiva x' è un *nuovo dato osservabile* (qui: l'esito del quarto lancio), non un parametro. Le due distribuzioni non vanno confuse: $P(\theta | D)$ è il posterior e vive nello spazio dei parametri; $P(x' | D) = \int P(x' | \theta)P(\theta | D) d\theta$ è la *posterior predittiva* e vive nello spazio dei dati. L'integrale è il ponte fra i due spazi: elimina θ mediando su di esso.

7.5 Approfondimenti

Domande d'Esame

1. Distinguere tra Maximum Likelihood, MAP e approccio Bayesiano. Quali sono i vantaggi e svantaggi di ciascuno?
2. Derivare lo stimatore ML per i parametri di una distribuzione categorica.
3. Spiegare come il Laplace smoothing affronta il problema delle probabilità zero nei classificatori Naive Bayes.
4. Perché nei classificatori Naive Bayes il denominatore $P(x)$ può essere omissso nella fase di predizione?
5. Come si generalizza il learning MLE a reti Bayesiane arbitrarie?

Chapter 8

Apprendimento con Variabili Nascoste: l'Algoritmo EM

Risorse video — GDL8 — EM e mixture of Gaussians

Videolezione del corso: registrazione della lezione GDL8 nella cartella *Recordings* del team del corso (v. Prefazione).

Probability 2: Maximum Likelihood, Gaussian Mixture Models and Expectation Maximization (*MLVU 2019, 1h*)

Stessa impostazione del Midterm 1: parte dal problema del “log della somma” e ci costruisce sopra EM. Se ti serve solo il meccanismo E-step/M-step, i primi 30 minuti bastano.

8.1 Il Problema delle Variabili Latenti

Molti modelli probabilistici contengono variabili nascoste (latenti) z che non sono osservate nei dati. Questo complica significativamente il learning.

Definizione 8.1. Date osservazioni $D = \{x_1, \dots, x_N\}$ e variabili nascoste $z = \{z_1, \dots, z_N\}$:

- *Dati completi:* (x, z) — entrambi osservati
- *Dati incompleti:* x soltanto — z è nascosto

La log-verosimiglianza dei dati completi è:

$$\ell_c(\theta) = \log P(x, z|\theta)$$

Ma dato che z non è osservato, non possiamo massimizzare direttamente. La log-verosimiglianza marginale è:

$$\ell(\theta) = \log P(x|\theta) = \log \sum_z P(x, z|\theta)$$

Il problema è che il logaritmo della somma non fattorizza, rendendo l'ottimizzazione difficile.

Intuizione — l'uovo e la gallina, risolto barando in modo onesto

Il problema con le variabili latenti è circolare. Se sapessi da quale componente viene ogni punto, stimare i parametri sarebbe banale: basterebbero medie e varianze per

gruppo, come nel caso completamente osservato. E se conoscessi i parametri, capire da dove viene ogni punto sarebbe altrettanto banale: basterebbe Bayes. Il guaio è che non ho né gli uni né gli altri.

EM rompe il circolo alternando, ma con un'accortezza decisiva: invece di *indovinare* l'assegnazione di ogni punto e poi trattarla come vera, si tiene la distribuzione completa sull'assegnazione. Nell'E-step ogni punto viene diviso fra le componenti in proporzione a quanto ciascuna lo spiega; nell'M-step ogni componente viene ristimata usando quelle frazioni come pesi.

È questa la differenza con k-means, che invece assegna ogni punto a un solo cluster. L'assegnazione soft è ciò che rende la log-verosimiglianza monotona: si sta massimizzando un lower bound che, alla fine dell'E-step, tocca esattamente la curva vera. Ogni salita sul bound è quindi anche una salita, o almeno un pareggio, sulla funzione obiettivo.

Il prezzo: monotono non vuol dire globale. EM sale sempre, ma sale verso il massimo locale più vicino all'inizializzazione. Ripartenze multiple non sono un dettaglio implementativo, sono parte dell'algoritmo.

8.2 L'Algoritmo Expectation-Maximization

L'EM è un algoritmo iterativo che alternativamente massimizza un'evidenza lower bound (ELBO).

Teorema 8.1 (Algoritmo EM). Inizializzare $\theta^{(0)}$. Ripetere fino a convergenza:

- **E-step:** Calcolare le responsabilità

$$Q(\theta, \theta^{\text{old}}) = \mathbb{E}_{z|x, \theta^{\text{old}}} [\log P(x, z|\theta)] = \sum_z P(z|x, \theta^{\text{old}}) \log P(x, z|\theta)$$

- **M-step:** Aggiornare i parametri

$$\theta^{\text{new}} = \arg \max_{\theta} Q(\theta, \theta^{\text{old}})$$

8.2.1 Proprietà di Convergenza

L'EM garantisce l'aumento monotono della log-verosimiglianza osservata:

Teorema 8.2. Per ogni iterazione dell'EM:

$$\log P(x|\theta^{\text{new}}) \geq \log P(x|\theta^{\text{old}})$$

Questo è perché l'ELBO $L(q, \theta)$ fornisce un lower bound sulla log-verosimiglianza, e EM massimizza questo lower bound ad ogni passo.

8.3 Mixture of Gaussians

Un'applicazione importante di EM è l'apprendimento di mixture of Gaussians (GMM).

Definizione 8.2. Un GMM con K componenti è:

$$P(x_n|\theta) = \sum_{k=1}^K \pi_k \mathcal{N}(x_n|\mu_k, \Sigma_k)$$

dove π_k sono i pesi, μ_k le medie, e Σ_k le covarianze.

Nel modello latente, $z_n \in \{1, \dots, K\}$ indica da quale componente viene generato x_n .

8.3.1 E-step di EM per GMM

Nel E-step, calcoliamo le *responsabilità* (probabilità posteriore che il punto n appartenga a cluster k):

$$r_{nk} = P(z_n = k | x_n, \theta) = \frac{\pi_k \mathcal{N}(x_n | \mu_k, \Sigma_k)}{\sum_{j=1}^K \pi_j \mathcal{N}(x_n | \mu_j, \Sigma_j)}$$

8.3.2 M-step di EM per GMM

Nel M-step, aggiorniamo i parametri:

$$\mu_k^{\text{new}} = \frac{\sum_{n=1}^N r_{nk} x_n}{\sum_{n=1}^N r_{nk}} \quad (8.1)$$

$$\Sigma_k^{\text{new}} = \frac{\sum_{n=1}^N r_{nk} (x_n - \mu_k)(x_n - \mu_k)^T}{\sum_{n=1}^N r_{nk}} \quad (8.2)$$

$$\pi_k^{\text{new}} = \frac{\sum_{n=1}^N r_{nk}}{N} \quad (8.3)$$

Esempio svolto — un giro completo di EM su quattro punti

Dati $x = \{1, 2, 3, 6\}$ in una dimensione, $K = 2$ componenti. Inizializziamo

$$\mu_1 = 1.5, \quad \mu_2 = 5.0, \quad \sigma_1^2 = \sigma_2^2 = 2, \quad \pi_1 = \pi_2 = 0.5.$$

E-step. Per ogni punto si calcola $\pi_k \mathcal{N}(x | \mu_k, \sigma_k^2)$ e si normalizza. Poiché π e σ^2 sono uguali per le due componenti, il rapporto si riduce a $\exp(-\frac{(x-\mu_1)^2 - (x-\mu_2)^2}{2\sigma^2})$. Per $x = 3$:

$$\frac{r_1}{r_2} = \exp\left(-\frac{(3-1.5)^2 - (3-5)^2}{4}\right) = \exp\left(\frac{4-2.25}{4}\right) = e^{0.4375} \approx 1.549 \Rightarrow r_1 = \frac{1.549}{2.549} \approx 0.608.$$

Ripetendo per tutti:

x	1	2	3	6
$r_{.1}$	0.981	0.899	0.608	0.008
$r_{.2}$	0.019	0.101	0.392	0.992

Si noti $x = 3$: la sua appartenenza è genuinamente divisa, 61% contro 39%. È il punto che k-means avrebbe assegnato d'ufficio alla prima componente, perdendo l'informazione che è un caso incerto.

M-step. Le masse effettive sono $N_1 = 2.496$ e $N_2 = 1.504$ (somme di colonna, e si noti che $N_1 + N_2 = 4$ come deve essere). Poi

$$\mu'_1 = \frac{0.981(1) + 0.899(2) + 0.608(3) + 0.008(6)}{2.496} \approx 1.863, \quad \mu'_2 \approx 4.886,$$

$$\sigma_1^{2'} \approx 0.670, \quad \sigma_2^{2'} \approx 2.497, \quad \pi'_1 = \frac{2.496}{4} = 0.624, \quad \pi'_2 = 0.376.$$

Verifica della monotonia. La log-verosimiglianza passa da -8.140 a -7.287 : è salita, come garantito dal teorema. E si vede *perché* è salita: la prima componente si è stretta (σ^2 da 2 a 0.67) attorno ai tre punti che la rivendicano, e si è presa un peso maggiore (π da 0.5 a 0.62) perché di punti ne ha attirati di più.

Da dire all'orale: il singolo numero che riassume l'E-step è r_{nk} , e la sua interpretazione è “il punto n conta per la frazione r_{nk} nella stima della componente k ”. L'M-step non è altro che la stima ML del caso completamente osservato, con i conteggi interi sostituiti da questi conteggi frazionari.

8.3.3 Hard vs Soft Assignment

EM fornisce un'assegnazione *soft* (probabilistica) ai cluster. Alternativamente, si può usare il *hard assignment*:

$$z_n = \arg \max_k r_{nk}$$

che corrisponde al clustering k-means.

8.4 EM per Reti Bayesiane con Variabili Nascoste

EM si generalizza a qualsiasi rete Bayesiana. L'E-step comporta l'inferenza della distribuzione posteriore sui nodi nascosti dato i dati osservati (usando algoritmi come belief propagation). L'M-step usa il posteriore per aggiornare i parametri come nel caso completamente osservato.

8.5 Approfondimenti

Domande d'Esame

1. Derivare l'algoritmo EM da principi variazionali. Come si relaziona al lower bound ELBO?
2. Spiegare come EM garantisce l'aumento monotono della log-verosimiglianza.
3. Derivare le equazioni di aggiornamento E-step e M-step per un GMM.
4. Quale è la differenza tra EM e k-means? Come si relazionano?
5. Come si estende EM a reti Bayesiane arbitrarie? Quali sono i costi computazionali?

Chapter 9

Modelli Markov Nascosti

Risorse video — GDL9–10 — HMM: forward-backward, Viterbi, Baum-Welch

Videolezione del corso: registrazione delle lezioni GDL9 e GDL10 nella cartella *Recordings* del team del corso (v. Prefazione).

MIT CompBio — Lecture 05: Hidden Markov Models II (*Manolis Kellis, 1h20m*)

Forward, backward, Viterbi e Baum-Welch svolti su un esempio concreto (rilevamento di isole CpG). Kellis fa girare gli algoritmi a mano sulle slide: è il modo più veloce per capire che cosa contengono davvero α e δ .

9.1 Modellazione di Sequenze

I Modelli Markov Nascosti (Hidden Markov Models, HMM) modellano sequenze di osservazioni dove la dinamica sottostante è governata da una catena Markov nascosta.

Definizione 9.1. Un HMM è definito da:

- Sequenza di stati nascosti: $z = (z_1, z_2, \dots, z_T)$, $z_t \in \{1, \dots, K\}$
- Sequenza di osservazioni: $x = (x_1, x_2, \dots, x_T)$
- Distribuzione iniziale: $\pi_i = P(z_1 = i)$
- Matrice di transizione: $A_{ij} = P(z_t = j | z_{t-1} = i)$
- Matrice di emissione: $B_{ik} = P(x_t = k | z_t = i)$

La distribuzione congiunta è:

$$P(x_{1:T}, z_{1:T}) = P(z_1)P(x_1|z_1) \prod_{t=2}^T P(z_t|z_{t-1})P(x_t|z_t)$$

9.2 Tre Problemi Fondamentali

9.2.1 Problema 1: Valutazione (Forward Algorithm)

Calcolare la probabilità delle osservazioni: $P(x_{1:T}|\lambda)$.

L'algoritmo *forward* usa la programmazione dinamica:

Definizione 9.2. Definiamo la variabile forward:

$$\alpha_t(i) = P(x_{1:t}, z_t = i | \lambda)$$

Teorema 9.1 (Forward Algorithm). • **Inizializzazione:** $\alpha_1(i) = \pi_i B_{ix_1}$

- **Ricorsione:** $\alpha_t(i) = \left[\sum_{j=1}^K \alpha_{t-1}(j) A_{ji} \right] B_{ix_t}$
- **Terminazione:** $P(x_{1:T} | \lambda) = \sum_{i=1}^K \alpha_T(i)$

Intuizione — α è una somma, δ è un massimo — tutta la differenza è lì

Forward e Viterbi hanno la stessa struttura, la stessa complessità $\mathcal{O}(K^2T)$ e ricorsioni che differiscono per un solo simbolo. Ma rispondono a domande diverse, e confonderle è l'errore più comune su questo argomento.

$\alpha_t(i) = P(x_{1:t}, z_t = i)$ raccoglie *tutti* i modi di arrivare nello stato i al tempo t avendo osservato quel prefisso, e li somma. Serve a valutare quanto la sequenza osservata sia probabile sotto il modello: è una marginalizzazione.

$\delta_t(i)$ tiene invece solo il *migliore* di quei modi, e memorizza da dove veniva. Serve a ricostruire una singola sequenza di stati: è un'ottimizzazione.

Il corollario che vale la pena saper dire: la sequenza di Viterbi non coincide in generale con la sequenza degli stati singolarmente più probabili $\arg \max_i \gamma_t(i)$. La prima massimizza $P(z_{1:T} | x)$ globalmente e per costruzione è un cammino percorribile; la seconda massimizza T marginali separate e può produrre una sequenza con probabilità congiunta *nulla*, se contiene una transizione vietata.

9.2.2 Problema 2: Decodifica (Viterbi Algorithm)

Trovare la sequenza più probabile di stati: $\arg \max_{z_{1:T}} P(z_{1:T} | x_{1:T})$.

L'algoritmo di Viterbi usa max-product invece di sum-product:

Definizione 9.3. Definiamo:

$$\delta_t(i) = \max_{z_{1:t-1}} P(x_{1:t}, z_t = i, z_{1:t-1} | \lambda)$$

Teorema 9.2 (Viterbi Algorithm). • **Inizializzazione:** $\delta_1(i) = \pi_i B_{ix_1}$

- **Ricorsione:** $\delta_t(i) = [\max_j \delta_{t-1}(j) A_{ji}] B_{ix_t}$
- **Traceback:** registrare $\psi_t(i) = \arg \max_j \delta_{t-1}(j) A_{ji}$ e ricostruire il path

9.2.3 Problema 3: Learning (Baum-Welch Algorithm)

Stimare i parametri $\lambda = (\pi, A, B)$ dai dati.

Esempio svolto — forward e Viterbi su tre osservazioni

Due stati $\{S_1, S_2\}$, due simboli $\{a, b\}$, con

$$\pi = (0.6, 0.4), \quad A = \begin{pmatrix} 0.7 & 0.3 \\ 0.4 & 0.6 \end{pmatrix}, \quad B = \begin{pmatrix} 0.5 & 0.5 \\ 0.1 & 0.9 \end{pmatrix},$$

dove $A_{ij} = P(z_t=j | z_{t-1}=i)$ e $B_{ik} = P(x_t=k | z_t=i)$. Osserviamo $x = (a, b, b)$.

Forward.

$$\alpha_1 = (0.6 \cdot 0.5, 0.4 \cdot 0.1) = (0.300, 0.040).$$

$$\alpha_2(1) = (0.300 \cdot 0.7 + 0.040 \cdot 0.4) \cdot 0.5 = 0.226 \cdot 0.5 = 0.1130,$$

$$\alpha_2(2) = (0.300 \cdot 0.3 + 0.040 \cdot 0.6) \cdot 0.9 = 0.114 \cdot 0.9 = 0.1026.$$

$$\alpha_3(1) = (0.1130 \cdot 0.7 + 0.1026 \cdot 0.4) \cdot 0.5 = 0.06007, \quad \alpha_3(2) = (0.1130 \cdot 0.3 + 0.1026 \cdot 0.6) \cdot 0.9 = 0.08591.$$

$$P(x) = 0.06007 + 0.08591 = \mathbf{0.14598}.$$

Viterbi. Identico, con max al posto di $\sum$ e tenendo il backpointer:

$$\delta_1 = (0.300, 0.040).$$

$$\delta_2(1) = \max(0.300 \cdot 0.7, 0.040 \cdot 0.4) \cdot 0.5 = 0.210 \cdot 0.5 = 0.1050 \quad (\text{da } S_1),$$

$$\delta_2(2) = \max(0.300 \cdot 0.3, 0.040 \cdot 0.6) \cdot 0.9 = 0.090 \cdot 0.9 = 0.0810 \quad (\text{da } S_1),$$

$$\delta_3(1) = \max(0.1050 \cdot 0.7, 0.0810 \cdot 0.4) \cdot 0.5 = 0.03675 \quad (\text{da } S_1),$$

$$\delta_3(2) = \max(0.1050 \cdot 0.3, 0.0810 \cdot 0.6) \cdot 0.9 = 0.04374 \quad (\text{da } S_2).$$

Il massimo finale è $\delta_3(2) = 0.04374$; risalendo i backpointer si ottiene il cammino

$$z^* = (S_1, S_2, S_2).$$

Che cosa confrontare. $P(x) = 0.146$ è la somma su tutti gli $2^3 = 8$ cammini; 0.04374 è il contributo del solo cammino migliore. Il rapporto $0.04374/0.14598 \approx 0.30$ dice che il cammino di Viterbi, per quanto ottimo, si porta dietro appena il 30% della probabilità totale: gli altri sette cammini insieme pesano più di lui. È l'argomento per cui in molte applicazioni si preferisce la marginale γ_t alla decodifica di Viterbi.

Nota pratica: per T grande questi numeri vanno a underflow. Nella pratica si lavora in log (Viterbi diventa somma di log e max) oppure si normalizza α_t a ogni passo tenendo da parte i fattori di scala.

9.3 Algoritmo Backward

Per il learning, abbiamo bisogno anche dell'algoritmo backward:

Definizione 9.4.

$$\beta_t(i) = P(x_{t+1:T} | z_t = i, \lambda)$$

Teorema 9.3 (Backward Algorithm). • **Inizializzazione:** $\beta_T(i) = 1$ per tutti gli i

• **Ricorsione:** $\beta_t(i) = \sum_{j=1}^K A_{ij} B_{j x_{t+1}} \beta_{t+1}(j)$

9.4 Baum-Welch: EM per HMM

Baum-Welch è l'applicazione di EM ai HMM.

9.4.1 E-step

Calcoliamo le probabilità posteriori sui stati:

$$P(x_{1:T} | \lambda) = \sum_i \alpha_T(i)$$

$$\gamma_t(i) = P(z_t = i | x_{1:T}, \lambda) = \frac{\alpha_t(i) \beta_t(i)}{P(x_{1:T} | \lambda)}$$

$$\xi_t(i, j) = P(z_t = i, z_{t+1} = j | x_{1:T}, \lambda) = \frac{\alpha_t(i) A_{ij} B_{j x_{t+1}} \beta_{t+1}(j)}{P(x_{1:T} | \lambda)}$$

9.4.2 M-step

Aggiornamenti dei parametri:

$$\pi_i^{\text{new}} = \gamma_1(i) \quad (9.1)$$

$$A_{ij}^{\text{new}} = \frac{\sum_{t=1}^{T-1} \xi_t(i, j)}{\sum_{t=1}^{T-1} \gamma_t(i)} \quad (9.2)$$

$$B_{ik}^{\text{new}} = \frac{\sum_{t=1}^T \gamma_t(i) \mathbb{I}[x_t = k]}{\sum_{t=1}^T \gamma_t(i)} \quad (9.3)$$

9.5 Reti Bayesiane Dinamiche

Gli HMM sono un caso speciale delle reti Bayesiane dinamiche, dove una slice temporale dipende dalla precedente. La struttura può essere più generale di una semplice catena di Markov.

9.6 Approfondimenti

Domande d'Esame

1. Derivare l'algoritmo forward per gli HMM e spiegare la complessità computazionale.
2. Spiegare la differenza tra forward e Viterbi algorithm. Quale problema risolve ciascuno?
3. Derivare le equazioni di Baum-Welch da EM applicato ai HMM.
4. Come si combina l'algoritmo forward e backward per il learning?
5. Discutere l'estensione degli HMM a reti Bayesiane dinamiche più generali.

Chapter 10

Inferenza Variazionale

Risorse video — GDL11 — Inferenza variazionale ed ELBO

Videolezione del corso: registrazione della lezione GDL11 nella cartella *Recordings* del team del corso (v. Prefazione).

Variational Inference: Foundations and Innovations (Part 1) (*David Blei, MLSS 2019, 1h30m*)

Blei è l'autore di riferimento sia per la VI moderna sia per LDA (capitolo seguente). Costruisce l'ELBO dalla KL e mostra il mean field come scelta di famiglia variazionale, non come approssimazione arbitraria.

10.1 Il Problema dell'Inferenza Intractable

In molti modelli, il posteriore $P(z|x, \theta)$ non è trattabile analiticamente. L'inferenza variazionale approssima il posteriore con una distribuzione dall'interno di una famiglia trattabile.

Definizione 10.1. L'inferenza variazionale approssima un posteriore intractable $P(z|x)$ con una distribuzione $q(z)$ da una famiglia $\mathcal{Q}$ più semplice, minimizzando una divergenza (tipicamente KL).

10.2 Evidence Lower Bound (ELBO)

La chiave è il seguente lower bound sulla log-verosimiglianza osservata:

Teorema 10.1. Per qualsiasi distribuzione $q(z)$:

$$\log P(x|\theta) = \mathbb{E}_q[\log P(x, z|\theta)] - \mathbb{E}_q[\log q(z)] + \text{KL}(q(z)||P(z|x, \theta))$$

Poiché il termine KL è non-negativo:

$$\log P(x|\theta) \geq \underbrace{\mathbb{E}_q[\log P(x, z|\theta)] - \mathbb{E}_q[\log q(z)]}_{\mathcal{L}(q, \theta) := \text{ELBO}}$$

Equivalentemente:

$$\mathcal{L}(q, \theta) = \mathbb{E}_q[\log P(x|z, \theta)] - \text{KL}(q(z)||P(z|\theta))$$

L'ELBO è il lower bound sulla verosimiglianza che vogliamo massimizzare.

Intuizione — l'ELBO è la log-evidenza meno quanto sbagli

L'identità da avere sempre in testa è

$$\log p(x) = \underbrace{\mathcal{L}(q)}_{\text{ELBO}} + \underbrace{D_{\text{KL}}(q(z) \| p(z | x))}_{\geq 0}.$$

Letta così, dice tre cose in una riga.

Primo: poiché la KL è non negativa, l'ELBO sta sempre *sotto* $\log p(x)$. Da qui il nome, lower bound.

Secondo: $\log p(x)$ non dipende da q . Quindi alzare l'ELBO e abbassare la KL sono *esattamente* la stessa operazione: la somma è vincolata a restare costante. È il motivo per cui possiamo massimizzare una quantità calcolabile (l'ELBO) e ottenere gratis ciò che ci interessa davvero (avvicinare q al posterior, che invece non sappiamo calcolare). Terzo: il gap fra ELBO e log-evidenza è l'errore di approssimazione. Se la famiglia variazionale contenesse il vero posterior, il massimo dell'ELBO sarebbe $\log p(x)$ esattamente, con KL nulla.

Perché mean field paga un prezzo: imporre $q(z) = \prod_i q_i(z_i)$ significa scegliere una famiglia che non può rappresentare correlazioni fra le componenti latenti. Se il vero posterior le ha, la KL non potrà mai annullarsi. E poiché si minimizza $D_{\text{KL}}(q \| p)$ e non $D_{\text{KL}}(p \| q)$, l'approssimazione risultante è *mode-seeking*: si stringe su una moda e sottostima la varianza, invece di allargarsi a coprire tutto il supporto.

Esempio svolto — l'identità dell'ELBO verificata su due stati

Modello minimo: $z \in \{1, 2\}$ con $p(z) = (0.5, 0.5)$ e, per la x osservata, $p(x | z=1) = 0.8$, $p(x | z=2) = 0.2$.

Evidenza e posterior esatti:

$$p(x) = 0.5(0.8) + 0.5(0.2) = 0.5, \quad \log p(x) = -0.6931,$$

$$p(z | x) = \left(\frac{0.4}{0.5}, \frac{0.1}{0.5} \right) = (0.8, 0.2).$$

Proviamo tre q diverse. Con $\mathcal{L}(q) = \sum_z q(z) \log \frac{p(x, z)}{q(z)}$ e $p(x, z) = (0.4, 0.1)$:

q	$\mathcal{L}(q)$	$D_{\text{KL}}(q \ p(z x))$	somma
(0.6, 0.4)	-0.7978	0.1046	-0.6931
(0.9, 0.1)	-0.7298	0.0367	-0.6931
(0.8, 0.2) = $p(z x)$	-0.6931	0	-0.6931

Il conto per la prima riga, esplicitato:

$$\mathcal{L} = 0.6 \log \frac{0.4}{0.6} + 0.4 \log \frac{0.1}{0.4} = 0.6(-0.4055) + 0.4(-1.3863) = -0.7978.$$

Che cosa mostra la tabella. La terza colonna è costante: qualunque q si scelga, ELBO e KL si scambiano massa ma la somma resta $\log p(x)$. E l'ottimo si raggiunge quando q coincide col posterior, dove il bound diventa un'uguaglianza. Qui la famiglia variazionale è tutto il simplex, quindi il posterior ci sta dentro; nei casi reali (mean field) non ci sta, e resta un gap incompressibile.

Collegamento da tenere pronto: questo è lo stesso bound dell'E-step di EM. In EM si sceglie $q = p(z | x, \theta^{\text{old}})$, cioè si azzera la KL esattamente, e per questo l'E-step rende il bound *tight*. Nella VI il posterior non è calcolabile e ci si accontenta della migliore q nella famiglia: EM è il caso fortunato della VI.

10.3 Approssimazione Mean-Field

Un'approssimazione comune è la *mean-field*, dove assumiamo che $q(z)$ fattorizza:

Definizione 10.2. L'approssimazione mean-field assume:

$$q(z) = \prod_{i=1}^M q_i(z_i)$$

cioè le variabili latenti sono indipendenti sotto q .

Ottimizzando coordinate-wise, la forma ottimale per ciascun fattore è:

Teorema 10.2.

$$q_j^*(z_j) \propto \exp(\mathbb{E}_{-j}[\log P(x, z)])$$

dove $\mathbb{E}_{-j}$ è l'attesa su tutte le altre variabili latenti.

Si usa *coordinate ascent variational inference* (CAVI): si aggiorna iterativamente ciascun q_j mantenendo gli altri fissi.

10.4 Divergenza KL

Definizione 10.3. La divergenza di Kullback-Leibler è:

$$\text{KL}(q\|p) = \sum_x q(x) \log \frac{q(x)}{p(x)} = \mathbb{E}_q[\log q - \log p]$$

Proprietà: $\text{KL}(q\|p) \geq 0$, con uguaglianza iff $q = p$.

10.5 EM Generalizzato

Variational inference è legato a EM:

- **Standard EM:** E-step: $q(z) = P(z|x, \theta^{\text{old}})$ (posteriore esatto)
- **Variational EM:** E-step: ottimizza $q(z)$ usando VI (posteriore approssimato)

In entrambi i casi, l'M-step massimizza $\mathbb{E}_q[\log P(x, z|\theta)]$ rispetto a θ .

10.6 Disuguaglianza di Jensen

Il lower bound ELBO si fonda sulla disuguaglianza di Jensen:

Teorema 10.3. Per una funzione concava f e variabile aleatoria X :

$$f(\mathbb{E}[X]) \geq \mathbb{E}[f(X)]$$

Con $\log$ (concava): $\log \mathbb{E}_q \left[\frac{p(x, z)}{q(z)} \right] \geq \mathbb{E}_q[\log p(x, z)] - \mathbb{E}_q[\log q(z)]$

10.7 Approfondimenti

Domande d'Esame

1. Derivare l'ELBO da principi primi. Spiegare il ruolo della divergenza KL.
2. Come si relaziona l'inferenza variazionale a EM?
3. Derivare la distribuzione ottimale q_j^* nel mean-field.
4. Spiegare l'algoritmo CAVI. Qual è la complessità?
5. Come scegliamo la famiglia variazionale $\mathcal{Q}$? Che tradeoff c'è tra flessibilità e trattabilità?

Chapter 11

Latent Dirichlet Allocation

Risorse video — GDL12 — Latent Dirichlet Allocation

Videolezione del corso: registrazione della lezione GDL12 nella cartella *Recordings* del team del corso (v. Prefazione).

Latent Dirichlet Allocation (Part 1 of 2) (*Luis Serrano, 20m*)

Il processo generativo spiegato visivamente, con il semplice e l'effetto di α resi in modo geometrico. Breve e molto efficace per fissare l'intuizione; l'inferenza è nella Part 2 dello stesso canale.

Variational Inference: Foundations and Innovations (*David Blei, 1h30m*)

Blei è il primo autore di LDA: nella seconda metà usa proprio il topic model come esempio guida della VI.

11.1 Topic Modeling

La Latent Dirichlet Allocation (LDA) è un modello Bayesiano generativo per collezioni di documenti testuali. L'obiettivo è scoprire topic nascosti (temi) che spiegano il corpus.

Definizione 11.1. Un topic è una distribuzione di probabilità su parole del vocabolario. Un documento è una miscela di topic.

11.2 Processo Generativo di LDA

Teorema 11.1 (Generative Process). Per ogni documento $d \in \{1, \dots, D\}$:

1. Estrarre la distribuzione su topic: $\theta_d \sim \text{Dirichlet}(\alpha)$
2. Per ogni position $n \in \{1, \dots, N_d\}$ nel documento:
 - (a) Estrarre l'assegnazione a topic: $z_{dn} \sim \text{Categorical}(\theta_d)$
 - (b) Estrarre la parola: $w_{dn} \sim \text{Categorical}(\beta_{z_{dn}})$

dove β_k è la distribuzione su vocabolario per topic k .

Il modello ha parametri:

- α : prior Dirichlet su distribuzioni documento-topic (hyperparameter)
- $\beta_k \in \mathbb{R}_+^V$, $\sum_v \beta_{kv} = 1$: distribuzione parola-topic per ogni topic k

Intuizione — α decide quanto un documento può “parlare di tutto”

Il pezzo di LDA che si dimentica per primo è a che cosa serva il prior Dirichlet. La risposta breve: a controllare la *sparsità* delle miscele.

Se $\alpha < 1$ (per esempio $\alpha = 0.1$), la Dirichlet mette massa vicino ai vertici del semplice. I θ_d estratti sono quindi sbilanciati: ogni documento parla di uno o due topic e ignora gli altri. Se $\alpha > 1$, la massa si concentra al centro e ogni documento diventa una miscela quasi uniforme di tutti i topic — di solito non è ciò che si vuole, perché i topic diventano indistinguibili.

Lo stesso ragionamento vale per il prior β sulle distribuzioni parola-topic: β piccolo produce topic con poche parole caratteristiche, β grande produce topic vaghi.

La differenza da pLSA sta esattamente qui: pLSA tratta θ_d come un parametro libero per documento, quindi il numero di parametri cresce col corpus e non c'è modo di assegnare θ a un documento mai visto. LDA lo tratta come una variabile latente estratta da un prior condiviso: i parametri restano α e β , il modello è generativo a livello di corpus, e un documento nuovo può essere inferito. È la stessa mossa che distingue la stima puntuale dall'approccio bayesiano nel primo capitolo di questa parte: invece di fissare un parametro, gli si mette sopra una distribuzione e la si integra.

Esempio svolto — un passo di collapsed Gibbs su un token

Due topic, vocabolario {sport, gol, borsa, titolo} ($V = 4$), iperparametri simmetrici $\alpha = 0.5$ e $\beta = 0.1$.

Il documento d contiene le parole (sport, gol, borsa) con assegnazioni correnti $z = (1, 1, 2)$. Vogliamo ricampionare il topic del token “gol”.

Passo 1 — togliere il token dai conteggi. Escludendo “gol”, il documento ha $n_{d,1}^- = 1$ (sport) e $n_{d,2}^- = 1$ (borsa). A livello di corpus, supponiamo i conteggi parola-topic

topic 1 : (sport=5, gol=3, borsa=0, titolo=0), $n_{1,\cdot}^- = 8$,

topic 2 : (sport=0, gol=1, borsa=4, titolo=6), $n_{2,\cdot}^- = 11$.

Passo 2 — applicare la formula.

$$P(z_{dn} = k \mid \text{resto}) \propto \underbrace{(n_{d,k}^- + \alpha)}_{\text{quanto } d \text{ ama } k} \cdot \underbrace{\frac{n_{k,w}^- + \beta}{n_{k,\cdot}^- + V\beta}}_{\text{quanto } k \text{ ama la parola } w}$$

$$k = 1 : (1 + 0.5) \cdot \frac{3 + 0.1}{8 + 0.4} = 1.5 \cdot 0.36905 = 0.5536,$$

$$k = 2 : (1 + 0.5) \cdot \frac{1 + 0.1}{11 + 0.4} = 1.5 \cdot 0.09649 = 0.1447.$$

Normalizzando: $P(z = 1) = 0.793$, $P(z = 2) = 0.207$. Si estrae da questa Categorical e si rimettono i conteggi aggiornati.

Che cosa dice la formula. È un prodotto di due pressioni. Il primo fattore è locale al documento: “in questo documento sto già parlando molto del topic k ?”. Il secondo è globale al corpus: “questa parola è tipica del topic k ?”. Un token viene attirato da un topic solo se *entrambe* le condizioni sono favorevoli. Qui “gol” va al topic 1 perché il topic 1 ha già visto “gol” tre volte e il documento ha già un token lì.

Perché si dice *collapsed*: θ e β sono stati integrati analiticamente sfruttando la coniugazione Dirichlet-Categorica, quindi non compaiono più. Si campionano solo le z , cioè le assegnazioni discrete. È questo che rende lo spazio di campionamento molto più piccolo e la catena molto meno correlata.

11.3 Inferenza in LDA

Il posteriore $P(\theta, z|w, \alpha, \beta)$ è intractable. Usiamo variational inference con approssimazione mean-field:

$$q(\theta, z) = q(\theta|\gamma) \prod_{n=1}^N q(z_n|\phi_n)$$

dove:

- γ : parametri Dirichlet variazionali per θ_d
- ϕ_{dn} : parametri categorici variazionali per z_{dn}

L'ELBO per LDA comporta il calcolo di attese sotto queste distribuzioni variazionali.

11.4 Learning di LDA

L'apprendimento di β (topic-word distributions) avviene con variational EM:

- **E-step:** Per ogni documento, ottimizzare (γ_d, ϕ_d) via CAVI
- **M-step:** Aggiornare β_k :

$$\beta_{kv} \propto \sum_{d,n:w_{dn}=v} \phi_{dnk}$$

dove ϕ_{dnk} è la probabilità variazionale che la parola n nel documento d sia assegnata a topic k .

11.5 Plate Notation

La struttura di LDA è rappresentata compattamente in plate notation:

- Box su D: una copia per documento
- Box su N annidato: una copia per parola
- Variabili: $\theta_d, z_{dn}, w_{dn}, \beta_k$

Trappola — LDA è bag of words, e questo si paga

Il processo generativo di LDA campiona ogni parola indipendentemente da θ_d : l'ordine delle parole non entra da nessuna parte. Il documento è un *multiset*, non una sequenza. Che cosa si perde: sintassi, negazione (“non è buono” e “è buono” hanno gli stessi conteggi), collocazioni e ogni polisemia risolvibile dal contesto locale. LDA trova temi, non significati.

Il ponte da fare se la domanda scava: i topic model neurali (ProdLDA e affini) sostituiscono l'inferenza variazionale per-documento con un *encoder ammortizzato*, cioè una rete che mappa direttamente il documento nei parametri variazionali. È

esattamente lo stesso salto che porta dalla VI generica al VAE: invece di ottimizzare γ_d, ϕ_d per ogni documento, si impara una funzione che li produce. Il costo di inferenza passa da “un’ottimizzazione per esempio” a “un forward pass”, ed è ciò che rende il modello utilizzabile su documenti nuovi.

Il bag of words però resta, finché la likelihood è quella: per superarlo servono modelli sequenziali, non un’inferenza migliore.

11.6 Applicazioni e Differenze da pLSA

LDA è usato per:

- Document clustering e topic discovery
- Semantic search
- Dimensionality reduction testuale

La principale differenza da probabilistic Latent Semantic Analysis (pLSA):

- **pLSA:** $P(\theta_d)$ è empirico (punto estimate per documento)
- **LDA:** $P(\theta_d) = \text{Dirichlet}(\alpha)$ è bayesiano (prior condiviso)

Questo rende LDA più robusto al overfitting e capace di generalizzare a nuovi documenti.

11.7 Approfondimenti

Domande d’Esame

1. Descrivere il processo generativo di LDA. Come si relaziona ai dati osservati?
2. Derivare l’ELBO per LDA sotto mean-field approximation.
3. Spiegare come funziona il CAVI per document-topic inference in LDA.
4. Qual è il ruolo del prior Dirichlet su θ_d ? Come influenza learning?
5. Confrontare LDA e pLSA. Quali sono i vantaggi bayesiani?

Chapter 12

Metodi di Campionamento

Risorse video — GDL13 — Campionamento, MCMC, Gibbs

Videolezione del corso: registrazione della lezione GDL13 nella cartella *Recordings* del team del corso (v. Prefazione).

Approximating Probability Distributions: Monte Carlo Methods (*Jakob Foerster, 1h*)

Basato sul capitolo di MacKay: rejection, importance, Metropolis-Hastings e Gibbs nella stessa cornice, con l'accento sul perché ciascuno degrada in alta dimensione.

12.1 Motivazione del Campionamento

Molte integrali nelle probabilità sono intractable:

$$\mathbb{E}_p[f(x)] = \int f(x)p(x)dx$$

Il campionamento approssima:

$$\mathbb{E}_p[f(x)] \approx \frac{1}{S} \sum_{s=1}^S f(x_s), \quad x_s \sim p(x)$$

Per il Teorema del Limite Centrale, l'errore scala come $1/\sqrt{S}$ indipendentemente dalla dimensionalità (evitando la maledizione della dimensionalità dei metodi numerici).

12.2 Campionamento Univariato

12.2.1 Inverse Transform Sampling

Se $U \sim \text{Uniform}(0, 1)$ e F è invertibile:

$$X = F^{-1}(U) \sim p(x)$$

dove $F(x) = P(X \leq x)$.

12.2.2 Box-Muller

Per la Gaussiana: dati $U_1, U_2 \sim \text{Uniform}(0, 1)$,

$$Z = \sqrt{-2 \log U_1} \cos(2\pi U_2) \sim \mathcal{N}(0, 1)$$

12.3 Rejection Sampling

Quando il CDF è intractable, usiamo rejection sampling da una proposal $q(x)$:

Teorema 12.1 (Rejection Sampling). 1. Campionare $x^* \sim q(x)$

2. Campionare $u \sim \text{Uniform}(0, Mq(x^*))$

3. Accettare x^* se $u < p^*(x^*)$, altrimenti rigettare e riprovare
dove M è una costante tale che $p^*(x) \leq Mq(x)$ per tutti gli x .

Lo svantaggio: in alte dimensioni, la probabilità di accettazione diventa esponenzialmente piccola.

12.4 Importance Sampling

Possiamo riscrivere attese sotto una proposal $q(x)$:

$$\mathbb{E}_p[f(x)] = \mathbb{E}_q \left[f(x) \frac{p(x)}{q(x)} \right] \approx \frac{\sum_s f(x_s) w(x_s)}{\sum_s w(x_s)}$$

dove $w(x_s) = p(x_s)/q(x_s)$ sono i pesi di importanza (self-normalized).

Intuizione — tre modi di aggirare lo stesso ostacolo

Il problema comune è sempre questo: si vuole $\mathbb{E}_p[f]$, ma da p non si sa campionare — tipicamente perché se ne conosce solo la forma non normalizzata $\tilde{p}$, e la costante Z è un integrale intrattabile.

Rejection sampling propone da q e butta via: campiona $x \sim q$, accetta con probabilità $\tilde{p}(x)/(Mq(x))$. Onesto e semplice, ma il tasso di accettazione è $1/M$, e in dimensione d la costante M tipicamente cresce esponenzialmente. Diventa inutilizzabile molto presto.

Importance sampling non butta via niente: tiene tutti i campioni e li *pesa* con $w = p/q$. Non spreca, ma sposta il problema sulla varianza: se q non copre bene p , pochi campioni si prendono quasi tutto il peso e la stima diventa rumorosa nonostante il numero di campioni sia alto.

MCMC rinuncia all'indipendenza. Invece di campioni i.i.d. costruisce una catena di Markov la cui distribuzione stazionaria è p : ogni campione dipende dal precedente, ma la catena esplora le regioni di alta densità invece di sperare di colpirle a caso. È il solo dei tre che scala in alta dimensione, e paga con burn-in, autocorrelazione e nessuna garanzia semplice su quando fermarsi.

Il filo comune: tutti e tre funzionano conoscendo p solo a meno di normalizzazione. È questo che li rende compatibili con le distribuzioni di Gibbs e le RBM del capitolo successivo, dove Z è precisamente ciò che non si sa calcolare.

Esempio svolto — importance sampling: perché i pesi vanno guardati

Target discreto $p = (0.1, 0.2, 0.7)$ su $\{1, 2, 3\}$, proposta uniforme $q = (1/3, 1/3, 1/3)$, funzione $f(x) = x$. Il valore esatto è

$$\mathbb{E}_p[f] = 0.1(1) + 0.2(2) + 0.7(3) = 2.6.$$

I pesi di importance sono $w(x) = p(x)/q(x)$:

$$w(1) = \frac{0.1}{1/3} = 0.3, \quad w(2) = 0.6, \quad w(3) = 2.1.$$

Prendendo un campione per ciascun valore, lo stimatore dà

$$\frac{1}{3}(0.3 \cdot 1 + 0.6 \cdot 2 + 2.1 \cdot 3) = \frac{7.8}{3} = 2.6. \quad \checkmark$$

Corretto, come garantito dal fatto che lo stimatore è non distorto.

Ora il lato scomodo. Guardiamo la *effective sample size*:

$$\text{ESS} = \frac{(\sum_i w_i)^2}{\sum_i w_i^2} = \frac{3^2}{0.3^2 + 0.6^2 + 2.1^2} = \frac{9}{4.86} \approx 1.85.$$

Con tre campioni in mano, ne stiamo usando l'equivalente statistico di 1.85: il campione con peso 2.1 domina, gli altri due contribuiscono poco. E questa è una situazione *facile*, con tre soli stati e una proposta ragionevole.

La lezione: la correttezza dello stimatore non dice nulla sulla sua varianza. In dimensione alta, o quando q ha code più leggere di p , l'ESS collassa verso 1 e la stima diventa effettivamente il valore di un solo campione — con un'incertezza che il numero nominale di campioni non lascia sospettare. Controllare l'ESS, o la distribuzione dei pesi, è la diagnostica minima da fare sempre.

12.5 Gibbs Sampling

Gibbs sampling è un metodo MCMC che campiona dalla distribuzione congiunta iterativamente:

Teorema 12.2 (Gibbs Sampling). 1. Inizializzare $x^{(0)}$

2. Per $t = 1, 2, \dots$:

(a) Per $i = 1, \dots, D$: campionare $x_i^{(t)} \sim P(x_i | x_{-i}^{(t)})$

dove x_{-i} denota tutte le variabili eccetto x_i .

La catena Markov converge alla distribuzione stazionaria (il target $p(x)$) sotto condizioni miti. Un *burn-in period* iniziale è raccomandato prima di raccogliere campioni.

12.6 MCMC Generali

Gli algoritmi MCMC mantengono una catena Markov la cui distribuzione stazionaria è il target $p(x)$.

Definizione 12.1. Una catena Markov è *reversibile* (detailed balance) se:

$$p(x)P(x'|x) = p(x')P(x|x')$$

Questa proprietà assicura che il target sia stazionario.

12.7 Metropolis-Hastings

Un algoritmo MCMC generale:

Teorema 12.3 (Metropolis-Hastings). 1. Inizializzare $x^{(t)}$

2. Proporre $x' \sim q(x'|x^{(t)})$

3. Accettare con probabilità $\alpha = \min \left(1, \frac{p(x')q(x^{(t)}|x')}{p(x^{(t)})q(x'|x^{(t)})} \right)$

Gibbs è un caso speciale dove la proposta è il posteriore condizionale ($\alpha = 1$).

12.8 Sampling per LDA

Per LDA, si usa collapsed Gibbs sampling, che marginalizza su θ e β :

$$P(z_{dn} = k | z_{-dn}, w, \alpha, \beta) \propto \frac{n_{d,k}^{-dn} + \alpha}{n_d^{-dn} + K\alpha} \cdot \frac{n_{k,v}^{-dn} + \beta}{n_k^{-dn} + V\beta}$$

dove i conteggi n escludono la posizione corrente (superscript $-dn$).

12.9 Approfondimenti

Domande d'Esame

1. Spiegare il principio di importance sampling e quando è utile.
2. Derivare il Gibbs sampler da Metropolis-Hastings.
3. Quale è la complessità computazionale di rejection sampling in alta dimensione?
4. Come si determina il burn-in period? Quali diagnostici di convergenza si possono usare?
5. Derivare il collapsed Gibbs sampler per LDA.

Chapter 13

Campi Casuali Markoviani

Risorse video — GDL14 — MRF, CRF, RBM, contrastive divergence

Videolezione del corso: registrazione della lezione GDL14 nella cartella *Recordings* del team del corso (v. Prefazione).

Undirected Graphical Models (*Bert Huang, 40m*)

Copertura più debole del resto della lista: va bene per fattorizzazione di Gibbs e Hammersley-Clifford, meno per le macchine di Boltzmann. È l'argomento con il buco più grande su YouTube, quindi qui le slide contano più del solito.

Neural networks 5.4: Restricted Boltzmann machine — contrastive divergence (*Hugo Larochelle, 15m*)

Quindici minuti secchi su CD, spiegata da chi lavorava nel gruppo che l'ha resa popolare. Se hai poco tempo, questo è il video da vedere del capitolo.

13.1 Modelli Grafici Indiretti

I Markov Random Fields (MRF) sono modelli grafici dove gli archi sono *indiretti* (no orientamento). Rappresentano indipendenze condizionali simmetriche.

Definizione 13.1. Un MRF è definito da un grafo indiretto $G = (V, E)$ e potenziali su clique $\psi_c(x_c) \geq 0$.

13.2 Fattorizzazione di Gibbs

Teorema 13.1 (Hammersley-Clifford). Una distribuzione $P(x)$ è Markov w.r.t. grafo G se e solo se fattorizza come:

$$P(x) = \frac{1}{Z} \prod_{c \in \mathcal{C}} \psi_c(x_c)$$

dove $\mathcal{C}$ è l'insieme delle clique massimali e $Z = \sum_x \prod_c \psi_c(x_c)$ è la partition function.

La partition function è generalmente intractable.

Intuizione — Z è il motivo per cui gli undirected sono difficili

La differenza operativa fra una rete Bayesiana e un MRF sta tutta in una costante. In una BN i fattori sono già distribuzioni condizionali normalizzate: ogni CPT somma a uno da sola, e la congiunta è normalizzata per costruzione. Si può campionare in avanti, calcolare probabilità e stimare parametri senza mai calcolare un integrale globale.

In un MRF i fattori sono *potenziali*: numeri non negativi senza vincolo di normalizzazione. La congiunta è normalizzata a posteriori dividendo per $Z = \sum_x \prod_c \psi_c(x_c)$, una somma su tutte le configurazioni — 2^N per N variabili binarie. Nessun ordinamento topologico aiuta, perché non c'è direzione.

Da qui tutto il resto. Il gradiente della log-verosimiglianza di una Boltzmann Machine è

$$\langle x_i x_j \rangle_{\text{data}} - \langle x_i x_j \rangle_{\text{model}},$$

dove il primo termine si calcola sui dati (facile) e il secondo richiede attese sotto il modello, cioè richiede Z (impossibile). Contrastive divergence non è un'idea elegante in cerca di un problema: è la risposta pragmatica a questo specifico ostacolo, e va presentata così.

Il vantaggio in cambio: gli undirected non hanno bisogno di scegliere una direzione causale, quindi modellano naturalmente vincoli simmetrici — pixel vicini che si somigliano, etichette adiacenti in una sequenza. È il motivo per cui i CRF hanno dominato il sequence labeling prima delle reti neurali.

13.3 Proprietà di Markov

Per un MRF, la proprietà di Markov locale è:

$$P(x_i | x_{-i}) = P(x_i | x_{N(i)})$$

dove $N(i)$ sono i vicini di i nel grafo. Solo i vicini influenzano la distribuzione condizionale.

13.4 CRF e Modelli Discriminativi

Una Conditional Random Field (CRF) modella il posteriore discriminativo:

$$P(y|x) = \frac{1}{Z(x)} \prod_{c \in \mathcal{C}} \psi_c(y_c, x)$$

Una CRF lineare è:

$$\log P(y|x) = \sum_c w^T \phi_c(x_c, y_c) - \log Z(x)$$

dove ϕ_c sono feature functions.

13.5 Macchina di Boltzmann

Una Boltzmann Machine è un MRF con variabili binarie completamente connesse:

Definizione 13.2. L'energia è:

$$E(x) = - \sum_{i < j} w_{ij} x_i x_j - \sum_i b_i x_i$$

La distribuzione è:

$$P(x) = \frac{1}{Z} \exp(-E(x))$$

L'apprendimento richiede di massimizzare la log-verosimiglianza, il cui gradiente è:

$$\frac{\partial \log P(x)}{\partial w_{ij}} = \langle x_i x_j \rangle_{\text{data}} - \langle x_i x_j \rangle_{\text{model}}$$

Il termine model richiede campionamento dalla distribuzione, rendendolo computazionalmente costoso.

13.6 Restricted Boltzmann Machine

Un RBM ha una struttura bipartita: variabili visibili v e nascoste h .

Definizione 13.3.

$$P(v, h) = \frac{1}{Z} \exp(v^T W h + b^T v + c^T h)$$

Crucialmente, i condizionali fattorizzano:

$$P(h|v) = \prod_j P(h_j|v) = \prod_j \sigma(c_j + W_j^T v)$$

$$P(v|h) = \prod_i P(v_i|h) = \prod_i \sigma(b_i + W_i h)$$

dove σ è la funzione logistica.

Esempio svolto — un passo di CD-1 su una RBM con tre unità

RBM minima: due unità visibili, una nascosta, con

$$W = \begin{pmatrix} 0.5 \\ -0.5 \end{pmatrix}, \quad b = (0.1, -0.1), \quad c = 0.2.$$

Dato osservato: $v^{(0)} = (1, 0)$.

Passo 1 — fase positiva.

$$P(h=1 | v^{(0)}) = \sigma(0.5(1) + (-0.5)(0) + 0.2) = \sigma(0.7) = 0.6682.$$

Passo 2 — ricostruzione. Campionato $h^{(0)} = 1$:

$$P(v_1=1 | h) = \sigma(0.5+0.1) = \sigma(0.6) = 0.6457, \quad P(v_2=1 | h) = \sigma(-0.5-0.1) = \sigma(-0.6) = 0.3543.$$

Usiamo le probabilità come ricostruzione: $v^{(1)} = (0.6457, 0.3543)$.

Passo 3 — fase negativa.

$$P(h=1 | v^{(1)}) = \sigma(0.5(0.6457) - 0.5(0.3543) + 0.2) = \sigma(0.3457) = 0.5856.$$

Passo 4 — gradiente.

$$\Delta W \propto v^{(0)} P(h=1 | v^{(0)}) - v^{(1)} P(h=1 | v^{(1)}) = \begin{pmatrix} 1(0.6682) - 0.6457(0.5856) \\ 0(0.6682) - 0.3543(0.5856) \end{pmatrix} = \begin{pmatrix} +0.2901 \\ -0.2075 \end{pmatrix}.$$

Analogamente $\Delta b \propto v^{(0)} - v^{(1)} = (0.3543, -0.3543)$ e $\Delta c \propto 0.6682 - 0.5856 = 0.0826$.

Come leggere il segno. Il peso verso v_1 cresce e quello verso v_2 cala: il modello viene spinto a rendere più probabile la configurazione osservata $(1, 0)$ e meno probabile la

sua ricostruzione, che aveva ancora troppa massa su v_2 . La regola è sempre “alza il dato, abbassa il sogno”.

Il punto critico, ed è quello che chiede l'esame. La fase negativa dovrebbe usare campioni dalla distribuzione del *modello*, cioè da una catena di Gibbs portata a convergenza. CD-1 la tronca dopo un solo passo, partendo dal dato. La conseguenza è che CD-1 non è il gradiente di nessuna funzione obiettivo: è un'approssimazione distorta. In pratica funziona perché la direzione è quasi sempre corretta anche se il modulo non lo è, ma il bias esiste, è misurabile, e cresce quando la distribuzione del modello si allontana dai dati. Le varianti CD- k e soprattutto Persistent CD (che mantiene la catena fra un mini-batch e l'altro invece di ripartire dal dato) servono esattamente a ridurlo.

13.7 Contrastive Divergence

L'apprendimento diretto è intrattabile. Contrastive Divergence (CD- k) approssima il gradiente:

1. Fissare v ai dati di training
2. Campionare $h \sim P(h|v)$ (E-step)
3. Campionare $v' \sim P(v|h)$ (ricampione visibili)
4. Ripetere per k passi Gibbs su (v', h)
5. Aggiornare i pesi usando la differenza tra statistiche dei dati e ricampionate

CD approssima il gradiente, rendendolo computazionalmente efficiente.

Trappola — la fase negativa è lo stesso problema che i GAN aggirano

Il gradiente della log-verosimiglianza di una Boltzmann Machine ha due termini:

$$\underbrace{\langle x_i x_j \rangle_{\text{data}}}_{\text{facile}} - \underbrace{\langle x_i x_j \rangle_{\text{model}}}_{\text{difficile}}.$$

Il primo è una media empirica sui dati. Il secondo è un'attesa *sotto il modello*, quindi richiede campioni dal modello stesso, quindi richiede Z . È tutta lì la difficoltà, ed è la ragione dell'esistenza di CD, PCD e affini.

Il ponte forte, che vale la pena avere pronto: i GAN non risolvono questo problema, lo *eliminano*. Rinunciando a una densità esplicita, un GAN non ha nessuna partition function da normalizzare e nessuna fase negativa da stimare: il discriminatore fornisce direttamente un segnale di “quanto i campioni del generatore sembrano veri”. Il prezzo è simmetrico e va detto — senza densità non c'è likelihood, quindi non c'è nemmeno un modo principiato di valutare il modello.

Detto in una riga: modelli espliciti pagano Z , modelli impliciti pagano la valutazione. È lo stesso compromesso, visto da due lati.

13.8 Approfondimenti

Domande d'Esame

1. Enunciare il Teorema di Hammersley-Clifford. Perché è importante?
2. Come si relazionano MRF e modelli grafici diretti? Quali indipendenze preservano?
3. Derivare l'algoritmo di aggiornamento per Gibbs sampling in un RBM.
4. Spiegare l'algoritmo Contrastive Divergence. Perché è un'approssimazione del gradiente?
5. Come si generalizzano RBM a variabili continue? Quali sono i cambiamenti?

Conclusioni

La Parte II ha coperto i fondamenti dell'apprendimento nei modelli probabilistici. Abbiamo visto come il Maximum Likelihood e l'EM forniscono framework principati per l'ottimizzazione dei parametri, da modelli semplici (Naive Bayes) a complessi (LDA, HMM). L'inferenza variazionale e i metodi di campionamento offrono soluzioni quando l'inferenza esatta è intractable. I Campi Casuali Markoviani concludono con modelli indiretti e applicazioni in deep learning.

La padronanza di questi concetti è essenziale per comprendere i modelli generativi moderni e le loro applicazioni in machine learning.

Part III

Deep Learning Fondamentale

Chapter 14

Convolutional Neural Networks (Lezioni 15–16)

Risorse video — GDL15–16 — Convolutional Neural Networks

Videolezione del corso: registrazione delle lezioni GDL15 e GDL16 nella cartella *Recordings* del team del corso (v. Prefazione).

CS231n — Lecture 5: Convolutional Neural Networks (*Stanford, 1h10m*)

La lezione da cui deriva mezzo mondo: convoluzione, stride, padding, pooling e il conto delle dimensioni fatto passo passo alla lavagna. È anche la base del Midterm 2.

CS231n — Lecture 9: CNN Architectures (*Stanford, 1h15m*)

LeNet, AlexNet, VGG, GoogLeNet, ResNet in ordine storico, con il ragionamento su *perché* ogni architettura ha cambiato qualcosa. Da vedere prima di studiare la sezione sulle architetture.

14.1 Contesto storico e motivazioni biologiche

Le reti neurali convoluzionali (CNN) derivano da osservazioni neuroscientifiche fondamentali. Nel 1959, Hubel e Wiesel scoprirono che i neuroni della corteccia visiva dei gatti rispondono a stimoli locali e specifici, con proprietà di invarianza traslazionale. Questa osservazione biologica ha ispirato il design di modelli artificiali.

Il **Neocognitron** di Fukushima (1980) introdusse il concetto di strati alternati per l'estrazione gerarchica di caratteristiche. Successivamente, **LeNet-5** di LeCun et al. (1989) dimostrò l'efficacia pratica delle CNN per il riconoscimento di cifre scritte a mano. Tuttavia, il vero punto di svolta fu **AlexNet** (Krizhevsky et al., 2012), che vinse la competizione ImageNet con un significativo margine di errore, segnando l'inizio della moderna era del deep learning.

14.2 Inductive biases nelle CNN

Le CNN incorporano tre importanti inductive biases che riducono lo spazio delle ipotesi e migliorano la generalizzazione:

Definizione 14.1 (Local Connectivity). I neuroni in uno strato convoluzionale non sono connessi a tutti gli input, ma solo a una piccola regione locale (receptive field). Questo riflette il principio biologico che i neuroni visivi rispondono a stimoli locali.

Definizione 14.2 (Parameter Sharing). Lo stesso filtro (kernel) è applicato a diverse posizioni nello spazio di input. Questo significa che i parametri riconoscono un motivo in

una posizione sono gli stessi che riconoscono lo stesso motivo in altre posizioni, favorendo l'equivarianza traslazionale.

Definizione 14.3 (Pooling e Invarianza). Gli operatori di pooling (max, average) riducono le dimensioni spaziali e forniscono una forma di invarianza traslazionale: piccole traslazioni dell'input non modificano significativamente l'output poolato.

Trappola — equivarianza, non invarianza

La frase “la CNN è invariante alla traslazione” si sente spesso ed è sbagliata, almeno per lo strato convoluzionale.

La convoluzione è **equivariante**: se trasli l'input, la feature map si trasla della stessa quantità, $f(T_v x) = T_v f(x)$. L'informazione sulla posizione non viene persa, viene *trasportata*. Ed è giusto che sia così: per la segmentazione o il rilevamento serve sapere *dove* si trova la cosa, non solo che c'è.

L'**invarianza**, $f(T_v x) = f(x)$, la introducono altri pezzi: il pooling la fornisce localmente e in modo approssimato, il global pooling finale la fornisce globalmente. È una scelta architetturale successiva, non una proprietà della convoluzione.

Perché è una trappola che conviene evitare: confondere le due cose vuol dire non aver capito perché una CNN funziona sia per la classificazione (dove serve invarianza) sia per la segmentazione (dove servirebbe il contrario). La risposta corretta è che i layer convoluzionali danno equivarianza, e l'architettura decide se e dove convertirla in invarianza.

14.3 Operazione di convoluzione

Formalmente, la convoluzione discreta 1D è definita come:

$$(f * g)[n] = \sum_{k=-\infty}^{\infty} f[k] g[n - k]$$

Per immagini 2D, la convoluzione è:

$$(I * K)[i, j] = \sum_m \sum_n I[i + m, j + n] K[m, n]$$

Nota: Cross-correlazione vs Convoluzione

Nelle reti neurali, l'operazione implementata è in realtà la **cross-correlazione**:

$$(I \star K)[i, j] = \sum_m \sum_n I[i + m, j + n] K[m, n]$$

A differenza della convoluzione, non si inverte il kernel. La denominazione “convoluzione” è mantenuta per convenzione, anche se tecnicamente si tratta di cross-correlazione.

Le dimensioni dell'output di una convoluzione sono calcolate come:

$$O = \left\lfloor \frac{I - K + 2P}{S} \right\rfloor + 1$$

dove I è la dimensione dell'input, K è la dimensione del kernel, P è il padding, e S è lo stride.

Intuizione — la convoluzione è un vincolo, non un'operazione

Il modo produttivo di vedere un layer convoluzionale non è “un filtro che scorre”, ma *un layer fully connected a cui sono stati imposti due vincoli*.

Il primo è la **località**: ogni unità di output può guardare solo una finestra di input, tutti gli altri pesi sono forzati a zero. Il secondo è la **condivisione dei pesi**: la stessa finestra di pesi viene riusata in ogni posizione, invece di averne una diversa per punto. Perché questi due vincoli e non altri? Perché corrispondono a due fatti veri sulle immagini. La località riflette che i pixel correlati sono vicini. La condivisione riflette che un bordo è un bordo ovunque si trovi: la statistica dell'immagine è approssimativamente stazionaria per traslazione.

Da qui discende tutto il resto. Il numero di parametri crolla, perché non dipende più dalla dimensione dell'input. L'equivarianza alla traslazione è automatica: se sposti l'input, la feature map si sposta uguale. E il modello generalizza meglio non perché sia “più potente” — è strettamente *meno* espressivo di una fully connected — ma perché il vincolo imposto è quello giusto per il dominio.

Il corollario da dire all'orale: se i dati non hanno struttura traslazionale (tabelle, feature non ordinate), quel bias diventa un handicap e la CNN perde il suo vantaggio. L'inductive bias aiuta solo quando è vero.

Esempio svolto — il conto dei parametri: 456 contro 14 milioni

Input $32 \times 32 \times 3$ (una immagine CIFAR). Applichiamo un layer convoluzionale con 6 filtri 5×5 , stride 1, nessun padding.

Dimensione dell'output.

$$O = \left\lfloor \frac{I - K + 2P}{S} \right\rfloor + 1 = \frac{32 - 5 + 0}{1} + 1 = 28 \Rightarrow 28 \times 28 \times 6.$$

Parametri del layer convoluzionale. Ogni filtro vede tutti e tre i canali di input:

$$\underbrace{(5 \times 5 \times 3)}_{\text{pesi per filtro}} + \underbrace{1}_{\text{bias}} \times 6 = 76 \times 6 = \mathbf{456}.$$

Parametri di una fully connected che producesse lo stesso output.

$$(32 \cdot 32 \cdot 3) \times (28 \cdot 28 \cdot 6) = 3072 \times 4704 = \mathbf{14\,450\,688}.$$

Un fattore $\approx 31\,700$. E si noti che i 456 parametri *non dipendono dalla dimensione dell'immagine*: su input 256×256 resterebbero 456, mentre la fully connected esploderebbe ancora.

Campo recettivo, sullo stesso esempio. Con la ricorrenza

$$\text{RF}_\ell = \text{RF}_{\ell-1} + (k_\ell - 1)j_{\ell-1}, \quad j_\ell = j_{\ell-1} \cdot s_\ell,$$

tre convoluzioni 3×3 stride 1 danno $\text{RF} = 1 \rightarrow 3 \rightarrow 5 \rightarrow 7$. Se invece si inserisce un pooling 2×2 in mezzo (conv3, pool2, conv3):

$$1 \xrightarrow{\text{conv3}} 3 \ (j=1) \xrightarrow{\text{pool2}} 4 \ (j=2) \xrightarrow{\text{conv3}} 8 \ (j=2).$$

Che cosa mostra: il downsampling è ciò che fa crescere il campo recettivo in fretta. Senza di esso servono molti layer per vedere l'immagine intera — ed è precisamente la ragione per cui esistono le convoluzioni dilate, che allargano l'RF senza pagare né in risoluzione né in parametri.

14.4 Pooling e Feature Maps

Il **max pooling** divide l'input in regioni rettangolari e restituisce il valore massimo per ogni regione:

$$\text{MaxPool}_{i,j} = \max_{m \in R_{i,j}} I[m]$$

dove $R_{i,j}$ è la regione di pooling.

L'**average pooling** calcola la media:

$$\text{AvgPool}_{i,j} = \frac{1}{|R_{i,j}|} \sum_{m \in R_{i,j}} I[m]$$

Ogni filtro produce una **feature map**, una rappresentazione 2D che evidenzia la presenza di un pattern specifico nell'input. Un layer convoluzionale con F filtri produce F feature maps.

14.4.1 Parametri per layer

Un layer convoluzionale che processa un input $H \times W \times C$ con F filtri di dimensione $K \times K$ ha:

$$\text{Parametri} = K \times K \times C \times F + F$$

Il primo termine conta i pesi del kernel (condivisi), il secondo termine i bias.

14.5 Architetture CNN fondamentali

14.5.1 LeNet-5 (1998)

LeNet-5 fu una delle prime architetture CNN di successo:

- Input: immagini 32×32 in scala di grigi
- C1: convoluzione 5×5 , 6 filtri $\rightarrow 28 \times 28 \times 6$
- S2: max pooling $2 \times 2 \rightarrow 14 \times 14 \times 6$
- C3: convoluzione 5×5 , 16 filtri $\rightarrow 10 \times 10 \times 16$
- S4: max pooling $2 \times 2 \rightarrow 5 \times 5 \times 16$
- F5, F6: fully connected con attivazioni tanh $\rightarrow$ output 10 classi

14.5.2 AlexNet (2012)

AlexNet introdusse diversi elementi chiave che divennero standard:

- Profondità maggiore (8 layer)
- Funzione ReLU per attivazioni (più veloce di tanh e sigmoid)
- Dropout per regolarizzazione
- Data augmentation (rotazioni, crop, flip)
- Addestramento su GPU multi-GPU (NVIDIA GTX 580)
- Local Response Normalization (LRN)

14.5.3 VGG Networks (2014)

VGG-16 e VGG-19 dimostrarono l'importanza della profondità:

- Architettura omogenea: only 3×3 convolutions
- Stacking di kernel piccoli $\approx$ kernel grandi ma con meno parametri e più non-linearità
- 3×3 è il kernel minimo che cattura vicini diagonali
- Receptive field di un kernel 7×7 si ottiene con 3 strati di 3×3

Parametri di VGG-16: ~ 138 milioni (molti nei layer FC).

14.5.4 ResNet (2015): Residual Connections

ResNet introdusse le **skip connections**, che permettono ai gradienti di fluire direttamente attraverso la rete:

Definizione 14.4 (Residual Block). Un blocco residuale implementa:

$$y = F(x, \{W_i\}) + x$$

dove $F(x, \{W_i\})$ è la trasformazione residua (il “percorso del residuo”) e x è l'identità (“skip connection”). Se le dimensioni non corrispondono, si usa una convoluzione lineare per l'identità.

Questo design risolve il **degradation problem**: con reti molto profonde senza skip connections, la perdita di addestramento aumenta (non è saturazione, ma una difficoltà intrinseca di apprendimento). Le skip connections alleviano questo problema.

ResNet utilizza **He initialization** per i pesi:

$$W_{ij} \sim \mathcal{N}(0, \sqrt{2/n_{in}})$$

dove n_{in} è il numero di input del neurone.

14.5.5 Inception e convoluzioni 1×1

L'architettura Inception utilizza rami paralleli con kernel di diverse dimensioni (1×1 , 3×3 , 5×5) concatenando i risultati. Questo cattura pattern a diverse scale.

Le convoluzioni 1×1 sono particolarmente utili per:

- **Channel mixing**: combinare informazioni da diversi canali
- **Dimensionality reduction**: ridurre il numero di canali prima di convoluzioni costose, riducendo i parametri e la computazione

Trappola — la convoluzione 1×1 non è un'operazione inutile

A prima vista un kernel 1×1 sembra non fare nulla: non guarda nessun vicino spaziale, quindi non può estrarre pattern locali. L'errore è pensare che una convoluzione agisca solo sullo spazio.

Una 1×1 su un input con C_{in} canali è una **combinazione lineare completa lungo i canali**, applicata indipendentemente in ogni posizione: è un layer fully connected sulla dimensione dei canali. Fa due cose preziose.

Mixing dei canali: ricombina le feature map fra loro, cosa che una convoluzione spaziale con gruppi separati non fa.

Riduzione dimensionale a costo bassissimo: portare 256 canali a 64 costa $256 \times 64 =$

16 384 pesi, contro i $256 \times 64 \times 9 = 147\,456$ di una 3×3 . È il *bottleneck* che rende praticabili i blocchi di ResNet ($1 \times 1 \rightarrow 3 \times 3 \rightarrow 1 \times 1$) e i moduli Inception: si comprime, si fa il lavoro spaziale costoso su pochi canali, si riespande.

Le depthwise separable convolution sono la stessa idea portata all'estremo: convoluzione spaziale per canale, poi una 1×1 per mescolarli.

14.6 Convoluzioni dilate e separabili

14.6.1 Dilated Convolutions

Una convoluzione dilatata inserisce zeri tra gli elementi del kernel, con parametro dilation rate d :

$$\text{Receptive field effettivo} = K + (K - 1)(d - 1)$$

Per $K = 3$ e $d = 2$, il receptive field è 5. Le convoluzioni dilate permettono di aumentare il receptive field senza aumentare il numero di parametri.

14.6.2 Depthwise Separable Convolutions

Una convoluzione depthwise separabile fattorizza una convoluzione standard in due operazioni:

Depthwise : ogni canale è convolto indipendentemente $K \times K \times 1 \times C$

Pointwise : convoluzioni 1×1 che mixano canali, $1 \times 1 \times C \times F$

Il numero di parametri passa da $K \times K \times C \times F$ a $K \times K \times C + C \times F$, una riduzione significativa quando F è grande.

14.7 Receptive field analysis

Il receptive field è l'area dell'input che influenza un neurone in uno strato superiore. Per un neurone in posizione (i, j) dopo ℓ layer:

$$\text{RF}_\ell = 1 + \sum_{l=1}^{\ell} (K_l - 1) \prod_{k=l+1}^{\ell} S_k$$

dove K_l è la dimensione del kernel al layer l e S_k è lo stride.

Un receptive field ampio è importante per catturare dipendenze a lungo raggio. Le architetture moderne come ResNet-50 hanno receptive field di centinaia di pixel.

14.8 Approfondimenti

14.8.1 Domande d'esame

1. Spiegare il concetto di parameter sharing nelle CNN e come contribuisce all'equivarianza traslazionale.
2. Qual è la differenza tra convoluzione e cross-correlazione? Perché nelle reti neurali si implementa la cross-correlazione?
3. Derivare la formula per le dimensioni dell'output di una convoluzione: $O = \frac{I-K+2P}{S} + 1$.
4. Descrivere il degradation problem in reti molto profonde e come ResNet lo risolve.

5. Spiegare il ruolo delle skip connections nel migliorare il flusso dei gradienti durante il backpropagation.
6. Quale architettura tra LeNet, VGG, ResNet sarebbe più adatta per un'immagine 224×224 RGB? Giustificare.
7. Quali sono i vantaggi computazionali delle convoluzioni depthwise separabili?
8. Come si calcola il receptive field di un neurone dopo 5 layer convoluzionali?

Chapter 15

Propagazione dell'informazione in reti profonde (Lezione 17)

Risorse video — GDL17 — RNN e vanishing/exploding gradient

Videolezione del corso: registrazione della lezione GDL17 nella cartella *Recordings* del team del corso (v. Prefazione).

EECS 498 — Lecture 12: Recurrent Networks (*Michigan Online, 1h10m*)
BPTT, flusso del gradiente e analisi del Jacobiano con le figure giuste. **Attenzione al nome:** GDL17 è “Information Propagation in Deep Networks”, cioè RNN ed EVGP — *non* belief propagation. È un equivoco che capita, visto il titolo.

15.1 RNN: basic architecture

Per dati sequenziali $\{x_t\}_{t=1}^T$ dove ogni $x_t \in \mathbb{R}^d$, una rete ricorrente mantiene uno stato nascosto $h_t \in \mathbb{R}^h$ aggiornato ad ogni passo:

Definizione 15.1 (RNN Vanilla).

$$h_t = \sigma(W_h h_{t-1} + W_x x_t + b)$$

$$y_t = f(h_t)$$

dove σ è un'attivazione (tanh, ReLU), $W_h \in \mathbb{R}^{h \times h}$ sono i pesi ricorrenti, $W_x \in \mathbb{R}^{h \times d}$ sono i pesi di input, e $b \in \mathbb{R}^h$ sono i bias.

La caratteristica fondamentale è il **parameter sharing** attraverso il tempo: gli stessi pesi W_h, W_x, b sono usati a tutti i passi temporali. Questo riduce drasticamente il numero di parametri rispetto a una rete fully connected “unrolled”.

15.2 Backpropagation Through Time (BPTT)

Per una loss $L = \sum_{t=1}^T L_t$, il gradiente rispetto ai pesi è:

$$\frac{\partial L}{\partial W} = \sum_{t=1}^T \frac{\partial L_t}{\partial W}$$

Per il calcolo di $\frac{\partial L_t}{\partial W}$, applichiamo la chain rule attraverso il tempo:

$$\frac{\partial L_t}{\partial h_t} = \frac{\partial L_t}{\partial y_t} \frac{\partial y_t}{\partial h_t}$$

$$\frac{\partial L_t}{\partial h_k} = \frac{\partial L_t}{\partial h_t} \prod_{j=k+1}^t \frac{\partial h_j}{\partial h_{j-1}} \quad (k < t)$$

Il gradiente totale è:

$$\frac{\partial L}{\partial h_t} = \frac{\partial L_t}{\partial h_t} + \frac{\partial L_{t+1}}{\partial h_t}$$

15.3 Gradient flow e Jacobian matrices

Analisi del flusso dei gradienti

La derivata dello stato nascosto rispetto a uno stato precedente è:

$$\frac{\partial h_t}{\partial h_{t-1}} = \text{diag}(\sigma'(W_h h_{t-1} + W_x x_t + b)) W_h$$

La derivata attraverso τ passi temporali è:

$$\frac{\partial h_t}{\partial h_{t-\tau}} = \prod_{k=1}^{\tau} \frac{\partial h_{t-k+1}}{\partial h_{t-k}}$$

Se denotiamo $J_k = \frac{\partial h_k}{\partial h_{k-1}}$, allora:

$$\frac{\partial h_t}{\partial h_{t-\tau}} = \prod_{k=t-\tau+1}^t J_k$$

Per la norma del gradiente lungo una traiettoria, consideriamo il caso semplificato dove σ' è costante (approssimazione):

$$\left\| \frac{\partial h_t}{\partial h_{t-\tau}} \right\| \approx |\sigma'|^\tau \|W_h\|^\tau$$

Se $\|W_h\| < 1$, il gradiente decade esponenzialmente con τ . Se $\|W_h\| > 1$, il gradiente cresce esponenzialmente.

15.4 Vanishing e exploding gradients

15.4.1 Vanishing Gradient Problem

Definizione 15.2 (Vanishing Gradient). Nel regime in cui $\|W_h\| < 1$ o σ' è piccolo (saturazione), il gradiente $\left\| \frac{\partial h_t}{\partial h_{t-\tau}} \right\|$ decresce esponenzialmente con τ :

$$\left\| \frac{\partial h_t}{\partial h_{t-\tau}} \right\| \lesssim \lambda^\tau, \quad \lambda < 1$$

Per lunghe sequenze ($\tau \gg 1$), il gradiente diventa praticamente zero, rendendo impossibile apprendere dipendenze a lungo termine.

Le cause principali sono:

- **Saturazione dell'attivazione:** sigmoid e tanh hanno derivate $\sigma' \in (0, 0.25]$ e $\sigma' \in (0, 1)$ rispettivamente
- **Piccoli pesi:** se gli elementi di W_h sono inizializzati male, $\|W_h\|$ può essere < 1
- **Lunghe sequenze:** il decadimento esponenziale è più grave per T grande

15.4.2 Exploding Gradient Problem

Definizione 15.3 (Exploding Gradient). Quando $\|W_h\| > 1$, il gradiente cresce esponenzialmente:

$$\left\| \frac{\partial h_t}{\partial h_{t-\tau}} \right\| \sim \lambda^\tau, \quad \lambda > 1$$

Questo causa instabilità numerica e aggiornamenti di peso eccessivi.

Esempio svolto — perché il raggio spettrale decide tutto: i numeri su 50 passi

Consideriamo la ricorrenza lineare più semplice possibile, $h_t = w h_{t-1}$, e chiediamoci quanto vale il gradiente che dal tempo $T = 50$ torna al tempo 1:

$$\frac{\partial h_{50}}{\partial h_1} = w^{49}.$$

w	w^{49}	effetto
0.5	1.8×10^{-15}	svanito: sotto la precisione utile di float32
0.9	5.7×10^{-3}	svanito in pratica: contributo 175 volte più piccolo di quello locale
1.0	1	conservato
1.1	1.07×10^2	esploso: due ordini di grandezza

Si noti quanto è stretta la finestra: fra 0.9 e 1.1 ci sono *diciotto ordini di grandezza* di differenza sul gradiente a lungo raggio. Il caso $w = 1$ non è un compromesso raggiungibile per caso, è un punto di misura nulla.

Il caso generale. Con $h_t = \tanh(\mathbf{W}h_{t-1} + \dots)$ il Jacobiano è $\mathbf{W}^\top \text{diag}(1 - \tanh^2)$, e la norma del prodotto su T passi è governata dal raggio spettrale $\rho(\mathbf{W})$. Poiché $\tanh'(\cdot) \leq 1$ sempre, il fattore diagonale può solo *peggiore* il vanishing: serve $\rho(\mathbf{W}) > 1$ anche solo per sperare di conservare il segnale, ma allora la dinamica in avanti tende a saturare o divergere. Il problema è strutturale, non un difetto di ottimizzazione.

Confronto con l'LSTM, che anticipa il capitolo seguente. Nell'LSTM il cammino privilegiato è la cella, con $\partial c_t / \partial c_{t-1} = f_t$. Con un forget gate a 0.95:

$$0.95^{49} = 0.081,$$

contro 5.7×10^{-3} della RNN con $w = 0.9$: un fattore 14. Con $f_t = 0.99$ si arriva a 0.61, cioè il gradiente attraversa 50 passi perdendo meno del 40%.

Il punto concettuale: l'LSTM non risolve il problema cambiando la matematica, ma rendendo f_t un parametro *appreso e vicino a 1 quando serve*, invece di una costante fissata dal raggio spettrale. La rete può decidere per quali dimensioni tenere il canale aperto e per quali chiuderlo.

E gli exploding gradients? Sono il caso più facile dei due: si vedono subito (loss a NaN) e si curano con il gradient clipping, che è un cerotto ma efficace. Il vanishing è insidioso perché non dà nessun sintomo: la rete addestra, la loss scende, semplicemente non impara mai dipendenze lunghe.

15.5 Soluzioni ai problemi di gradiente

15.5.1 Gradient Clipping

Gradient Clipping

Se la norma del gradiente supera una soglia θ :

$$g \leftarrow g \cdot \frac{\theta}{\|g\|} \quad \text{if } \|g\| > \theta$$

Questo limita la norma senza cambiare la direzione. È particolarmente efficace per l'exploding gradient.

15.5.2 Proper Weight Initialization

L'inizializzazione **ortogonale** dei pesi ricorrenti mantiene il singular spectrum di W_h vicino a 1:

$$W_h = QR \quad \text{dove } Q \text{ è ortogonale}$$

Questo preserva la norma e facilita il gradient flow.

15.5.3 Alternative Activation Functions

ReLU riduce la saturazione, ma introduce nuove sfide (dead neurons). Le attivazioni leaky-ReLU ($f(x) = \alpha x$ for $x < 0$) migliorano il flusso dei gradienti.

15.5.4 Architetture specializzate

La soluzione più efficace è modificare l'architettura stessa:

- **LSTM** (Lesson 18): porte esplicite che gestiscono il flusso di informazione
- **Skip connections**: permettono ai gradienti di saltare i layer ricorrenti
- **Layer normalization**: stabilizza i gradienti normalizzando input ai layer

15.6 Echo State Network perspective

Un **Echo State Network** (ESN) è una RNN con reservoir ricorrente casuale e peso di output addestrabile:

$$h_t = \sigma(W_{\text{res}}h_{t-1} + W_{\text{in}}x_t)$$

$$y_t = W_{\text{out}}h_t$$

Solo W_{out} è addestrabile; W_{res} è fisso ma casuale. L'Echo State Property richiede che il comportamento asintoticamente dipenda solo dall'input attuale e passato, non dalle condizioni iniziali. Ciò è garantito dal:

Definizione 15.4 (Spectral Radius Condition).

$$\rho(W_{\text{res}}) < 1$$

dove ρ è il raggio spettrale (massimo valore assoluto degli autovalori).

Con $\rho < 1$, i perturbazioni iniziali decadono, e il sistema dimentica lo stato iniziale. Questo fornisce una prospettiva diversa sul problema del vanishing gradient: con spectral radius ben controllato, i gradienti non spariscono automaticamente.

15.7 Approfondimenti

15.7.1 Domande d'esame

1. Scrivere la formula per $\frac{\partial h_t}{\partial h_{t-\tau}}$ in una RNN vanilla e analizzare quando il gradiente svanisce.
2. Qual è la differenza tra vanishing e exploding gradients? Quali condizioni su W_h le causano?
3. Spiegare il meccanismo del gradient clipping e quando è più efficace.
4. Perché l'inizializzazione ortogonale dei pesi ricorrenti aiuta il gradient flow?
5. Descrivere l'Echo State Property e il ruolo della spectral radius.
6. In una sequenza di 100 passi temporali con $\|W_h\| = 0.95$, quanto è ridotto il gradiente per un evento al passo 1 visto dal passo 100?
7. Quali attivazioni sono preferibili per le RNN e perché?

Chapter 16

Modelli ricorrenti con porte (Lezione 18)

Risorse video — GDL18 — LSTM, GRU, echo state network

Videolezione del corso: registrazione della lezione GDL18 nella cartella *Recordings* del team del corso (v. Prefazione).

CMU 11-785 — Lecture 14: Recurrent Networks II, Stability Analysis and LSTMs (1h20m)

Parte dall'analisi di stabilità e ci costruisce sopra l'LSTM: è il video che rende ovvio *perché* le porte sono fatte così invece di essere una lista di formule da memorizzare.

Buco dichiarato: sugli *Echo State Network* non esiste un video decente in circolazione. Su quella sezione le slide di Bacciu sono la fonte, e non a caso: è lui uno degli autori di riferimento del reservoir computing.

16.1 LSTM: long-short term memory

Le **LSTM** (Hochreiter & Schmidhuber, 1997) risolvono il vanishing gradient problem introducendo un meccanismo di **memory cell** esplicito e porte che regolano il flusso di informazione.

Definizione 16.1 (LSTM Cell). Una cella LSTM mantiene:

- **Cell state** c_t (memoria a lungo termine)
- **Hidden state** h_t (output / memoria a breve termine)

A ogni passo, quattro trasformazioni aggiornano questi stati:

Forget gate: quale parte della cell state dimenticare

$$f_t = \sigma(W_f[h_{t-1}, x_t] + b_f)$$

Input gate: quali nuove informazioni scrivere

$$i_t = \sigma(W_i[h_{t-1}, x_t] + b_i)$$

Candidate cell state: cosa aggiungere

$$\tilde{c}_t = \tanh(W_c[h_{t-1}, x_t] + b_c)$$

Cell state update: combinazione additiva (cruciale!)

$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t$$

Output gate: quale parte della cell state esporre

$$o_t = \sigma(W_o[h_{t-1}, x_t] + b_o)$$

Hidden state: output della cella

$$h_t = o_t \odot \tanh(c_t)$$

dove $\odot$ denota elemento-wise multiplication e $[h_{t-1}, x_t]$ è la concatenazione.

16.2 Gradient flow in LSTM

La chiave del successo delle LSTM è il percorso additivo della cell state:

$$\frac{\partial c_t}{\partial c_{t-1}} = f_t$$

Questo è in contrasto con la RNN vanilla dove:

$$\frac{\partial h_t}{\partial h_{t-1}} = \text{diag}(\sigma'(\cdot))W_h$$

Nel caso LSTM, il gradiente non subisce la moltiplicazione matriciale; è solo una moltiplicazione elemento-wise con $f_t \in [0, 1]$ (poiché f_t è output di una sigmoid). Sebbene i valori di f_t possano ancora causare decadimento, il meccanismo delle porte fornisce un controllo esplicito e apprendibile su cosa ritenere e cosa dimenticare.

Vanishing gradient mitigation

Anche se in teoria $\prod_k f_t$ potrebbe ancora decadere, in pratica le porte sono apprese per mantenere $f_t \approx 1$ quando le dipendenze a lungo termine sono importanti, permettendo ai gradienti di fluire.

Intuizione — le porte non “ricordano”, decidono quanto lasciar passare

Il nome “memoria” inganna. Nell’LSTM non c’è nessun meccanismo di memorizzazione: c’è un’autostrada, la cella c_t , e tre caselli che regolano il traffico.

Il *forget gate* decide quanta parte del contenuto attuale sopravvive: $c_t = f_t \odot c_{t-1} + \dots$. Se $f_t \approx 1$ il contenuto passa intatto; se $f_t \approx 0$ viene azzerato. L’*input gate* decide quanta della nuova informazione candidata entra. L’*output gate* decide quanto della cella viene esposto verso l’esterno — la cella può quindi trattenere qualcosa senza usarlo subito, il che è la vera differenza con una RNN semplice.

La ragione per cui questo risolve il vanishing è che l’aggiornamento della cella è *additivo*, non moltiplicativo per una matrice. La derivata lungo quel cammino è f_t , un numero fra 0 e 1 scelto dalla rete, invece di un prodotto di Jacobiani il cui modulo nessuno controlla. È il “constant error carousel” dell’articolo originale.

E la GRU? Fonde cella e stato nascosto e riduce a due porte, legando input e forget in un unico z_t : quello che entra è esattamente quello che non resta. Meno parametri, spesso prestazioni comparabili, a volte peggiori quando servono memorie molto lunghe — perché non può più tenere qualcosa in cella senza esporlo.

Il collegamento in avanti: l’idea di un cammino additivo che protegge il gradiente è la stessa delle skip connection nelle ResNet, e la stessa delle connessioni residue dentro il blocco Transformer. Cambia il contesto, non il meccanismo — ed è il tipo di collegamento trasversale che vale punti all’orale.

16.3 GRU: gated recurrent unit

Definizione 16.2 (GRU). La GRU (Cho et al., 2014) è una variante semplificata della LSTM con 3 porte invece di 4:

Reset gate: controllare quanto passato stato nascosto usare

$$r_t = \sigma(W_r[h_{t-1}, x_t] + b_r)$$

Update gate: bilanciare tra stato precedente e candidato

$$z_t = \sigma(W_z[h_{t-1}, x_t] + b_z)$$

Candidate hidden state:

$$\tilde{h}_t = \tanh(W[r_t \odot h_{t-1}, x_t] + b)$$

Output:

$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t$$

La GRU ha meno parametri di LSTM ($3W + 3b$ vs $4W + 4b$), mantenendo performance comparabili.

16.4 Confronto LSTM vs GRU

- **LSTM:** cell state separato, controllo fine con 4 porte, preferito per task critiche
- **GRU:** più leggero computazionalmente, meno parametri, spesso sufficiente per molti problemi
- Empiricamente, le performance sono simili su molti task; la scelta spesso dipende da considerazioni pratiche di velocità e memoria

16.5 Bidirectional RNNs

Per task dove il contesto futuro è disponibile (classificazione di sequenza, tagging), conviene usare:

$$h_t^{\rightarrow} = \text{RNN}_{\rightarrow}(x_t, h_{t-1}^{\rightarrow})$$

$$h_t^{\leftarrow} = \text{RNN}_{\leftarrow}(x_t, h_{t+1}^{\leftarrow})$$

$$h_t = [h_t^{\rightarrow}; h_t^{\leftarrow}]$$

La concatenazione cattura informazioni da entrambe le direzioni.

16.6 Autoregressive modeling

Un modello generativo di sequenze fattorizza la probabilità congiunta come:

$$P(x_{1:T}) = \prod_{t=1}^T P(x_t | x_{1:t-1})$$

Una RNN apprende distribuzioni condizionali:

$$P(x_t | x_{1:t-1}) = \text{softmax}(Wh_t + b)$$

16.6.1 Teacher forcing

Durante l'addestramento, il vero x_{t-1} è fornito come input (non la predizione), permettendo parallelizzazione e accelerando la convergenza:

$$h_t = \text{RNN}(x_{t-1}, h_{t-1}) \quad (\text{durante addestramento})$$

Durante la generazione (inference), la predizione è usata come input:

$$\tilde{x}_t \sim P(x_t | x_{1:t-1}), \quad h_t = \text{RNN}(\tilde{x}_{t-1}, h_{t-1})$$

Questo causa un **exposure bias**: il modello non vede i propri errori durante l'addestramento.

16.7 Approfondimenti

16.7.1 Domande d'esame

1. Descrivere la struttura completa di una cella LSTM e il ruolo di ciascuna porta.
2. Perché l'aggiornamento della cell state in LSTM è additivo e non moltiplicativo? Quale vantaggio porta?
3. Derivare $\frac{\partial c_t}{\partial c_{t-1}}$ in LSTM e confrontare con la RNN vanilla.
4. Quali sono le differenze principali tra LSTM e GRU? Quale scegliereste per un task a risorse limitate?
5. Spiegare il concetto di teacher forcing e l'exposure bias che causa.
6. Come si estendono LSTM e GRU a bidirectional RNNs? Quando è utile?
7. In un LSTM, se tutte le forget gates imparano a outputs $f_t \approx 0$, cosa succede alle informazioni a lungo termine? Quando sarebbe desiderabile?

Chapter 17

Encoder-Decoder e Cross-Attention (Lezione 19)

Risorse video — GDL19 — Seq2seq e cross-attention

Videolezione del corso: registrazione della lezione GDL19 nella cartella *Recordings* del team del corso (v. Prefazione).

EECS 498 — Lecture 13: Attention (*Michigan Online, 1h10m*)

Costruisce l'attenzione di Bahdanau partendo dal bottleneck del seq2seq e arriva fino alla self-attention: è esattamente il percorso di questo capitolo e del successivo, in un video solo.

17.1 Sequence-to-sequence tasks

Molti problemi nel NLP e nella visione richiedono una trasformazione da una sequenza variabile-lunghezza a un'altra:

- **Machine translation:** “Hello world” → “Ciao mondo”
- **Summarization:** articolo lungo → sommario breve
- **Image captioning:** immagine → descrizione testuale
- **Question answering:** domanda → risposta

17.2 Encoder-decoder con bottleneck

L'approccio classico usa un'architettura encoder-decoder:

Definizione 17.1 (Encoder-Decoder Architecture). **Encoder:** elabora la sequenza di input e produce uno stato finale

$$h_t = \text{RNN}_{\text{enc}}(x_t, h_{t-1}), \quad c = h_T$$

Decoder: genera la sequenza di output, condizionata sul contesto c

$$s_t = \text{RNN}_{\text{dec}}(y_{t-1}, s_{t-1}, c)$$

$$P(y_t | \dots) = \text{softmax}(W s_t + b)$$

Il vettore $c = h_T$ è un **bottleneck**: contiene tutta l'informazione sull'input.

17.3 Information bottleneck problem

Problema del bottleneck

Comprimere un'intera sequenza di lunghezza variabile in un singolo vettore c è problematico:

- Per input lunghi, il primo token “dimenticato” prima di generare l'output
- La capacità di memoria di c è limitata
- Distanza tra input e output aumenta

Soprattutto per coppie di sequenze lunghe (traduzioni di articoli), il modello struggles.

17.4 Attention mechanism

La soluzione è permettere al decoder di **attentare** (fare focus) su diverse parti dell'input ad ogni step:

Definizione 17.2 (Attention Mechanism (Bahdanau et al., 2015)). Anziché usare solo il contesto finale $c = h_T$, il decoder calcola un contesto dinamico c_t a ogni step:

$$c_t = \sum_{j=1}^T \alpha_{t,j} h_j$$

dove i pesi di attention $\alpha_{t,j}$ indicano quanto il decoder dovrebbe “guardare” all'encoder output h_j :

$$\alpha_{t,j} = \frac{\exp(e_{t,j})}{\sum_{k=1}^T \exp(e_{t,k})}$$

Lo score di alignment $e_{t,j}$ misura la “compatibilità” tra lo stato del decoder s_{t-1} e l'encoder output h_j . Una scelta comune è l'**additive attention**:

$$e_{t,j} = v^\top \tanh(W_a s_{t-1} + U_a h_j)$$

dove $v \in \mathbb{R}^{d_a}$, $W_a \in \mathbb{R}^{d_a \times d_s}$, $U_a \in \mathbb{R}^{d_a \times d_h}$.

Intuizione — l'attenzione è un dizionario morbido

Il modo più economico per non perdersi fra query, key e value è pensare a una `dict` di Python.

In un dizionario normale si presenta una chiave, si cerca l'unica chiave identica, si restituisce il valore associato. L'accesso è *secco*: una chiave, un valore.

L'attenzione fa la stessa cosa in versione differenziabile. La *query* è ciò che sto cercando; le *key* sono le etichette di ciò che è disponibile; i *value* sono i contenuti. Solo che invece di cercare la chiave uguale, si misura la somiglianza fra query e ogni key (prodotto scalare), si trasforma in una distribuzione (softmax) e si restituisce la *media pesata* di tutti i value. Non si prende un elemento: si prende un po' di tutti, in proporzione a quanto sono pertinenti.

Questo spiega perché l'attenzione risolve il bottleneck. Nel seq2seq classico il decoder riceve un solo vettore $c = h_T$, e tutta la frase deve starci dentro. Con l'attenzione il decoder, a ogni passo, si costruisce il *suo* contesto c_t interrogando l'intera sequenza di encoder. Non c'è più compressione forzata, e la distanza fra un token di input e il

punto in cui serve diventa un cammino di lunghezza 1 invece che T .

Cross vs self, in una riga: se le query vengono da una sequenza e key/value da un'altra, è *cross*-attention (decoder che guarda l'encoder). Se tutte e tre vengono dalla stessa sequenza, è *self*-attention. Il meccanismo è identico, cambia solo da dove si pescano i tre ingredienti.

17.5 Cross-attention vs self-attention

Definizione 17.3 (Cross-Attention). Nel framework di attention generale:

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^\top}{\sqrt{d_k}} \right) V$$

dove:

- **Query Q :** viene dal decoder (“cosa sto cercando?”)
- **Key K e Value V :** vengono dall'encoder (“dove cercare?”)

In forma matriciale:

$$Q = SW_Q, \quad K = HW_K, \quad V = HW_V$$

dove $S \in \mathbb{R}^{B \times T_s \times d_s}$ è la sequenza di stati del decoder, $H \in \mathbb{R}^{B \times T_x \times d_h}$ è la sequenza di output dell'encoder, B è batch size.

Esempio svolto — un passo di attenzione con tre chiavi

Sia $d_k = 2$, query $\mathbf{q} = (1, 0)$, e tre coppie chiave-valore:

$$\mathbf{k}_1 = (1, 0), \quad \mathbf{k}_2 = (0, 1), \quad \mathbf{k}_3 = (0.7, 0.7), \quad \mathbf{v}_1 = (1, 0), \quad \mathbf{v}_2 = (0, 2), \quad \mathbf{v}_3 = (3, 3).$$

Punteggi scalati.

$$e_j = \frac{\mathbf{q} \cdot \mathbf{k}_j}{\sqrt{d_k}} = \frac{(1, 0, 0.7)}{\sqrt{2}} = (0.7071, 0, 0.4950).$$

Softmax.

$$\exp(e) = (2.0281, 1, 1.6405), \quad \sum = 4.6686 \Rightarrow \boldsymbol{\alpha} = (0.4344, 0.2142, 0.3514).$$

Output.

$$\sum_j \alpha_j \mathbf{v}_j = 0.4344(1, 0) + 0.2142(0, 2) + 0.3514(3, 3) = (1.489, 1.483).$$

Da leggere così: $\mathbf{k}_1$ è perfettamente allineata alla query e prende il peso maggiore; $\mathbf{k}_2$ è ortogonale e prende il minimo; $\mathbf{k}_3$ è a metà strada. Ma anche la chiave ortogonale contribuisce per il 21% — il softmax non azzera mai nulla, ed è per questo che si parla di *soft* alignment.

Perché si divide per $\sqrt{d_k}$: gli stessi numeri, senza scalatura. Supponiamo $d_k = 64$ e prodotti scalari grezzi $(8, 0, 5.6)$, cioè gli stessi valori scalati moltiplicati per $\sqrt{64} = 8$. Allora

$$\exp(8, 0, 5.6) = (2981, 1, 270) \Rightarrow \boldsymbol{\alpha} = (0.917, 0.0003, 0.083).$$

L'entropia della distribuzione crolla da 1.026 a 0.289 nat (il massimo su tre elementi è $\log 3 = 1.099$).

Il problema non è l'asimmetria in sé, è la *saturazione*: quando il softmax è quasi one-hot, la sua derivata $\alpha_i(\delta_{ij} - \alpha_j)$ è vicina a zero ovunque, e il gradiente non passa più. Poiché con componenti a varianza unitaria il prodotto scalare in d_k dimensioni ha deviazione standard $\sqrt{d_k}$, dividere per $\sqrt{d_k}$ riporta i logit a scala unitaria *indipendentemente dalla dimensione*. Non è un fattore cosmetico: senza, i Transformer con d_k grande non addestrano.

17.6 Attention come soft alignment

Interpretazione

L'attention può essere vista come un **soft alignment** tra source e target. Nel machine translation, $\alpha_{t,j}$ indica quale token source influenza il token target al passo t .

Visualizzando la matrice α (attention weights), si osservano spesso pattern diagonali leggermente shiftati, corrispondenti all'ordine naturale della traduzione. Deviazioni (ad es., salti) riflettono fenomeni linguistici come inversioni di ordine.

17.7 Encoder-decoder con attention

Integrando attention nel decoder:

$$c_t = \text{Attention}(s_{t-1}, H, H)$$

$$\tilde{s}_t = \tanh(W_c[s_{t-1}, c_t])$$

$$s_t = \text{RNN}_{\text{dec}}(\tilde{s}_t, s_{t-1})$$

Oppure più comunemente, si combina context e decoder state:

$$\tilde{s}_t = \text{RNN}_{\text{dec}}(y_{t-1}, s_{t-1})$$

$$c_t = \sum_j \alpha_{t,j} h_j, \quad \alpha_{t,j} \propto \exp(e_{t,j}(s_t, h_j))$$

$$\text{logits}_t = W[\tilde{s}_t; c_t] + b$$

17.8 Applicazioni dell'attention

Oltre al machine translation, l'attention è efficace in:

- **Image captioning**: visual attention su regioni della immagine
- **Document summarization**: attention su paragrafi source
- **Visual question answering**: cross-modal attention tra immagine e domanda

17.9 Approfondimenti

17.9.1 Domande d'esame

1. Descrivere il problema del bottleneck nel encoder-decoder senza attention.
2. Derivare la formula per i pesi di attention $\alpha_{t,j}$ e spiegare l'uso della softmax.
3. Qual è la differenza tra attention additiva e moltiplicativa (dot-product)? Quali sono i vantaggi di ciascuna?
4. In cross-attention, quale componente (Q, K, V) proviene dal decoder e quale dall'encoder?
5. Come si interpreta geometricamente l'attention matrix in machine translation?
6. Spiegare come l'attention mitiga il vanishing information problem per sequenze lunghe.
7. In image captioning, come differisce l'attention rispetto al machine translation?

Chapter 18

Transformers (Lezione 20)

Risorse video — GDL20 — Transformers

Videolezione del corso: registrazione della lezione GDL20 nella cartella *Recordings* del team del corso (v. Prefazione).

Attention is All You Need — spiegazione del modello, inferenza e training (*Umar Jamil, 1h*)

Il paper letto riga per riga, con le dimensioni dei tensori sempre a schermo. È il video che chiarisce i dettagli che le slide comprimono: masking, positional encoding, e che cosa cambia fra training e inferenza.

EECS 498 — Lecture 13: Attention (*Michigan Online, 1h10m*)

La seconda metà arriva alla self-attention e al blocco Transformer: utile come ripasso rapido prima di questo capitolo.

18.1 Self-attention

Il Transformer (Vaswani et al., 2017) abbandona la ricorrenza a favore di **self-attention**, dove una sequenza attende a se stessa:

Definizione 18.1 (Self-Attention).

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V$$

dove **tutte** le query, key, e value vengono dalla stessa sequenza:

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V$$

dove $X \in \mathbb{R}^{n \times d}$ è la sequenza di input (n token, dimensione d), e $W_Q, W_K, W_V \in \mathbb{R}^{d \times d_k}$.

L'output è $n \times d$ dove ogni posizione i attende a tutte le posizioni j , con peso proporzionale a $\exp(\frac{Q_i K_j^\top}{\sqrt{d_k}})$.

Scala $1/\sqrt{d_k}$

Il fattore di scala $\frac{1}{\sqrt{d_k}}$ previene la saturazione della softmax per dimensioni grandi. Se d_k è grande, i prodotti $Q_i K_j^\top$ possono essere molto grandi, causando softmax a collassare in una distribuzione sharp (one-hot approssimativamente) con gradienti piccoli. Dividendo per $\sqrt{d_k}$ si normalizza la varianza: $\mathbb{E}[Q_i K_j^\top] = 0$ e $\text{Var}[Q_i K_j^\top] = 1$.

18.2 Multi-head attention

Una singola testa di attention cattura un tipo di dipendenza. Più teste, in parallelo, permettono di catturare diversi aspetti:

Definizione 18.2 (Multi-Head Attention).

$$\text{head}_i = \text{Attention}(XW_{Q,i}, XW_{K,i}, XW_{V,i})$$

$$\text{MultiHead}(X) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W_O$$

dove $W_{Q,i}, W_{K,i}, W_{V,i} \in \mathbb{R}^{d \times d_k}$, e $W_O \in \mathbb{R}^{hd_k \times d}$.

Tipicamente $h = 8$ o $h = 16$, e $d_k = d/h$ per mantenere la dimensione totale costante.

18.3 Positional encoding

A differenza delle RNN, il Transformer non ha nozione intrinseca di ordine. Per preservare l'informazione posizionale, si aggiungono **positional encodings**:

Definizione 18.3 (Sinusoidal Positional Encoding).

$$PE(\text{pos}, 2i) = \sin\left(\frac{\text{pos}}{10000^{2i/d}}\right)$$

$$PE(\text{pos}, 2i + 1) = \cos\left(\frac{\text{pos}}{10000^{2i/d}}\right)$$

dove pos è la posizione nella sequenza $(0, 1, \dots, n-1)$ e i è la dimensione $(0, 1, \dots, d/2-1)$.

Le funzioni sinusoidali hanno la proprietà che $PE(\text{pos} + \Delta, \cdot)$ può essere espressa come una combinazione lineare di $PE(\text{pos}, \cdot)$, permettendo al modello di apprendere offset relativi.

Un'alternativa è usare **learnable positional embeddings**, vettori $e_{\text{pos}} \in \mathbb{R}^d$ addestrati durante l'apprendimento.

18.4 Transformer block

Un blocco Transformer consiste di due sub-layer con residual connections e normalization:

Definizione 18.4 (Transformer Block).

$$X' = X + \text{MultiHeadAttention}(X)$$

$$X'' = \text{LayerNorm}(X')$$

$$Z = X'' + \text{FFN}(X'')$$

dove (assumendo post-norm):

$$\text{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2$$

In pre-norm (più stabile):

$$X' = X + \text{MultiHeadAttention}(\text{LayerNorm}(X))$$

$$Z = X' + \text{FFN}(\text{LayerNorm}(X'))$$

Tipicamente $d_{\text{ff}} = 4 \times d$ (feed-forward hidden dimension è 4 volte la dimensione del modello).

18.5 Full Transformer architecture

Definizione 18.5 (Transformer Encoder-Decoder). **Encoder:** stack di N blocchi, ciascuno con self-attention su tutta la sequenza source:

$$H = \text{Encoder}(X)$$

Decoder: stack di N blocchi, ciascuno con:

1. **Masked self-attention** sulla sequenza target (non attende a posizioni future)
2. **Cross-attention** sulla sequenza source
3. **Feed-forward**

$$Y = \text{Decoder}(y_{1:t-1}, H)$$

La masked self-attention ha matrice di attenzione:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top + M}{\sqrt{d_k}}\right)V$$

dove M è una matrice che setta le posizioni future a $-\infty$:

$$M_{ij} = \begin{cases} 0 & \text{se } j \leq i \\ -\infty & \text{se } j > i \end{cases}$$

18.6 Complexity analysis

18.6.1 Self-attention complexity

La matrice di attenzione è $n \times n$ (dove n è lunghezza sequenza), quindi:

$$\text{Time complexity: } O(n^2d)$$

$$\text{Space complexity: } O(n^2 + nd)$$

per immagazzinare la matrice di attenzione e gli input.

18.6.2 RNN complexity

Le RNN processano sequenzialmente:

$$\text{Time complexity: } O(nd^2)$$

$$\text{Space complexity: } O(d^2)$$

per una RNN con dimensione nascosta d .

Per sequenze lunghe, self-attention è più efficiente se $d > n$; per sequenze corte RNN è competitivo. Tuttavia, self-attention è completamente parallelizzabile (tutti i passi temporali contemporaneamente), mentre le RNN sono sequenziali. In pratica, Transformer è molto più veloce grazie alla parallelizzazione GPU.

Intuizione — il Transformer scambia complessità per profondità di cammino

Il confronto con la RNN si riassume in due colonne, e conviene averle in testa esattamente così.

Per una sequenza di lunghezza n e dimensione d , la self-attention costa $\mathcal{O}(n^2d)$ e la RNN $\mathcal{O}(nd^2)$. Quindi la self-attention è più economica finché $n < d$, e più costosa quando $n > d$. Con $n = 512$ e $d = 64$ i conti danno 1.7×10^7 contro 2.1×10^6 : la self-attention costa 8 volte tanto.

Sembra una sconfitta, ma le colonne sono due. Il *maximum path length* — quanti passi deve fare l'informazione per andare dal token 1 al token n — è $\mathcal{O}(n)$ per la RNN e $\mathcal{O}(1)$ per la self-attention. E le operazioni sequenziali obbligate sono $\mathcal{O}(n)$ per la RNN, $\mathcal{O}(1)$ per il Transformer.

È qui che si decide tutto. La RNN ha un costo teorico più basso ma è *intrinsecamente sequenziale*: non si può calcolare h_t senza h_{t-1} , quindi le GPU restano in gran parte inutilizzate. Il Transformer fa più FLOP ma li fa tutti in parallelo, quindi il tempo di parete crolla. E il cammino di lunghezza 1 significa che il gradiente fra due token qualsiasi non attraversa n moltiplicazioni: il vanishing gradient a lungo raggio semplicemente non si pone.

Il prezzo, che va detto: il termine n^2 in memoria è il vero limite, ed è la ragione dell'intera letteratura sugli efficient Transformer (Linformer, Performer, sparse attention). E il Transformer ha un inductive bias molto più debole della CNN o della RNN — nessuna località, nessuna nozione di ordine se non quella iniettata dal positional encoding — quindi ha bisogno di molti più dati per compensare ciò che le altre architetture avevano gratis.

18.7 Gradient flow advantage

Nel Transformer, any two positions are connected in $\mathcal{O}(1)$ layers:

$$\text{Distanza massima} = 1 \quad (\text{direct attention})$$

Nelle RNN, la distanza è $\mathcal{O}(n)$, causando exploding/vanishing gradient per lunghe dipendenze. Questo è un grande vantaggio del Transformer.

18.8 Self-supervised training

18.8.1 BERT (Bidirectional Encoder)

BERT usa un Transformer encoder con **masked language modeling**:

- 15% dei token sono sostituiti da [MASK]
- Model deve predire i token originali da contesto biadirezionale
- Addestrato su miliardi di token non etichettati

18.8.2 GPT (Generative Pre-training)

GPT usa un Transformer decoder con **autoregressive language modeling**:

- Predict il prossimo token data la sequenza passata
- Masked self-attention previene l'accesso a token futuri
- Addestrato su miliardi di token

18.8.3 Pre-training e fine-tuning

Entrambi gli approcci seguono il paradigma:

Pre-training (non supervisionato, dati massicci)

Fine-tuning (supervisionato, dati task-specific)

Il pre-training cattura pattern linguistici generali; il fine-tuning adatta al task specifico.

18.9 Inductive biases

A differenza delle CNN e RNN, il Transformer ha inductive biases deboli:

- **No locality:** self-attention attende a tutte le posizioni ugualmente (senza bias verso posizioni vicine)
- **No translation equivariance:** l'architettura è permutation-equivariant (con positional encodings)
- **Global dependencies:** preferenza per dipendenze globali

Questo richiede molti dati e paramet per imparare, ma fornisce flessibilità per catturare relazioni complesse (es., coreference resolution in NLP).

18.10 Appendix: Efficient Transformers

Per sequenze molto lunghe, la complessità $O(n^2)$ diventa proibitiva. Approcci alternativi:

- **Sparse attention:** attendance solo a un subset di posizioni (es., sparse patterns, local windows)
- **Linear attention:** approssimazione kernel che riduce a $O(n)$
- **Hierarchical attention:** windowed attention multi-scala
- **Recurrent Transformer:** combine attention con ricorrenza selettiva

18.11 Approfondimenti

18.11.1 Domande d'esame

1. Spiegare il meccanismo di self-attention e come differ dalla cross-attention del capitolo precedente.
2. Derivare la formula per multi-head attention e spiegare perché more heads permettono di catturare diverse dipendenze.
3. Perché il fattore di scala $\frac{1}{\sqrt{d_k}}$ è importante nella self-attention?
4. Descrivere le positional encodings sinusoidali. Quali proprietà hanno?
5. Disegnare un blocco Transformer (pre-norm variant) e etichettare tutti i componenti.
6. Qual è la differenza tra attention mascherato (nel decoder) e non mascherato (nell'encoder)?

7. Confrontare complexity time e space di Transformer vs RNN. Quando è Transformer più efficiente?
8. Spiegare come il Transformer affronta il problema di vanishing/exploding gradients meglio delle RNN.
9. Descrivere il processo di pre-training e fine-tuning in BERT e GPT.
10. In BERT, perché usiamo bidirectional attention durante pre-training, mentre in GPT usiamo masked attention?

Part IV

Generative Deep Learning

Questa parte introduce i modelli generativi profondi: dall'autoencoder regolarizzato ai modelli moderni di trasporto di probabilità (flow matching). Il filo conduttore è duplice. Da un lato, una tassonomia comune (figura nella slide GDL23) divide i modelli in *esplicitamente densi* (VAE, normalizing flows, autoregressivi, diffusion via ELBO) e *implicitamente densi* (GAN, modelli che imparano un processo di campionamento). Dall'altro, la famiglia di modelli vista dal punto di vista della *trasformazione di rumore in dati*: VAE $\rightarrow$ GAN $\rightarrow$ Flows $\rightarrow$ Diffusion $\rightarrow$ Flow Matching costituiscono uno spettro in cui ogni modello rilassa o specializza il precedente.

Chapter 19

Autoencoders e Manifold Learning

Risorse video — GDL23 — Autoencoder e manifold hypothesis

Videolezione del corso: registrazione della lezione GDL23 nella cartella *Recordings* del team del corso (v. Prefazione).

Autoencoders — Deep Learning Animated (*Deepia*, 20m)

Divulgativo e visivamente ottimo per l'intuizione geometrica (bottleneck, manifold), ma non copre il taglio “regularized AE” delle slide: match dichiarato parziale.

Regularized Autoencoders: Denoising, Contractive, Variational (*Charu Aggarwal*, 40m)

Questo invece copre esattamente sparse, denoising e contractive con le rispettive regolarizzazioni: è il complemento del primo.

19.1 Motivazione e posizionamento

L'obiettivo della modellazione generativa è apprendere, da un dataset $\mathcal{D} = \{\mathbf{x}_1, \dots, \mathbf{x}_N\}$ campionato da una distribuzione sconosciuta $P(\mathbf{x})$, un modello $P_\theta(\mathbf{x}) \approx P(\mathbf{x})$ oppure un processo di campionamento che produca esemplari indistinguibili da quelli in $\mathcal{D}$. Esistono due famiglie:

Modelli espliciti apprendono $P_\theta(\mathbf{x})$ in forma chiusa o tramite un *lower bound* (VAE, flows, diffusion, autoregressivi).

Modelli impliciti apprendono solo una procedura di campionamento $\mathbf{x} \sim P_\theta$ senza esprimere la densità (GAN, generative stochastic networks).

L'autoencoder (AE) è il punto di partenza: storicamente nato come modello *non* generativo per riduzione di dimensionalità, viene rimodulato per generare attraverso interpretazioni probabilistiche (DAE come stimatore della score, VAE come modello a variabile latente differenziabile).

19.2 Autoencoder neurale

Definizione 19.1 (Autoencoder). Un autoencoder è una coppia di funzioni (f_ϕ, g_θ) con $f_\phi : \mathcal{X} \rightarrow \mathcal{Z}$ (encoder) e $g_\theta : \mathcal{Z} \rightarrow \mathcal{X}$ (decoder), addestrata a minimizzare un errore di ricostruzione

$$\mathcal{L}_{\text{AE}}(\theta, \phi) = \frac{1}{N} \sum_{i=1}^N L(\mathbf{x}_i, g_\theta(f_\phi(\mathbf{x}_i))),$$

tipicamente con $L = \|\cdot\|_2^2$. La rappresentazione $\mathbf{z} = f_\phi(\mathbf{x})$ vive nel *bottleneck*, con $\dim(\mathcal{Z}) \ll \dim(\mathcal{X})$, oppure (in AE sovracompleti) con un vincolo di regolarizzazione che impedisce l'apprendimento dell'identità.

Osservazione 19.1 (Manifold hypothesis). Si assume che i dati osservati giacciono (approssimativamente) su una sottovarietà $\mathcal{M} \subset \mathcal{X}$ di dimensione molto inferiore a $\dim(\mathcal{X})$. L'AE impara una parametrizzazione di $\mathcal{M}$ via f_ϕ (immersione) e g_θ (ricostruzione). La regolarizzazione spinge $g_\theta \circ f_\phi$ a essere sensibile solo alle variazioni *tangenti* alla varietà.

Intuizione — il bottleneck non è l'unico modo di impedire la copia

Il rischio strutturale di un autoencoder è banale: se encoder e decoder sono abbastanza espressivi, la soluzione ottima è l'identità. Ricostruzione perfetta, rappresentazione inutile.

Il bottleneck (latente più piccolo dell'input) è il rimedio ovvio: se lo spazio non basta, la rete deve buttare via qualcosa, e conviene che butti via il rumore invece del segnale. Ma è un rimedio grezzo, perché la capacità va scelta a mano e non c'è garanzia che ciò che sopravvive sia semanticamente interessante.

Le varianti regolarizzate sostituiscono il vincolo di *dimensione* con un vincolo sul *comportamento*, e questo permette di usare anche latenti sovracompleti:

- **Sparse**: penalizza $\|\mathbf{h}\|_1$. Il latente può essere grande, ma solo poche unità si accendono per input — capacità limitata per campione, non in assoluto.
- **Denoising**: corrompe l'input e chiede di ricostruire l'originale. L'identità non è più una soluzione, perché copierebbe anche il rumore. La rete è costretta a imparare che cosa *non* appartiene ai dati.
- **Contractive**: penalizza $\|\partial \mathbf{h} / \partial \mathbf{x}\|_F^2$. Chiede esplicitamente che la rappresentazione sia insensibile a piccole perturbazioni, tranne che nelle direzioni tangenti alla varietà dei dati.

Il filo che porta al resto della Parte IV: DAE e CAE arrivano alla stessa conclusione da direzioni diverse, cioè che il modello deve imparare come i dati si comportano *attorno* alla varietà, non solo sopra. Il DAE in particolare, si dimostrerà, stima implicitamente $\nabla_{\mathbf{x}} \log p(\mathbf{x})$ — la *score*. È il motivo per cui il capitolo su diffusion e score matching non è un argomento nuovo, ma il seguito di questo.

19.3 Varianti regolarizzate

19.3.1 Sparse autoencoder (SAE)

Aggiunge una penalità sull'attivazione latente

$$\mathcal{L}_{\text{SAE}} = L(\mathbf{x}, \tilde{\mathbf{x}}) + \lambda \Omega(\mathbf{z}), \quad \Omega(\mathbf{z}) = \|\mathbf{z}\|_1 = \sum_j |z_j|.$$

Osservazione 19.2 (Interpretazione MAP). $\Omega(\mathbf{z}) = \lambda \|\mathbf{z}\|_1$ corrisponde a un prior di Laplace $P(\mathbf{z}) \propto \exp(-\lambda \|\mathbf{z}\|_1/2)$. Allenare con regolarizzazione equivale (a meno di costanti) a $\max_{\theta} \log P(\mathbf{x}|\mathbf{z}) + \log P(\mathbf{z})$, ossia inferenza MAP.

19.3.2 Denoising autoencoder (DAE)

Si corrompe l'input con un processo di rumore $C(\hat{\mathbf{x}}|\mathbf{x})$, tipicamente $\hat{\mathbf{x}} = \mathbf{x} + \epsilon$ con $\epsilon \sim \mathcal{N}(0, \sigma^2 \mathbf{I})$, e si minimizza

$$\mathcal{L}_{\text{DAE}} = \mathbb{E}_{\mathbf{x}, \hat{\mathbf{x}}} \|\mathbf{x} - g_\theta(f_\phi(\hat{\mathbf{x}}))\|_2^2.$$

Teorema 19.1 (DAE come stima della score, Vincent 2011). Sotto rumore gaussiano $\hat{\mathbf{x}} = \mathbf{x} + \boldsymbol{\epsilon}$ con $\boldsymbol{\epsilon} \sim \mathcal{N}(0, \sigma^2 \mathbf{I})$ e per σ piccolo, il minimo della loss DAE è il *conditional mean estimator* $g(f(\hat{\mathbf{x}})) = \mathbb{E}[\mathbf{x} | \hat{\mathbf{x}}]$. Applicando la formula di Tweedie

$$\mathbb{E}[\mathbf{x} | \hat{\mathbf{x}}] = \hat{\mathbf{x}} + \sigma^2 \nabla_{\hat{\mathbf{x}}} \log p(\hat{\mathbf{x}}),$$

si ottiene

$$\nabla_{\hat{\mathbf{x}}} \log p(\hat{\mathbf{x}}) = \frac{g_{\theta}(f_{\phi}(\hat{\mathbf{x}})) - \hat{\mathbf{x}}}{\sigma^2}.$$

Il campo vettoriale $g(f(\hat{\mathbf{x}})) - \hat{\mathbf{x}}$ approssima il gradiente della log-densità (la *score function*).

Osservazione 19.3 (Ponte con diffusion). Questa identità è il seme del legame fra DAE, score matching e modelli di diffusione (Cap. 23 e 24): un DAE già impara la score; un diffusion model è un DAE gerarchico a livelli multipli di σ .

19.3.3 Contractive autoencoder (CAE)

Penalizza la sensibilità dell'encoder

$$\mathcal{L}_{\text{CAE}} = L(\mathbf{x}, \tilde{\mathbf{x}}) + \lambda \left\| \frac{\partial f_{\phi}(\mathbf{x})}{\partial \mathbf{x}} \right\|_F^2.$$

La rappresentazione $\mathbf{z}$ varia poco rispetto a perturbazioni infinitesime di $\mathbf{x}$: si apprende un'immersione locale a varietà.

19.4 Deep autoencoders e pretraining

Un deep AE è una composizione gerarchica di encoder $f_1, \dots, f_L$ e decoder simmetrici $g_L, \dots, g_1$. Storicamente, prima di Adam e batch normalization, l'addestramento end-to-end soffriva del vanishing gradient (Cap. 15). La soluzione era il *layerwise unsupervised pretraining*:

1. Addestra (W_1) come AE su $\mathbf{x}$;
2. Congela W_1 , addestra (W_2) come AE su $\mathbf{z}_1 = f_1(\mathbf{x})$;
3. Continua fino al livello L ;
4. (Opzionale) fine-tuning end-to-end di tutta la pila.

Connessione con RBM. Riscrivendo lo stack di autoencoder come una struttura a strati di unità stocastiche bidirezionali si ottiene la *Deep Belief Network* (DBN): una pila di RBM (Cap. 13) addestrate con contrastive divergence. La DBN è (in larga parte) un grafo diretto top-down con uno strato superiore non diretto (l'ultimo RBM). La *Deep Boltzmann Machine* (DBM), invece, è un modello completamente non diretto, e richiede il *double counting trick* (dimezzare i pesi degli strati intermedi durante il pretraining per evitare di contare due volte i contributi $\mathbf{x} \rightarrow \mathbf{z}_1$).

19.5 Applicazioni

Visualizzazione e indexing proiezione su 2D/3D per esplorazione dati.

Anomaly detection punti con errore di ricostruzione $\|\mathbf{x} - g(f(\mathbf{x}))\|$ sopra una soglia sono potenziali anomalie.

Pretraining supervisionato iniziare un classificatore profondo dai pesi di un deep AE.

Compressione e denoising usando CNN/U-Net come encoder/decoder.

Multimodal learning DBM multimodale (Srivastava & Salakhutdinov 2014) fonde più modalità (immagini + testo) in uno spazio condiviso.

Quando usare un autoencoder

Si vuole una rappresentazione compatta non supervisionata

DAE o CAE: la regolarizzazione fissa la varietà.

Anomaly detection

usare la distribuzione degli errori di ricostruzione sui normali.

Generazione di qualità

Non l'AE classico: l'output del bottleneck non è una distribuzione campionabile. Passare a VAE (Cap. 20) o diffusion (Cap. 23).

Pretraining moderno

generalmente sostituito da self-supervised contrastivo (SimCLR, BYOL) per le immagini, ma DAE rimane competitivo per dati strutturati e biomedici.

Connessioni

- DAE $\leftrightarrow$ score matching: il DAE apprende il campo $\nabla_{\mathbf{x}} \log p(\mathbf{x})$ (Tweedie).
- AE regolarizzato $\leftrightarrow$ MAP: la regolarizzazione $\Omega(\mathbf{z})$ implementa un prior sulla latente.
- Deep AE $\leftrightarrow$ DBN/DBM: la stessa architettura, ma con unità stocastiche e training contrastivo.
- AE $\rightarrow$ VAE: aggiungere un encoder stocastico e una loss ELBO trasforma l'AE in un modello latente generativo.

Domande d'esame — Autoencoders

1. Cosa significa che il DAE impara un'approssimazione della score function? Deriva l'identità di Tweedie e mostra come si arriva a $\nabla_{\hat{\mathbf{x}}} \log p(\hat{\mathbf{x}}) = (g(f(\hat{\mathbf{x}})) - \hat{\mathbf{x}})/\sigma^2$.
2. Differenza concettuale e formale fra sparse AE, denoising AE e contractive AE. Quando ne preferiresti uno rispetto agli altri?
3. Spiega il *layerwise pretraining* di un deep AE. Perché era necessario in passato e perché oggi è meno usato?
4. Differenza fra DBN e DBM: quale è diretto, quale no, come differisce l'inferenza.
5. Perché un AE classico non è un buon modello generativo, anche se ha un bottleneck? (suggerimento: lo spazio latente non è distribuito secondo una distribuzione campionabile).
6. Confronto fra Tweedie's formula e reparametrization trick: entrambi permettono di trattare distribuzioni gaussiane in modi che facilitano l'apprendimento — sono lo stesso strumento?

Chapter 20

Variational Autoencoders

Risorse video — GDL24 — Variational Autoencoders

Videolezione del corso: registrazione della lezione GDL24 nella cartella *Recordings* del team del corso (v. Prefazione).

CS294-158 SP24 — L4: Latent Variable Models and Variational AutoEncoders (*Pieter Abbeel, UC Berkeley, 2h*)

Deriva l'ELBO senza saltare passaggi e spiega il reparametrization trick come soluzione a un problema preciso di stima del gradiente. Lungo, ma è la trattazione più completa disponibile.

Variational Inference: Foundations and Innovations (*David Blei, 1h30m*)

Da rivedere se l'ELBO della Parte II non è ancora automatico: il VAE è quella stessa ELBO con q parametrizzata da una rete.

20.1 Dalla DAE al modello a variabile latente

L'osservazione cruciale (Cap. 19, formula di Tweedie) è che il DAE stima $\nabla_x \log p(\mathbf{x})$ solo nell'intorno dei training point. Vogliamo invece un modello esplicito di $p_\theta(\mathbf{x})$, ben definito su tutto $\mathcal{X}$.

L'idea è un modello a variabile latente continua:

$$p_\theta(\mathbf{x}) = \int p_\theta(\mathbf{x} | \mathbf{z}) p(\mathbf{z}) d\mathbf{z}, \quad (20.1)$$

con $p(\mathbf{z}) = \mathcal{N}(0, \mathbf{I})$ prior semplice e $p_\theta(\mathbf{x} | \mathbf{z})$ rete neurale (decoder). L'integrale è intrattabile in dimensione alta: serve un'approssimazione variazionale.

20.2 ELBO

Teorema 20.1 (Evidence Lower BOund). Per qualunque distribuzione $q_\phi(\mathbf{z} | \mathbf{x})$ vale

$$\log p_\theta(\mathbf{x}) = \underbrace{\mathbb{E}_{q_\phi(\mathbf{z} | \mathbf{x})} \left[\log \frac{p_\theta(\mathbf{x}, \mathbf{z})}{q_\phi(\mathbf{z} | \mathbf{x})} \right]}_{\mathcal{L}(\mathbf{x}; \theta, \phi)} + D_{\text{KL}}(q_\phi(\mathbf{z} | \mathbf{x}) \| p_\theta(\mathbf{z} | \mathbf{x})).$$

Proof. $\log p(\mathbf{x}) = \log \int p(\mathbf{x}, \mathbf{z}) d\mathbf{z} = \log \mathbb{E}_q \left[\frac{p(\mathbf{x}, \mathbf{z})}{q(\mathbf{z} | \mathbf{x})} \right]$. La disuguaglianza di Jensen su $\log$ (concavo) dà $\log p(\mathbf{x}) \geq \mathbb{E}_q \left[\log \frac{p(\mathbf{x}, \mathbf{z})}{q(\mathbf{z} | \mathbf{x})} \right]$. L'identità con il gap-KL si ottiene scrivendo $p(\mathbf{x}, \mathbf{z}) = p(\mathbf{z} | \mathbf{x}) p(\mathbf{x})$ e separando i termini. $\square$

Poiché $D_{\text{KL}}(\geq \|0\|)$:

$$\log p_{\theta}(\mathbf{x}) \geq \mathcal{L}(\mathbf{x}; \theta, \phi). \quad (20.2)$$

Massimizzare $\mathcal{L}$ rispetto a (θ, ϕ) *contemporaneamente* (i) aumenta la verosimiglianza del modello, (ii) avvicina $q_{\phi}(\mathbf{z}|\mathbf{x})$ alla vera posteriori intrattabile $p_{\theta}(\mathbf{z}|\mathbf{x})$.

Una riscrittura utile:

$$\boxed{\mathcal{L}(\mathbf{x}; \theta, \phi) = \underbrace{\mathbb{E}_{q_{\phi}(\mathbf{z}|\mathbf{x})}[\log p_{\theta}(\mathbf{x}|\mathbf{z})]}_{\text{ricostruzione}} - \underbrace{D_{\text{KL}}(q_{\phi}(\mathbf{z}|\mathbf{x})\|p(\mathbf{z}))}_{\text{regolarizzazione}}}. \quad (20.3)$$

20.3 Architettura VAE

Encoder $q_{\phi}(\mathbf{z}|\mathbf{x}) = \mathcal{N}(\boldsymbol{\mu}_{\phi}(\mathbf{x}), \text{diag}(\boldsymbol{\sigma}_{\phi}^2(\mathbf{x})))$: una rete neurale produce $\boldsymbol{\mu}, \boldsymbol{\sigma}$.

Decoder $p_{\theta}(\mathbf{x}|\mathbf{z})$ è tipicamente $\mathcal{N}(g_{\theta}(\mathbf{z}), \sigma_x^2 \mathbf{I})$ (output continuo) o Bernoulli (binari) o categoriale (multinomiali).

Prior $p(\mathbf{z}) = \mathcal{N}(0, \mathbf{I})$.

20.3.1 Reparametrization trick

Il problema: $\mathbb{E}_{q_{\phi}}[\cdot]$ richiede campionare $\mathbf{z}$ da q_{ϕ} che dipende da ϕ . Il sampling non è differenziabile. La soluzione:

$$\mathbf{z} = \boldsymbol{\mu}_{\phi}(\mathbf{x}) + \boldsymbol{\sigma}_{\phi}(\mathbf{x}) \odot \boldsymbol{\epsilon}, \quad \boldsymbol{\epsilon} \sim \mathcal{N}(0, \mathbf{I}). \quad (20.4)$$

Ora $\boldsymbol{\epsilon}$ non dipende dai parametri, e $\mathbf{z}$ è una funzione differenziabile di $(\boldsymbol{\mu}, \boldsymbol{\sigma}, \boldsymbol{\epsilon})$. Quindi

$$\nabla_{\phi} \mathbb{E}_{q_{\phi}}[f(\mathbf{z})] = \mathbb{E}_{\boldsymbol{\epsilon}}[\nabla_{\phi} f(\boldsymbol{\mu}_{\phi}(\mathbf{x}) + \boldsymbol{\sigma}_{\phi}(\mathbf{x}) \odot \boldsymbol{\epsilon})],$$

che si stima con Monte Carlo (in pratica un solo $\boldsymbol{\epsilon}$ per esempio).

Intuizione — il reparametrization trick risolve un problema di gradiente, non di campionamento

Campionare $\mathbf{z} \sim q_{\phi}(\mathbf{z} | \mathbf{x})$ non è difficile: è una gaussiana, si sa fare. Il problema è un altro: l'operazione di campionamento è *stocastica e non differenziabile rispetto a ϕ* . Il gradiente che dovrebbe tornare all'encoder trova un nodo casuale sulla strada e si ferma.

Esisterebbe una via alternativa, lo stimatore REINFORCE (score function estimator), che è non distorto ma ha varianza altissima — in pratica inutilizzabile per addestrare un encoder.

Il trucco della riparametrizzazione sposta la casualità fuori dal cammino del gradiente. Invece di campionare $\mathbf{z}$ da una distribuzione che dipende da ϕ , si campiona $\boldsymbol{\epsilon} \sim \mathcal{N}(0, \mathbf{I})$ — che non dipende da niente — e si costruisce

$$\mathbf{z} = \boldsymbol{\mu}_{\phi}(\mathbf{x}) + \boldsymbol{\sigma}_{\phi}(\mathbf{x}) \odot \boldsymbol{\epsilon}.$$

Ora $\mathbf{z}$ è una funzione *deterministica e differenziabile* di ϕ e di un rumore esterno. Il grafo computazionale è continuo da capo a fondo e la backpropagation passa normalmente.

Il modo di dirlo all'orale: la casualità non è stata eliminata, è stata spostata da dentro il grafo a un ingresso del grafo. La differenza in varianza dello stimatore è di ordini di grandezza, ed è ciò che rende il VAE addestrabile.

Limite implicito: il trucco funziona perché la gaussiana ammette una riparametrizzazione location-scale. Per latenti discreti non esiste, ed è la ragione per cui servono

Gumbel-Softmax, straight-through estimator o VQ-VAE.

20.3.2 KL in forma chiusa

Per $q = \mathcal{N}(\boldsymbol{\mu}, \text{diag}(\boldsymbol{\sigma}^2))$ e $p = \mathcal{N}(0, \mathbf{I})$:

$$D_{\text{KL}}(q||p) = \frac{1}{2} \sum_{j=1}^d (\mu_j^2 + \sigma_j^2 - 1 - \log \sigma_j^2). \quad (20.5)$$

Sketch. $D_{\text{KL}}(\mathcal{N}(\mu_1, \sigma_1^2) || \mathcal{N}(\mu_2, \sigma_2^2)) = \log \frac{\sigma_2}{\sigma_1} + \frac{\sigma_1^2 + (\mu_1 - \mu_2)^2}{2\sigma_2^2} - \frac{1}{2}$. Per gaussiane indipendenti la KL è somma componente per componente. Specializzando con $\mu_2 = 0, \sigma_2 = 1$ si ottiene (20.5). $\square$

20.3.3 Loss finale

Combinando (20.3), (20.5) e la reparametrizzazione:

$$-\mathcal{L} \approx \frac{1}{2\sigma_x^2} \|\mathbf{x} - g_\theta(\mathbf{z})\|^2 + \frac{1}{2} \sum_j (\mu_j^2 + \sigma_j^2 - 1 - \log \sigma_j^2),$$

con $\mathbf{z} = \boldsymbol{\mu}_\phi(\mathbf{x}) + \boldsymbol{\sigma}_\phi(\mathbf{x}) \odot \boldsymbol{\epsilon}$.

Esempio svolto — i due termini dell'ELBO, con numeri

La loss del VAE per un campione è

$$-\mathcal{L} = \underbrace{-\mathbb{E}_q[\log p_\theta(\mathbf{x} | \mathbf{z})]}_{\text{ricostruzione}} + \underbrace{D_{\text{KL}}(q_\phi(\mathbf{z}|\mathbf{x})||p(\mathbf{z}))}_{\text{regolarizzazione}}.$$

Il termine KL, in forma chiusa. Per $q = \mathcal{N}(\mu, \sigma^2)$ e prior $p = \mathcal{N}(0, 1)$ in una dimensione:

$$D_{\text{KL}}(q||p) = \frac{1}{2} (\mu^2 + \sigma^2 - 1 - \log \sigma^2).$$

Con $\mu = 0.5$ e $\sigma = 0.8$:

$$\frac{1}{2} (0.25 + 0.64 - 1 + 0.4463) = \frac{1}{2} (0.3363) = \mathbf{0.168}.$$

Si verifichino i due casi limite: con $\mu = 0, \sigma = 1$ la KL è esattamente 0 (il posterior coincide col prior); con $\sigma \rightarrow 0$ il termine $-\log \sigma^2$ diverge, quindi il modello è *penalizzato all'infinito* se prova a rendere l'encoder deterministico.

Che cosa fa quel $-\log \sigma^2$. È il termine che impedisce al VAE di collassare in un autoencoder normale. Senza rumore nel latente il modello ricostruirebbe meglio, ma lo spazio latente resterebbe pieno di buchi e il campionamento da $p(\mathbf{z})$ produrrebbe spazzatura. La KL forza le gaussiane posteriori a sovrapporsi, riempiendo lo spazio.

Il tiro alla fune, e le due patologie. I due termini si oppongono:

- se la KL vince, si ha *posterior collapse*: $q_\phi(\mathbf{z}|\mathbf{x}) \rightarrow p(\mathbf{z})$ per ogni $\mathbf{x}$, la KL va a 0, il latente non porta più informazione sull'input e il decoder impara a ignorarlo. Si riconosce guardando la KL: se tende a zero mentre la ricostruzione non migliora, è questo.
- se vince la ricostruzione, il latente si sparpaglia e il campionamento a priori peggiora.

Il β -VAE non fa che mettere un peso esplicito su questo equilibrio: $\beta > 1$ spinge verso il disentanglement pagando in qualità di ricostruzione, $\beta < 1$ il contrario.

Nota sull'immagine sfocata. Le ricostruzioni del VAE sono tipicamente sfocate perché il termine di ricostruzione, con decoder gaussiano a varianza fissa, è un errore quadratico: di fronte a un'ambiguità, l'MSE preferisce la *media* di tutte le possibilità plausibili a una qualsiasi di esse. La media di più volti nitidi è un volto sfocato. Il GAN del capitolo seguente non ha questo problema proprio perché non ottimizza una verosimiglianza per pixel.

20.4 Generazione e CVAE

Campionamento (test). Si stacca l'encoder: $\mathbf{z} \sim \mathcal{N}(0, \mathbf{I})$, $\tilde{\mathbf{x}} = g_\theta(\mathbf{z})$.

Conditional VAE. Si aggiunge un input $\mathbf{y}$ (etichetta o condizione) a encoder e decoder:

$$\mathcal{L}_{\text{CVAE}}(\mathbf{x}, \mathbf{y}) = \mathbb{E}_{q_\phi(\mathbf{z}|\mathbf{x}, \mathbf{y})}[\log p_\theta(\mathbf{x}|\mathbf{z}, \mathbf{y})] - D_{\text{KL}}(q_\phi(\mathbf{z}|\mathbf{x}, \mathbf{y}) \| p(\mathbf{z}|\mathbf{y})).$$

20.5 Interpretazione info-teorica

Riscrivendo l'ELBO:

$$\mathcal{L} = \mathbb{E}_{\mathbf{x} \sim \mathcal{D}} \left[\mathbb{E}_{q_\phi(\mathbf{z}|\mathbf{x})}[\log p_\theta(\mathbf{x}|\mathbf{z})] - D_{\text{KL}}(q_\phi(\mathbf{z}|\mathbf{x}) \| p(\mathbf{z})) \right].$$

- Il primo termine è il numero di bit necessari per ricodificare $\mathbf{x}$ dato $\mathbf{z}$ con $p_\theta(\mathbf{x}|\mathbf{z})$.
- Il secondo è l'*information gain*: quanti bit servono per trasmettere $\mathbf{z}$ usando $q_\phi(\mathbf{z}|\mathbf{x})$ rispetto al prior. È anche un costo di codifica.

Il VAE è quindi un codec a tasso minimo (compressione lossy con prior gaussiano).

20.6 Patologie e varianti

20.6.1 β -VAE

$$\mathcal{L}_\beta = \mathbb{E}_{q_\phi}[\log p_\theta(\mathbf{x}|\mathbf{z})] - \beta D_{\text{KL}}(q_\phi(\mathbf{z}|\mathbf{x}) \| p(\mathbf{z})), \quad \beta > 1.$$

Spinge i fattori latenti a essere più indipendenti (*disentanglement*). Vedi Cap. 25.

20.6.2 Posterior collapse

Se il decoder $p_\theta(\mathbf{x}|\mathbf{z})$ è troppo espressivo (es. autoregressive PixelCNN), può ricostruire $\mathbf{x}$ senza usare $\mathbf{z}$. Allora $q_\phi(\mathbf{z}|\mathbf{x}) \rightarrow p(\mathbf{z})$, il KL va a zero e la latente diventa inutile. Mitigazioni: KL annealing, free bits, $\beta < 1$ iniziale.

20.6.3 Confronto con AE classico e DAE

- L'AE classico non ha vincoli sul prior di $\mathbf{z}$: campionare random non ha senso.
- Il DAE apprende la score *solo* attorno ai training point. Il VAE definisce $p_\theta(\mathbf{x})$ globalmente.

Quando usare un VAE

Representation learning con prior strutturato

sì, è il *best in class* per spazi latenti regolari, interpolazioni, manipolazione semantica.

Generazione fotorealistica

no: campioni tipicamente sfocati. Usare diffusion o GAN.

Backbone modulare

sì, il VAE è il building block di latent diffusion (Stable Diffusion), CITRIS, e di buona parte dei modelli causali.

Anomaly detection

sì, l'ELBO è una misura naturale di tipicità.

Conditional generation

facile via CVAE.

Connessioni

- ELBO = stessa quantità di Variational Inference (Cap. 10); il reparametrization trick rende l'EM differenziabile e parametrizza encoder/decoder con reti neurali.
- Il VAE è il padre di diffusion (visto come VAE gerarchico con encoder fisso) e di Causal Representation Learning (iVAE, β -VAE, CITRIS).
- DAE $\rightarrow$ VAE: passare da score locale a densità globale.
- Reparametrization trick $\rightarrow$ flow matching: in entrambi si differenzia attraverso un campionamento.

Domande d'esame — VAE

1. Deriva l'ELBO partendo da $\log p_{\theta}(\mathbf{x}) = \log \int p(\mathbf{x}, \mathbf{z}) d\mathbf{z}$ usando la disuguaglianza di Jensen. Mostra esplicitamente come compaiono i termini di ricostruzione e KL.
2. Cos'è il reparametrization trick e perché è necessario? Cosa succederebbe se rimuovi questo trucco e provi a backpropagare attraverso il campionamento?
3. Deriva $D_{\text{KL}}(\mathcal{N}(\boldsymbol{\mu}, \text{diag}(\boldsymbol{\sigma}^2)) \parallel \mathcal{N}(0, \mathbf{I}))$ in forma chiusa.
4. Cos'è il *posterior collapse*? In quali condizioni si verifica e come si mitiga?
5. Confronta VAE con DAE: entrambi usano rumore gaussiano, dov'è la differenza?
6. Perché i campioni VAE sono tipicamente sfocati? È un problema dell'ELBO o della parametrizzazione gaussiana del decoder?
7. Come modifichereesti l'ELBO per condizionare la generazione su un'etichetta $\mathbf{y}$? Scrivi la loss CVAE.

Chapter 21

Generative Adversarial Networks

Risorse video — GDL25 — Generative Adversarial Networks

Videolezione del corso: registrazione della lezione GDL25 nella cartella *Recordings* del team del corso (v. Prefazione).

CS294-158 SP24 — L5: GANs (Pieter Abbeel, UC Berkeley, 2h)

Deriva D^* e la connessione con la Jensen-Shannon esattamente come le slide, e prosegue con WGAN, mode collapse e le varianti condizionali. È il riferimento completo del capitolo.

21.1 Idea: imparare a campionare

A differenza del VAE che apprende esplicitamente $p_\theta(\mathbf{x})$ via ELBO, una GAN *non* apprende mai la densità. Invece, apprende un *generator* $G_{\theta_G} : \mathcal{Z} \rightarrow \mathcal{X}$ (rete deterministica) tale che $G(\mathbf{z})$ con $\mathbf{z} \sim p(\mathbf{z}) = \mathcal{N}(0, \mathbf{I})$ produca campioni *indistinguibili* dai dati reali agli occhi di un *discriminatore* $D_{\theta_D} : \mathcal{X} \rightarrow [0, 1]$.

21.2 Formulazione minimax

$$\min_{\theta_G} \max_{\theta_D} V(D, G) = \mathbb{E}_{\mathbf{x} \sim p_{\text{data}}} [\log D(\mathbf{x})] + \mathbb{E}_{\mathbf{z} \sim p(\mathbf{z})} [\log(1 - D(G(\mathbf{z})))] \quad (21.1)$$

D massimizza la probabilità di etichettare correttamente reali e falsi; G minimizza la probabilità che D smascheri i suoi campioni.

21.2.1 Discriminatore ottimale

Teorema 21.1 (Goodfellow et al. 2014). Per G fisso, il discriminatore ottimale di (21.1) è

$$D^*(\mathbf{x}) = \frac{p_{\text{data}}(\mathbf{x})}{p_{\text{data}}(\mathbf{x}) + p_G(\mathbf{x})},$$

dove p_G è la densità implicita indotta da G .

Proof. Per ogni $\mathbf{x}$ fissato, il funzionale è $f(d) = p_{\text{data}}(\mathbf{x}) \log d + p_G(\mathbf{x}) \log(1 - d)$. Derivando rispetto a d e azzerando: $p_{\text{data}}/d - p_G/(1 - d) = 0 \Rightarrow d = p_{\text{data}}/(p_{\text{data}} + p_G)$. La seconda derivata è negativa, dunque è un massimo. $\square$

21.2.2 Connessione con Jensen-Shannon

Sostituendo D^* in V :

$$\begin{aligned} V(D^*, G) &= \mathbb{E}_{p_{\text{data}}} \left[\log \frac{p_{\text{data}}}{p_{\text{data}} + p_G} \right] + \mathbb{E}_{p_G} \left[\log \frac{p_G}{p_{\text{data}} + p_G} \right] \\ &= -\log 4 + D_{\text{KL}} \left(p_{\text{data}} \parallel \frac{p_{\text{data}} + p_G}{2} \right) + D_{\text{KL}} \left(p_G \parallel \frac{p_{\text{data}} + p_G}{2} \right) \\ &= -\log 4 + 2 \cdot \text{JS}(p_{\text{data}} \parallel p_G). \end{aligned}$$

Quindi il problema GAN minimizza la divergenza di Jensen-Shannon fra p_{data} e p_G . L'ottimo globale è $p_G = p_{\text{data}}$, raggiunto con $V = -\log 4$ e $D^* \equiv 1/2$ (indistinguibile).

Intuizione — la GAN non minimizza una loss, cerca un equilibrio

La differenza radicale con tutto il resto della Parte IV: VAE, flow e diffusion minimizzano una funzione obiettivo. Una GAN no. Il problema

$$\min_G \max_D V(D, G)$$

è la ricerca di un *punto di sella*, e questo cambia tutto ciò che si può dire sul training. Non esiste una quantità che scende monotonamente e che si possa guardare per capire se le cose vanno bene: la loss del generatore può salire mentre i campioni migliorano, e viceversa. Non c'è un criterio di stop naturale. La dinamica può oscillare o entrare in ciclo, perché due gradienti che si contrastano non convergono necessariamente — anche nel caso quadratico più semplice il gradient descent-ascent può orbitare intorno alla soluzione senza raggiungerla.

Perché comunque funziona (quando funziona): sostituendo D^* nella V si ottiene

$$V(D^*, G) = -\log 4 + 2 \text{JSD}(p_{\text{data}} \parallel p_G),$$

quindi *all'ottimo del discriminatore* il generatore sta effettivamente minimizzando una divergenza, e il minimo $-\log 4 \approx -1.386$ si raggiunge sse $p_G = p_{\text{data}}$. Il punto sottile è quel “all'ottimo”: nella pratica D non è mai ottimo, quindi il generatore minimizza un'approssimazione della JSD, e le garanzie teoriche non si applicano.

E il vantaggio, che va detto insieme ai difetti: proprio perché non ottimizza una verosimiglianza per pixel, la GAN non è spinta a mediare fra le alternative plausibili. È il motivo per cui i suoi campioni sono nitidi dove quelli del VAE sono sfocati — e anche il motivo per cui può ignorare interi modi della distribuzione senza pagare pegno, che è il mode collapse.

21.3 Problemi pratici

21.3.1 Gradient saturation

Quando il generatore è scarso, $D(G(\mathbf{z})) \rightarrow 0$ e $\log(1 - D(G(\mathbf{z}))) \rightarrow 0$ con gradiente quasi piatto. La fix (Goodfellow 2014) è il *non-saturating loss*: il generatore massimizza $\mathbb{E}_{\mathbf{z}}[\log D(G(\mathbf{z}))]$ invece di minimizzare $\mathbb{E}_{\mathbf{z}}[\log(1 - D(G(\mathbf{z})))]$. Stesso punto fisso, gradiente meno piatto.

21.3.2 Mode collapse

Il generatore impara a produrre pochi modi della distribuzione che *ingannano* D , ignorando il resto. Sintomo: campioni simili tra loro.

21.3.3 Instabilità del training

Il problema minimax è un punto sella, non un minimo. Senza heuristiche (batch normalization, label smoothing, spectral norm, two-timescale rules) il training oscilla.

Esempio svolto — gradient saturation: perché si cambia la loss del generatore

La formulazione originale chiede al generatore di minimizzare $\log(1 - D(G(\mathbf{z})))$. Vediamo che cosa succede all'inizio del training, quando D smaschera facilmente i campioni falsi e quindi $D(G(\mathbf{z})) \approx 0.01$.

Loss saturante. La derivata rispetto all'uscita del discriminatore è

$$\left| \frac{d}{dD} \log(1 - D) \right| = \frac{1}{1 - D} = \frac{1}{0.99} \approx 1.01.$$

Loss non saturante. Si massimizza invece $\log D(G(\mathbf{z}))$, cioè si minimizza $-\log D$:

$$\left| \frac{d}{dD} (-\log D) \right| = \frac{1}{D} = \frac{1}{0.01} = 100.$$

Un fattore ≈ 99 nel segnale che arriva al generatore, *nello stesso punto operativo*. E il rapporto peggiora ulteriormente man mano che D migliora: a $D = 10^{-3}$ diventa circa 1000.

Perché è esattamente il momento sbagliato per avere gradiente piccolo.

All'inizio del training il generatore produce spazzatura, quindi D vince facilmente: è proprio la fase in cui G avrebbe più bisogno di un segnale forte, ed è la fase in cui la loss originale gliene dà meno. Le due loss hanno lo stesso punto fisso ma dinamiche opposte dove conta.

Il collegamento con WGAN. Il problema di fondo è che la JSD fra due distribuzioni con supporti disgiunti è costante a $\log 2$, quindi il suo gradiente è nullo: se p_G e p_{data} vivono su varietà che non si intersecano — situazione normale in alta dimensione — non esiste una direzione di discesa. La distanza di Wasserstein non ha questo difetto perché misura *quanto* le distribuzioni sono lontane e non solo *se* si sovrappongono, e fornisce gradienti utili anche a supporti disgiunti. Il critico 1-Lipschitz (weight clipping, poi gradient penalty) serve a rendere calcolabile la forma duale di Kantorovich-Rubinstein.

Riscontro sperimentale: l'esercizio `12_gan_optimal_d` misura questa differenza su un caso concreto e ottiene norme del gradiente di 1.4×10^{-5} contro 1.6×10^1 — sei ordini di grandezza, ancora più netto del conto analitico qui sopra, perché nel caso reale D è molto più sicuro di 0.01.

21.4 WGAN

Wasserstein GAN (Arjovsky et al. 2017) sostituisce la JS divergence con la *Earth Mover's Distance* (Wasserstein-1):

$$W(p_{\text{data}}, p_G) = \inf_{\gamma \in \Pi(p_{\text{data}}, p_G)} \mathbb{E}_{(\mathbf{x}, \mathbf{y}) \sim \gamma} \|\mathbf{x} - \mathbf{y}\|.$$

Per la dualità di Kantorovich-Rubinstein:

$$W(p_{\text{data}}, p_G) = \sup_{\|f\|_L \leq 1} \mathbb{E}_{p_{\text{data}}}[f(\mathbf{x})] - \mathbb{E}_{p_G}[f(\mathbf{x})],$$

con f funzioni 1-Lipschitz. Il discriminatore (*critic*) impara f :

$$\min_G \max_{\|D\|_L \leq 1} \mathbb{E}_{p_{\text{data}}} [D(\mathbf{x})] - \mathbb{E}_{\mathbf{z}} [D(G(\mathbf{z}))].$$

Il vincolo Lipschitz si impone via *weight clipping* (originale, lento) o *gradient penalty* (WGAN-GP). Vantaggio: gradiente non saturante anche quando le distribuzioni sono disgiunte.

Trappola — un GAN non si può valutare con la verosimiglianza

Domanda tipica: “come confronti due GAN?”. La risposta sbagliata è “guardo la likelihood sul test set”. Un GAN è un modello **implicito**: definisce una procedura di campionamento, non una densità. Non esiste una $p_G(\mathbf{x})$ da valutare, né in forma chiusa né come bound — a differenza del VAE, che ha l’ELBO, e dei flow, che hanno la likelihood esatta.

Da qui nascono le metriche basate su campioni: *Inception Score* e soprattutto *Fréchet Inception Distance*, che confronta media e covarianza delle feature di una rete pre-addestrata sui campioni reali e generati.

Ed è bene conoscerne i difetti, perché è lì che la domanda scava: dipendono da una rete pre-addestrata su ImageNet, quindi non hanno senso fuori dal dominio delle immagini naturali; sono sensibili al numero di campioni; il FID assume gaussianità delle feature; e soprattutto **nessuna delle due rileva il mode collapse in modo affidabile**, perché un generatore che produce pochi modi ma di ottima qualità può ottenere un FID eccellente.

Quindi la valutazione dei GAN è un problema aperto, non una questione di scegliere la metrica giusta. È anche uno dei motivi per cui i diffusion model hanno preso il sopravvento: avendo un bound sulla verosimiglianza, offrono almeno un criterio principiato accanto alle metriche percettive.

21.5 Varianti notevoli

DCGAN convoluzioni transposte + batch norm; primo a generare immagini realistiche.

Progressive GAN genera prima a bassa risoluzione, poi aumenta progressivamente.

CycleGAN style transfer non appaiato: due generatori e due discriminatori con cycle consistency loss $\|F(G(\mathbf{x})) - \mathbf{x}\| + \|G(F(\mathbf{y})) - \mathbf{y}\|$.

Conditional GAN $G(\mathbf{z}, \mathbf{y})$, $D(\mathbf{x}, \mathbf{y})$: condizionamento su label/immagine/testo.

StyleGAN mapping network + AdaIN: controllo fine dello stile.

21.6 Adversarial Autoencoder (AAE)

L’AAE è il ponte VAE-GAN: l’encoder f_ϕ produce $\mathbf{z}$, e invece di forzare $D_{\text{KL}}(q_\phi(\mathbf{z}|\mathbf{x})\|p(\mathbf{z}))$ in forma chiusa (richiede p tractable), si addestra un discriminatore D a distinguere $\mathbf{z}$ campionati dal prior $p(\mathbf{z})$ (arbitrario!) da $\mathbf{z}$ prodotti da f_ϕ :

$$\mathcal{L}_{\text{AAE}} = \underbrace{\mathbb{E}_{q_\phi} [\log p_\theta(\mathbf{x}|\mathbf{z})]}_{\text{ricostruzione}} - \underbrace{\text{loss adversarial su } \mathbf{z}}_{\text{sostituisce la KL}}.$$

Vantaggio: $p(\mathbf{z})$ può essere arbitrario (mistura di gaussiane, distribuzione empirica, etc.) e fornire una copertura più liscia dello spazio latente.

Quando usare una GAN

Generazione di alta qualità ma rapida

sì, le GAN moderne (StyleGAN3, BigGAN) sono ancora competitive in qualità con generazione one-shot vs many-steps dei diffusion.

Inferenza, density estimation, anomaly detection via likelihood

no: la GAN non fornisce $p_\theta(x)$.

Conditional generation

sì (cGAN, CycleGAN, pix2pix).

Training stabile e veloce su dataset piccoli

no: training notoriamente instabile, richiede tuning.

Style transfer non appaiato

CycleGAN.

Connessioni

- Stesso problema del VAE (imparare p_{data}) con strategia opposta: implicita (sample-based) anziché esplicita (density-based).
- AAE = VAE con KL sostituita da loss adversarial.
- Diffusion ha sostituito le GAN come SOTA per qualità immagine (2020+), ma le GAN restano competitive per velocità di inferenza.
- La distanza di Wasserstein è anche la base di flow matching e optimal transport (Cap. 24).

Domande d'esame — GAN

1. Scrivi l'obiettivo minimax di Goodfellow e ricava il discriminatore ottimale D^* per G fisso.
2. Mostra che, sostituendo D^* nella loss, il problema è equivalente a minimizzare la divergenza di Jensen-Shannon fra p_{data} e p_G . Qual è il punto di ottimo globale?
3. Cos'è il *mode collapse*? Cosa lo causa intuitivamente?
4. Perché la WGAN funziona meglio della GAN classica quando le distribuzioni hanno supporti disgiunti?
5. Qual è l'enunciato del teorema di dualità Kantorovich-Rubinstein, e come si usa per derivare la loss WGAN?
6. In una GAN il generatore vede mai i dati reali? Spiega.
7. Differenza concettuale fra AAE e VAE: cosa cambia nel termine di regolarizzazione e quali vantaggi/svantaggi porta.
8. Confronta VAE e GAN su likelihood, qualità campioni, stabilità training, possibilità di inferenza.

Chapter 22

Normalizing Flows

Risorse video — GDL27–28 — Normalizing Flows

Videolezione del corso: registrazione delle lezioni GDL27 e GDL28 nella cartella *Recordings* del team del corso (v. Prefazione).

CS294-158 SP20 — L3: Flow Models (Pieter Abbeel, UC Berkeley, 2h)

Cambio di variabile, coupling layer, NICE/RealNVP/Glow e flussi autoregressivi, con l'accento sul vincolo che governa tutto: rendere calcolabile il determinante dello Jacobiano.

22.1 Idea: cambiare variabile in modo invertibile

Sia $\mathbf{z} \sim p_Z(\mathbf{z})$ una distribuzione base semplice (gaussiana). Cerchiamo una funzione *invertibile e differenziabile* $\mathbf{x} = f_\theta(\mathbf{z})$ tale che $\mathbf{x} \sim p_{\text{data}}$. A differenza del VAE (che usa un decoder *stocastico* e un encoder approssimato) e della GAN (che fa solo sampling), un flow *computa la likelihood esatta* via cambio di variabile.

22.2 Formula del cambio di variabile

Teorema 22.1 (Cambio di variabile multidimensionale). Se $f : \mathbb{R}^d \rightarrow \mathbb{R}^d$ è C^1 , invertibile, con Jacobiano $J_f(\mathbf{z}) = \partial f / \partial \mathbf{z}$ non singolare, allora la densità di $\mathbf{x} = f(\mathbf{z})$ è

$$p_X(\mathbf{x}) = p_Z(f^{-1}(\mathbf{x})) |\det J_{f^{-1}}(\mathbf{x})| = p_Z(\mathbf{z}) |\det J_f(\mathbf{z})|^{-1}.$$

In log-form:

$$\log p_X(\mathbf{x}) = \log p_Z(\mathbf{z}) - \log |\det J_f(\mathbf{z})|.$$

22.3 Composizione di flussi

Sia $f = f_K \circ f_{K-1} \circ \dots \circ f_1$, $\mathbf{z}_0 \sim p_Z$, $\mathbf{z}_k = f_k(\mathbf{z}_{k-1})$, $\mathbf{x} = \mathbf{z}_K$. Allora:

$$\log p_X(\mathbf{x}) = \log p_Z(\mathbf{z}_0) - \sum_{k=1}^K \log |\det J_{f_k}(\mathbf{z}_{k-1})|.$$

Requisiti per un layer f_k pratico:

1. invertibilità (per sampling: $\mathbf{x} \rightarrow \mathbf{z}$ via f^{-1} ; per likelihood: $\mathbf{x} \rightarrow \mathbf{z}$);

2. Jacobiano con determinante *efficientemente calcolabile* (idealmente $\mathcal{O}(d)$);
3. espressività sufficiente a comporre distribuzioni complesse.

Intuizione — il termine di volume è un vincolo contabile

La formula del cambio di variabile ha due pezzi e il secondo è quello che si dimentica:

$$\log p_X(\mathbf{x}) = \log p_Z(\mathbf{z}) - \log |\det J_f(\mathbf{z})|.$$

Il primo pezzo dice “quanto era probabile il punto di partenza”. Il secondo corregge per il fatto che la trasformazione ha *stirato o compresso* lo spazio in quel punto. Se f espande il volume, la stessa massa di probabilità si spalma su una regione più grande e la densità cala: da qui il segno meno.

Senza quel termine la “densità” non integrerebbe a 1, e il modello potrebbe barare aumentando la likelihood semplicemente comprimendo tutto verso un punto. È un vincolo di conservazione, non un dettaglio tecnico.

Da qui discende l'intero design dei flow: calcolare $\det J$ per una rete arbitraria costa $\mathcal{O}(d^3)$, proibitivo. Quindi tutte le architetture di flow sono risposte alla stessa domanda — *come costruire trasformazioni espressive il cui Jacobiano abbia determinante calcolabile in $\mathcal{O}(d)$* ? I coupling layer rispondono rendendolo triangolare; i flussi autoregressivi pure; i residual flow usano una stima stocastica; i continuous flow sostituiscono il determinante con una traccia lungo un'ODE.

Il prezzo, che va detto all'orale: l'invertibilità impone $\dim \mathbf{z} = \dim \mathbf{x}$. Un flow non può comprimere, quindi non ha uno spazio latente più piccolo dei dati — a differenza di VAE e latent diffusion. Ottiene in cambio la likelihood *esatta*, che né il VAE (che ha un bound) né la GAN (che non ha densità) possono dare.

22.4 Layer notevoli

22.4.1 Affine flow

$f(\mathbf{z}) = \mathbf{W}\mathbf{z} + \mathbf{b}$ con $\mathbf{W}$ invertibile. Determinante $|\det \mathbf{W}|$. Espressivo solo per casi speciali (triangolare, permutazione).

22.4.2 Coupling layer (NICE, RealNVP)

Si partiziona $\mathbf{z} = (\mathbf{z}_A, \mathbf{z}_B)$:

$$\begin{aligned} \mathbf{z}'_A &= \mathbf{z}_A, \\ \mathbf{z}'_B &= \mathbf{z}_B \odot \exp(\mathbf{s}_\theta(\mathbf{z}_A)) + \mathbf{t}_\theta(\mathbf{z}_A), \end{aligned}$$

con $\mathbf{s}, \mathbf{t}$ reti neurali arbitrarie (non devono essere invertibili!). Il Jacobiano è triangolare per blocchi:

$$J = \begin{pmatrix} \mathbf{I} & 0 \\ * & \text{diag}(\exp(\mathbf{s}_\theta(\mathbf{z}_A))) \end{pmatrix}, \quad \log |\det J| = \sum_j s_{\theta}(\mathbf{z}_A)_j.$$

Inversione: $\mathbf{z}_A = \mathbf{z}'_A$, $\mathbf{z}_B = (\mathbf{z}'_B - \mathbf{t}_\theta(\mathbf{z}'_A)) \odot \exp(-\mathbf{s}_\theta(\mathbf{z}'_A))$. Costo $\mathcal{O}(d)$ in entrambe le direzioni.

NICE ($\mathbf{s} \equiv 0$, solo shift): volume-preserving ($\det = 1$), poco espressivo da solo ma stackabile.

RealNVP affine: scale + shift. Si alterna la suddivisione (masking/squeezing) per coprire tutte le dimensioni.

GLOW introduce convoluzioni 1×1 invertibili $\mathbf{W}$ con decomposizione LU per efficienza del determinante.

Esempio svolto — cambio di variabile in 1D e un coupling layer a due dimensioni

Parte 1 — verifica numerica in una dimensione. Sia $z \sim \mathcal{N}(0, 1)$ e $x = f(z) = 2z + 1$. La formula dà

$$\log p_X(3) = \log p_Z(1) - \log |2| = \left(-\frac{1}{2}(1)^2 - \frac{1}{2} \log 2\pi\right) - \log 2 = -0.5 - 0.9189 - 0.6931 = -2.1121.$$

Verifichiamo per via diretta: $x \sim \mathcal{N}(1, 4)$, quindi

$$p_X(3) = \frac{1}{\sqrt{8\pi}} e^{-4/8} = 0.12099, \quad \log p_X(3) = -2.1121. \quad \checkmark$$

Il termine $-\log 2$ è esattamente la penalità per aver raddoppiato la scala: la densità si dimezza.

Parte 2 — perché il coupling layer è furbo. Un coupling layer divide il vettore in due blocchi e trasforma solo il secondo, condizionando sul primo:

$$\mathbf{x}_1 = \mathbf{z}_1, \quad \mathbf{x}_2 = \mathbf{z}_2 \odot \exp(s(\mathbf{z}_1)) + t(\mathbf{z}_1).$$

Con $\mathbf{z} = (0.5, -1)$, $s(0.5) = 0.4$ e $t(0.5) = 0.2$:

$$x_1 = 0.5, \quad x_2 = -1 \cdot e^{0.4} + 0.2 = -1.4918 + 0.2 = -1.2918.$$

Lo Jacobiano è

$$J = \begin{pmatrix} 1 & 0 \\ * & e^{s(\mathbf{z}_1)} \end{pmatrix} \Rightarrow \log |\det J| = s(\mathbf{z}_1) = 0.4.$$

I tre fatti che rendono questo layer il mattone standard. Primo: il determinante è la *somma* degli s , calcolabile in $\mathcal{O}(d)$, perché la matrice è triangolare — e questo vale qualunque cosa siano s e t , che possono essere reti profonde arbitrarie. Secondo: l'inversione è esplicita e altrettanto economica, $\mathbf{z}_2 = (\mathbf{x}_2 - t(\mathbf{x}_1)) \odot \exp(-s(\mathbf{x}_1))$, perché $\mathbf{x}_1 = \mathbf{z}_1$ è disponibile subito. Terzo: metà delle componenti resta ferma, quindi si alternano le partizioni fra un layer e il successivo — altrimenti quelle dimensioni non verrebbero mai trasformate.

Nota: s e t non devono essere invertibili. È il layer a esserlo, per costruzione. È questo che permette di usare reti arbitrariamente espressive dentro un flow.

22.4.3 Autoregressive flow (MAF/IAF)

Si fattorizza f in modo autoregressivo:

$$x_i = \mu_i(\mathbf{x}_{1:i-1}) + z_i \exp(s_i(\mathbf{x}_{1:i-1})),$$

con Jacobiano triangolare $\log |\det J| = \sum_i s_i$. **MAF** (Masked Autoregressive Flow): efficiente in *density evaluation* (parallelizzabile), lento in sampling (sequenziale). **IAF** (Inverse AF): opposto. Si usano in combinazione (Parallel Wavenet) per avere entrambi efficienti.

22.4.4 Residual flow

$f(\mathbf{z}) = \mathbf{z} + g_\theta(\mathbf{z})$ con g_θ contraente ($\|g\|_L < 1$). Per il teorema di Banach f è invertibile e l'inversa si calcola con iterazione di punto fisso $\mathbf{z}^{(i+1)} = \mathbf{z}' - g_\theta(\mathbf{z}^{(i)})$. Il log-determinante si

stima con la serie di Neumann

$$\log |\det J_f| = \log |\det(\mathbf{I} + J_g)| = \sum_{k=1}^{\infty} \frac{(-1)^{k+1}}{k} \text{tr}(J_g^k),$$

e tr si stima con *Hutchinson trace estimator*: $\text{tr}(\mathbf{A}) = \mathbb{E}_{\epsilon \sim \mathcal{N}(0, \mathbf{I})}[\epsilon^\top \mathbf{A} \epsilon]$.

22.4.5 Continuous Normalizing Flow (Neural ODE)

Limite continuo di un residual flow: definendo $\mathbf{z}(t)$ come la soluzione di

$$\frac{d\mathbf{z}(t)}{dt} = \theta_t(\mathbf{z}(t)), \quad \mathbf{z}(0) \sim p_Z, \quad \mathbf{z}(T) = \mathbf{x},$$

la formula di *instantaneous change of variables* (Chen et al. NeurIPS 2018) dà

$$\log p_X(\mathbf{x}) = \log p_Z(\mathbf{z}(0)) - \int_0^T \text{tr} \left(\frac{\partial \theta_t}{\partial \mathbf{z}} \right) dt.$$

Senza determinante! Solo la traccia del Jacobiano, e l'ODE si integra numericamente. Costo dipende dal solver (Euler, RK4, Dopri).

Trappola — likelihood esatta, e allora perché i flow non hanno vinto?

È la domanda cattiva su questo capitolo, e ha una risposta precisa: la likelihood esatta si paga in capacità espressiva per parametro.

Vincolo di invertibilità: ogni layer deve essere biiettivo. Questo esclude a priori tutto ciò che scarta informazione — niente ReLU non invertibili, niente pooling, niente riduzione di dimensione. Le architetture disponibili sono un sottoinsieme stretto di ciò che si userebbe altrimenti.

Vincolo di dimensione preservata: $\dim \mathbf{z} = \dim \mathbf{x}$ ovunque. Non esiste un collo di bottiglia, quindi non esiste una rappresentazione compressa, e ogni layer lavora sempre alla piena dimensione dei dati. Su immagini ad alta risoluzione questo costa memoria e calcolo in modo diretto.

Vincolo sul determinante: lo Jacobiano deve avere determinante calcolabile in tempo lineare, il che forza strutture triangolari (coupling, autoregressive) e quindi trasformazioni che a ogni layer toccano solo metà delle componenti. Servono molti layer per ottenere l'espressività che un VAE o un diffusion ottengono con pochi.

Il risultato pratico: a parità di budget computazionale, la qualità dei campioni resta indietro. La diffusion ha fatto la scelta opposta — rinunciare alla likelihood esatta accontentandosi di un bound — e ha guadagnato la libertà architetturale che i flow non hanno.

La coda della storia va però raccontata: il probability flow ODE e il flow matching recuperano l'idea del trasporto continuo *senza* il vincolo di invertibilità layer per layer, perché l'invertibilità viene dall'ODE e non dall'architettura. In quel senso i flow non hanno perso, si sono trasformati.

22.5 Dequantizzazione

I dati discreti (pixel in $\{0, \dots, 255\}$) violano l'assunzione di densità continua. Si aggiunge rumore uniforme $\mathbf{u} \in [0, 1]^d$: $\mathbf{x}' = \mathbf{x} + \mathbf{u}$. La likelihood sul continuo è un *lower bound* di quella sul discreto. Varianti più sofisticate: *variational dequantization* (Flow++).

22.6 Universalità

Sotto ipotesi blande (Lipschitz, densità con supporto pieno), un flow sufficientemente profondo è un *universal density approximator*: può approssimare qualunque distribuzione target con accuratezza arbitraria.

Quando usare un Normalizing Flow

Density estimation esatta

sì, è l'unico modello generativo profondo che fornisce $p_\theta(\mathbf{x})$ esatta (no ELBO, no JS implicita).

Anomaly detection con likelihood

sì, score = $\log p_\theta(\mathbf{x})$ ben definito.

Generazione di alta qualità su immagini complesse

meno competitivo di diffusion/GAN moderne, ma utilizzato come testa di latent diffusion / preprocessing.

Modellare dati strutturati (audio, fisica, scienza)

sì, ad es. Glow per audio (Parallel Wavenet).

Conditional generation

facile (condizionare f_θ).

Limite: il flow deve preservare dimensioni

non riduce, non comprime.

Connessioni

- Cambio di variabile $\leftrightarrow$ VAE: il VAE è un cambio di variabile *stocastico* con dimensioni diverse; il flow è deterministico e dim-preserving.
- Residual flow $\rightarrow$ ODE neurale $\rightarrow$ flow matching: il limite continuo apre la strada al Cap. 24.
- Autoregressive flow $\leftrightarrow$ PixelRNN/PixelCNN: stessa fattorizzazione $p(\mathbf{x}) = \prod_i p(x_i | \mathbf{x}_{<i})$.
- Flow $\leftrightarrow$ diffusion: entrambi possono essere visti come trasporti, ma il flow è deterministico, la diffusion stocastica.

Domande d'esame — Normalizing Flows

1. Scrivi e dimostra la formula del cambio di variabile multidimensionale. Spiega perché compare $|\det J|^{-1}$.
2. Scrivi la log-likelihood di un flow composto da K layer. Quali sono i requisiti pratici sui layer?
3. Spiega come funziona un coupling layer (RealNVP). Perché il Jacobiano è triangolare? Quanto costa l'inversione?
4. Differenza fra MAF e IAF: in quali condizioni preferiresti uno o l'altro?
5. Cos'è un Continuous Normalizing Flow? Mostra la formula per il log-determinante

(instantaneous change of variables). Vantaggi e costi.

6. Perché serve la dequantizzazione su immagini? Cosa succede se non la fai?
7. Confronta normalizing flow, VAE e diffusion sulla capacità di calcolare $p_{\theta}(\mathbf{x})$ esatta.
8. Cos'è la stima Hutchinson della traccia, e perché è utile per i residual flows?

Chapter 23

Diffusion Models

Risorse video — GDL29 — Diffusion Models

Videolezione del corso: registrazione della lezione GDL29 nella cartella *Recordings* del team del corso (v. Prefazione).

CS294-158 SP24 — L6: Diffusion Models (*Pieter Abbeel, UC Berkeley, 2h*)

Forward e reverse process, derivazione dell'ELBO, parametrizzazione ϵ , DDIM e guidance: copre l'intero capitolo nello stesso ordine.

Autoencoders — Deep Learning Animated (*Deepia, 20m*)

Da rivedere per il collegamento concettuale: il forward process è un denoising autoencoder gerarchico, e l'intuizione visiva del DAE aiuta a leggere la diffusion in quella chiave.

23.1 Idea: corrompere e ricostruire

Un modello di diffusione (DDPM, Ho et al. 2020) definisce un processo stocastico *forward* fisso che gradualmente corrompe i dati in rumore gaussiano, e impara un processo *reverse* parametrico che denoisi step-by-step. Il sampling consiste nel partire da rumore puro e iterare il reverse.

23.2 Forward process

Dato $\mathbf{x}_0 \sim p_{\text{data}}$ e una schedule $\{\beta_t\}_{t=1}^T \subset (0, 1)$:

$$q(\mathbf{z}_t | \mathbf{z}_{t-1}) = \mathcal{N}(\sqrt{1 - \beta_t} \mathbf{z}_{t-1}, \beta_t \mathbf{I}), \quad \mathbf{z}_0 = \mathbf{x}_0. \quad (23.1)$$

La catena è markoviana. Definendo $\alpha_t = 1 - \beta_t$ e $\bar{\alpha}_t = \prod_{s=1}^t \alpha_s$, si ottiene la *forma chiusa* per t qualsiasi:

$$q(\mathbf{z}_t | \mathbf{x}_0) = \mathcal{N}(\sqrt{\bar{\alpha}_t} \mathbf{x}_0, (1 - \bar{\alpha}_t) \mathbf{I}). \quad (23.2)$$

Derivazione. Per induzione su t . Caso base $t = 1$: ovvio da (23.1). Passo: se $\mathbf{z}_{t-1} = \sqrt{\bar{\alpha}_{t-1}} \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_{t-1}} \boldsymbol{\epsilon}'$, allora $\mathbf{z}_t = \sqrt{\alpha_t} \mathbf{z}_{t-1} + \sqrt{\beta_t} \boldsymbol{\epsilon}'' = \sqrt{\alpha_t \bar{\alpha}_{t-1}} \mathbf{x}_0 + \sqrt{\alpha_t (1 - \bar{\alpha}_{t-1})} \boldsymbol{\epsilon}' + \sqrt{1 - \alpha_t} \boldsymbol{\epsilon}''$. Le ultime due gaussiane si combinano: $\sqrt{\alpha_t (1 - \bar{\alpha}_{t-1})} \boldsymbol{\epsilon}' + \sqrt{1 - \alpha_t} \boldsymbol{\epsilon}'' = \sqrt{1 - \bar{\alpha}_t} \boldsymbol{\epsilon}$. $\square$

In particolare per T grande con schedule appropriata, $\bar{\alpha}_T \approx 0$ e $\mathbf{z}_T \sim \mathcal{N}(0, \mathbf{I})$.

Intuizione — molti problemi facili invece di uno difficile

Il salto concettuale della diffusion sta tutto qui: generare un'immagine da zero è difficile, ma *togliere un po' di rumore* da un'immagine quasi pulita è facile. Se si decompone la generazione in mille passi di denoising locale, ogni singolo passo diventa un problema di regressione banale, e la difficoltà si distribuisce lungo la catena invece di concentrarsi in un salto solo.

Da qui il resto segue in modo quasi obbligato:

- il processo forward deve essere *semplice e fisso*, perché non deve essere imparato: aggiungere gaussiano un po' alla volta è la scelta minima;
- con incrementi piccoli, la vera reverse $q(\mathbf{z}_{t-1}|\mathbf{z}_t)$ è approssimativamente gaussiana (Feller), quindi è lecito parametrizzarla come gaussiana;
- la distribuzione finale $q(\mathbf{z}_T)$ deve essere semplice e nota, altrimenti non si saprebbe da dove partire a campionare.

Il punto sottile che l'esame cerca: la reverse vera $q(\mathbf{z}_{t-1}|\mathbf{z}_t)$ è intrattabile, perché denoising è *ambiguo* — molti dati puliti possono aver prodotto lo stesso $\mathbf{z}_t$, e sapere quale richiederebbe conoscere p_{data} . Ma se si *condiziona anche su $\mathbf{x}_0$* , allora $q(\mathbf{z}_{t-1}|\mathbf{z}_t, \mathbf{x}_0)$ è gaussiana in forma chiusa. È esattamente su questo che si costruisce l'ELBO: in training $\mathbf{x}_0$ ce l'abbiamo, quindi possiamo confrontare due gaussiane; a inferenza non ce l'abbiamo, e per questo serve la rete.

Il prezzo, da dire sempre: la stessa decomposizione che rende facile ogni passo rende costosa la generazione, perché i passi vanno fatti tutti. È il trade-off strutturale della diffusion, e DDIM, distillazione e flow matching sono tre modi diversi di attaccarlo.

23.3 Reverse process

Si parametrizza la reverse con una gaussiana:

$$p_\theta(\mathbf{z}_{t-1} | \mathbf{z}_t) = \mathcal{N}(\boldsymbol{\mu}_\theta(\mathbf{z}_t, t), \sigma_t^2 \mathbf{I}).$$

La giustificazione (Feller 1949): per β_t piccolo, la *vera* reverse $q(\mathbf{z}_{t-1}|\mathbf{z}_t)$ è approssimativamente gaussiana.

23.4 Training: ELBO e parametrizzazione del rumore

23.4.1 ELBO per la catena di diffusione

$$\log p_\theta(\mathbf{x}_0) \geq \mathbb{E}_q \left[\log \frac{p_\theta(\mathbf{x}_0, \mathbf{z}_{1:T})}{q(\mathbf{z}_{1:T}|\mathbf{x}_0)} \right].$$

Espandendo e usando l'identità $q(\mathbf{z}_t|\mathbf{z}_{t-1}) = q(\mathbf{z}_t|\mathbf{z}_{t-1}, \mathbf{x}_0)$:

$$-\text{ELBO} = \mathbb{E}_q \left[\underbrace{D_{\text{KL}}(q(\mathbf{z}_T|\mathbf{x}_0)||p(\mathbf{z}_T))}_{L_T \text{ (costante)}} + \sum_{t=2}^T \underbrace{D_{\text{KL}}(q(\mathbf{z}_{t-1}|\mathbf{z}_t, \mathbf{x}_0)||p_\theta(\mathbf{z}_{t-1}|\mathbf{z}_t))}_{L_{t-1}} - \underbrace{\log p_\theta(\mathbf{x}_0|\mathbf{z}_1)}_{L_0} \right].$$

La *posterior reverse condizionata su $\mathbf{x}_0$* è gaussiana in forma chiusa:

$$q(\mathbf{z}_{t-1}|\mathbf{z}_t, \mathbf{x}_0) = \mathcal{N}(\tilde{\boldsymbol{\mu}}_t(\mathbf{z}_t, \mathbf{x}_0), \tilde{\beta}_t \mathbf{I}),$$

con

$$\tilde{\boldsymbol{\mu}}_t = \frac{\sqrt{\bar{\alpha}_{t-1}}\beta_t}{1 - \bar{\alpha}_t} \mathbf{x}_0 + \frac{\sqrt{\alpha_t}(1 - \bar{\alpha}_{t-1})}{1 - \bar{\alpha}_t} \mathbf{z}_t, \quad \tilde{\beta}_t = \frac{1 - \bar{\alpha}_{t-1}}{1 - \bar{\alpha}_t} \beta_t.$$

23.4.2 Parametrizzazione ϵ

Dalla diffusion kernel (23.2), $\mathbf{x}_0 = (\mathbf{z}_t - \sqrt{1 - \bar{\alpha}_t}\epsilon)/\sqrt{\bar{\alpha}_t}$. Si riscrive $\tilde{\mu}_t$ in funzione di ϵ e si parametrizza

$$\mu_\theta(\mathbf{z}_t, t) = \frac{1}{\sqrt{\alpha_t}} \left(\mathbf{z}_t - \frac{\beta_t}{\sqrt{1 - \bar{\alpha}_t}} \epsilon_\theta(\mathbf{z}_t, t) \right).$$

Il termine L_{t-1} diventa allora $\propto \|\epsilon - \epsilon_\theta(\mathbf{z}_t, t)\|^2$. Ignorando i coefficienti (DDPM *simplified loss*):

$$\mathcal{L}_{\text{simple}}(\theta) = \mathbb{E}_{t, \mathbf{x}_0, \epsilon} \left[\|\epsilon - \epsilon_\theta(\sqrt{\bar{\alpha}_t} \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon, t)\|^2 \right]. \quad (23.3)$$

È una loss di *denoising score matching* a T livelli di rumore.

Esempio svolto — che cosa fa davvero la schedule: $\bar{\alpha}_t$ e il rapporto segnale-rumore

Prendiamo una schedule giocattolo con $T = 4$ e $\beta = (0.1, 0.2, 0.3, 0.4)$. Allora $\alpha_t = 1 - \beta_t$ e $\bar{\alpha}_t = \prod_{s \leq t} \alpha_s$:

t	1	2	3	4
α_t	0.9	0.8	0.7	0.6
$\bar{\alpha}_t$	0.900	0.720	0.504	0.302
$\sqrt{\bar{\alpha}_t}$ (segnale)	0.949	0.849	0.710	0.550
$\sqrt{1 - \bar{\alpha}_t}$ (rumore)	0.316	0.529	0.704	0.835

A $t = 4$ si ha $\mathbf{z}_4 = 0.550 \mathbf{x}_0 + 0.835 \epsilon$ e $\text{SNR}_4 = \bar{\alpha}_4 / (1 - \bar{\alpha}_4) = 0.434$: il rumore ha già superato il segnale.

Con la schedule vera. DDPM usa $T = 1000$ e β lineare da 10^{-4} a 0.02:

t	1	100	500	1000
$\bar{\alpha}_t$	0.9999	0.897	0.0786	4.0×10^{-5}
SNR_t	10^4	8.71	0.085	4.0×10^{-5}

A $t = 1000$ resta lo 0.6% del segnale originale: $\mathbf{z}_T$ è indistinguibile da $\mathcal{N}(0, \mathbf{I})$, che è la condizione perché il sampling possa partire dal rumore puro.

Perché la schedule è un iperparametro serio. Se il rumore cresce troppo in fretta, i primi passi di reverse devono ricostruire troppo e il problema torna difficile. Se cresce troppo lentamente, $\bar{\alpha}_T$ non arriva abbastanza vicino a zero e la distribuzione terminale non è davvero gaussiana, quindi il sampling parte dal punto sbagliato. L'idea guida è che i primi passi debbano *corrompere i dettagli fini preservando la struttura semantica*, e gli ultimi portare tutto verso il rumore isotropo. La schedule coseno di Nichol & Dhariwal nasce proprio dall'osservazione che quella lineare distrugge il segnale troppo presto.

Una nota sulle varianze della reverse: nella pratica si pone spesso $\sigma_t^2 = \beta_t$, oppure $\sigma_t^2 = \tilde{\beta}_t = \frac{1 - \bar{\alpha}_t - 1}{1 - \bar{\alpha}_t} \beta_t$. Sono i due estremi teorici (entropia massima e minima per la reverse); alcune varianti la fanno apprendere alla rete.

23.5 Sampling: DDPM e DDIM

23.5.1 DDPM (stocastico)

$$z_T \sim \mathcal{N}(0, \mathbf{I}),$$

$$z_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left(z_t - \frac{\beta_t}{\sqrt{1 - \bar{\alpha}_t}} \epsilon_\theta(z_t, t) \right) + \sigma_t \mathbf{w}, \quad \mathbf{w} \sim \mathcal{N}(0, \mathbf{I}).$$

T tipicamente 1000. Lento.

23.5.2 DDIM (deterministico)

Song et al. 2021. Si reinterpreta la dinamica come ODE e si campiona deterministicamente con meno step (anche 50). DDIM è un caso particolare con $\sigma_t = 0$: stesso modello ϵ_θ , sampling diverso.

23.6 Architettura: U-Net

Per immagini, ϵ_θ è una U-Net con:

- ResNet blocks down/up sampling;
- Self-attention nei layer di media risoluzione;
- *Time embedding*: posizionale sinusoidale di t iniettato in tutti i ResBlock;
- (Opzionale) condizionamento testuale via cross-attention.

23.7 Conditional generation

23.7.1 Classifier guidance

Si addestra un classificatore $p_\phi(\mathbf{y}|\mathbf{z}_t)$ su immagini rumorose. Durante il sampling si modifica la mean:

$$\boldsymbol{\mu}'_\theta(\mathbf{z}_t, t, \mathbf{y}) = \boldsymbol{\mu}_\theta(\mathbf{z}_t, t) + \gamma \sigma_t^2 \nabla_{\mathbf{z}_t} \log p_\phi(\mathbf{y}|\mathbf{z}_t).$$

γ è il *guidance scale*. Mappa Bayesiana: $\nabla \log p(\mathbf{z}_t|\mathbf{y}) = \nabla \log p(\mathbf{z}_t) + \nabla \log p(\mathbf{y}|\mathbf{z}_t)$.

23.7.2 Classifier-free guidance

Si addestra un singolo modello $\epsilon_\theta(\mathbf{z}_t, t, \mathbf{y})$ con dropout della condizione (a volte $\mathbf{y} = \emptyset$). A inference:

$$\hat{\epsilon}(\mathbf{z}_t, t, \mathbf{y}) = (1 + \gamma) \epsilon_\theta(\mathbf{z}_t, t, \mathbf{y}) - \gamma \epsilon_\theta(\mathbf{z}_t, t, \emptyset).$$

Nessun classificatore separato. È la base di Stable Diffusion, Imagen, DALL-E 2.

Osservazione 23.1 (Due scritture equivalenti della classifier-free guidance). La formula usata sopra,

$$\hat{\epsilon} = (1 + \gamma) \epsilon_\theta(\mathbf{z}_t, t, \mathbf{y}) - \gamma \epsilon_\theta(\mathbf{z}_t, t, \emptyset),$$

si trova spesso anche nella forma a combinazione affine

$$\hat{\epsilon} = (1 - w) \epsilon_\theta(\mathbf{z}_t, t, \emptyset) + w \epsilon_\theta(\mathbf{z}_t, t, \mathbf{y}),$$

che è la scrittura adottata negli handout del corso. Sono la stessa espressione con $w = 1 + \gamma$: in particolare $w = 1$ (cioè $\gamma = 0$) è il caso puramente condizionale senza guidance, e $w > 1$

amplifica la direzione condizionale. Vale la pena riconoscere entrambe le forme, perché all'orale può comparire l'una o l'altra.

Il trade-off da nominare: aumentando w i campioni aderiscono di più al prompt ma diventano meno diversi, perché il sampling viene spinto verso i pochi modi più compatibili con la condizione. Guidance alta migliora la fedeltà e riduce la varietà — è il motivo per cui i sistemi text-to-image espongono quel parametro all'utente.

Osservazione 23.2 (Come si inietta la condizione). Il condizionamento entra nella U-Net in modi diversi a seconda della natura del segnale: condizioni scalari o vettoriali (classe, timestep) tramite *embedding sommati* dentro i blocchi residui; condizioni-immagine (inpainting, ControlNet, super-risoluzione) tramite *concatenazione sui canali*; condizioni testuali tramite *cross-attention* sugli embedding del testo, che è il meccanismo di Stable Diffusion e Imagen. È la stessa cross-attention della Parte III (capitolo su encoder-decoder e cross-attention): query dalla mappa di feature dell'immagine, key e value dagli embedding testuali.

23.8 Latent diffusion

Eseguire diffusion direttamente in pixel space è costoso. *Latent diffusion* (Rombach et al. 2022, Stable Diffusion): pre-addestra un VAE che comprime $\mathbf{x}$ (es. 512×512) in latente $\mathbf{z}$ (es. 64×64), poi addestra diffusion in latente. Sampling: $\mathbf{z}_T \rightarrow \mathbf{z}_0 \rightarrow \mathbf{x} = \text{decoder}(\mathbf{z}_0)$.

Quando usare un Diffusion Model

Generazione di immagini/audio di altissima qualità

SOTA dal 2021. Stable Diffusion, DALL-E 2, Imagen.

Conditional generation text-to-image

sì, con classifier-free guidance.

Density estimation esatta

no (ma bound ELBO).

Sampling velocissimo

no: 50-1000 step. Flow matching e DDIM mitigano.

Editing controllato

sì, via inpainting / SDEdit / ControlNet.

Spazio latente strutturato

meno del VAE, ma latent diffusion offre un compromesso.

Connessioni

- Forward process = DAE gerarchico con T livelli di rumore.
- Training objective = denoising score matching (Cap. 24).
- Diffusion = VAE gerarchico con encoder *fisso* (non parametrizzato) e T latenti della stessa dimensione di $\mathbf{x}$.
- Reverse SDE $\rightarrow$ probability flow ODE $\rightarrow$ flow matching: connessione completa in Cap. 24.
- Latent diffusion = VAE encoder + diffusion in latente.

Domande d'esame — Diffusion

1. Scrivi il forward process $q(\mathbf{z}_t|\mathbf{z}_{t-1})$ di DDPM. Deriva la forma chiusa $q(\mathbf{z}_t|\mathbf{x}_0)$ (diffusion kernel).
2. Perché possiamo assumere che la reverse $p_\theta(\mathbf{z}_{t-1}|\mathbf{z}_t)$ sia gaussiana?
3. Deriva l'ELBO per la catena di diffusione. Quali sono i tre tipi di termini?
4. Mostra come si arriva alla loss simplified $\mathbb{E}\|\epsilon - \epsilon_\theta(\mathbf{z}_t, t)\|^2$ a partire dall'ELBO. Cosa si trascura?
5. Differenza fra classifier guidance e classifier-free guidance. Vantaggi e svantaggi.
6. Perché DDIM è più veloce di DDPM senza riaddestrare il modello?
7. Cos'è il latent diffusion e perché è importante computazionalmente?
8. Confronta VAE, GAN, Flow e Diffusion su: likelihood, qualità campioni, stabilità, velocità.
9. In che senso un diffusion model è un denoising autoencoder gerarchico?

Chapter 24

Score Matching e Flow Matching

Risorse video — GDL31 — Score matching e Flow matching

Videolezione del corso: registrazione della lezione GDL31 nella cartella *Recordings* del team del corso (v. Prefazione).

MIT 6.S184 — Lecture 03: Training Flow and Diffusion Models (*Peter Holderrieth, 1h20m*)

Il corso MIT 2025 costruisce flow matching e diffusion nella stessa cornice, che è esattamente il taglio di questa lezione. La lezione 3 è quella sul training, cioè su CFM e denoising score matching.

CS294-158 SP24 — L6: Diffusion Models (*Pieter Abbeel, 2h*)

Da rivedere per la parte SDE: la connessione fra reverse SDE e probability flow ODE è trattata con più calma che nelle slide.

24.1 Score function e diffusione

Per ogni distribuzione $p(\mathbf{x})$ definiamo la *score function*

$$\mathbf{s}(\mathbf{x}) = \nabla_{\mathbf{x}} \log p(\mathbf{x}).$$

È un campo vettoriale che, in ogni punto, indica la direzione di massimo incremento della log-densità. Conoscere $\mathbf{s}$ permette il sampling via *Langevin dynamics*:

$$\mathbf{x}_{k+1} = \mathbf{x}_k + \frac{\eta}{2} \mathbf{s}(\mathbf{x}_k) + \sqrt{\eta} \boldsymbol{\epsilon}_k, \quad \boldsymbol{\epsilon}_k \sim \mathcal{N}(0, \mathbf{I}), \quad (24.1)$$

che ha $p(\mathbf{x})$ come distribuzione stazionaria per $\eta \rightarrow 0$.

24.1.1 Vantaggio della score

Per una densità in forma esponenziale $p(\mathbf{x}) = \exp(-f_{\theta}(\mathbf{x}))/Z_{\theta}$, la *partition function* Z_{θ} è intrattabile in alta dimensione. Ma:

$$\nabla_{\mathbf{x}} \log p(\mathbf{x}) = -\nabla_{\mathbf{x}} f_{\theta}(\mathbf{x}) - \nabla_{\mathbf{x}} \log Z_{\theta} = -\nabla_{\mathbf{x}} f_{\theta}(\mathbf{x}),$$

poiché Z_{θ} non dipende da $\mathbf{x}$. La score è *indipendente dalla normalizzazione*: training molto più facile per modelli energy-based.

24.2 Formula di Tweedie: il ponte fra denoising e score

Il legame fra “predire il rumore” e “imparare la score” non è una coincidenza notazionale: si ricava in tre righe dalla formula di Tweedie, ed è il primo dei due risultati di unificazione di questo capitolo.

Teorema 24.1 (Tweedie). Sia $\mathbf{Z}_t \mid \mathbf{Z}_0 = \mathbf{z}_0 \sim \mathcal{N}(\sqrt{\bar{\alpha}_t} \mathbf{z}_0, (1 - \bar{\alpha}_t) \mathbf{I})$ e sia p_t la marginale di $\mathbf{Z}_t$. Allora

$$\mathbb{E}[\sqrt{\bar{\alpha}_t} \mathbf{Z}_0 \mid \mathbf{Z}_t = \mathbf{z}_t] = \mathbf{z}_t + (1 - \bar{\alpha}_t) \nabla_{\mathbf{z}_t} \log p_t(\mathbf{z}_t).$$

Risolvendo per la stima del dato pulito:

$$\hat{\mathbf{z}}_0 = \frac{\mathbf{z}_t + (1 - \bar{\alpha}_t) \nabla_{\mathbf{z}_t} \log p_t(\mathbf{z}_t)}{\sqrt{\bar{\alpha}_t}}.$$

Il denoising ottimo, cioè la media a posteriori del dato pulito, è governato interamente dalla score.

Esempio svolto — da “predici il rumore” a “impara la score”, in tre righe

Dal kernel di diffusione sappiamo che $\mathbf{z}_t = \sqrt{\bar{\alpha}_t} \mathbf{z}_0 + \sqrt{1 - \bar{\alpha}_t} \boldsymbol{\epsilon}_t$, da cui

$$\mathbf{z}_0 = \frac{\mathbf{z}_t - \sqrt{1 - \bar{\alpha}_t} \boldsymbol{\epsilon}_t}{\sqrt{\bar{\alpha}_t}}.$$

La formula di Tweedie dà invece

$$\mathbf{z}_0 = \frac{\mathbf{z}_t + (1 - \bar{\alpha}_t) \nabla_{\mathbf{z}_t} \log p_t(\mathbf{z}_t)}{\sqrt{\bar{\alpha}_t}}.$$

Uguagliando le due espressioni e moltiplicando per $\sqrt{\bar{\alpha}_t}$:

$$\mathbf{z}_t - \sqrt{1 - \bar{\alpha}_t} \boldsymbol{\epsilon}_t = \mathbf{z}_t + (1 - \bar{\alpha}_t) \nabla_{\mathbf{z}_t} \log p_t(\mathbf{z}_t),$$

e semplificando $\mathbf{z}_t$ da entrambi i lati si ottiene

$$\nabla_{\mathbf{z}_t} \log p_t(\mathbf{z}_t) = -\frac{\boldsymbol{\epsilon}_t}{\sqrt{1 - \bar{\alpha}_t}}.$$

Come leggerla. Il rumore aggiunto è proporzionale alla score cambiata di segno. Una rete addestrata a predire $\boldsymbol{\epsilon}$ sta quindi imparando, a meno di un fattore che dipende solo da t , il campo $\nabla \log p_t$. Le due formulazioni non sono equivalenti “moralmente”: sono la stessa rete con una riscalatura dell’uscita.

$$\boldsymbol{\epsilon}_\theta(\mathbf{z}_t, t) = -\sqrt{1 - \bar{\alpha}_t} \mathbf{s}_\theta(\mathbf{z}_t, t).$$

Verifica del segno con un numero. Con $\bar{\alpha}_t = 0.5$ si ha $\mathbf{s} = -\boldsymbol{\epsilon}/\sqrt{0.5} = -1.414 \boldsymbol{\epsilon}$. Se il rumore aggiunto puntava “verso l’alto”, la score punta verso il basso: cioè verso la regione di densità più alta, che è da dove il campione era stato allontanato. Il segno è quello giusto — la score punta sempre *verso i dati*.

Perché conta all’orale: questa è la risposta alla domanda “in che senso i diffusion model sono modelli score-based?”. La risposta non è “sono simili”, è “sono la stessa cosa, e la formula di Tweedie lo dimostra”.

24.3 Score matching

24.3.1 Score matching esplicito

$$\mathcal{L}_{\text{SM}}(\theta) = \mathbb{E}_{\mathbf{x} \sim p_{\text{data}}} \left[\|\mathbf{s}_{\theta}(\mathbf{x}) - \nabla_{\mathbf{x}} \log p_{\text{data}}(\mathbf{x})\|^2 \right].$$

Inattuabile: non conosciamo p_{data} .

24.3.2 Implicit Score Matching (Hyvärinen 2005)

Per integrazione per parti:

$$\mathcal{L}_{\text{ISM}} = \mathbb{E}_{p_{\text{data}}} \left[\frac{1}{2} \|\mathbf{s}_{\theta}(\mathbf{x})\|^2 + \text{tr}(\nabla_{\mathbf{x}} \mathbf{s}_{\theta}(\mathbf{x})) \right].$$

Costo: $\text{tr}(\nabla \mathbf{s}_{\theta})$ richiede $\mathcal{O}(d^2)$ in dimensione alta.

24.3.3 Denoising Score Matching (Vincent 2011)

Si perturba $\mathbf{x}$ con rumore gaussiano $\tilde{\mathbf{x}} = \mathbf{x} + \sigma \boldsymbol{\epsilon}$:

$$\mathcal{L}_{\text{DSM}}(\theta) = \mathbb{E}_{p(\mathbf{x}) p(\tilde{\mathbf{x}}|\mathbf{x})} \left[\|\mathbf{s}_{\theta}(\tilde{\mathbf{x}}) - \nabla_{\tilde{\mathbf{x}}} \log p(\tilde{\mathbf{x}}|\mathbf{x})\|^2 \right].$$

Per $p(\tilde{\mathbf{x}}|\mathbf{x}) = \mathcal{N}(\mathbf{x}, \sigma^2 \mathbf{I})$ si ha $\nabla_{\tilde{\mathbf{x}}} \log p(\tilde{\mathbf{x}}|\mathbf{x}) = -(\tilde{\mathbf{x}} - \mathbf{x})/\sigma^2 = -\boldsymbol{\epsilon}/\sigma$. Quindi

$$\mathcal{L}_{\text{DSM}} = \mathbb{E} \left[\left\| \mathbf{s}_{\theta}(\tilde{\mathbf{x}}) + \frac{\boldsymbol{\epsilon}}{\sigma} \right\|^2 \right].$$

È esattamente la loss DDPM riparametrizzata: $\boldsymbol{\epsilon}_{\theta}(\mathbf{z}_t, t) = -\sqrt{1 - \bar{\alpha}_t} \mathbf{s}_{\theta}(\mathbf{z}_t, t)$.

24.4 Score-based SDE

Song & Ermon (NeurIPS 2019, 2021): unificano diffusion e score matching tramite SDE.

24.4.1 Forward SDE

$$d\mathbf{x} = \mathbf{f}(\mathbf{x}, t) dt + g(t) d\mathbf{w},$$

con $\mathbf{w}$ Brownian motion. Casi notevoli:

- *Variance Exploding* (NCSN): $\mathbf{f} = 0$, $g(t)$ crescente.
- *Variance Preserving* (DDPM): $\mathbf{f}(\mathbf{x}, t) = -\frac{1}{2}\beta(t)\mathbf{x}$, $g(t) = \sqrt{\beta(t)}$.

24.4.2 Reverse SDE (Anderson 1982)

Per ogni forward SDE esiste una reverse:

$$d\mathbf{x} = [\mathbf{f}(\mathbf{x}, t) - g^2(t) \nabla_{\mathbf{x}} \log p_t(\mathbf{x})] dt + g(t) d\bar{\mathbf{w}}. \quad (24.2)$$

Il termine $\nabla \log p_t$ è la *time-dependent score*, che la rete neurale apprende.

24.4.3 Probability flow ODE

Rimuovendo il termine Brownian:

$$\frac{d\mathbf{x}}{dt} = \mathbf{f}(\mathbf{x}, t) - \frac{1}{2}g^2(t)\nabla_{\mathbf{x}} \log p_t(\mathbf{x}). \quad (24.3)$$

Stesse *marginal distributions* della SDE, ma traiettorie deterministiche $\Rightarrow$ è un Continuous Normalizing Flow! Si può:

- calcolare likelihood esatta via instantaneous change of variables,
- invertire il flusso per encoding $\mathbf{x} \rightarrow \mathbf{z}$.

24.5 Flow Matching

L'idea (Lipman et al. ICLR 2023): saltare la SDE e regredire direttamente un *campo di velocità* $\mathbf{v}_\theta(\mathbf{z}_\tau, \tau)$ che trasporti il prior p_0 nella distribuzione dei dati p_1 in tempo $\tau \in [0, 1]$.

Osservazione 24.1 (Convenzione di tempo). In diffusion: $t = 0$ è dato, $t = T$ è rumore. In flow matching si usa τ con $\tau = 0$ rumore (sorgente), $\tau = 1$ dato (target).

24.5.1 Probabilità path e velocità

Si sceglie una famiglia $\{p_\tau\}_{\tau \in [0,1]}$ con p_0 semplice e $p_1 = p_{\text{data}}$. La *marginal velocity* $\mathbf{u}_\tau(\mathbf{z})$ è il campo vettoriale che genera p_τ via l'ODE

$$\frac{d\mathbf{z}_\tau}{d\tau} = \mathbf{u}_\tau(\mathbf{z}_\tau).$$

L'obiettivo ideale è

$$\mathcal{L}_{\text{FM}} = \mathbb{E}_{\tau, \mathbf{z}_\tau \sim p_\tau} \left[\|\mathbf{v}_\theta(\mathbf{z}_\tau, \tau) - \mathbf{u}_\tau(\mathbf{z}_\tau)\|^2 \right],$$

ma $\mathbf{u}_\tau$ non è osservabile.

24.5.2 Conditional Flow Matching (CFM)

Si sceglie un *conditional path* $q_\tau(\mathbf{z}|\mathbf{x})$ con velocità $\mathbf{u}_\tau(\mathbf{z}|\mathbf{x})$ noti. Loss:

$$\mathcal{L}_{\text{CFM}}(\theta) = \mathbb{E}_{\tau, \mathbf{x}, \mathbf{z} \sim q_\tau(\cdot|\mathbf{x})} \left[\|\mathbf{v}_\theta(\mathbf{z}, \tau) - \mathbf{u}_\tau(\mathbf{z}|\mathbf{x})\|^2 \right]. \quad (24.4)$$

Teorema 24.2 (Lipman et al. 2023). Sotto ipotesi tecniche, $\nabla_\theta \mathcal{L}_{\text{CFM}} = \nabla_\theta \mathcal{L}_{\text{FM}}$. Le velocità condizionate *si mediano* sulla velocità marginale corretta.

Sketch. Si mostra che $\mathbf{u}_\tau(\mathbf{z}) = \mathbb{E}[\mathbf{u}_\tau(\mathbf{z}|\mathbf{x}) \mid \mathbf{z}_\tau = \mathbf{z}]$. La differenza fra le due loss è una costante indipendente da θ . $\square$

24.5.3 Path notevoli

Rectified flow.

$$\mathbf{z}_\tau = (1 - \tau)\mathbf{z}_0 + \tau\mathbf{x}, \quad \mathbf{z}_0 \sim p_0, \quad \mathbf{x} \sim p_{\text{data}}.$$

Velocità conditional costante:

$$\mathbf{u}_\tau(\mathbf{z}_\tau|\mathbf{x}, \mathbf{z}_0) = \frac{d\mathbf{z}_\tau}{d\tau} = \mathbf{x} - \mathbf{z}_0.$$

Traiettorie rettilinee. Sampling: pochi step di Euler bastano.

Gaussian probability path.

$$\mathbf{z}_\tau = \alpha_\tau \mathbf{x} + \sigma_\tau \boldsymbol{\epsilon}, \quad \alpha_0 = 0, \sigma_0 = 1, \alpha_1 = 1, \sigma_1 = 0.$$

$$\mathbf{u}_\tau(\mathbf{z}_\tau | \mathbf{x}, \boldsymbol{\epsilon}) = \dot{\alpha}_\tau \mathbf{x} + \dot{\sigma}_\tau \boldsymbol{\epsilon}.$$

Per appropriate scelte di α_τ, σ_τ si recuperano le path della diffusion. *Flow matching gaussiano e diffusion sono lo stesso modello sotto parametrizzazioni e pesi diversi.*

24.6 Sampling

Integrazione dell'ODE:

$$\frac{d\mathbf{z}_\tau}{d\tau} = \mathbf{v}_\theta(\mathbf{z}_\tau, \tau), \quad \mathbf{z}_0 \sim p_0.$$

Solver Euler:

$$\mathbf{z}_{\tau+\Delta\tau} = \mathbf{z}_\tau + \Delta\tau \mathbf{v}_\theta(\mathbf{z}_\tau, \tau).$$

Numero di step tipicamente molto inferiore a DDPM (anche 4-8 step con rectified flow + distillation).

Esempio svolto — rectified flow: la velocità è costante, e si vede

Caso unidimensionale. Sorgente $z_0 = 0.5$, dato $x = 2.0$, cammino lineare

$$z_\tau = (1 - \tau)z_0 + \tau x.$$

Il bersaglio della regressione.

$$u_\tau = \frac{dz_\tau}{d\tau} = x - z_0 = 2.0 - 0.5 = 1.5,$$

costante rispetto a τ . Non serve né una score, né un kernel reverse, né un'ELBO: la coppia (sorgente, dato) determina direttamente il bersaglio.

Sampling con Euler, $K = 4$ passi, $\Delta\tau = 0.25$. Supponendo che la rete abbia imparato $v_\theta \equiv 1.5$:

$$0.5 \rightarrow 0.875 \rightarrow 1.25 \rightarrow 1.625 \rightarrow 2.0.$$

Quattro passi, arrivo *esatto*. Non è una coincidenza: il metodo di Euler è esatto quando la traiettoria è una retta, perché l'errore di troncamento dipende dalla derivata seconda, che qui è nulla.

Ecco perché flow matching campiona in pochi passi. Un DDPM ha bisogno di centinaia o migliaia di passi perché le sue traiettorie sono curve e stocastiche, e ogni passo di Euler introduce un errore. Rectified flow costruisce deliberatamente traiettorie il più possibile rettilinee, e su traiettorie rettilinee l'integratore non sbaglia. Le procedure di *reflow* iterano la costruzione per raddrizzare ulteriormente i cammini, arrivando a generare in 1–4 passi.

L'onestà del caso reale: con un solo punto le traiettorie sono banalmente rette. Su una distribuzione vera il campo marginale $\mathbf{u}_\tau(\mathbf{z})$ è la *media* delle velocità condizionali che passano per $\mathbf{z}$, e quella media in generale curva. Il teorema del CFM garantisce che regredire i bersagli condizionali dia il gradiente giusto per il campo marginale — non che il campo marginale sia rettilineo.

Le due ricette di training, affiancate

La distanza concettuale fra diffusion e flow matching è grande; quella fra le due implementazioni è minima. Vale la pena saperlo dire, perché è una domanda che si presta bene a distinguere chi ha capito da chi ha memorizzato.

Diffusion

1. campiona $\mathbf{x} \sim p_{\text{data}}$
2. campiona il tempo t
3. campiona $\epsilon \sim \mathcal{N}(0, \mathbf{I})$
4. costruisci $\mathbf{z}_t = \sqrt{\bar{\alpha}_t} \mathbf{x} + \sqrt{1 - \bar{\alpha}_t} \epsilon$
5. predici ϵ (o la score)

Flow matching

1. campiona $\mathbf{x} \sim p_{\text{data}}$
2. campiona la sorgente $\mathbf{z}_0 \sim p_0$ (o ϵ)
3. campiona il tempo τ
4. costruisci $\mathbf{z}_\tau = (1 - \tau)\mathbf{z}_0 + \tau \mathbf{x}$
5. predici la velocità $\mathbf{v}$

Stessa struttura, stessa backbone (U-Net o Transformer con time embedding), stessa loss quadratica. Cambia *il bersaglio della regressione*: ϵ , $\mathbf{s}$ oppure $\mathbf{v}$. E cambia chi decide il cammino: nella diffusion lo detta il processo di rumore, nel flow matching lo si sceglie.

24.7 Riepilogo unificante

Quattro lenti sullo stesso oggetto

Lo stesso modello generativo si può vedere come:

- **Diffusion** (DDPM): denoising step-by-step di una catena di Markov.
- **Score matching**: regressione del campo $\nabla \log p_t$.
- **Probability flow ODE**: continuous normalizing flow deterministico.
- **Flow matching**: regressione del campo di velocità lungo un cammino di probabilità.

La differenza pratica è nella *parametrizzazione della testa* (predice ϵ , $\mathbf{s}$ o $\mathbf{v}$) e nelle scelte di path/scheduling, non nell'architettura.

Quando usare Flow Matching

Generazione veloce ad alta qualità

sì, $\mathcal{O}(10)$ step vs $\mathcal{O}(1000)$ di DDPM.

Path flessibili per dati con struttura nota

sì, si può scegliere il path che fitta il problema (rectified, OT, schroedinger bridge).

Optimal transport

connessione naturale, rectified flow approssima OT.

Density estimation esatta

sì, tramite probability flow ODE.

Quando preferire diffusion classico?

solo per maggiore maturità del software / pretrained models. Concettualmente flow matching è preferibile.

Domande d'esame — Score & Flow Matching

1. Cos'è la score function di una distribuzione e perché è utile per modelli energy-based?
2. Deriva la loss del Denoising Score Matching per rumore gaussiano. Mostra che è equivalente alla loss simplified di DDPM.
3. Scrivi forward e reverse SDE. Cosa rappresenta il termine $g^2(t)\nabla \log p_t$ nella reverse?
4. Cos'è la probability flow ODE e perché collega diffusion e normalizing flows?
5. Spiega l'idea di Flow Matching: cosa si regredisce e perché si usa un *conditional* flow matching?
6. Mostra il path rectified flow e calcola la velocità conditional. Perché le traiettorie sono rettilinee?
7. In che senso flow matching e diffusion sono lo stesso modello sotto diverse parametrizzazioni?
8. Vantaggi di flow matching in termini di numero di passi di sampling.

Chapter 25

Causal Representation Learning

Risorse video — GDL30 — Disentanglement e Causal Representation Learning

Videolezione del corso: registrazione della lezione GDL30 nella cartella *Recordings* del team del corso (v. Prefazione).

UAI 2023 Tutorial: Causal Representation Learning (2h)

Copre il risultato di impossibilità di Locatello, l'identificabilità tramite interventi e i modelli temporali tipo CITRIS: è la mappa completa del capitolo, tenuta da chi lavora sul tema.

Causal Models (Brady Neal, 1h)

Da rivedere per riallacciare la Parte I: il CRL è la causalità del Cap. 5 applicata a variabili latenti anziché osservate.

25.1 Disentanglement: definizione e motivazione

Si assume che le osservazioni $\mathbf{x}$ siano generate da *fattori di variazione* $\mathbf{s} = (s_1, \dots, s_k)$ *indipendenti*:

$$\mathbf{x} = f^*(\mathbf{s}), \quad p(\mathbf{s}) = \prod_j p(s_j).$$

Una rappresentazione $\mathbf{z} = h(\mathbf{x})$ è *disentangled* se ciascuna componente z_j corrisponde a un unico fattore $s_{\pi(j)}$ (permutazione π) e ad un'eventuale trasformazione monotona componente-per-componente. Intervenire su z_j cambia solo $s_{\pi(j)}$.

Disentanglement è auspicabile per: interpretabilità, equità, generalizzazione compositiva, transfer learning.

25.2 Approcci unsupervised

25.2.1 β -VAE (Higgins et al. 2017)

$$\mathcal{L}_\beta = \mathbb{E}_{q_\phi}[\log p_\theta(\mathbf{x}|\mathbf{z})] - \beta D_{\text{KL}}(q_\phi(\mathbf{z}|\mathbf{x})\|p(\mathbf{z})), \quad \beta > 1.$$

La regolarizzazione forte spinge $q_\phi(\mathbf{z}|\mathbf{x})$ verso il prior fattorizzato $p(\mathbf{z}) = \mathcal{N}(\mathbf{0}, \mathbf{I})$, sperando in componenti indipendenti.

25.2.2 FactorVAE (Kim & Mnih 2018)

Decomponi la KL e penalizza specificamente la *total correlation* (TC) del marginale $q_\phi(\mathbf{z}) = \mathbb{E}_{\mathbf{x}}[q_\phi(\mathbf{z}|\mathbf{x})]$:

$$\text{TC}(\mathbf{z}) = D_{\text{KL}}\left(q_\phi(\mathbf{z}) \parallel \prod_j q_\phi(z_j)\right).$$

La TC si stima con un discriminatore.

25.2.3 β -TCVAE (Chen et al. 2018)

Decomponi $D_{\text{KL}}(q_\phi(\mathbf{z}|\mathbf{x})\parallel p(\mathbf{z}))$ in tre termini (mutual info $\mathbf{x} \leftrightarrow \mathbf{z}$, TC, KL dimensionwise) e pesa solo quello che penalizza la TC.

Intuizione — perché il disentanglement non supervisionato è impossibile

L'argomento di Locatello è più semplice di quanto la sua fama suggerisca, e conviene saperlo raccontare in tre passaggi.

Supponiamo di avere un modello generativo con latente $\mathbf{z} \sim \mathcal{N}(0, \mathbf{I})$ e decoder g , che riproduce perfettamente la distribuzione dei dati. Prendiamo ora una rotazione $\mathbf{R}$ dello spazio latente e definiamo $\tilde{\mathbf{z}} = \mathbf{R}\mathbf{z}$, $\tilde{g} = g \circ \mathbf{R}^{-1}$.

Poiché la gaussiana standard isotropa è invariante per rotazioni, $\tilde{\mathbf{z}}$ ha *esattamente* la stessa distribuzione di $\mathbf{z}$, e il nuovo modello genera *esattamente* la stessa distribuzione sui dati. I due modelli sono indistinguibili da qualunque quantità calcolata sui dati: stessa likelihood, stesso ELBO, stessa qualsiasi cosa.

Ma se $\mathbf{z}$ era disentangled — una coordinata per il colore, una per la posizione — allora $\tilde{\mathbf{z}}$ mescola colore e posizione in ogni coordinata. Le due rappresentazioni hanno lo stesso valore per ogni criterio osservativo e opposto valore di disentanglement. Quindi *nessun* obiettivo non supervisionato può preferire l'una all'altra: le rotazioni sono un'intera famiglia infinita di soluzioni equivalenti.

Da cui la morale, che è costruttiva e non nichilista: serve rompere quella simmetria dall'esterno. Le tre vie sono supervisione debole (coppie, gruppi, etichette parziali), struttura temporale (TCL: se le sorgenti evolvono in modo indipendente nel tempo, la storia le identifica), oppure *interventi* — ed è esattamente qui che la causalità entra nel discorso. Un intervento su un fattore cambia una sola coordinata vera e lascia ferme le altre; sotto rotazione ne cambierebbe molte. L'intervento distingue ciò che l'osservazione non distingue, esattamente come nella Parte I.

Nota su β -VAE: alza β e ottiene rappresentazioni più fattorizzate, ma questo non contraddice il teorema. Sta introducendo un bias induttivo implicito (verso assi allineati alle coordinate), non identificando i fattori veri — e infatti il risultato dipende fortemente dal seed, che è proprio la verifica sperimentale di Locatello.

25.3 Impossibilità (Locatello et al. 2019)

Teorema 25.1 (Indeterminatezza non-supervisionata). Per qualunque $\mathbf{z}$ con componenti marginalmente indipendenti, esiste un'infinità di trasformazioni $\mathbf{z}' = f(\mathbf{z})$ che preservano la indipendenza marginale ma producono rappresentazioni *entangled* rispetto ai fattori ground-truth, con *stessa loss ELBO*.

In altre parole, marginale indipendenza è necessaria ma non sufficiente. Il successo empirico di β -VAE e FactorVAE dipende da *inductive bias* (architettura, prior, regolarizzazione), non da garanzie formali.

25.4 Disentanglement supervisionato debolmente

Per ottenere *identifiability* servono informazioni ausiliarie $\mathbf{u}$ (tempo, ambiente, classe) tali che $p(\mathbf{s}|\mathbf{u})$ vari con $\mathbf{u}$.

25.4.1 Time-Contrastive Learning (TCL, Hyvärinen & Morioka 2016)

Si divide una serie temporale in segmenti e si allena un classificatore a riconoscere il segmento da $\mathbf{x}$. Sotto ipotesi (sorgenti exp-family, mixing iniettivo, segmenti "abbastanza diversi"), si recuperano le sorgenti a meno di permutazione e trasformazione monotona componente-per-componente.

25.4.2 iVAE (Khemakhem et al. 2020)

Variante condizionale del VAE in cui il prior dipende dalla variabile ausiliaria:

$$p_\theta(\mathbf{z}|\mathbf{u}) = \prod_j p_j(z_j|\boldsymbol{\lambda}(\mathbf{u})).$$

ELBO con $p(\mathbf{z}|\mathbf{u})$ al posto di $p(\mathbf{z})$:

$$\mathcal{L}_{\text{iVAE}} = \mathbb{E}_{q_\phi}[\log p_\theta(\mathbf{x}|\mathbf{z})] - D_{\text{KL}}(q_\phi(\mathbf{z}|\mathbf{x}, \mathbf{u}) \| p_\theta(\mathbf{z}|\mathbf{u})).$$

Identifiability: sotto condizioni su $\boldsymbol{\lambda}$ e sull'iniettività del decoder, si recuperano le sorgenti a meno di trasformazioni componente-wise.

25.5 Causal Representation Learning

25.5.1 Setup

I fattori $\mathbf{s}$ non sono indipendenti ma seguono uno *Structural Causal Model* (SCM) (Pearl, Cap. 5):

$$s_i := f_i(\text{Pa}(s_i), \epsilon_i), \quad \epsilon_i \perp \epsilon_j.$$

Si osserva $\mathbf{x} = h^*(\mathbf{s})$ con h^* mixing iniettivo. L'obiettivo è recuperare sia le variabili causali $\mathbf{s}$ sia il loro *causal graph* G a partire dalle osservazioni.

25.5.2 Identifiability via interventi

Senza informazione extra il problema è impossibile (analogamente alla causalità classica): le *Markov equivalence classes* rendono indistinguibili più grafi. Servono *interventi*.

25.5.3 CITRIS (Lippe et al. 2022)

Setting: *Temporal Causal Triplets* $(\mathbf{x}_t, \mathbf{x}_{t+1}, \mathbf{I}_{t+1})$ dove $\mathbf{I}_{t+1}$ è una maschera binaria che indica quali variabili causali sono state intervenute al tempo $t+1$ (non si conoscono i valori dell'intervento).

Architettura:

- Encoder $\mathbf{z}_t = g_\phi(\mathbf{x}_t)$;
- Assignment function $\psi : \mathcal{Z} \rightarrow \{1, \dots, k\}^m$ che mappa ogni dimensione latente a una variabile causale;
- Transition prior $p(\mathbf{z}_{t+1}|\mathbf{z}_t, \mathbf{I}_{t+1})$ parametrizzato in modo che ogni gruppo $\mathbf{z}^{(i)}$ dipenda da $\mathbf{z}_t$ tramite il proprio modulo, modulato da I_i ;

- Decoder $p(\mathbf{x}_t|\mathbf{z}_t)$.

Training: ELBO temporale + (opzionale) target classifier che predice $\mathbf{I}_{t+1}$ dai latenti.

Teorema 25.2 (Identifiability CITRIS). Nel limite di dati infiniti, CITRIS recupera le variabili causali ground-truth se per ogni C_i :

1. esiste un regime in cui C_i è intervenuto e non sempre congiunto a un altro C_j ;
2. C_i dipende sempre dalla sua variabile di intervento I_i .

25.5.4 Varianti

CITRIS-NF disentangle una rappresentazione pre-addestrata via normalizing flows.

BISCUIT generalizza a interventi binari sconosciuti.

25.6 Triplet evaluation

Test: dato un modello CITRIS addestrato, si genera un *interchange intervention* sostituendo il valore di una variabile causale da un'immagine con quello dell'altra. Se la generazione cambia in modo coerente solo sulla variabile sostituita, il modello è disentangled.

Quando usare CRL

Sistemi con interventi disponibili

(RL, scienze sperimentali) sì.

Dati osservazionali puri

CRL puro è impossibile (Locatello). Si può ancora usare β -VAE per interpretabilità empirica, senza garanzie.

Time-series con regimi non stazionari

iVAE/TCL.

Generative model che generalizzi a interventi distribuzionali

sì, CRL è il giusto framework.

Connessioni

- β -VAE $\rightarrow$ FactorVAE $\rightarrow$ β -TCVAE: progressione di obiettivi per disentanglement.
- Locatello 2019 $\rightarrow$ TCL/iVAE $\rightarrow$ CITRIS: dalla impossibilità unsupervised al requirements di interventi.
- CITRIS = VAE temporale con prior strutturato per intervento (parente del CVAE, Cap. 20).
- Connette Causalità (Cap. 5) e Deep Generative Learning.

Domande d'esame — Causal Representation Learning

1. Definisci formalmente cosa significa che una rappresentazione è *disentangled*.
2. Cos'è il β -VAE e perché aumentare β dovrebbe promuovere il disentanglement?
3. Enuncia il risultato di impossibilità di Locatello et al. 2019. Quale assunto viene

violato dall'esistenza di trasformazioni che preservano la indipendenza marginale?

4. Cos'è una variabile ausiliaria $\mathbf{u}$ in iVAE/TCL? Perché permette di superare l'impossibilità?
5. Cos'è il problema di Causal Representation Learning? In cosa differisce da disentanglement classico?
6. Descrivi il setup di CITRIS (input, output, ipotesi su interventi).
7. Cosa garantisce il teorema di identifiability di CITRIS?
8. Differenza pratica fra CITRIS-VAE e CITRIS-NF.
9. Confronta causalità classica (Cap. 5) e CRL: quale è il contributo del deep learning?

Confronto finale Parte IV

Confronto modelli generativi profondi

Proprietà	VAE	GAN	Flows	Diffusion	Flow Match.
Likelihood esatta	ELBO	no	sì	ELBO/no	no
Score implicita	no	no	sì (PF-ODE)	sì	sì
Qualità campioni	media	alta	media	molto alta	alta
Velocità sampling	veloce	veloce	veloce	lenta ($T \sim 10^3$)	media ($T \sim 10$)
Stabilità training	stabile	instabile	stabile	stabile	stabile
Latente strutturato	sì	no	sì	parziale	sì
Cond. generation	facile	possibile	facile	facile (CFG)	facile
Encoder esplicito	sì	no	sì (inverso)	no/fisso	no
Dim. latente	$\neq$ dato	$\neq$ dato	$=$ dato	$=$ dato	$=$ dato
Anno (paper seminale)	2013	2014	2014–15	2020	2023

Famiglia generativa — la storia in 5 passi

1. **Autoencoder** (1980s, deep AE 2006): latente, ma non probabilistico.
2. **VAE** (Kingma 2013): aggiunge un encoder stocastico, ELBO, reparametrization $\Rightarrow$ il primo modello latente differenziabile end-to-end.
3. **GAN** (Goodfellow 2014): cambia paradigma — impara a campionare, non a stimare la densità.
4. **Normalizing Flow** (Dinh 2014, 2017): cambio di variabile esplicito, likelihood esatta, ma vincoli di invertibilità.
5. **Diffusion** (Ho 2020): VAE gerarchico con encoder fisso a T livelli, training stabile, qualità SOTA, ma sampling lento.
6. **Flow Matching** (Lipman 2023): regressione diretta del campo di velocità $\Rightarrow$ diffusion senza SDE, sampling efficiente.

Le 5 lenti unificanti del corso si manifestano qui:

- *Message passing*: nessuno (questi modelli operano su tensor euclidei).

- *Generative family* (questo capitolo).
- *Attention*: usato come building block (U-Net dei diffusion, Transformer per latent diffusion).
- *EM/Latent variable*: il VAE è il discendente diretto di EM (Cap. 8).
- *Causality*: CRL applica il framework generativo al recupero di SCM.

Part V

Graph Neural Networks

Osservazione 25.1 (Materia d'esame). Come esplicitamente indicato dal prof nella lezione finale (GDL35), il modulo *Deep Learning for Graphs II — Advanced Models* (GDL33) **non** è materia d'esame. Questa Parte copre solo il modulo fondamentale (GDL32): formalizzazione del task, message passing, varianti convolutive, attention-based e graph transformer.

Chapter 26

Fondamenti di Deep Learning per Grafi

Risorse video — GDL32 — Deep learning per grafi: fondamenti

Videolezione del corso: registrazione della lezione GDL32 nella cartella *Recordings* del team del corso (v. Prefazione).

Theoretical Foundations of Graph Neural Networks (*Petar Veličković, 1h*)

Tenuto dall'autore di GAT: costruisce le GNN partendo dall'invarianza per permutazione, cioè dalla stessa logica di inductive bias usata per le CNN nella Parte III. È la fonte migliore per capire *perché* il message passing ha quella forma.

Physics-inspired Graph Neural Networks (*Michael Bronstein, LMS/IMA 2023, 1h*)

Riguarda soprattutto oversmoothing e oversquashing, quindi la parte “avanzata”. **Attenzione:** la lezione GDL33 *Deep Learning for Graphs II — Advanced Models* è stata esplicitamente esclusa dal materiale d'esame (slide GDL35, “Deep learning for graph II: Advanced models lecture is not part of exam materials”). Questo video resta utile per capire meglio le patologie citate nel Cap. 28, ma non è materiale da studiare per l'orale.

26.1 Perché i grafi?

Molti dati reali sono *intrinsecamente relazionali*: molecole (atomi e legami), reti sociali, knowledge graph, point cloud, sistemi software, mesh 3D, sistemi fisici interagenti. Un modello che ignora la struttura relazionale (es. trattando i nodi come iid) perde informazione cruciale: il *contesto* di un nodo è il suo intorno.

26.2 Formalizzazione del task

Definizione 26.1 (Grafo). Un grafo è una tupla $G = (V, E, X, X^E)$ dove V è l'insieme dei nodi, $E \subseteq V \times V$ degli archi, $X = \{\mathbf{x}_v\}_{v \in V}$ feature di nodo, X^E feature di arco (opzionali). Il grafo può essere diretto/non diretto, ciclico/aciclico, con/senza etichette.

26.2.1 Tassonomia dei task

Node-level data una graph singolo G e un nodo v , prediciamo una label y_v (classificazione/regressione di nodo). Esempio: identificazione di account fraudolenti in una social network.

Link prediction data una coppia (u, v) , prevediamo l'esistenza/peso dell'arco. Esempio: recommender system.

Community detection partizione di V in cluster.

Graph-level dato un dataset $\{(G_i, y_i)\}$ di grafi iid, classifichiamo/regressiamo l'intero grafo. Esempio: predizione di proprietà molecolari.

Trasduttivo vs induttivo trasduttivo: training e test sullo stesso grafo (i nodi di test sono visibili in training senza label). Induttivo: test su grafi mai visti.

Strutturato output stesso un grafo (generazione). Vedi modelli generativi su grafi (cenni in GDL33, fuori esame).

26.3 Prospettiva storica

26.3.1 Modelli contractive (Scarselli et al. 2009)

La prima Graph Neural Network. Si calcola uno stato $\mathbf{h}_v$ per ogni nodo come punto fisso di un'iterazione contractive:

$$\mathbf{h}_v = f(\mathbf{x}_v, \{\mathbf{h}_u\}_{u \in N(v)}, \{\mathbf{x}_{uv}\}),$$

con f scelta in modo che la trasformazione globale sia contraente (vincolando i pesi). Il punto fisso esiste per il teorema di Banach.

26.3.2 Modelli contextual (Micheli 2009, NN4G)

Approccio feedforward: si compone una sequenza di layer dove il layer ℓ usa solo informazioni *congelate* del layer $\ell - 1$. La ricorsione viene quindi sostituita da gerarchia (layering), e il training è layerwise.

26.3.3 Deep Graph Network — la sintesi moderna

Encoder $G \mapsto \{\mathbf{h}_v\}$ via L layer di propagazione, seguito da un decoder per il task specifico. Architetture moderne (GCN, GraphSAGE, GAT, GIN, Graph Transformer) sono varianti della stessa idea.

Domande d'esame — Fondamenti di DL per grafi

1. Perché non si può applicare direttamente una CNN a un grafo? Elenca gli ostacoli strutturali (nessun ordine canonico dei vicini, grado variabile, assenza di una griglia).
2. Che cosa significa *invarianza per permutazione* e perché è un requisito e non una comodità? Distingui dalla *equivarianza* e di quale serve a livello di nodo e quale a livello di grafo.
3. Descrivi la tassonomia dei task su grafi: node-level, edge-level, graph-level. Per ciascuno un esempio e che cosa cambia nel readout.
4. Che differenza c'è fra setting *trasduttivo* e *induttivo*? Quali modelli funzionano solo nel primo caso e perché.
5. Confronta i modelli *contractive* di Scarselli (2009) e quelli *contextual* di Micheli (NN4G, 2009). Che cosa impone il vincolo di contrattività e come lo si evita impilando layer.

6. Che cos'è un Deep Graph Network nella sintesi moderna, e in che senso unifica i due filoni storici?

Chapter 27

Message Passing e GNN Convolutionali

Risorse video — GDL32 — Message passing, GCN, GraphSAGE, GIN

Videolezione del corso: registrazione della lezione GDL32 nella cartella *Recordings* del team del corso (v. Prefazione).

Theoretical Foundations of Graph Neural Networks (*Petar Veličković, 1h*)

La parte centrale è esattamente questo capitolo: derivazione dello schema di message passing dall'invarianza per permutazione, e la gerarchia convolutional / attentional / message-passing.

27.1 Schema generale: message passing

Definizione 27.1 (Message Passing Neural Network, Gilmer 2017). A ogni layer $\ell = 0, 1, \dots, L - 1$:

$$\mathbf{m}_v^{(\ell+1)} = \text{AGGREGATE}^{(\ell)}(\{\mathbf{h}_u^{(\ell)} : u \in N(v)\}) \quad (27.1)$$

$$\mathbf{h}_v^{(\ell+1)} = \text{UPDATE}^{(\ell)}(\mathbf{h}_v^{(\ell)}, \mathbf{m}_v^{(\ell+1)}) \quad (27.2)$$

con $\mathbf{h}_v^{(0)} = \mathbf{x}_v$. Per task graph-level si aggrega:

$$\mathbf{h}_G = \text{READOUT}(\{\mathbf{h}_v^{(L)} : v \in V\}).$$

Requisiti su AGGREGATE: **invariante per permutazione** dei vicini (altrimenti il modello dipenderebbe dall'ordine arbitrario di numerazione dei nodi). Esempi: somma, media, max, attention weighted sum.

Intuizione — message passing è belief propagation con i pesi imparati

Questo è il collegamento trasversale più redditizio dell'intero corso, ed è il tipo di domanda che il prof usa come terza domanda dell'orale.

Nella belief propagation (Parte I) ogni nodo di un grafo probabilistico raccoglie messaggi dai vicini, li combina e aggiorna la propria credenza. I messaggi sono determinati dai potenziali del modello, la regola di combinazione è fissata dalle regole della probabilità, e si itera fino a convergenza.

In una GNN succede la stessa cosa con tre sostituzioni: i messaggi sono *funzioni apprese* invece che determinate dai potenziali; la combinazione è una rete neurale invece che un

prodotto di fattori; e non si itera fino a convergenza ma per un numero fisso L di layer. Da questo parallelo si legge subito il resto:

- l'**invarianza per permutazione** di AGGREGATE non è una scelta di comodo: senza, il modello dipenderebbe dalla numerazione arbitraria dei nodi, che non è un'informazione reale del grafo;
- il numero di layer L è il *raggio* entro cui l'informazione può viaggiare, esattamente come il campo recettivo di una CNN. Da qui l'under-reaching: con L layer non si vedono nodi a distanza $> L$;
- la stessa struttura ricompare nel Transformer, che è una GNN su grafo completo con attenzione come funzione di aggregazione. CNN, GNN e Transformer differiscono per *quale grafo* si assume, non per il meccanismo.

27.2 Convolutional Graph Networks

27.2.1 Approccio spettrale

Esiste una nozione di convoluzione su grafi basata sul *Laplaciano* $L = D - A$ (o sua versione normalizzata $\tilde{L} = I - D^{-1/2}AD^{-1/2}$). Decomponendo $\tilde{L} = U\Lambda U^\top$:

$$\mathbf{f} *_G \mathbf{g} = U\mathbf{W}(\Lambda)U^\top \mathbf{f},$$

dove $\mathbf{W}(\Lambda)$ è una funzione diagonale dei autovalori, parametrizzata. **Limite:** U dipende dal grafo, quindi non si generalizza a grafi diversi (poco utile in setting induttivo).

27.2.2 Graph Convolutional Network (GCN, Kipf & Welling 2017)

Approssimazione di primo ordine di un filtro spettrale di Chebyshev:

$$\mathbf{H}^{(\ell+1)} = \sigma\left(\tilde{D}^{-1/2}\tilde{A}\tilde{D}^{-1/2}\mathbf{H}^{(\ell)}\mathbf{W}^{(\ell)}\right), \quad (27.3)$$

con $\tilde{A} = A + I$ (self-loop), $\tilde{D}$ matrice diagonale dei gradi di $\tilde{A}$. Per nodo:

$$\mathbf{h}_v^{(\ell+1)} = \sigma\left(\mathbf{W}^{(\ell)} \sum_{u \in N(v) \cup \{v\}} \frac{1}{\sqrt{\tilde{d}_v \tilde{d}_u}} \mathbf{h}_u^{(\ell)}\right).$$

È un caso particolare di message passing con AGGREGATE = media pesata Laplacian-normalized.

Limiti GCN:

- normalization fissata: non discrimina vicini in modo adattivo;
- over-smoothing dopo molti layer (vedi sotto);
- assume grafo omofilo (vicini simili).

Esempio svolto — un layer di GCN calcolato a mano

Grafo a catena su quattro nodi: $1 - 2 - 3 - 4$. Feature scalari $\mathbf{h}^{(0)} = (1, 2, 3, 4)$, $\mathbf{W} = 1$ per semplicità.

Passo 1 — self-loop e gradi. $\tilde{A} = A + I$ dà gradi $\tilde{d} = (2, 3, 3, 2)$: i nodi interni

hanno due vicini più sé stessi, quelli terminali uno più sé stessi.

Passo 2 — propagazione normalizzata. Con $h'_i = \sum_{j \in N(i) \cup \{i\}} \frac{h_j}{\sqrt{d_i d_j}}$:

$$h'_1 = \frac{1}{\sqrt{2 \cdot 2}} + \frac{2}{\sqrt{2 \cdot 3}} = 0.500 + 0.816 = 1.316,$$

$$h'_2 = \frac{1}{\sqrt{6}} + \frac{2}{3} + \frac{3}{3} = 0.408 + 0.667 + 1.000 = 2.075,$$

$$h'_3 = \frac{2}{3} + \frac{3}{3} + \frac{4}{\sqrt{6}} = 3.300, \quad h'_4 = \frac{3}{\sqrt{6}} + \frac{4}{2} = 3.225.$$

Si osservi che h_4 è *sceso* da 4 a 3.225 e h_1 è *salito* da 1 a 1.316: un layer di GCN è un'operazione di media locale, cioè un passo di smoothing sul grafo. Il self-loop serve proprio a evitare che il nodo dimentichi sé stesso, e la normalizzazione $1/\sqrt{d_i d_j}$ a evitare che i nodi ad alto grado dominino.

Passo 3 — l'over-smoothing, misurato. Iterando lo stesso operatore, l'energia di Dirichlet $\sum_{(i,j) \in E} (h_i/\sqrt{d_i} - h_j/\sqrt{d_j})^2$, che misura quanto le rappresentazioni dei nodi adiacenti differiscono, crolla:

layer	0	1	2	10	20
energia	1.736	0.712	0.370	2.3×10^{-3}	4.2×10^{-6}

E la cosa peggiore è la prossima. Partendo da due input *completamente diversi*, $(1, 2, 3, 4)$ e $(10, -5, 0, 3)$, dopo 30 layer entrambi convergono alla stessa direzione $(0.447, 0.548, 0.548, 0.447) \propto \sqrt{\tilde{d}}$. La rappresentazione finale dipende solo dai *gradi*: tutta l'informazione contenuta nelle feature è stata cancellata.

Il punto da dire: l'over-smoothing non è un difetto di ottimizzazione, è una proprietà spettrale dell'operatore di propagazione. $\mathbf{S} = \tilde{\mathbf{D}}^{-1/2} \tilde{\mathbf{A}} \tilde{\mathbf{D}}^{-1/2}$ ha autovalore dominante 1 con autovettore $\sqrt{\tilde{d}}$, e iterare $\mathbf{S}^L$ proietta tutto su quell'autovettore. È per questo che le GNN profonde non funzionano “e basta”, e perché i rimedi (residual connection, jumping knowledge, PairNorm) agiscono tutti impedendo alla dinamica di raggiungere quel punto fisso.

Riscontro: l'esercizio 14_gnn misura la stessa quantità su un grafo più grande e ottiene $2.21 \rightarrow 0.110 \rightarrow 1.7 \times 10^{-5}$.

27.2.3 GraphSAGE (Hamilton et al. 2017)

Pensato esplicitamente per setting induttivo:

$$\mathbf{h}_v^{(\ell+1)} = \sigma\left(\mathbf{W}^{(\ell)} [\mathbf{h}_v^{(\ell)} \parallel \text{AGG}(\{\mathbf{h}_u^{(\ell)}\}_{u \in N(v)})]\right).$$

AGG può essere mean, max-pool, LSTM. Si campiona un subset di vicini per scalabilità.

27.2.4 Graph Isomorphism Network (GIN, Xu et al. 2019)

Studio dell'*espressività* delle GNN.

Teorema 27.1 (Limite Weisfeiler-Lehman). Una GNN basata su message passing distingue al massimo le coppie di grafi che il test di Weisfeiler-Lehman (1-WL) distingue.

Per raggiungere il limite, AGGREGATE deve essere *iniettiva*. Sum-aggregation + MLP è iniettiva su multiset (sotto ipotesi tecniche), mean/max no. GIN:

$$\mathbf{h}_v^{(\ell+1)} = \text{MLP}^{(\ell)} \left((1 + \epsilon^{(\ell)}) \mathbf{h}_v^{(\ell)} + \sum_{u \in N(v)} \mathbf{h}_u^{(\ell)} \right).$$

GCN vs GraphSAGE vs GIN

Modello	AGGREGATE	Note
GCN	media Laplacian-norm	implicito, semplice, soffre over-smoothing
GraphSAGE	mean/max/LSTM su sample	induttivo, scalabile
GIN	sum + MLP	massima espressività (1-WL)

Esempio svolto — perché GIN è più espressivo: due multiset, un aggregatore

La domanda dietro GIN è: quando due nodi diversi ricevono la stessa rappresentazione, e quindi il modello non riesce a distinguerli?

Consideriamo un nodo u con tre vicini tutti di feature 1, e un nodo v con due vicini tutti di feature 1. I due multiset dei vicini sono

$$\{\{1, 1, 1\}\} \quad \text{e} \quad \{\{1, 1\}\}.$$

aggregatore	su $\{\{1, 1, 1\}\}$	su $\{\{1, 1\}\}$	li distingue?
media	1	1	no
max	1	1	no
somma	3	2	sì

Media e max sono *invarianti alla molteplicità*: perdono l'informazione su quante volte un elemento compare. Poiché GCN usa una media (normalizzata) e GraphSAGE-mean pure, entrambi non riescono a distinguere questi due intorni — e quindi non distinguono grafi che differiscono solo per il numero di vicini identici.

La somma conserva la molteplicità. GIN sfrutta questo fatto:

$$\mathbf{h}_v^{(\ell+1)} = \text{MLP} \left((1 + \epsilon) \mathbf{h}_v^{(\ell)} + \sum_{u \in N(v)} \mathbf{h}_u^{(\ell)} \right),$$

e Xu et al. dimostrano che con una MLP sufficientemente espressiva questo schema è *tanto potente quanto il test di Weisfeiler-Lehman a 1 dimensione*, che è il limite superiore di ciò che il message passing può distinguere.

Il limite superiore va detto insieme al risultato: 1-WL non distingue tutto. Due grafi regolari con lo stesso grado — per esempio un ciclo a sei nodi e due triangoli disgiunti — sono indistinguibili da *qualunque* GNN a message passing, GIN incluso, perché ogni nodo ha esattamente lo stesso intorno a ogni distanza. Per superare questa barriera servono feature posizionali, conteggi di sottostrutture, o attenzione globale.

E il termine ϵ ? Serve a distinguere il contributo del nodo stesso da quello dei vicini. Con $\epsilon = 0$ il nodo verrebbe sommato al pari di un vicino, e due situazioni diverse potrebbero collassare.

27.3 Edge features

Quando gli archi hanno feature $\mathbf{x}_{uv}$, si estende il messaggio:

$$\mathbf{m}_v^{(\ell+1)} = \text{AGG}\left(\{\phi(\mathbf{h}_u^{(\ell)}, \mathbf{x}_{uv}) : u \in N(v)\}\right),$$

con ϕ una piccola rete. Esempi: *Relational GCN* per knowledge graph (diversi $\mathbf{W}_r$ per tipo di relazione), *MPNN* per chimica.

Domande d'esame — Message passing e GNN convoluzionali

1. Definisci formalmente il message passing. Quali requisiti deve soddisfare AGGREGATE, e che cosa succede se non li soddisfa?
2. Scrivi la regola di aggiornamento della GCN in forma matriciale e per singolo nodo. Da quale approssimazione spettrale deriva, e da dove viene $\tilde{\mathbf{D}}^{-1/2} \tilde{\mathbf{A}} \tilde{\mathbf{D}}^{-1/2}$?
3. Perché l'approccio spettrale puro non si generalizza a grafi diversi da quello di training?
4. A che cosa serve il self-loop in GCN? E la normalizzazione simmetrica?
5. Che cosa cambia GraphSAGE rispetto a GCN, e quale problema pratico risolve il sampling dei vicini?
6. Enuncia il legame fra GNN a message passing e il test di Weisfeiler-Lehman a una dimensione. Quale aggregazione raggiunge quel limite, e perché somma e media non sono equivalenti?
7. Fai un esempio di due grafi che nessuna GNN a message passing standard, GIN incluso, riesce a distinguere. Che cosa servirebbe per superare la barriera?
8. A che cosa serve il termine ϵ nella formulazione di GIN?
9. Come si incorporano le feature sugli archi nello schema di message passing?
10. Confronta message passing nelle GNN e belief propagation nei modelli grafici probabilistici: che cosa è analogo e che cosa è diverso.
11. Confronta GCN, GraphSAGE e GIN su AGGREGATE, espressività e scalabilità.

Chapter 28

Graph Attention e Graph Transformer

Risorse video — GDL32 — Graph attention e graph transformer

Videolezione del corso: registrazione della lezione GDL32, parte finale nella cartella *Recordings* del team del corso (v. Prefazione).

Theoretical Foundations of Graph Neural Networks (*Petar Veličković, 1h*)
Veličković è il primo autore di GAT: la parte su attenzione sui grafi è raccontata da chi l'ha proposta.

Attention is All You Need — spiegazione del modello (*Umar Jamil, 1h*)
Da rivedere per il ponte concettuale: il graph transformer è self-attention su grafo completo più un positional encoding strutturale. Se il meccanismo di attenzione è solido, questo capitolo si riduce a “dove va la struttura del grafo”.

28.1 Graph Attention Network (GAT, Veličković et al. 2018)

Idea: pesare i vicini in modo adattivo via attention. Per ogni coppia $(v, u \in N(v) \cup \{v\})$:

$$e_{vu}^{(\ell)} = \text{LeakyReLU}\left(\mathbf{a}^\top [\mathbf{W}\mathbf{h}_v^{(\ell)} \parallel \mathbf{W}\mathbf{h}_u^{(\ell)}]\right) \quad (28.1)$$

$$\alpha_{vu}^{(\ell)} = \frac{\exp(e_{vu}^{(\ell)})}{\sum_{u' \in N(v) \cup \{v\}} \exp(e_{vu'}^{(\ell)})} \quad (28.2)$$

$$\mathbf{h}_v^{(\ell+1)} = \sigma \left(\sum_{u \in N(v) \cup \{v\}} \alpha_{vu}^{(\ell)} \mathbf{W}\mathbf{h}_u^{(\ell)} \right). \quad (28.3)$$

Multi-head: K teste indipendenti, output concatenato (layer intermedi) o medio (output layer).

Differenza con il self-attention del Transformer. Nel Transformer standard ogni token attiva su tutti gli altri (grafo completo). Nel GAT l'attention è ristretta ai vicini effettivi $N(v)$ del grafo: si rispetta l'*inductive bias* strutturale.

Esempio svolto — GAT contro GCN sullo stesso intorno

Nodo v con vicinato $\{v, u_1, u_2\}$ (self-loop incluso) e feature trasformate $\mathbf{W}\mathbf{h} = (1, 2, 3)$.

GCN. I coefficienti sono *fissati dalla struttura*: con gradi uguali si ottiene una media semplice,

$$\mathbf{h}'_v = \frac{1 + 2 + 3}{3} = 2.0.$$

GAT. I coefficienti sono *calcolati dalle feature*. Supponiamo che il meccanismo di attenzione produca i punteggi non normalizzati

$$e = (\underbrace{0.5}_{\text{self}}, \underbrace{1.2}_{u_1}, \underbrace{-0.3}_{u_2}), \quad e_{vu} = \text{LeakyReLU}(\mathbf{a}^\top [\mathbf{W}\mathbf{h}_v \parallel \mathbf{W}\mathbf{h}_u]).$$

Softmax sul vicinato:

$$\exp(e) = (1.649, 3.320, 0.741), \quad \sum = 5.710 \Rightarrow \alpha = (0.289, 0.582, 0.130).$$

$$\mathbf{h}'_v = 0.289(1) + 0.582(2) + 0.130(3) = 1.841.$$

La differenza in una frase. GCN pesa i vicini in base a *quanti* sono (e ai loro gradi); GAT in base a *chi* sono. Qui u_1 prende il 58% del peso e u_2 il 13%, mentre GCN avrebbe dato 33% a ciascuno.

Le tre conseguenze da nominare all'orale. Primo: GAT è *induttivo* — i coefficienti dipendono dalle feature, non dalla matrice di adiacenza, quindi funziona su grafi mai visti, a differenza dei metodi spettrali. Secondo: è interpretabile, perché gli α dicono a chi il nodo ha guardato. Terzo, e va detto: essendo l'attenzione una media pesata normalizzata, GAT eredita il limite di espressività della media — non distingue le molteplicità, quindi resta sotto il potere di GIN sul test di Weisfeiler-Lehman. Attenzione e potere discriminativo sono due assi diversi.

Multi-head: come nel Transformer, si calcolano K attenzioni indipendenti e si concatenano (nei layer intermedi) o si mediano (nel layer finale). Serve a stabilizzare il training e a permettere di guardare a più relazioni contemporaneamente.

28.2 Graph Transformer

Si applica self-attention completa a tutti i nodi (non solo i vicini), pagando il costo $\mathcal{O}(|V|^2)$. La struttura del grafo viene ri-iniettata via *positional/structural encoding*.

28.2.1 Positional encoding per grafi

Locali random-walk landing probabilities di lunghezza k .

Globali autovettori del Laplaciano $\tilde{L}$ (analoghi alle frequenze di Fourier).

Relativi distanze pairwise (shortest path, resistance distance).

28.2.2 GPS (Rampášek et al. 2022)

Combina message passing locale (per inductive bias strutturale) e self-attention globale (per long-range dependencies):

$$\mathbf{h}_v^{(\ell+1)} = \text{MPNN}_v(\mathbf{h}^{(\ell)}) + \text{GlobalAttn}_v(\mathbf{h}^{(\ell)}).$$

28.3 Pooling e gerarchia

Per task graph-level la sola READOUT (sum/mean/max) ignora la struttura gerarchica. Alternative:

DiffPool (Ying et al. 2018) apprende un assignment soft $S \in \mathbb{R}^{|V| \times K}$ che mappa nodi a cluster:

$$X' = S^\top X, \quad A' = S^\top A S.$$

Top-K Pooling mantiene i nodi con score più alto.

Edge contraction coarsening basato su criteri grafici.

28.4 Patologie

28.4.1 Over-smoothing

Dopo molti layer di message passing, le rappresentazioni dei nodi diventano simili. Le AGGREGATE basate su media tendono a un punto fisso costante. Mitigazioni: residual connection, jumping knowledge, normalization (PairNorm), modelli con bias verso identità.

28.4.2 Over-squashing

Un nodo deve aggregare informazioni da una vicinanza che cresce esponenzialmente con il numero di layer, attraverso un vettore di dimensione fissata. Informazione "schiacciata" in un canale stretto. Mitigazioni: edge rewiring, virtual nodes, attention globale.

Trappola — over-smoothing e over-squashing non sono la stessa cosa

Vengono nominati insieme e confusi di continuo. Sono due patologie diverse, con cause diverse e rimedi diversi.

Over-smoothing è omogeneizzazione: dopo molti layer le rappresentazioni dei nodi convergono allo stesso punto e diventano indistinguibili. La causa è spettrale — iterare l'operatore di propagazione proietta tutto sull'autovettore dominante. Il sintomo è che nodi diversi ricevono la stessa rappresentazione. I rimedi agiscono sulla dinamica: residual connection, jumping knowledge, PairNorm.

Over-squashing è perdita di banda: un nodo deve comprimere l'informazione proveniente da un vicinato che cresce esponenzialmente col numero di layer dentro un vettore di dimensione fissa. La causa è la topologia — colli di bottiglia nel grafo attraverso cui deve passare troppa informazione. Il sintomo è che le dipendenze a lungo raggio non arrivano, anche con abbastanza layer. I rimedi agiscono sul grafo o sul canale: rewiring, virtual node, attenzione globale.

La verifica rapida: se aggiungere layer peggiora, è over-smoothing. Se aggiungere layer non basta mai perché l'informazione lontana non arriva comunque, è over-squashing. E se il problema è semplicemente che con L layer non si raggiungono nodi a distanza maggiore di L , non è nessuna delle due: è *under-reaching*, ed è l'unica delle tre che si risolve davvero aggiungendo layer — pagando però in over-smoothing.

28.4.3 Under-reaching

Con L layer si raggiungono solo i nodi a distanza $\leq L$. Per dipendenze a lungo raggio servono molti layer (che amplificano over-smoothing/squashing) o attention globale.

28.5 Modelli randomized: Reservoir per grafi

Estensione delle Echo State Network (Parte III, RNN) ai grafi: i pesi di message passing sono fissi (random, contractive), si addestra solo il readout lineare:

$$\mathbf{h}(v) = \tanh\left(\mathbf{V}\mathbf{x}_v + \sum_{u \in N(v)} \mathbf{W}\mathbf{h}(u)\right),$$

con $\rho(\mathbf{W}) < 1$. Training closed-form (regressione ridge). Vantaggi: addestramento veloce, pochi iperparametri. Limiti: meno espressivo di un GNN trained end-to-end.

Quando usare quale GNN

Grafo omofilo (vicini simili)

GCN o GraphSAGE.

Bisogno di pesare i vicini in modo adattivo

GAT.

Massima espressività (chimica computazionale)

GIN o MPNN custom.

Long-range dependencies

Graph Transformer (GPS).

Dataset piccoli / training rapido

Graph Echo State Network.

Grafi con archi tipizzati (knowledge graph)

R-GCN.

Task graph-level con gerarchia

DiffPool o Top-K Pooling.

Connessioni

- Message passing $\leftrightarrow$ Belief Propagation: la stessa idea di aggregazione locale dei messaggi (Parte I, reti Bayesiane). La differenza: in PGM i messaggi sono distribuzioni, in GNN sono embedding.
- GAT $\leftrightarrow$ self-attention del Transformer (Parte III): stessa formula, ristretta ai vicini.
- GNN $\leftrightarrow$ CNN: una CNN è un GNN su grafo regolare (griglia). La GCN generalizza la convoluzione a grafi.
- Graph generation $\leftrightarrow$ modelli generativi (Parte IV): graph VAE, graph diffusion (fuori esame).

Domande d'esame — Graph attention, transformer e patologie

1. Scrivi le equazioni del GAT. Qual è la differenza concettuale fra l'attenzione di un GAT e la self-attention di un Transformer?
2. Quando preferiresti un GAT a una GCN, e viceversa? Che cosa guadagni e che cosa perdi.

3. Un GAT è più espressivo di un GIN sul test 1-WL? Motiva la risposta.
4. A che cosa serve il multi-head in un GAT, e perché nei layer intermedi si concatena mentre nell'ultimo si media?
5. Che cos'è un Graph Transformer? Perché serve un positional encoding *strutturale* e che forme può prendere?
6. Come si fa pooling gerarchico in una GNN? Spiega DiffPool.
7. Che cos'è l'*over-smoothing*? Da che cosa è causato a livello spettrale e come si mitiga.
8. Che cos'è l'*over-squashing*? Distingilo con precisione dall'*over-smoothing* e di' come un'attenzione globale lo attenua.
9. Che cos'è l'*under-reaching*? Perché non si risolve semplicemente aggiungendo layer.
10. Che cosa sono i modelli reservoir per grafi? Che cosa si addestra e che cosa resta fisso, e quale condizione deve soddisfare $\mathbf{W}$.
11. Il denoiser di GrIDDD (Parte VII) è un graph transformer: le patologie di questo capitolo valgono anche lì? Che cosa succede a un nodo appena inserito, che ha pochissimi archi?

Part VI

Reinforcement Learning

Chapter 29

Fondamenti di Reinforcement Learning

Risorse video — GDL34 — Reinforcement Learning

Videolezione del corso: registrazione della lezione GDL34 nella cartella *Recordings* del team del corso (v. Prefazione).

Reinforcement Learning: Machine Learning Meets Control Theory (*Steve Brunton, 30m*)

Mezz'ora che inquadra MDP, value function e la tassonomia model-based / model-free dalla prospettiva del controllo. Ottimo come primo passaggio, prima di entrare nelle equazioni di Bellman.

29.1 Posizionamento

Il Reinforcement Learning (RL) differisce dagli altri paradigmi ML:

- **Nessun supervisore:** solo un segnale di reward scalare R_t .
- **Feedback ritardato e asincrono:** una scelta al tempo t può portare reward dopo k step.
- **Sequenzialità:** i dati non sono iid; le azioni dell'agente influenzano la distribuzione dei dati futuri.
- **Non-stazionarietà:** la policy in via di apprendimento cambia la distribuzione.

Reward hypothesis. Tutti gli obiettivi possono essere descritti come massimizzazione del reward cumulativo atteso.

29.2 Markov Decision Process

Definizione 29.1 (MDP). Un *Markov Decision Process* è una tupla $(\mathcal{S}, \mathcal{A}, P, \mathcal{R}, \gamma)$ dove

- $\mathcal{S}$ insieme finito degli stati,
- $\mathcal{A}$ insieme finito delle azioni,
- $P_{ss'}^a = P(S_{t+1} = s' \mid S_t = s, A_t = a)$ matrice di transizione,

- $\mathcal{R}_s^a = \mathbb{E}[R_{t+1} \mid S_t = s, A_t = a]$ funzione reward,
- $\gamma \in [0, 1]$ fattore di sconto.

29.2.1 Ritorno e value functions

Definizione 29.2 (Ritorno).

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}.$$

Definizione 29.3 (Policy). Una policy π è una distribuzione condizionata $\pi(a|s) = P(A_t = a \mid S_t = s)$. Per policy deterministiche scriviamo $a = \pi(s)$.

Definizione 29.4 (State-value e action-value).

$$v_\pi(s) = \mathbb{E}_\pi[G_t \mid S_t = s] \quad (29.1)$$

$$q_\pi(s, a) = \mathbb{E}_\pi[G_t \mid S_t = s, A_t = a] \quad (29.2)$$

Relazione:

$$v_\pi(s) = \sum_{a \in \mathcal{A}} \pi(a|s) q_\pi(s, a).$$

Intuizione — l'equazione di Bellman è la chain rule del futuro

Tutto il RL poggia su un'osservazione quasi banale: il valore di uno stato è la ricompensa immediata più il valore scontato di dove ci si ritrova dopo.

$$V^\pi(s) = \mathbb{E}[r + \gamma V^\pi(s')].$$

L'osservazione è banale, la conseguenza non lo è: una quantità definita su traiettorie *infinite* viene espressa in termini di sé stessa a un passo di distanza. È esattamente la stessa mossa della programmazione dinamica di Viterbi nella Parte II — una ricorsione che sostituisce un'enumerazione esponenziale.

Il ruolo di γ , che spesso si liquida come “preferiamo il presente”: serve anche a garantire che la somma converga su orizzonte infinito, e rende l'operatore di Bellman una contrazione di fattore γ in norma infinito. È da lì che discende la convergenza di value iteration: si sta iterando una contrazione, quindi per il teorema del punto fisso di Banach converge, e converge in modo geometrico.

La differenza fra V e Q , e perché quasi tutti gli algoritmi usano Q : da V non si può estrarre una politica senza conoscere le probabilità di transizione, perché bisognerebbe calcolare $\mathbb{E}_{s'}[V(s')]$ per ogni azione. Da Q sì: $\pi(s) = \arg \max_a Q(s, a)$, senza sapere nulla della dinamica. È questo che rende Q -learning *model-free*.

29.2.2 Equazioni di Bellman

Teorema 29.1 (Bellman Expectation Equation).

$$v_\pi(s) = \sum_a \pi(a|s) \left[\mathcal{R}_s^a + \gamma \sum_{s'} P_{ss'}^a v_\pi(s') \right] \quad (29.3)$$

$$q_\pi(s, a) = \mathcal{R}_s^a + \gamma \sum_{s'} P_{ss'}^a \sum_{a'} \pi(a'|s') q_\pi(s', a') \quad (29.4)$$

Proof. $v_\pi(s) = \mathbb{E}_\pi[R_{t+1} + \gamma G_{t+1} \mid S_t = s]$. Espandendo l'aspettativa su a, s' : $\mathbb{E}_\pi[R_{t+1} \mid S_t = s] = \sum_a \pi(a|s) \mathcal{R}_s^a$ e $\mathbb{E}_\pi[G_{t+1} \mid S_t = s] = \sum_a \pi(a|s) \sum_{s'} P_{ss'}^a v_\pi(s')$. $\square$

Definizione 29.5 (Optimal value function).

$$v_*(s) = \max_{\pi} v_{\pi}(s), \quad q_*(s, a) = \max_{\pi} q_{\pi}(s, a).$$

Teorema 29.2 (Bellman Optimality).

$$v_*(s) = \max_{a \in \mathcal{A}} \left[\mathcal{R}_s^a + \gamma \sum_{s'} P_{ss'}^a v_*(s') \right] \quad (29.5)$$

$$q_*(s, a) = \mathcal{R}_s^a + \gamma \sum_{s'} P_{ss'}^a \max_{a'} q_*(s', a'). \quad (29.6)$$

Osservazione 29.1 (Politica ottimale deterministica). Per ogni MDP finito esiste una policy ottimale deterministica:

$$\pi_*(s) = \arg \max_{a \in \mathcal{A}} q_*(s, a).$$

Domande d'esame — Fondamenti di RL

1. Definisci un MDP $(\mathcal{S}, \mathcal{A}, P, R, \gamma)$. Che cosa significa esattamente “Markov” qui, e su che cosa è un’assunzione forte?
2. Che cosa fa γ ? Dai le due giustificazioni, quella economica e quella matematica.
3. Definisci il ritorno G_t , la state-value $v_{\pi}(s)$ e la action-value $q_{\pi}(s, a)$. Che relazione lega le due value function?
4. Deriva la Bellman expectation equation per $v_{\pi}(s)$ partendo dalla definizione di ritorno.
5. Differenza fra Bellman expectation e Bellman optimality. Perché la seconda non è lineare?
6. Perché l’operatore di Bellman è una contrazione, e che cosa se ne deduce sulla convergenza di value iteration?
7. Perché quasi tutti gli algoritmi model-free lavorano su Q e non su V ? Che cosa servirebbe per estrarre una politica da V ?
8. Che cos’è una politica ottima, e perché ne esiste sempre almeno una deterministica in un MDP finito?

Chapter 30

Algoritmi RL

Risorse video — GDL34 — Algoritmi di RL

Videolezione del corso: registrazione della lezione GDL34, seconda parte nella cartella *Recordings* del team del corso (v. Prefazione).

Reinforcement Learning: Machine Learning Meets Control Theory (*Steve Brunton, 30m*)

La tassonomia model-based / model-free e il posizionamento di Q-learning, policy gradient e actor-critic, senza entrare nei dettagli implementativi. Buona mappa da tenere accanto mentre si studiano le formule.

30.1 Tassonomia

Model-based $P, \mathcal{R}$ noti $\Rightarrow$ planning (policy iteration, value iteration).

Model-free $P, \mathcal{R}$ ignoti $\Rightarrow$ apprendimento da esperienza (Monte Carlo, TD-learning, Q-learning).

Value-based si apprende v o q , la policy è derivata greedy.

Policy-based si parametrizza direttamente π .

Actor-critic entrambi.

30.2 Model-based: Policy Iteration

Dato π :

1. *Policy evaluation*: iterazione fino a convergenza di (29.3):

$$v_{k+1}(s) \leftarrow \sum_a \pi(a|s) \left[\mathcal{R}_s^a + \gamma \sum_{s'} P_{ss'}^a v_k(s') \right].$$

2. *Policy improvement*: $\pi'(s) = \arg \max_a q_\pi(s, a)$.

3. Ripeti fino a $\pi' = \pi$.

Teorema 30.1 (Policy improvement). Se π' è greedy rispetto a v_π , allora $v_{\pi'}(s) \geq v_\pi(s)$ per ogni s . Iterando si converge alla policy ottimale π_* .

30.3 Model-based: Value Iteration

Si applica direttamente Bellman optimality come aggiornamento:

$$v_{k+1}(s) \leftarrow \max_a \left[\mathcal{R}_s^a + \gamma \sum_{s'} P_{ss'}^a v_k(s') \right].$$

A convergenza, $v_k \rightarrow v_*$ e $\pi_*(s) = \arg \max_a [\dots]$.

Confronto. Policy iteration: due loop annidati ma converge in pochi step esterni. Value iteration: un solo loop ma molti step prima di policy stabile.

30.4 Model-free: Temporal Difference e Q-learning

In assenza di $P, \mathcal{R}$ si usano *sample updates*: l'agente esplora, osserva transizioni (S, A, R, S') e aggiorna stime.

30.4.1 TD(0) per value

$$v(S_t) \leftarrow v(S_t) + \alpha [R_{t+1} + \gamma v(S_{t+1}) - v(S_t)].$$

Il termine in parentesi è il *TD error*: differenza fra target $R + \gamma v(S')$ e stima corrente.

30.4.2 Q-learning (off-policy)

$$Q(S, A) \leftarrow Q(S, A) + \alpha \left[R + \gamma \max_{a'} Q(S', a') - Q(S, A) \right]. \quad (30.1)$$

Target policy (quella che impariamo) $\pi(s) = \arg \max_a Q(s, a)$, *behavior policy* (quella che esegue azioni durante esplorazione) ϵ -greedy:

$$\pi'(a|s) = \begin{cases} \epsilon/|\mathcal{A}| + (1 - \epsilon) & a = \arg \max_{a'} Q(s, a') \\ \epsilon/|\mathcal{A}| & \text{altrimenti.} \end{cases}$$

Off-policy perché si apprende Q della greedy policy mentre si esegue l' ϵ -greedy.

Teorema 30.2 (Convergenza Q-learning, Watkins & Dayan 1992). Sotto condizioni standard (visite infinite di ogni (s, a) , learning rate decrescente $\sum \alpha = \infty$, $\sum \alpha^2 < \infty$), Q converge a q_* con probabilità 1.

Esempio svolto — lo stesso passo, Q-learning e SARSA a confronto

Uno stesso istante di esperienza: dallo stato s si prende l'azione a , si riceve $r = 1$ e si finisce in s' . Sia $\alpha = 0.1$, $\gamma = 0.9$, $Q(s, a) = 2.0$. Nello stato s' :

$$\max_{a'} Q(s', a') = 3.0 \quad (\text{azione golosa}), \quad Q(s', a_{\text{presa}}) = 1.0 \quad (\text{azione effettivamente scelta, esplorativa}).$$

Q-learning (off-policy). Usa il massimo, cioè quello che farebbe la politica *golosa*, non quella che sta guidando:

$$Q(s, a) \leftarrow 2.0 + 0.1(1 + 0.9(3.0) - 2.0) = 2.0 + 0.1(1.7) = \mathbf{2.17}.$$

SARSA (on-policy). Usa il valore dell'azione che è stata *davvero* presa:

$$Q(s, a) \leftarrow 2.0 + 0.1(1 + 0.9(1.0) - 2.0) = 2.0 + 0.1(-0.1) = \mathbf{1.99}.$$

Stessa transizione, aggiornamenti di segno opposto.

Che cosa significa la differenza. Q-learning stima il valore della politica ottima *indipendentemente* da come si sta esplorando: impara la politica migliore anche seguendo una peggiore. SARSA stima il valore della politica che si sta effettivamente eseguendo, esplorazione inclusa.

L'esempio classico è il *cliff walking*: lungo un burrone, Q-learning impara il cammino ottimo rasente il bordo, perché nel calcolo assume che poi si comporterà in modo goloso. Ma se durante l'esecuzione si continua a esplorare con ϵ -greedy, ogni tanto si cade. SARSA impara un cammino più lungo e più lontano dal bordo, perché il valore che stima tiene conto del rischio dell'esplorazione — e nella pratica, con esplorazione attiva, ottiene un ritorno migliore.

La formula generale: il TD error $\delta = r + \gamma \hat{V}(s') - \hat{V}(s)$ è lo stesso in entrambi; cambia solo che cosa si mette al posto di $\hat{V}(s')$. Bootstrapping (usare una stima per aggiornare un'altra stima) è ciò che distingue il TD dai metodi Monte Carlo, che aspettano la fine dell'episodio: il TD ha bias ma varianza molto più bassa, e aggiorna online.

30.4.3 SARSA (on-policy)

$$Q(S, A) \leftarrow Q(S, A) + \alpha[R + \gamma Q(S', A') - Q(S, A)],$$

con A' campionato dalla policy corrente. Più conservativo di Q-learning ma robusto in ambienti rischiosi.

30.5 Function approximation

Per spazi di stato grandi (es. pixel) non si può tabulare Q . Si parametrizza $\hat{Q}(s, a; \mathbf{w})$ con una rete neurale.

30.5.1 Deep Q-Network (DQN, Mnih et al. 2015)

$$\mathcal{L}_i(\mathbf{w}_i) = \mathbb{E}_{(s,a,r,s') \sim \mathcal{D}_i} \left[\left(r + \gamma \max_{a'} Q(s', a'; \mathbf{w}_i^-) - Q(s, a; \mathbf{w}_i) \right)^2 \right]. \quad (30.2)$$

Tre trucchi cruciali:

Experience replay buffer $\mathcal{D}$ di transizioni passate; campionamento iid per rompere correlazione temporale.

Target network copia $\mathbf{w}^-$ aggiornata ogni C step; stabilizza il target.

Reward clipping $r \in [-1, 1]$ per stabilità.

DQN ha imparato a giocare 49 giochi Atari da soli pixel + reward, una stessa architettura su tutti.

30.6 Policy gradient

Alternativa: parametrizzare direttamente la policy $\pi_\theta(a|s)$.

Obiettivo.

$$J(\theta) = \mathbb{E}_{\pi_\theta} \left[\sum_{t=0}^{\infty} \gamma^t R_{t+1} \right].$$

Teorema 30.3 (Policy Gradient Theorem, Sutton et al. 2000).

$$\boxed{\nabla_\theta J(\theta) = \mathbb{E}_{\pi_\theta} [\nabla_\theta \log \pi_\theta(a|s) Q^{\pi_\theta}(s, a)]}. \quad (30.3)$$

Sketch. Per la log-derivative trick: $\nabla_\theta \pi_\theta = \pi_\theta \nabla_\theta \log \pi_\theta$. Espandendo $J(\theta)$ come somma di reward attesa lungo traiettorie e differenziando si arriva a (30.3). $\square$

30.6.1 REINFORCE

1. Generare una traiettoria $(s_0, a_0, r_1, \dots, s_T)$ da π_θ ;
2. Calcolare $G_t = \sum_{k=t}^T \gamma^{k-t} r_{k+1}$;
3. Update: $\theta \leftarrow \theta + \alpha \sum_t \nabla_\theta \log \pi_\theta(a_t|s_t) G_t$.

Stima Monte Carlo, alta varianza.

Intuizione — value-based e policy-based rispondono a due domande diverse

I metodi value-based (Q -learning, DQN) imparano “quanto vale ogni azione” e poi ricavano la politica prendendo l’arg max. I metodi policy-based (REINFORCE, PPO) imparano direttamente “che cosa fare”, parametrizzando $\pi_\theta(a | s)$ e salendo il gradiente del ritorno atteso.

La differenza pratica si vede in tre punti.

Azioni continue: l’arg max su uno spazio continuo è a sua volta un problema di ottimizzazione, da risolvere a ogni passo. Una politica parametrica invece emette direttamente l’azione (o i parametri di una distribuzione su di essa). È la ragione per cui il controllo continuo è dominio dei metodi policy-based.

Politiche stocastiche: l’arg max è deterministico per costruzione. Ma in ambienti parzialmente osservabili, o in giochi dove essere prevedibili è fatale, l’ottimo è stocastico. Solo i metodi policy-based possono rappresentarlo.

Stabilità: un cambiamento infinitesimo nei valori Q può ribaltare l’arg max e quindi cambiare la politica di colpo. Il gradiente di policy la muove con continuità, il che rende la convergenza più regolare — al prezzo di convergere tipicamente a un ottimo locale, e con varianza del gradiente molto più alta.

Actor-critic è la sintesi: l’attore è una politica parametrica (quindi gestisce azioni continue e stocasticità), il critico è una value function che fornisce una stima a bassa varianza del ritorno al posto del ritorno campionato. Si tiene la flessibilità dei policy gradient e si recupera parte della stabilità dei metodi value-based.

Esempio svolto — la baseline: perché toglie varianza senza introdurre bias

Tre traiettorie con ritorni $R = (10, 12, 11)$. Tutte e tre positive.

Senza baseline. Il gradiente REINFORCE è $\nabla_\theta \log \pi_\theta(a|s) R$. Poiché tutti gli R sono positivi, *tutte* le azioni vengono rese più probabili — anche la peggiore delle tre. La distinzione fra buono e cattivo si gioca solo sulle differenze relative dei moduli, sepolte in un segnale grande.

La grandezza che governa la varianza dello stimatore è $\mathbb{E}[R^2]$:

$$\mathbb{E}[R^2] = \frac{100 + 144 + 121}{3} = 121.7.$$

Con baseline $b = \bar{R} = 11$. Gli advantage diventano

$$A = R - b = (-1, +1, 0).$$

Ora il segno è informativo: la prima azione viene resa *meno* probabile, la seconda più probabile, la terza lasciata stare. E

$$\mathbb{E}[A^2] = \frac{1 + 1 + 0}{3} = 0.667,$$

cioè un fattore **182** in meno sulla scala che governa la varianza.

Perché non introduce bias. Il termine sottratto ha valore atteso nullo:

$$\mathbb{E}_{a \sim \pi_\theta} [\nabla_\theta \log \pi_\theta(a|s) b(s)] = b(s) \sum_a \pi_\theta(a|s) \frac{\nabla_\theta \pi_\theta(a|s)}{\pi_\theta(a|s)} = b(s) \nabla_\theta \underbrace{\sum_a \pi_\theta(a|s)}_{=1} = 0.$$

Il passaggio chiave è che b non dipende dall'azione, quindi esce dalla somma; e la somma delle probabilità è costante, quindi il suo gradiente è nullo. Qualunque funzione del solo stato può fare da baseline.

La scelta naturale è $b(s) = V^\pi(s)$, da cui l'advantage $A^\pi(s, a) = Q^\pi(s, a) - V^\pi(s)$: “quanto questa azione è migliore della media in questo stato”. Stimare V con una rete è precisamente il ruolo del critico in actor-critic.

30.6.2 Riduzione di varianza: baseline e advantage

Si sottrae una baseline $b(s)$ che non dipende da a :

$$\nabla_\theta J(\theta) = \mathbb{E}[\nabla_\theta \log \pi_\theta(a|s) (Q^\pi(s, a) - b(s))].$$

Tipica scelta: $b(s) = v^\pi(s)$, che dà l'advantage $A^\pi(s, a) = Q^\pi(s, a) - v^\pi(s)$.

30.6.3 Vantaggi/svantaggi policy-based

Pro: convergenza migliore, gestione spazi azione continui, policy stocastiche.

Contro: minimo locale, alta varianza, policy evaluation costosa.

30.6.4 Policy gradient deterministico (DPG)

Per azione continua $a = \pi_\theta(s)$:

$$\nabla_\theta J(\theta) = \mathbb{E} \left[\nabla_a Q^\pi(s, a) \Big|_{a=\pi_\theta(s)} \nabla_\theta \pi_\theta(s) \right].$$

30.7 Actor-Critic

Combina:

Critic $Q_w(s, a)$ o $V_w(s)$, addestrato via TD;

Actor $\pi_\theta(a|s)$, addestrato via policy gradient con il critic come stima di Q .

Update simultaneo:

$$\begin{aligned}\delta &= r + \gamma V_{\mathbf{w}}(s') - V_{\mathbf{w}}(s) \quad (\text{TD error/advantage}) \\ \mathbf{w} &\leftarrow \mathbf{w} + \alpha_w \delta \nabla_{\mathbf{w}} V_{\mathbf{w}}(s) \\ \theta &\leftarrow \theta + \alpha_\theta \delta \nabla_{\theta} \log \pi_\theta(a|s).\end{aligned}$$

Riduce la varianza di REINFORCE mantenendo bias basso.

30.7.1 Algoritmi notevoli

A3C / **A2C** asynchronous / synchronous actor-critic, parallel workers.

DDPG / **TD3** DPG con replay buffer e target network (action continuous).

PPO proximal policy optimization, clip del ratio $\pi_\theta/\pi_{\theta_{\text{old}}}$ per stabilità.

SAC soft actor-critic, entropy regularization.

PPO objective.

$$\mathcal{L}^{\text{CLIP}}(\theta) = \mathbb{E}_t \left[\min(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t) \right],$$

con $r_t(\theta) = \pi_\theta(a_t|s_t)/\pi_{\theta_{\text{old}}}(a_t|s_t)$. È diventato lo standard de facto in policy-based deep RL.

Quando usare quale algoritmo RL

Spazio di stato e azione piccolo, MDP noto
policy iteration / value iteration.

Spazio finito, model-free, azioni discrete
Q-learning, SARSA.

Spazio grande, action discreta (Atari, gioco)
DQN.

Spazio continuo (robotica)
DDPG / TD3 / SAC.

Stabilità e semplicità
PPO.

Esplorazione complicata, sparse reward
tecniche avanzate (intrinsic motivation, model-based con AlphaZero-style planning).

Connessioni

- Bellman equations $\leftrightarrow$ programmazione dinamica.
- Q-learning $\leftrightarrow$ TD-learning $\leftrightarrow$ stochastic approximation.
- Policy gradient $\leftrightarrow$ score function estimator (log-derivative trick): stessa identità usata nei VAE per gradienti attraverso campionamento.
- Actor-critic $\leftrightarrow$ GAN: due reti, una stima valori, l'altra agisce, addestrate alterna-

tivamente (anche se la struttura adversarial non è esattamente la stessa).

- RL su grafi $\leftrightarrow$ Parte V: graph RL per molecular design, combinatorial optimization.

Domande d'esame — RL

1. Confronta policy iteration e value iteration: che cosa itera ciascuno, quale converge in meno iterazioni e quale ha passi più costosi.
2. Che differenza c'è fra metodi Monte Carlo e metodi Temporal Difference? Che cos'è il bootstrapping e che compromesso bias-varianza introduce.
3. Scrivi l'aggiornamento di Q-learning. Perché è *off-policy*?
4. Differenza fra Q-learning e SARSA. Quando preferiresti uno o l'altro?
5. Cos'è il *deadly triad*? (function approximation + bootstrapping + off-policy $\rightarrow$ instabilità) Come DQN mitiga?
6. Enuncia il policy gradient theorem e ricavalo via log-derivative trick.
7. Cos'è REINFORCE? Perché ha alta varianza e come si riduce?
8. Architettura actor-critic: ruolo di actor e critic. Vantaggio rispetto a REINFORCE puro.
9. PPO: cos'è il clipped objective e perché stabilizza il training?
10. Confronta value-based, policy-based e actor-critic: quando serve l'uno e quando l'altro (azioni continue, politiche stocastiche, stabilità).
11. Il policy gradient stima un gradiente attraverso un campionamento, esattamente come il VAE (Cap. 20). Perché nel VAE si può usare il reparametrization trick e nel policy gradient no? (score function estimator contro pathwise estimator.)

Part VII

Il paper del quarto assignment: GrIDDD

Questa parte non corrisponde a una lezione del corso: tratta il paper oggetto del quarto assignment. **Correzione rispetto alla prima stesura:** qui era scritto che l'orale “tipicamente prende avvio” dal paper. Il resoconto di un appello reale (Appendice C) mostra che in sei esami consecutivi il paper *non è mai stato chiesto*. Resta da chiarire se quei candidati avessero già presentato separatamente il quarto assignment; finché non è chiaro, questa parte va considerata materiale da conoscere ma non il punto di partenza probabile. Il paper è utile anche come banco di prova trasversale, perché tocca contemporaneamente la Parte IV (diffusion, classifier-free guidance), la Parte V (message passing e graph transformer) e la Parte II (catene di Markov, stati assorbenti, ELBO).

Chapter 31

GrIDDD: Graph Diffusion that can Insert and Delete

Fonti

M. Ninniri, M. Podda, D. Bacciu, *Graph Diffusion that can Insert and Delete*, 39th Conference on Neural Information Processing Systems (NeurIPS 2025). Preprint arXiv:2506.15725v2, 30 ottobre 2025.

Codice: <https://github.com/mninniri/GrIDDD>

Materiali locali: `midterms/fourth/GrIDDD_paper.pdf`, il poster `GrIDDD_poster.pdf`, e i due riassunti in `utils` and `paper/`. **Bacciu è coautore**, quindi se il paper viene toccato è ragionevole che le domande scavino da lì verso il programma. Ma si veda la correzione in apertura di parte: nell'appello osservato non è mai comparso.

31.1 Il problema: i diffusion model su grafi hanno taglia fissa

Un DDPM discreto su grafi molecolari (DiGress, MiDi, FreeGress) parte da un grafo latente rumoroso e lo denoizza fino a una molecola valida. Ma il numero di nodi va deciso *prima* di iniziare la diffusione, e da quel momento non cambia più: il processo agisce sui *tipi* di atomo e di legame, non sulla cardinalità del grafo.

Intuizione — perché la taglia fissa è un problema per le molecole e non per le immagini

Per un'immagine la risoluzione è una scelta a monte: nessuno chiede a un modello di generare “un'immagine il cui numero di pixel sia ottimale per la proprietà richiesta”. Il numero di pixel non è una proprietà dell'oggetto, è una proprietà del formato.

Per una molecola no. Molte proprietà chimiche **correlano direttamente col numero di atomi**: il peso molecolare in modo quasi definitorio, ma anche LogP e QED in modo forte. Se il target è “peso molecolare 250”, la taglia del grafo non è un iperparametro da fissare da fuori: è parte della risposta.

La differenza si vede meglio confrontando i due task del paper. Nel *property targeting* si genera da zero una molecola con una proprietà voluta: qui si può ancora barare, indovinando la taglia con un modello ausiliario e poi generando a taglia fissa. Nella *property optimization* si parte da una molecola esistente e la si modifica per migliorarne una proprietà mantenendo una similarità minima: qui la taglia è legata alla struttura che si sta modificando, e fissarla dall'esterno significa vietarsi metà dello spazio delle

soluzioni.

Il claim del paper in una frase: rendere la taglia del grafo una variabile del processo di diffusione, permettendo inserimenti e cancellazioni *monotone* di nodi sia nel forward sia nel reverse, e incorporandoli *durante il training* anziché come post-processing.

31.2 Che cosa facevano gli altri, e perché non basta

DiGress, MiDi

campionano il numero di nodi dalla distribuzione empirica delle taglie del training set. La taglia è quindi indipendente dal condizionamento.

FreeGress

(stesso gruppo, lavoro precedente) predice la taglia con un classificatore ausiliario a partire dall'informazione di condizionamento. Meglio, ma resta una decisione presa *prima* e mai rivista.

Entrambe le strategie aggirano il problema invece di risolverlo, e la differenza si vede nei numeri: su peso molecolare FreeGress dimezza il proprio errore *grazie* alla rete che indovina la taglia, mentre GrIDDD fa meglio senza conoscere a priori la distribuzione delle taglie.

31.3 Lo schema degli edit: quanti, e quando

Sia G^0 il grafo dei dati con n^0 nodi e G^T il latente con n^T nodi. Il numero di operazioni di edit è

$$|\Delta^T| = n^T - n^0,$$

con $\Delta^T > 0$ inserimenti (nel forward), $\Delta^T < 0$ cancellazioni, $\Delta^T = 0$ il caso classico. Gli edit sono **monotoni**: un nodo inserito non viene poi cancellato, e viceversa.

Il *quando* è governato dal valore assoluto della densità di una distribuzione logistica, normalizzata in modo che $\sum_{t=0}^T \zeta'(t) = 1$ e $\zeta'(0) = \zeta'(T) = 0$:

$$\zeta'(t) = \left| \frac{e^{(t-D)/w}}{w (e^{(t-D)/w} + 1)^2} \right|.$$

D indica il timestep in cui gli edit sono massimizzati, w la ripidità della curva. Nel paper $T = 500$, $D = T/2 = 250$, $w = 0.05$: la gran parte degli inserimenti e delle cancellazioni avviene a *metà* del processo di diffusione. L'integrale $\zeta(t)$ è la cumulativa corrispondente e compare dentro le matrici di transizione modificate.

Intuizione — perché gli edit stanno in mezzo e non agli estremi

Inserire un nodo quando il grafo è quasi pulito significa piazzare un atomo estraneo dentro una struttura già formata, che il denoising residuo non ha il tempo di riassorbire. Cancellarne uno in quel momento apre un buco che nessuno richiude.

All'estremo opposto, agire quando il grafo è già rumore puro è inutile: si sta modificando la cardinalità di un oggetto che non ha ancora nessuna struttura, e l'informazione si perde nei passi successivi.

Il punto utile è a metà, dove esiste già una struttura grossolana da rispettare ma restano abbastanza passi di denoising per integrare la modifica. È lo stesso ragionamento che governa le schedule del rumore nel Cap. 23: i primi passi preservano la semantica, gli ultimi rifiniscono, e le decisioni strutturali vanno prese quando c'è ancora margine per

correggerle.

31.4 Forward process: tre casi

31.4.1 Caso base ($\Delta^T = 0$): il forward ereditato

Come in DiGress e FreeGress, le quantità sono categoriche e il rumore è una *matrice di transizione* anziché un'aggiunta gaussiana:

$$q(G^t | G^{t-1}) = (X^{t-1}Q_X^t, E^{t-1}Q_E^t), \quad Q_X^t = \alpha^t A_X + (1 - \alpha^t)B_X,$$

con $A_X = \mathbf{I}$ e $B_X = \mathbf{1} \mathbf{m}_X^\top$, dove $\mathbf{m}_X$ è la distribuzione marginale dei tipi di atomo nel training set. Le multi-step si compongono come $Q_X^{t|s} = \prod_{i=s+1}^t Q_X^i = \alpha^{t|s} A_X + (1 - \alpha^{t|s})B_X$, con schedule coseno alla MiDi ($\nu = 1$ per i nodi, $\nu = 1.5$ per gli archi).

Intuizione — nel discreto, il “rumore” è la marginale del dataset

Vale la pena fermarsi su questo punto, perché è il primo posto in cui il diffusion discreto si stacca da quello gaussiano.

Con $\alpha^t \approx 1$ si ha $Q_X^t \approx \mathbf{I}$: nulla cambia. Con $\alpha^t \approx 0$ si ha $Q_X^t \approx B_X$: ogni nodo viene rimpiazzato da un tipo campionato dalla marginale del training set. Quindi lo stato terminale non è “rumore bianco”, è una molecola con *composizione atomica plausibile* ma struttura completamente casuale — più carbonio e azoto che atomi rari, nelle proporzioni giuste.

È la scelta corretta per lo stesso motivo per cui nel continuo si sceglie $\mathcal{N}(0, \mathbf{I})$: si vuole una distribuzione terminale semplice e nota da cui partire a campionare. Solo che nel discreto “semplice e nota” vuol dire la marginale empirica, non l'uniforme — usare l'uniforme renderebbe il primo passo di reverse molto più difficile.

31.4.2 Cancellazioni monotone ($\Delta^T < 0$): il trucco DEL/DEL*

La cancellazione è modellata come un *tipo di atomo aggiuntivo* DEL, nello spirito dell'absorbing diffusion. Perché la monotonicità sia garantita, DEL deve essere uno stato **assorbente**:

$$p(x^t \neq \text{DEL} | x^{t-1} = \text{DEL}) = 0.$$

Intuizione — perché servono due stati e non uno

Qui c'è il passaggio più elegante del paper, ed è quasi certamente la domanda che arriva se si nomina il meccanismo.

Il vincolo è chiaro: DEL deve essere assorbente, altrimenti un nodo cancellato potrebbe “resuscitare” nel forward e la monotonicità salterebbe.

Ma un assorbente puro rompe il reverse. Il reverse percorre la catena al contrario, quindi deve poter *re-inserire* un nodo che il forward aveva cancellato. Se da DEL non si esce mai, quel nodo resta morto per sempre e il modello non può generare nulla che il forward avesse tolto.

La soluzione è spezzare la cancellazione in due stati. DEL* è **transiente**: un nodo ci passa attraverso e, nel forward, da lì scivola deterministicamente in DEL; nel reverse, invece, da DEL* può uscire verso un tipo di atomo proprio. DEL resta l'assorbente vero e proprio.

Detto in altri termini: DEL* è la *soglia* e DEL è la stanza chiusa. La monotonicità è garantita dalla stanza chiusa, la reversibilità dalla soglia. Un solo stato non può fare

entrambe le cose, perché le due proprietà sono in contraddizione diretta.

Per garantire che al tempo T siano in stato DEL *esattamente* $|\Delta^T|$ nodi, si usa uno schema ibrido: solo $|\Delta^T|$ nodi seguono la matrice di transizione speciale

$$Q^{*t} = \zeta(t)(\alpha^t A^* + (1 - \alpha^t) B^*) + (1 - \zeta(t)) C^*,$$

mentre tutti gli altri seguono il forward standard. Le matrici A^*, B^*, C^*, D^* codificano la logica delle transizioni verso DEL/DEL*; poiché ζ è una cumulativa, a $t = T$ si ha $Q^{*T} = C^*$.

31.4.3 Inserimenti monotoni ($\Delta^T > 0$)

Concettualmente più semplici: un nodo viene *attivato* al timestep $s + 1$ se il modello ha campionato s come tempo di inserimento. La categoria iniziale del nodo si campiona dalla marginale $\mathbf{m}_X$, gli archi incidenti dalla marginale $\mathbf{m}_E$.

31.5 Il posterior generalizzato

Esempio svolto — perché il posterior classico non è definito, e come lo si aggiusta

Nel Cap. 23 il training si regge su un fatto preciso: la reverse vera $q(z_{t-1} | z_t)$ è intrattabile, ma *condizionandola su x^0* diventa trattabile, ed è su $q(z_{t-1} | z_t, x^0)$ che si costruisce l'ELBO.

Dove si rompe in GrIDDD. Un nodo inserito durante il forward al tempo $\hat{s}$ *non esiste* in G^0 . Per quel nodo la quantità x^0 non è definita: non c'è nessun valore originale su cui condizionare, perché il nodo non aveva un originale. Il termine $p_\theta(x^{t-1} | x^t, x^0)$ è quindi mal posto per una parte dei nodi.

La riparazione. Si sostituisce il condizionamento su x^0 con una marginalizzazione sul *tempo di attivazione* $\hat{s}$ del nodo:

$$p_\theta(x^{t-1} | x^t, y) = \sum_{x \in \mathcal{X}} \frac{q(x^t | x^{t-1}) q(x^{t-1} | x^{\hat{s}} = x)}{q(x^t | x^{\hat{s}} = x)} p_\theta(x^{\hat{s}} = x | x^t, y).$$

Per un nodo esistente fin dall'inizio si ha $\hat{s} = 0$ e la formula collassa nel posterior classico: la generalizzazione è conservativa, non sostituisce nulla di quanto già funzionava.

Il costo: la rete non predice più solo il tipo originale dei nodi e degli archi, ma anche *quando* ciascun nodo è stato attivato. È un'uscita in più della testa del denoiser, ed è per questo che la loss ha un termine dedicato.

31.6 Rete ausiliaria, loss, guidance

Per le cancellazioni serve una seconda rete $g_\phi(G^t)$, che predice quanti DEL* aggiungere al passo t prima di passare il grafo al denoiser principale. È addestrata in parallelo e *solo nei timestep in cui $\zeta'(t) > 0$* , cioè dove gli edit sono effettivamente possibili.

La loss è una somma di **quattro cross-entropy**, pesate da $\lambda_X, \lambda_E, \lambda_S, \lambda_{\text{DEL}}$:

1. sui tipi dei nodi X^{*0} ;
2. sui tipi degli archi E^{*0} ;
3. sui tempi di attivazione S^* ;

4. sul numero di DEL* predetti dalla rete ausiliaria.

La generazione condizionata usa la **classifier-free guidance** di Ho e Salimans, identica a quella del Cap. 23: durante il training, con probabilità ρ il vettore di condizionamento y viene sostituito da un placeholder parametrizzato $\bar{y}$ (conditional dropout), e a inferenza si combinano predizione condizionata e non condizionata pesando per λ . È lo stesso meccanismo di FreeGress: GrIDDD eredita la pipeline di condizionamento e ci innesta sopra gli edit di taglia.

31.7 Risultati

31.7.1 Property targeting

QM9	MAE μ ↓	validità	MAE HOMO ↓	validità
FreeGress	0.74 ± 0.08	83.7%	0.32 ± 0.04	90.1%
GrIDDD	0.66 ± 0.07	79.0%	0.37 ± 0.07	94%

ZINC-250k	LogP ↓	val.	QED ↓	val.	MW ↓	val.
DiGress	0.74 ± 0.08	74.6%	0.15 ± 0.01	85.1%	20.92 ± 2.90	40.4%
FreeGress	0.17 ± 0.01	84.9%	0.04 ± 0.01	84.9%	8.96 ± 1.93	79.7%
GrIDDD	0.19 ± 0.02	87.9%	0.04 ± 0.00	87.2%	4.89 ± 0.57	84.2%

Il caso da raccontare: il peso molecolare

Su LogP e QED GrIDDD è alla pari con FreeGress; su μ e HOMO il quadro è misto. **Su MW il divario è netto:** 4.89 contro 8.96, cioè quasi la metà dell'errore, con validità più alta.

Non è un risultato casuale, è il risultato per cui il modello è stato progettato. MW correla in modo quasi definitorio con il numero di atomi, quindi è il caso in cui la flessibilità di taglia conta di più. E l'aspetto decisivo è *come* ci arriva: FreeGress dimezza il proprio MAE su MW grazie a una rete ausiliaria che indovina la taglia ottimale dal target, mentre GrIDDD ottiene un errore inferiore **senza conoscere a priori la distribuzione delle taglie**.

31.7.2 Property optimization (ZINC-250k)

	LogP (sim ≥ 0.4)	LogP (sim ≥ 0.6)	QED	DRD2
CG-VAE	0.61 ± 1.09	0.25 ± 0.74	4.8%	—
GCPN	2.49 ± 1.30	0.79 ± 0.63	9.4%	4.4%
GrIDDD	2.70 ± 0.94	1.33 ± 0.61	45.1%	5.0%

Il 45.1% contro il 9.4% di GCPN su QED — quasi $5\times$ — è il numero più forte del paper, ed è esattamente il task in cui la taglia è vincolata alla struttura di partenza.

31.7.3 Out-of-distribution e ablation

Addestrando su QM9 (molecole fino a 9 atomi pesanti) ma forzando GrIDDD a inserire fino a 14 nodi, e chiedendo poi a entrambi i modelli di generare fino a 15 atomi: fino a 12 atomi GrIDDD e DiGress sono comparabili, oltre la validità di DiGress crolla mentre **GrIDDD**

mantiene il 35% di validità a 15 nodi — cioè oltre la propria capacità di training. Il modello ha imparato la flessibilità di taglia, non semplicemente a restare nel range visto.

L’ablation conferma l’attribuzione: disabilitando insert e delete, il success rate su QED scende da 45.1% a **33.8%**. Un baseline naive che simula gli edit con una classe PAD genera un 3% di molecole valide in più ma è nettamente peggiore su tutto il resto (numero di componenti connesse, cross-entropy).

Costo: circa il 30% più lento di FreeGress in training, per l’overhead delle due reti; in sampling spesso più veloce, perché può adattare il numero di passi al task.

31.8 Limiti dichiarati dagli autori

Sono due e precisi: non improvvisarne altri

1. **Le due reti possono entrare in conflitto.** Occasionalmente il modello principale predice un’attivazione e la rete ausiliaria una cancellazione nello stesso timestep, il che è formalmente illegale nel framework.
2. **Il modello tende a produrre molecole “spezzate”**, cioè con più componenti connesse rispetto a FreeGress e DiGress. La causa è strutturale: i nodi inseriti tardi nel processo hanno pochi archi e faticano a connettersi prima della fine del denoising. Gli autori suggeriscono di dare in input il numero di componenti connesse come feature.

Il secondo limite è anche un ottimo ponte verso la Parte V. Un nodo appena inserito ha grado quasi nullo, quindi nell’aggregazione di message passing riceve pochissimi messaggi: è un caso estremo di *under-reaching*, in cui l’informazione strutturale non ha né i vicini né i layer per raggiungerlo. Se la domanda va in questa direzione, è il collegamento da fare.

31.9 Dove si colloca nel programma

Connessioni

- **Tassonomia generativa** (Cap. 23): explicit $\rightarrow$ latent $\rightarrow$ variational, lo stesso ramo dei VAE, perché entrambi massimizzano un ELBO della log-verosimiglianza. La differenza dei diffusion è che lo spazio latente ha la *stessa dimensionalità dei dati*; la torsione di GrIDDD è che ora quella cardinalità può *cambiare durante il processo*.
- **Diffusion come VAE gerarchico con encoder fisso**: in GrIDDD l’encoder non è più del tutto fisso, perché la parte di edit di taglia è appresa. È la risposta pronta se la domanda arriva da lì.
- **Classifier-free guidance** (Cap. 23): identica, ereditata da FreeGress.
- **Stati assorbenti** (Parte II, catene di Markov): DEL assorbente più DEL* transiente.
- **Message passing e graph transformer** (Parte V): il denoiser è un graph transformer sulla scia di DiGress/MiDi. La scelta è motivata: i DDPM su grafi

raffinano iterativamente interazioni a lungo raggio come le chiusure di anelli, che un decoder one-shot dovrebbe imparare in una sola trasformazione.

- **Attention spectrum:** il graph transformer è l'ultimo anello della linea Bahdanau → self-attention → GAT applicata ai grafi.

31.10 Domande d'esame — GrIDDD

Livello 1 — sai raccontarlo

1. Racconta GrIDDD in tre minuti: problema, idea, risultato principale.
2. Qual è il problema concreto che risolve?
3. Perché la taglia fissa è un problema serio per le molecole e non, per esempio, per le immagini?
4. Come lo risolvevano prima DiGress, MiDi e FreeGress?
5. Perché quelle soluzioni bastano per il property targeting ma non per la property optimization?
6. Qual è il claim in una frase?
7. Su quali dataset è valutato, e qual è il numero più forte?

Livello 2 — sai come funziona

1. Che cosa vuol dire che gli edit sono *monotoni*?
2. Quanti edit servono e come si distribuiscono nel tempo? Che ruolo hanno D e w ?
3. Spiega il trucco DEL/DEL*: perché servono **due** stati e non uno?
4. Come si garantisce che esattamente $|\Delta^T|$ nodi siano in DEL al tempo T ?
5. Scrivi il forward standard ereditato da DiGress/FreeGress. Che cos'è il “rumore” nel caso discreto?
6. Come funzionano gli inserimenti, e perché sono più semplici delle cancellazioni?
7. Qual è il problema del reverse process, e come lo risolve il posterior generalizzato?
8. A che cosa serve la rete ausiliaria $g_\phi(G^t)$, e quando viene addestrata?
9. Da che cosa è composta la loss?
10. Come si fa generazione condizionata? È un meccanismo nuovo o ereditato?

Livello 3 — sai criticarlo e collocarlo

1. Colloca GrIDDD nella tassonomia generativa del corso.
2. Se un diffusion model è un VAE gerarchico con encoder fisso, che cosa cambia in GrIDDD?

3. Perché il caso MW è quello che racconta meglio il paper?
4. Discuti il trade-off MAE/validità su QM9: è un miglioramento uniforme?
5. Che cosa dimostra l'esperimento out-of-distribution?
6. Che cosa dice l'ablation, e perché è importante per l'attribuzione del risultato?
7. Perché il baseline naive con classe PAD non basta?
8. Quali sono i due limiti dichiarati dagli autori?
9. Quanto costa in training e in sampling?
10. Come cambieresti il paper? (Il conflitto fra le due reti si potrebbe evitare con un'unica testa che predice l'edit come variabile categorica a tre valori, invece di due reti indipendenti che possono contraddirsi.)
11. Se dovessi fare la stessa cosa con un flow matching su grafi invece che con un DDPM, che cosa cambierebbe? (Il path è deterministico e la cardinalità variabile diventa più scomoda: non c'è una catena di stati discreti su cui appoggiare uno stato assorbente.)
12. Ponte con la Parte V: il denoiser fa message passing, quindi valgono oversmoothing e oversquashing anche qui? Che cosa succede a un nodo appena inserito, che ha pochi archi, rispetto all'aggregazione dai vicini?

Appendix A

Raccolta domande d'esame — preparazione orale

Il professore (vedi GDL35) struttura l'orale in ~ 3 domande di complessità crescente: voto correlato alla profondità raggiunta. Aspettati che possa chiederti di:

- *derivare* formule alla lavagna (es. EM per GMM, ELBO, discriminatore ottimale GAN, change-of-variables per flows);
- *scegliere e giustificare* il modello giusto per un task descritto a parole;
- *connettere* parti diverse del corso (le 5 lenti unificanti);
- *discutere proprietà e trade-off*: stabilità training, qualità campioni, scalabilità, identificabilità.

Regole dell'orale, dalle slide GDL35 (verbatim, tradotto)

Vale la pena conoscerle alla lettera, perché cambiano la strategia con cui si affronta la prova.

- **Circa tre domande di complessità crescente**, e il voto è legato alla complessità raggiunta: fermarsi presto significa un voto più basso, non un voto “da rifare”.
- **Se non si risponde alla domanda corrente in un tempo ragionevole** (indicativamente 5–10 minuti) **l'orale si conclude lì**, con il voto corrispondente alle domande già affrontate.
- **Non si cambia domanda su richiesta**: non c'è la possibilità di chiedere “me ne fa un'altra?” finché non ne capita una che si sa.
- **Soglia minima per passare**: saper *formalizzare* i modelli visti a lezione e saperne discutere in modo convincente proprietà e utilizzo.
- Le domande sono costruite per distinguere il ragionamento dalla memorizzazione. Può essere chiesto di *derivare* enunciati e di descrivere i modelli formalmente.
- **La lezione GDL33** (*Deep Learning for Graphs II — Advanced Models*) **non fa parte del materiale d'esame**.
- L'orale può essere sostenuto in italiano o in inglese, a scelta.

Che cosa se ne deduce sulla preparazione. Poiché l'orale si interrompe al primo blocco, la profondità conta più della copertura marginale: è meglio saper derivare

bene dieci cose che riconoscerne trenta. E poiché la prima domanda è la più semplice, conviene che le definizioni di base siano automatiche — non è lì che si guadagna, ma è lì che si può perdere tutto.

A.1 Domande trasversali (le più difficili)

Famiglia generativa: traccia l'evoluzione dal VAE al Flow Matching

Punti chiave per la risposta:

1. VAE (2013): ELBO + reparametrization $\Rightarrow$ primo modello latente differenziabile; latente strutturato, campioni sfocati.
2. GAN (2014): cambio di paradigma — impara a campionare, non a stimare densità; alta qualità, instabilità.
3. Normalizing Flows (2014–15): cambio di variabile esplicito; likelihood esatta ma vincolo invertibilità + dim-preserving.
4. Diffusion (2020): VAE gerarchico con encoder fisso; training stabile, qualità SOTA, sampling lento.
5. Flow Matching (2023): regressione di velocità lungo path scelto; deterministico, pochi step, equivalente a diffusion sotto path gaussiani.

ELBO: come collega EM, Variational Inference e VAE?

1. Definizione: $\log p(\mathbf{x}) \geq \mathbb{E}_q[\log p(\mathbf{x}, \mathbf{z})] - \mathbb{E}_q[\log q(\mathbf{z})]$ (Jensen).
2. EM (Cap. 8): l'E-step pone $q(\mathbf{z}) = p(\mathbf{z}|\mathbf{x}, \theta^{\text{old}})$ rendendo l'ELBO tight; M-step massimizza $\mathbb{E}_q[\log p(\mathbf{x}, \mathbf{z}; \theta)]$.
3. Variational Inference (Cap. 10): q ristretto a una famiglia (mean-field) per cui $p(\mathbf{z}|\mathbf{x})$ è intrattabile. CAVI alterna ottimizzazione coordinate-wise.
4. VAE (Cap. 20): $q_\phi(\mathbf{z}|\mathbf{x})$ parametrizzato da una rete neurale (amortized inference), reparametrization rende differenziabile, ottimizzazione gradient-based su (θ, ϕ) congiunti.
5. Diffusion (Cap. 23): ELBO gerarchico su catena di T latenti.

La storia è la stessa quantità (ELBO) con vincoli progressivamente più rilassati su q .

Message passing: collega PGM, GNN e Transformer

1. Belief Propagation (Cap. 8, Parte I): aggregare messaggi probabilistici dai vicini in un fattore graph per calcolare marginali.
2. GNN message passing (Parte V): aggregare embedding dai vicini in un grafo per costruire rappresentazioni.
3. Transformer self-attention (Parte III): grafo completo (ogni token attiva su tutti), pesi via softmax dot-product.

4. GAT: self-attention ristretto ai vicini effettivi $\Rightarrow$ ponte fra GNN e Transformer.
5. Graph Transformer (GPS): combina message passing locale + global attention.

La stessa formula $h_v^{(\ell+1)} = \text{UPDATE}(h_v^{(\ell)}, \text{AGG}(\{h_u^{(\ell)}\}))$ varia solo per: (i) la natura dei messaggi (densità vs vettori), (ii) la struttura del grafo (esplicito vs implicito completo), (iii) la AGG (sum, mean, attention).

Attention: dall'attenzione di Bahdanau ai GAT

1. Bahdanau cross-attention (2014, Cap. Encoder-Decoder Parte III): attention "additiva" su tokens encoder dal decoder RNN. Pesi $\alpha_t = \text{softmax}(\mathbf{v}^\top \tanh(\mathbf{W}_h \mathbf{h}_t + \mathbf{W}_s \mathbf{s}))$.
2. Self-attention (2017, Cap. Transformer): ogni token attende su tutti. $\text{Attention}(Q, K, V) = \text{softmax}(QK^\top / \sqrt{d_k})V$.
3. Multi-head: h teste indipendenti concatenate $\Rightarrow$ subspaces di rappresentazione.
4. GAT: stessa formula applicata su vicini di un grafo.

Filo conduttore: *indicizzare contenuto per somiglianza*, non per posizione.

Causalità classica vs Causal Representation Learning

1. Causalità classica (Cap. 5): si parte da variabili *osservate* $X_1, \dots, X_d$ con semantica chiara. Si infiscia un SCM, si fa do-calculus, structure learning.
2. CRL (Cap. 25): le variabili causali $\mathbf{s}$ sono *latenti*; si osserva solo $\mathbf{x} = h^*(\mathbf{s})$ (immagini, video). Bisogna prima *ricavare* le variabili, poi inferire il grafo.
3. Locatello 2019: unsupervised non identifiable $\Rightarrow$ servono interventi/regimi (TCL, iVAE) o auxiliary variables.
4. CITRIS: identifiability sotto interventi temporali noti.

Il deep learning aggiunge: rappresentazioni apprese, mixing non lineare, mappatura immagini $\rightarrow$ fattori.

Modelli a variabile latente: una panoramica EM-style

Risponderesti:

GMM (mistura di gaussiane, latente discreta) $\rightarrow$ HMM (sequenza di latenti discreti, Baum-Welch è EM specializzato) $\rightarrow$ Variational Inference (latenti generiche, q in famiglia mean-field) $\rightarrow$ VAE (latenti continue, q amortizzato neurale, reparametrization) $\rightarrow$ Diffusion (gerarchia di latenti, encoder fisso). Ogni passo rilassa un'assunzione del precedente.

A.2 Parte I — PGM e Causalità

Definisci e disegna un esempio di Bayesian Network. Specifica: variabili, archi diretti aciclici, CPT. Spiega la fattorizzazione $p(\mathbf{x}) = \prod_i p(x_i | \text{Pa}(x_i))$.

Cos'è la d-separation? Quando due variabili sono d-separate? Definisci il criterio: cammino bloccato da chain/fork con osservato, collider non osservato senza discendenti osservati.

Markov blanket di un nodo X . $MB(X) = Pa(X) \cup Ch(X) \cup Pa(Ch(X))$. È il set minimo che rende X indipendente dal resto.

Differenza fra rete Bayesiana e rete causale. BN: rappresenta indipendenze osservazionali. Causale: gli archi rappresentano relazioni causa-effetto e supportano do-interventi.

Gerarchia di Pearl (associativo, intervenzionale, controfattuale). Tre livelli: $p(y|x)$ (osservazione) $\subset p(y|do(x))$ (intervento) $\subset p(y_x|x', y')$ (controfattuale).

Cos'è il do-operator? Quando $p(y|x) = p(y|do(x))$? $do(X = x)$ fissa X rimuovendo gli archi entranti. Le due quantità coincidono se non ci sono backdoor path da X a Y .

Backdoor criterion. Una insieme Z soddisfa il backdoor criterion se: nessun nodo in Z è discendente di X , e Z blocca tutti i backdoor path da X a Y . Allora $p(y|do(x)) = \sum_z p(y|x, z)p(z)$.

Structure learning: score-based vs constraint-based. Score-based: ottimizza una funzione (BIC, BDe) sulla struttura. Constraint-based (PC, GES): usa test di indipendenza condizionale per ricostruire il grafo (entro Markov equivalence class).

A.3 Parte II — Learning Probabilistico

MLE vs MAP vs Bayesian. MLE: $\theta^* = \arg \max_{\theta} p(\mathcal{D}|\theta)$. MAP: $\arg \max_{\theta} p(\theta|\mathcal{D}) = \arg \max_{\theta} p(\mathcal{D}|\theta)p(\theta)$. Bayesian: posterior completa $p(\theta|\mathcal{D})$, non un singolo punto.

Deriva EM per Gaussian Mixture Model. E-step: $\gamma_{ik} = \frac{\pi_k \mathcal{N}(x_i|\mu_k, \Sigma_k)}{\sum_j \pi_j \mathcal{N}(x_i|\mu_j, \Sigma_j)}$. M-step: $\mu_k = \frac{\sum_i \gamma_{ik} x_i}{\sum_i \gamma_{ik}}$, Σ_k analogo, $\pi_k = \sum_i \gamma_{ik}/N$. **Domanda iconica del prof.**

Deriva l'ELBO e mostra che $\log p(x) = \mathcal{L} + D_{KL}(q||p(z|x))$. Vedi Cap. 20, sezione ELBO.

Forward-Backward HMM. $\alpha_t(i) = p(x_{1:t}, S_t = i)$ con ricorrenza $\alpha_t(i) = b_i(x_t) \sum_j \alpha_{t-1}(j) a_{ji}$; $\beta_t(i) = p(x_{t+1:T}|S_t = i)$ con $\beta_t(i) = \sum_j a_{ij} b_j(x_{t+1}) \beta_{t+1}(j)$. Smoothing $\gamma_t(i) = \alpha_t(i) \beta_t(i) / Z$.

Algoritmo di Viterbi. DP per il path più probabile: $\delta_t(i) = \max_{s_{1:t-1}} p(s_{1:t-1}, S_t = i, x_{1:t})$ con $\delta_t(j) = b_j(x_t) \max_i \delta_{t-1}(i) a_{ij}$, e backpointer.

Baum-Welch. EM specializzato per HMM: E-step calcola $\gamma_t(i), \xi_t(i, j) = p(S_t = i, S_{t+1} = j|x_{1:T})$. M-step aggiorna π, A, B come medie pesate.

Mean-field variational inference (CAVI). $q(z) = \prod_i q_i(z_i)$. Coordinate ascent: $q_i^*(z_i) \propto \exp(\mathbb{E}_{q_{-i}}[\log p(z, \mathbf{x})])$.

LDA: generative process e inference. Per ogni doc d : $\theta_d \sim \text{Dir}(\alpha)$. Per ogni parola: $z_{dn} \sim \text{Cat}(\theta_d)$, $w_{dn} \sim \text{Cat}(\beta_{z_{dn}})$. Inference: collapsed Gibbs o variational.

Gibbs sampling vs Metropolis-Hastings. Gibbs: campiona $z_i|z_{-i}$ in sequenza. MH: propone z' da $q(z'|z)$, accetta con prob $\min(1, p(z')q(z|z')/(p(z)q(z'|z)))$. Gibbs è caso speciale con acceptance 1.

Restricted Boltzmann Machine: definizione e contrastive divergence. $p(\mathbf{v}, \mathbf{h}) \propto \exp(\mathbf{v}^\top \mathbf{W} \mathbf{h} + \mathbf{a}^\top \mathbf{v} + \mathbf{b}^\top \mathbf{h})$. CD-1: $\Delta W \propto \langle \mathbf{v} \mathbf{h}^\top \rangle_{\text{data}} - \langle \mathbf{v} \mathbf{h}^\top \rangle_{1\text{-step Gibbs}}$.

A.4 Parte III — Deep Learning Fondamentale

Convoluzione: parameter sharing, sparsità, equivariance. Filtri condivisi $\Rightarrow$ pochi parametri. Sparsità $\Rightarrow$ ogni output dipende da pochi input. Equivariance alla traslazione: $f(T_v x) = T_v f(x)$.

LeNet $\rightarrow$ AlexNet $\rightarrow$ VGG $\rightarrow$ ResNet. LeNet (1998, primo CNN per cifre); AlexNet (2012, ReLU + dropout + GPU); VGG (2014, depth con kernel 3×3); ResNet (2015, residual connection, oltre 100 layer).

Residual connection: perché funziona? $\mathbf{h}^{(\ell+1)} = \mathbf{h}^{(\ell)} + F(\mathbf{h}^{(\ell)})$. Backprop ha un cammino diretto in cui il gradiente non si attenua; permette di addestrare reti molto profonde.

Vanishing/exploding gradient (EVGP). In una rete profonda i prodotti di Jacobiani $\prod_{\ell} J_{\ell}$ esplodono o vanno a zero. Mitigazioni: init (Xavier, He), normalization, ReLU, residual, LSTM gates.

Batch Normalization: cosa fa e perché. Normalizza ogni mini-batch a media 0 e varianza 1 per layer, poi affine shift apprendibile. Stabilizza training, permette LR più alti, regolarizza.

Dilated/causal convolutions. Dilated: kernel con buchi $\Rightarrow$ receptive field esponenziale (WaveNet). Causal: kernel asimmetrico, output a t dipende solo da input $\leq t$ (sequenze).

BPTT e troncamento. Backpropagation Through Time: unroll della RNN e backprop normale. Truncated BPTT: limita la lunghezza dell'unroll.

LSTM: gates e architettura. Forget gate $f_t = \sigma(\mathbf{W}_f[h_{t-1}, x_t])$, input gate i_t , cell state $c_t = f_t c_{t-1} + i_t \tilde{c}_t$, output gate o_t , $h_t = o_t \tanh(c_t)$. Risolve il vanishing gradient via il cell state.

GRU vs LSTM. GRU: due gate (reset, update), no cell state separato. Più parametri-efficienti, prestazioni comparabili.

Echo State Network. Reservoir random $\mathbf{W}$ con $\rho(\mathbf{W}) < 1$, training solo del readout lineare. Veloce, pochi iperparametri.

Bahdanau attention. Pesi additivi sul context vector dell'encoder, calcolati dal decoder. Risolve il bottleneck dei seq2seq classici.

Self-attention: formula e complessità. $\text{softmax}(QK^{\top}/\sqrt{d_k})V$. Complessità $\mathcal{O}(n^2d)$ in lunghezza sequenza.

Multi-head attention: perché? h teste indipendenti permettono di attendere a sottospazi diversi. Output concatenato + proiezione.

Positional encoding. Necessario perché self-attention è permutation invariant. Sinusoidale (Vaswani) o appreso. Variants: relative (Shaw), RoPE.

Encoder-only (BERT) vs Decoder-only (GPT) vs Encoder-Decoder (T5). BERT: bidirezionale, masked LM, classificazione/embedding. GPT: autoregressivo causale, generazione. T5: seq2seq generico.

Vision Transformer. Patches $\rightarrow$ token $\rightarrow$ Transformer. Pre-training scala meglio di CNN su dataset enormi.

A.5 Parte IV — Generative Deep Learning

Definisci un autoencoder e i suoi varianti regolarizzati. Vedi Cap. 19.

DAE come stimatore della score. Tweedie + Vincent 2011; $\nabla \log p = (g \circ f - x)/\sigma^2$.

Deriva l'ELBO del VAE. Vedi Cap. 20.

Reparametrization trick: cos'è e perché serve. $z = \mu + \sigma \odot \epsilon$, gradient flow attraverso μ, σ . Senza, non si può backpropare nel sampling.

Cos'è il posterior collapse? Decoder troppo potente ignora z ; $KL \rightarrow 0$; latente inutile. Mitigazioni: KL annealing, β -VAE.

GAN: minimax e discriminatore ottimale. $D^* = p_d/(p_d + p_g)$. A D^* fisso, $V = JS$ divergence $-\log 4$.

Perché WGAN funziona meglio della GAN classica? Wasserstein è ben definita anche se i supporti sono disgiunti, gradiente non saturante.

Mode collapse: cos'è e perché. Il generatore impara pochi modi che ingannano D . Causa: punto sella, D non riconosce mancanza di copertura.

Cycle consistency in CycleGAN. $\|F(G(x)) - x\| + \|G(F(y)) - y\|$. Permette style transfer senza dati appaiati.

Normalizing flow: cambio di variabile e composizione. $\log p_X = \log p_Z - \sum \log |\det J_{f_k}|$.

Coupling layer (RealNVP). Jacobiano triangolare, inversione $\mathcal{O}(d)$.

MAF vs IAF. MAF: density evaluation parallela, sampling sequenziale. IAF: opposto.

Continuous Normalizing Flow. ODE neurale, traccia al posto del determinante.

DDPM: forward e training objective. Vedi Cap. 23.

Classifier-free guidance: come funziona? Stesso modello con/senza condizione, miscela a inference.

Latent diffusion. Diffusion in latente di VAE per efficienza.

Score matching: ESM, ISM, DSM. Esplicito (intrattabile), implicito (costoso), denoising (efficiente, equivalente a DDPM).

Probability flow ODE. Versione deterministica della reverse SDE; stesse marginali; permette likelihood esatta.

Flow Matching e CFM. Si regredisce velocità; conditional path. Rectified flow = traiettorie rettilinee, pochi step.

Disentanglement: definizione e impossibilità. Locatello 2019: marginale indipendenza non è sufficiente.

iVAE: come supera l'impossibilità? Auxiliary variable u rompe l'invarianza non-identificabile.

CITRIS: setup, identificabilità, varianti. Triplet (x_t, x_{t+1}, I_{t+1}) , transition prior dipendente da intervento.

A.6 Parte V — Graph Neural Networks

Message passing framework. $h_v^{(\ell+1)} = U(h_v^{(\ell)}, \text{AGG}\{h_u^{(\ell)}\})$. AGG permutation invariant.

Deriva la GCN da approssimazione spettrale. Chebyshev di ordine 1 sulla normalized Laplacian.

GraphSAGE: scalabilità. Sampling di vicini, mean/max/LSTM aggregator.

GIN: espressività. Sum + MLP iniettivo $\Rightarrow$ raggiunge il limite 1-WL.

GAT: differenza con Transformer. Attention ristretta ai vicini effettivi.

Over-smoothing. Mean aggregation $\rightarrow$ punto fisso costante; mitigazioni: residual, jumping knowledge.

Over-squashing. Vicinanza esponenziale schiacciata in vettore fisso; mitigazioni: global attention, rewiring.

DiffPool. Soft assignment $S \in \mathbb{R}^{n \times k}$, coarsening $X' = S^\top X$, $A' = S^\top A S$.

Graph Echo State Network. Reservoir di message passing random, training lineare.

Confronta GCN/GraphSAGE/GAT/GIN. Vedi tabella in Cap. 27.

A.7 Parte VI — Reinforcement Learning

Definisci MDP. $(\mathcal{S}, \mathcal{A}, P, R, \gamma)$.

Deriva Bellman expectation per v_π . Espansione di $G_t = R_{t+1} + \gamma G_{t+1}$.

Bellman optimality. $v_*(s) = \max_a [R + \gamma \sum P v_*]$.

Policy iteration vs Value iteration. Pro/contro.

Q-learning: scrivi update e spiega off-policy. Vedi (30.1).

SARSA vs Q-learning. On-policy vs off-policy.

DQN: i tre trucchi. Experience replay, target network, reward clipping.

Policy gradient theorem. Log-derivative trick, $\nabla J = \mathbb{E}[\nabla \log \pi Q]$.

REINFORCE: pro e contro. Stima MC, no bias, alta varianza.

Actor-Critic: ruolo di actor e critic. Critic stima value/Q, actor aggiorna policy con advantage.

PPO: clipped objective. $\min(r\hat{A}, \text{clip}(r, 1 \pm \epsilon)\hat{A})$.

Quando policy-based vs value-based? Azioni continue, stochastic policy $\rightarrow$ policy. Azioni discrete, Q-rappresentabile $\rightarrow$ value.

Appendix B

Cheat sheet delle formule

Riferimento rapido per le formule chiave del corso. Le formule sono raggruppate per tema; per ciascuna è indicato il capitolo in cui è derivata.

B.1 Probabilità e PGM

Bayes.

$$p(A|B) = \frac{p(B|A)p(A)}{p(B)}, \quad p(\theta|\mathcal{D}) \propto p(\mathcal{D}|\theta)p(\theta).$$

Fattorizzazione BN.

$$p(\mathbf{x}) = \prod_{i=1}^n p(x_i | \text{Pa}(x_i)).$$

Markov blanket. $\text{MB}(X) = \text{Pa}(X) \cup \text{Ch}(X) \cup \text{Pa}(\text{Ch}(X)).$

d-separation. $X \perp\!\!\!\perp Y \mid Z$ in G se ogni cammino $X \rightarrow Y$ è bloccato: chain/fork con un nodo in Z , oppure collider $X \rightarrow W \leftarrow Y$ con W e i suoi discendenti $\notin Z$.

Backdoor adjustment.

$$p(y \mid \text{do}(x)) = \sum_z p(y \mid x, z) p(z), \quad Z \text{ blocca i backdoor path.}$$

Gerarchia di Pearl. $p(y|x) \subset p(y|\text{do}(x)) \subset p(y_x \mid x', y').$

B.2 Learning probabilistico

MLE.

$$\theta_{\text{MLE}} = \arg \max_{\theta} \sum_{i=1}^N \log p(x_i|\theta).$$

MAP.

$$\theta_{\text{MAP}} = \arg \max_{\theta} \left[\sum_i \log p(x_i|\theta) + \log p(\theta) \right].$$

ELBO.

$$\log p(\mathbf{x}) \geq \mathbb{E}_{q(\mathbf{z})}[\log p(\mathbf{x}, \mathbf{z})] - \mathbb{E}_q[\log q(\mathbf{z})] = \mathbb{E}_q[\log p(\mathbf{x}|\mathbf{z})] - D_{\text{KL}}(q(\mathbf{z})\|p(\mathbf{z})).$$

$$\log p(\mathbf{x}) = \mathcal{L}(q) + D_{\text{KL}}(q(\mathbf{z})\|p(\mathbf{z}|\mathbf{x})).$$

EM.

- E-step: $q^*(\mathbf{z}) = p(\mathbf{z}|\mathbf{x}, \theta^{\text{old}})$ (rende l'ELBO tight).
- M-step: $\theta^{\text{new}} = \arg \max_{\theta} \mathbb{E}_{q^*}[\log p(\mathbf{x}, \mathbf{z}; \theta)]$.

GMM E-step.

$$\gamma_{ik} = \frac{\pi_k \mathcal{N}(x_i|\mu_k, \Sigma_k)}{\sum_j \pi_j \mathcal{N}(x_i|\mu_j, \Sigma_j)}.$$

GMM M-step.

$$\mu_k = \frac{\sum_i \gamma_{ik} x_i}{\sum_i \gamma_{ik}}, \quad \Sigma_k = \frac{\sum_i \gamma_{ik} (x_i - \mu_k)(x_i - \mu_k)^\top}{\sum_i \gamma_{ik}}, \quad \pi_k = \frac{1}{N} \sum_i \gamma_{ik}.$$

HMM Forward.

$$\alpha_t(i) = p(x_{1:t}, S_t = i) = b_i(x_t) \sum_j \alpha_{t-1}(j) a_{ji}.$$

HMM Backward.

$$\beta_t(i) = p(x_{t+1:T} | S_t = i) = \sum_j a_{ij} b_j(x_{t+1}) \beta_{t+1}(j).$$

HMM Smoothing.

$$\gamma_t(i) = p(S_t = i | x_{1:T}) = \frac{\alpha_t(i) \beta_t(i)}{\sum_j \alpha_t(j) \beta_t(j)}.$$

Viterbi.

$$\delta_t(j) = b_j(x_t) \max_i \delta_{t-1}(i) a_{ij}.$$

Mean-field CAVI.

$$q_i^*(z_i) \propto \exp(\mathbb{E}_{q_{-i}}[\log p(z, \mathbf{x})]).$$

Gibbs sampling. Itera $z_i^{(t+1)} \sim p(z_i | \mathbf{z}_{-i}^{(t)})$.

Metropolis-Hastings acceptance.

$$\alpha = \min\left(1, \frac{p(z') q(z|z')}{p(z) q(z'|z)}\right).$$

RBM joint.

$$p(\mathbf{v}, \mathbf{h}) = \frac{1}{Z} \exp(\mathbf{v}^\top \mathbf{W} \mathbf{h} + \mathbf{a}^\top \mathbf{v} + \mathbf{b}^\top \mathbf{h}).$$

Contrastive Divergence (CD-1).

$$\Delta W \propto \langle \mathbf{v} \mathbf{h}^\top \rangle_{\text{data}} - \langle \mathbf{v} \mathbf{h}^\top \rangle_{1\text{-step}}.$$

B.3 Deep Learning fondamentale

Convoluzione (1D, valid). $y_i = \sum_{k=0}^{K-1} x_{i+k} w_k$. Output size: $n_{\text{in}} - K + 1$ (no pad).

Dimensione output (padding p , stride s).

$$n_{\text{out}} = \left\lfloor \frac{n_{\text{in}} + 2p - K}{s} \right\rfloor + 1.$$

Receptive field. Per stack di L conv $K \times K$ con stride 1: $R = 1 + L(K - 1)$.

ReLU. $\text{ReLU}(x) = \max(0, x)$.

Softmax.

$$\text{softmax}(\mathbf{z})_i = \frac{e^{z_i}}{\sum_j e^{z_j}}.$$

Cross-entropy.

$$\mathcal{L} = - \sum_i y_i \log \hat{y}_i.$$

Batch normalization.

$$\hat{x}_i = \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}, \quad y_i = \gamma \hat{x}_i + \beta.$$

Residual block. $\mathbf{h}^{(\ell+1)} = \mathbf{h}^{(\ell)} + F(\mathbf{h}^{(\ell)})$.

LSTM.

$$\begin{aligned} \mathbf{f}_t &= \sigma(\mathbf{W}_f[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_f) \\ \mathbf{i}_t &= \sigma(\mathbf{W}_i[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_i) \\ \tilde{\mathbf{c}}_t &= \tanh(\mathbf{W}_c[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_c) \\ \mathbf{c}_t &= \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t \\ \mathbf{o}_t &= \sigma(\mathbf{W}_o[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_o) \\ \mathbf{h}_t &= \mathbf{o}_t \odot \tanh(\mathbf{c}_t). \end{aligned}$$

GRU.

$$\begin{aligned} \mathbf{z}_t &= \sigma(\mathbf{W}_z[\mathbf{h}_{t-1}, \mathbf{x}_t]) \\ \mathbf{r}_t &= \sigma(\mathbf{W}_r[\mathbf{h}_{t-1}, \mathbf{x}_t]) \\ \tilde{\mathbf{h}}_t &= \tanh(\mathbf{W}[\mathbf{r}_t \odot \mathbf{h}_{t-1}, \mathbf{x}_t]) \\ \mathbf{h}_t &= (1 - \mathbf{z}_t) \odot \mathbf{h}_{t-1} + \mathbf{z}_t \odot \tilde{\mathbf{h}}_t. \end{aligned}$$

Bahdanau attention.

$$e_{t,i} = \mathbf{v}^\top \tanh(\mathbf{W}_h \mathbf{h}_i + \mathbf{W}_s \mathbf{s}_t), \quad \alpha_{t,i} = \text{softmax}_i(e_{t,i}), \quad \mathbf{c}_t = \sum_i \alpha_{t,i} \mathbf{h}_i.$$

Scaled dot-product attention.

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}}\right) \mathbf{V}.$$

Multi-head attention.

$$\text{MHA}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = [\text{head}_1 \parallel \dots \parallel \text{head}_h] \mathbf{W}_O.$$

Positional encoding (sinusoidale).

$$\text{PE}_{(p, 2i)} = \sin\left(\frac{p}{10000^{2i/d}}\right), \quad \text{PE}_{(p, 2i+1)} = \cos\left(\frac{p}{10000^{2i/d}}\right).$$

B.4 Generative Deep Learning

VAE ELBO.

$$\mathcal{L}(\mathbf{x}; \theta, \phi) = \mathbb{E}_{q_\phi(\mathbf{z}|\mathbf{x})} [\log p_\theta(\mathbf{x}|\mathbf{z})] - D_{\text{KL}}(q_\phi(\mathbf{z}|\mathbf{x}) \| p(\mathbf{z})).$$

Reparametrization trick.

$$\mathbf{z} = \boldsymbol{\mu}_\phi(\mathbf{x}) + \boldsymbol{\sigma}_\phi(\mathbf{x}) \odot \boldsymbol{\epsilon}, \quad \boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I}).$$

KL gaussiane.

$$D_{\text{KL}}(\mathcal{N}(\boldsymbol{\mu}, \text{diag}(\boldsymbol{\sigma}^2)) \| \mathcal{N}(\mathbf{0}, \mathbf{I})) = \frac{1}{2} \sum_j (\mu_j^2 + \sigma_j^2 - 1 - \log \sigma_j^2).$$

GAN minimax.

$$\min_G \max_D \mathbb{E}_{p_{\text{data}}} [\log D(\mathbf{x})] + \mathbb{E}_{p_z} [\log(1 - D(G(\mathbf{z})))].$$

GAN discriminatore ottimale.

$$D^*(\mathbf{x}) = \frac{p_{\text{data}}(\mathbf{x})}{p_{\text{data}}(\mathbf{x}) + p_G(\mathbf{x})}.$$

GAN $\leftrightarrow$ JS. $V(D^*, G) = 2\text{JS}(p_{\text{data}} \| p_G) - \log 4.$

WGAN.

$$\min_G \max_{\|D\|_L \leq 1} \mathbb{E}_{p_{\text{data}}} [D(\mathbf{x})] - \mathbb{E}_{p_z} [D(G(\mathbf{z}))].$$

Cambio di variabile.

$$\log p_X(\mathbf{x}) = \log p_Z(f^{-1}(\mathbf{x})) + \log |\det J_{f^{-1}}(\mathbf{x})|.$$

Composizione di flow.

$$\log p_X(\mathbf{x}) = \log p_Z(\mathbf{z}_0) - \sum_{k=1}^K \log |\det J_{f_k}(\mathbf{z}_{k-1})|.$$

Coupling (RealNVP).

$$\mathbf{z}'_B = \mathbf{z}_B \odot \exp(\mathbf{s}_\theta(\mathbf{z}_A)) + \mathbf{t}_\theta(\mathbf{z}_A), \quad \log |\det J| = \sum_j \mathbf{s}_\theta(\mathbf{z}_A)_j.$$

Continuous flow (CNF).

$$\frac{dz_t}{dt} = \mathbf{f}_\theta(z_t, t), \quad \log p_X(\mathbf{x}) = \log p_Z(z_0) - \int_0^T \text{tr} \left(\frac{\partial \mathbf{f}_\theta}{\partial \mathbf{z}} \right) dt.$$

DDPM forward.

$$q(z_t|z_{t-1}) = \mathcal{N}(\sqrt{1 - \beta_t}z_{t-1}, \beta_t \mathbf{I}), \quad q(z_t|x_0) = \mathcal{N}(\sqrt{\bar{\alpha}_t}x_0, (1 - \bar{\alpha}_t)\mathbf{I}).$$

DDPM simplified loss.

$$\mathcal{L}_{\text{simple}} = \mathbb{E}_{t, x_0, \epsilon} \left\| \epsilon - \epsilon_\theta(\sqrt{\bar{\alpha}_t}x_0 + \sqrt{1 - \bar{\alpha}_t}\epsilon, t) \right\|^2.$$

DDPM sampling.

$$z_{t-1} = \frac{1}{\sqrt{\bar{\alpha}_t}} \left(z_t - \frac{\beta_t}{\sqrt{1 - \bar{\alpha}_t}} \epsilon_\theta(z_t, t) \right) + \sigma_t \mathbf{w}.$$

Classifier-free guidance.

$$\hat{\epsilon}(z_t, t, \mathbf{y}) = (1 + \gamma) \epsilon_\theta(z_t, t, \mathbf{y}) - \gamma \epsilon_\theta(z_t, t, \emptyset).$$

Score.

$$\mathbf{s}(\mathbf{x}) = \nabla_{\mathbf{x}} \log p(\mathbf{x}).$$

Denoising Score Matching.

$$\mathcal{L}_{\text{DSM}} = \mathbb{E} \left\| \mathbf{s}_\theta(\tilde{\mathbf{x}}) + \frac{\epsilon}{\sigma} \right\|^2, \quad \tilde{\mathbf{x}} = \mathbf{x} + \sigma \epsilon.$$

Forward SDE.

$$d\mathbf{x} = \mathbf{f}(\mathbf{x}, t) dt + g(t) d\mathbf{w}.$$

Reverse SDE.

$$d\mathbf{x} = [\mathbf{f}(\mathbf{x}, t) - g^2(t) \nabla_{\mathbf{x}} \log p_t(\mathbf{x})] dt + g(t) d\bar{\mathbf{w}}.$$

Probability flow ODE.

$$\frac{d\mathbf{x}}{dt} = \mathbf{f}(\mathbf{x}, t) - \frac{1}{2} g^2(t) \nabla_{\mathbf{x}} \log p_t(\mathbf{x}).$$

Langevin dynamics.

$$\mathbf{x}_{k+1} = \mathbf{x}_k + \frac{\eta}{2} \mathbf{s}(\mathbf{x}_k) + \sqrt{\eta} \epsilon_k.$$

Flow Matching (CFM).

$$\mathcal{L}_{\text{CFM}} = \mathbb{E}_{\tau, \mathbf{x}, \mathbf{z} \sim q_\tau(\cdot|\mathbf{x})} \left\| \mathbf{v}_\theta(\mathbf{z}, \tau) - \mathbf{u}_\tau(\mathbf{z}|\mathbf{x}) \right\|^2.$$

Rectified flow.

$$\mathbf{z}_\tau = (1 - \tau)\mathbf{z}_0 + \tau\mathbf{x}, \quad \mathbf{u}_\tau = \mathbf{x} - \mathbf{z}_0.$$

β -VAE.

$$\mathcal{L}_\beta = \mathbb{E}_{q_\phi} [\log p_\theta(\mathbf{x}|\mathbf{z})] - \beta D_{\text{KL}}(q_\phi \| p(\mathbf{z})).$$

Total Correlation (FactorVAE).

$$\text{TC}(z) = D_{\text{KL}} \left(q(z) \parallel \prod_j q(z_j) \right).$$

B.5 Graph Neural Networks

Message passing.

$$\mathbf{h}_v^{(\ell+1)} = U^{(\ell)}(\mathbf{h}_v^{(\ell)}, \text{AGG}^{(\ell)}\{\mathbf{h}_u^{(\ell)} : u \in N(v)\}).$$

GCN (Kipf-Welling).

$$\mathbf{H}^{(\ell+1)} = \sigma \left(\tilde{D}^{-1/2} (\mathbf{A} + \mathbf{I}) \tilde{D}^{-1/2} \mathbf{H}^{(\ell)} \mathbf{W}^{(\ell)} \right).$$

GraphSAGE.

$$\mathbf{h}_v^{(\ell+1)} = \sigma(\mathbf{W}^{(\ell)}[\mathbf{h}_v^{(\ell)} \parallel \text{AGG}\{\mathbf{h}_u^{(\ell)}\}]).$$

GIN.

$$\mathbf{h}_v^{(\ell+1)} = \text{MLP}((1 + \epsilon)\mathbf{h}_v^{(\ell)} + \sum_{u \in N(v)} \mathbf{h}_u^{(\ell)}).$$

GAT.

$$\alpha_{vu} = \text{softmax}_u \left(\text{LeakyReLU}(\mathbf{a}^\top [\mathbf{W}\mathbf{h}_v \parallel \mathbf{W}\mathbf{h}_u]) \right), \quad \mathbf{h}'_v = \sigma \sum_u \alpha_{vu} \mathbf{W}\mathbf{h}_u.$$

DiffPool.

$$\mathbf{X}' = \mathbf{S}^\top \mathbf{X}, \quad \mathbf{A}' = \mathbf{S}^\top \mathbf{A} \mathbf{S}, \quad \mathbf{S} \in \mathbb{R}^{|V| \times K}.$$

B.6 Reinforcement Learning

Ritorno. $G_t = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}$.

Bellman expectation (state value).

$$v_\pi(s) = \sum_a \pi(a|s) \left[\mathcal{R}_s^a + \gamma \sum_{s'} P_{ss'}^a v_\pi(s') \right].$$

Bellman optimality.

$$v_*(s) = \max_a \left[\mathcal{R}_s^a + \gamma \sum_{s'} P_{ss'}^a v_*(s') \right], \quad q_*(s, a) = \mathcal{R}_s^a + \gamma \sum_{s'} P_{ss'}^a \max_{a'} q_*(s', a').$$

TD(0). $v(S_t) \leftarrow v(S_t) + \alpha[R_{t+1} + \gamma v(S_{t+1}) - v(S_t)]$.

Q-learning.

$$Q(S, A) \leftarrow Q(S, A) + \alpha \left[R + \gamma \max_{a'} Q(S', a') - Q(S, A) \right].$$

SARSA.

$$Q(S, A) \leftarrow Q(S, A) + \alpha[R + \gamma Q(S', A') - Q(S, A)].$$

DQN loss.

$$\mathcal{L}(\mathbf{w}) = \mathbb{E}_{(s,a,r,s') \sim \mathcal{D}} [(r + \gamma \max_{a'} Q(s', a'; \mathbf{w}^-) - Q(s, a; \mathbf{w}))^2].$$

Policy gradient theorem.

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\pi_{\theta}} [\nabla_{\theta} \log \pi_{\theta}(a|s) Q^{\pi_{\theta}}(s, a)].$$

REINFORCE update.

$$\theta \leftarrow \theta + \alpha \sum_t \nabla_{\theta} \log \pi_{\theta}(a_t|s_t) G_t.$$

Advantage.

$$A^{\pi}(s, a) = Q^{\pi}(s, a) - v^{\pi}(s).$$

Actor-Critic update.

$$\delta = r + \gamma V_{\mathbf{w}}(s') - V_{\mathbf{w}}(s), \quad \theta \leftarrow \theta + \alpha_{\theta} \delta \nabla_{\theta} \log \pi_{\theta}(a|s).$$

PPO clip.

$$\mathcal{L}^{\text{CLIP}} = \mathbb{E} [\min(r_t \hat{A}_t, \text{clip}(r_t, 1 - \epsilon, 1 + \epsilon) \hat{A}_t)], \quad r_t = \frac{\pi_{\theta}(a|s)}{\pi_{\theta_{\text{old}}}(a|s)}.$$

Appendix C

Che cosa è stato chiesto davvero

Questa appendice raccoglie i resoconti degli orali degli appelli di **giugno e luglio 2026**, ricostruiti da due fonti: il racconto diretto di un candidato che ha assistito a sei esami consecutivi, e l'archivio del gruppo Telegram del corso (3128 messaggi, settembre 2023 – agosto 2026, di cui 1230 nell'anno GDL).

Come leggere questa appendice

È l'unica parte della dispensa **non** basata sui materiali ufficiali del corso. Sono racconti di studenti, riportati a memoria, talvolta di seconda mano. Vanno usati per capire *come* il professore interroga e per calibrare le priorità — non come elenco di ciò che uscirà.

Dove due fonti si contraddicono lo segnalo invece di scegliere. Dove una notizia è riportata come “ufficiale” ma non l'ho vista sulle slide, lo dico.

C.1 Le regole del gioco

Questa è la parte che vale più di tutte le domande messe insieme, perché è consistente su tre appelli e più fonti indipendenti.

La prima domanda è eliminatoria

“I primi sono usciti tutti per la policy o lo sai alla prima o esci” (1 luglio 2026).

Un candidato riferisce di essere stato mandato via in pochi secondi: gli è stata chiesta l'ultima lezione, non l'aveva vista, e il professore ha dichiarato di “seguire una policy per la quale non va neanche avanti con le altre domande”. Un altro studente conferma il comportamento anche per l'anno precedente: *“se non sapevi la prima domanda tendenzialmente bocciava subito, se arrivavi alla terza al massimo dava un voto molto basso”*.

Il consiglio operativo che ne deriva, formulato da chi l'ha vissuto: *“dai priorità a quello che è molto più certo saranno le prime domande, perché quelle determineranno la bocciatura; poi se le superi è solo questione di alzare il voto”*.

Conseguenza sulla strategia: la copertura conta più della profondità *fino a* superare la prima domanda; dopo, si inverte. Non esiste un argomento su cui ci si possa permettere di non saper partire.

Il segnale che il voto sta scendendo

“Se vedi che comincia a fare domande facili facili, tipo roba sulle CNN, allora il voto sta scendendo: di solito fa così” (gennaio 2026).

Se le domande si semplificano, non è benevolenza: sta cercando il livello a cui sai rispondere per assegnare il voto corrispondente. Simmetricamente, se le domande si fanno più difficili sta cercando il tetto — e gli argomenti “da lode” riportati sono **Bellman optimality, reinforcement learning** in generale, e i temi dell’ultima parte del corso.

Formule: le vuole, ma la memoria non basta

Il punto è riportato con insistenza da più persone, ed è più sottile di “impara le formule”.

“Se provi a impararti a memoria tutte le formule ti ammazzi e lui capisce che hai imparato a memoria — ho fatto questo errore per un paio di formule e mi ci ha beccato. Se sai una formula a memoria lui lo vede e ti ci fotte; se sai i passaggi ma sbagli a mettere un indice o simile, magari ti aiuta.”

Lo stesso studente descrive il **metodo corretto** su HMM, ed è generalizzabile:

1. parti dal *disegno* del grafo;
2. scrivi la likelihood *leggendola dal grafo*, con sommatorie e marginalizzazioni;
3. introduci le *variabili indicatrici* che trasformano le sommatorie in produttorie;
4. prendi il logaritmo, così le produttorie diventano sommatorie;
5. risolvi le derivate parziali (con i moltiplicatori di Lagrange dove i parametri sono distribuzioni).

Cioè: non si ricorda il risultato, si ricorda *la strada*. Ed è coerente con la frase delle slide GDL35 sul focus sul ragionamento invece che sulla memorizzazione.

Due avvertenze pratiche riportate dal gruppo: *“a volte le formule nelle slide hanno typo o errori: conviene sempre provarsele a derivare da soli”*; e si può *spiegare* una formula a parole per evitare errori di indice, ma bisogna comunque saperla scrivere se richiesta.

La routine su HMM, osservata più volte

Un candidato ha ricostruito lo schema ricorrente:

1. Che cos’è un HMM?
2. Com’è fatta la congiunta?
3. Una domanda più libera: forward-backward, smoothing, oppure Viterbi.

E il commento: *“se descrivi bene il modello e scrivi bene la congiunta, sia in forma generale $p(s_1)p(x_1|s_1)\dots$ sia parametrizzata con variabili indicatrici e matrice di transizione, l’esame è passato abbondantemente”*.

Il legame con il quarto assignment

“Quando era ISPR, una domanda tendeva spesso a farla nell’area dell’ultimo midterm: se fai il paper su GNN ti chiede GNN.”

Se la regola vale ancora, per chi ha fatto il poster su **GrIDDD** (graph diffusion) gli argomenti esposti sono *grafi* e *diffusion* — entrambi già fra i più frequenti. Non è una garanzia, ma è un motivo in più per non lasciarli scoperti.

Slide, handout, OneNote

Domanda ricorrente nel gruppo. La sintesi delle risposte: chiede dalle slide, ma *“in caso facesse domande specifiche per assicurarsi che tu abbia capito, potrebbe far riferimento a cose presenti su OneNote o negli handout. Una volta è capitato.”* Sugli handout i pareri si dividono: c'è chi li trova più chiari delle slide e ci ha studiato sopra, chi consiglia il contrario. Il caso concreto citato è l'M-step di HMM, dove la slide dice solo che servono i moltiplicatori di Lagrange e l'handout svolge la derivazione per π , A e B .

C.2 Gli orali riportati

Appello di giugno 2026

#	Argomenti	Esito
1	Bayesian Network, regole della rete; Loss GAN; Wasserstein GAN	26 (rifiutato)
2	RNN, problemi e gated RNN; generative matching; Flow Matching	29
3	CNN; HMM e il termine α ; Bellman optimality (RL)	30L
4	LDA; LSTM	—
5	Convolution on graphs; Diffusion Models	—

Sintesi riportata da chi ha assistito: *“ha chiesto formule specifiche agli orali di giugno: α della HMM e Bellman optimality (questo per la lode), e alcune Loss”.*

Appello di inizio luglio 2026

#	Argomenti	Esito
1	MRF; Normalizing Flow	20
2	HMM; Wasserstein e GAN; conditional generation (“how would you do it”)	26
3	Self-attention	bocciato
4	RNN; ELBO	28–30
5	Bayesian Networks + <i>esercizio</i> sulla rete; VAE	28
6	CNN; RBM	25
7	LSTM; Diffusion Models	27
8	(con progetto) MDP; Q-learning; LDA	—
9	Boltzmann machine; Restricted Boltzmann; GAN	26
10	Self-attention e Transformers; GAN	25
11	GMM; CNN	18 (+26 prog. = 22)
12	HMM; VAE	<i>ritirato</i>
13	(ultima lezione, non preparata)	bocciato

Due dettagli che vale la pena estrarre.

L'**esercizio** del caso 5 non è una domanda teorica: gli è stato dato un grafo e chiesto *rispetto a quali nodi C e D sono condizionalmente indipendenti*. È esattamente il tipo di esercizio del Cap. 4, e conferma che la d-separazione può essere chiesta *operativamente* e non solo definita.

Il caso 9 mostra fino a dove scava sulle GAN: non sapeva che cosa fosse l'operatore $\langle \cdot \rangle$ nella Boltzmann machine, non aveva scritto bene il *supremo* nel discriminatore, e da lì è partita una catena — differenza fra max e sup, *perché* nella Wasserstein si usa sup, e **perché** D deve essere **Lipschitz** (con la precisazione: non “perché è contrattiva”, ma perché *deve* esserlo).

Appello di fine luglio 2026

#	Argomenti	Esito
1	Restricted Boltzmann Machine; Diffusion models (ELBO scritta male)	25
2	Self-attention (non sapeva che cos'è un token né come si calcolano Q, K, V)	bocciato
3	Transformers: architettura, normalizzazioni, connessioni residue	<i>ritirata</i>
4	RNN e vanishing gradient; VAE e reparametrization trick	25
5	LSTM (schema); Normalizing flows (idea, metodi, residual flow)	30L
6	LDA (schema, learning, posterior approssimato, Gibbs); GNN (aggregazione)	30L
7	LSTM: grafico e formule; Normalizing Flow: tutto tranne il continuous	—

C.3 Distribuzione degli argomenti (rivista)

Correzione rispetto alla versione precedente di questa appendice

Sulla base dei soli sei esami di fine luglio avevo scritto che le Parti I e VI *non erano mai comparse*. Con venticinque orali è **falso**: le reti Bayesiane compaiono tre volte (una con un esercizio di d-separazione), e il reinforcement learning tre volte, di cui una decisiva per la lode. Sei osservazioni erano troppo poche, e la conclusione che ne avevo tratto era sbagliata.

Su circa venticinque orali riportati, contando ogni argomento una volta per esame:

Argomento	Volte	Parte
GAN / Wasserstein	4	IV
HMM	4	II
RNN	4	III
LSTM	4	III
VAE / ELBO	4	IV
Diffusion	4	IV
Self-attention / Transformer	4	III
Boltzmann / RBM / MRF	4	II
Bayesian Networks	3	I
CNN	3	III
Normalizing Flow	3	IV
LDA	3	II
MDP / Q-learning / Bellman	3	VI
GNN / grafi	2	V
GMM	1	II
Flow matching	1	IV

La distribuzione è molto più *piatta* di quanto sembrasse: nessun argomento domina, e praticamente ogni parte del corso è comparsa. Il messaggio è l'opposto di quello che avevo scritto prima — non c'è una zona sicura da cui pescare, e la copertura conta.

Restano non osservati: causalità e do-operator, structure learning, score matching, causal representation learning, e il paper del quarto assignment. Su structure learning un candidato osserva: “*della prima parte del Massidda chiede principalmente le Bayesian networks, nemmeno le causali*”. Su GDL30 nessuno ha memoria di una domanda, e il consiglio del gruppo è: “*guardati giusto il disentanglement e il β -VAE, non si sa mai; però penso siano argomenti da terza domanda*”, con l'aggiunta che saltare la lezione per intero è “*estremamente sconsigliato*”.

C.4 Che cosa è escluso

Due esclusioni riportate, con una tensione irrisolta

Il 25 luglio 2026 un candidato riferisce che il professore avrebbe detto **ufficialmente** di *non* chiedere:

- la **lezione 33** (*Deep Learning for Graphs II — Advanced Models*) — coerente con le slide GDL35, p. 34, che è la fonte ufficiale e va considerata dirimente;
- il **training di LDA**.

La tensione: nel resoconto di fine luglio il caso 6 riferisce di aver avuto esattamente “LDA: schema, learning, posterior approssimato” e di aver preso 30 e lode. Le due notizie sono a pochi giorni di distanza.

Tre spiegazioni possibili, e non ho modo di decidere fra loro: l’esclusione è stata annunciata *dopo* quell’orale; oppure “training” si riferisce alla derivazione variazionale completa e non allo schema generale del learning; oppure una delle due testimonianze è imprecisa. **Fino a chiarimento, conviene sapere il learning di LDA:** il costo di saperlo è basso, quello di non saperlo se viene chiesto è l’esame.

C.5 Domande verbatim

Il blocco seguente è la trascrizione delle domande fatte in una singola giornata di orali (3 luglio 2026), riportate da un candidato che vi ha assistito. È il materiale più prezioso di tutta l’appendice, perché sono le *formulazioni reali*, non le nostre parafrasi.

Domande verbatim — giornata del 3 luglio 2026

1. Il normalizing flow è basato sul cambio di variabile: qual è l’idea?
2. Perché lo Jacobiano va considerato?
3. Nelle distribuzioni di un HMM, quale può *non* essere multinomiale?
4. Perché in un HMM il calcolo della joint distribution parte da s_1 ?
5. What is a generator in a GAN?
6. And the discriminator?
7. Conditional generation: how would you do it?
8. Talk me about self-attention.
9. **What is a token?**
10. **How are computed Q , K , V ?** (*qui ha bocciato un candidato che non ha saputo rispondere*)
11. What is the advantage of weight sharing in RNN?
12. What is the assumption that makes weight sharing usable?
13. Why does tanh/sigmoid make me lose information?

14. What does the spectral radius inform me about?
15. In a Bayesian network, do we have only local properties?
16. What is the loss of a VAE?

Dallo stesso appello, sulla catena GAN/Wasserstein: che cos'è l'operatore $\langle \cdot \rangle$ in una Boltzmann machine; scrivi il sup nel discriminatore; differenza fra max e sup; perché nella Wasserstein si usa il sup; perché D deve essere Lipschitz.

Le risposte: (1–2) cap. 22 D1; (3–4) cap. 9 D1 e Cap. 3; (5–7) cap. 21 D1 e cap. 20 D7; (8–10) cap. 18 D1; (11–12) cap. 15 D7 e cap. 14 D1; (13–14) cap. 15 D1–D2; (15) cap. 3 D2; (16) cap. 20 D1; catena Wasserstein: cap. 21 D4–D5.

La domanda 9 merita una riga a sé

“Che cos'è un token” ha fatto fuori due candidati in due appelli diversi. Non è una domanda difficile: è una domanda *elementare* su un argomento che tutti studiano. La risposta è che è un **vettore** — l'embedding di un elemento della sequenza, non la parola e non l'indice nel vocabolario. E subito dopo arriva “come si calcolano $\mathbf{Q}$, $\mathbf{K}$, $\mathbf{V}$ ”, che è $\mathbf{Q} = \mathbf{XW}^Q$, $\mathbf{K} = \mathbf{XW}^K$, $\mathbf{V} = \mathbf{XW}^V$ con matrici apprese. È il promemoria più utile di questa appendice: si viene bocciati sulle definizioni operative, non sulle derivazioni difficili.

Bibliography

- [1] D. Barber. *Bayesian Reasoning and Machine Learning*. Cambridge University Press, 2012.
- [2] S.J.D. Prince. *Understanding Deep Learning*. MIT Press, 2023.
- [3] C.M. Bishop. *Pattern Recognition and Machine Learning*. Springer, 2006.
- [4] K.P. Murphy. *Probabilistic Machine Learning: An Introduction*. MIT Press, 2022.
- [5] I. Goodfellow, Y. Bengio, A. Courville. *Deep Learning*. MIT Press, 2016.
- [6] J. Pearl. *Causality*. 2nd ed., Cambridge University Press, 2009.
- [7] A. Vaswani et al. Attention Is All You Need. *NeurIPS*, 2017.
- [8] S. Hochreiter, J. Schmidhuber. Long Short-Term Memory. *Neural Computation*, 1997.
- [9] Y. LeCun et al. Gradient-Based Learning Applied to Document Recognition. *Proc. IEEE*, 1998.
- [10] D.M. Blei, A.Y. Ng, M.I. Jordan. Latent Dirichlet Allocation. *JMLR*, 2003.
- [11] A.P. Dempster, N.M. Laird, D.B. Rubin. Maximum Likelihood from Incomplete Data via the EM Algorithm. *JRSS-B*, 1977.
- [12] D.P. Kingma, M. Welling. Auto-Encoding Variational Bayes. *ICLR*, 2014.
- [13] I. Goodfellow et al. Generative Adversarial Nets. *NeurIPS*, 2014.
- [14] M. Arjovsky, S. Chintala, L. Bottou. Wasserstein GAN. *ICML*, 2017.
- [15] L. Dinh, D. Krueger, Y. Bengio. NICE: Non-linear Independent Components Estimation. *ICLR-WS*, 2014.
- [16] L. Dinh, J. Sohl-Dickstein, S. Bengio. Density Estimation using Real-Valued Non-Volume Preserving Transformations. *ICLR*, 2017.
- [17] D.P. Kingma, P. Dhariwal. Glow: Generative Flow with Invertible 1×1 Convolutions. *NeurIPS*, 2018.
- [18] R.T.Q. Chen, Y. Rubanova, J. Bettencourt, D. Duvenaud. Neural Ordinary Differential Equations. *NeurIPS*, 2018.
- [19] J. Ho, A. Jain, P. Abbeel. Denoising Diffusion Probabilistic Models. *NeurIPS*, 2020.
- [20] J. Song, C. Meng, S. Ermon. Denoising Diffusion Implicit Models. *ICLR*, 2021.
- [21] Y. Song, S. Ermon. Generative Modeling by Estimating Gradients of the Data Distribution. *NeurIPS*, 2019.

- [22] Y. Song, J. Sohl-Dickstein, D.P. Kingma, A. Kumar, S. Ermon, B. Poole. Score-Based Generative Modeling through Stochastic Differential Equations. *ICLR*, 2021.
- [23] Y. Lipman, R.T.Q. Chen, H. Ben-Hamu, M. Nickel, M. Le. Flow Matching for Generative Modeling. *ICLR*, 2023.
- [24] R. Rombach et al. High-Resolution Image Synthesis with Latent Diffusion Models. *CVPR*, 2022.
- [25] I. Higgins et al. β -VAE: Learning Basic Visual Concepts with a Constrained Variational Framework. *ICLR*, 2017.
- [26] F. Locatello et al. Challenging Common Assumptions in the Unsupervised Learning of Disentangled Representations. *ICML*, 2019.
- [27] I. Khemakhem, D.P. Kingma, R.P. Monti, A. Hyvärinen. Variational Autoencoders and Nonlinear ICA: A Unifying Framework. *AISTATS*, 2020.
- [28] B. Schölkopf et al. Toward Causal Representation Learning. *Proc. IEEE*, 2021.
- [29] P. Lippe et al. CITRIS: Causal Identifiability from Temporal Intervened Sequences. *ICML*, 2022.
- [30] T.N. Kipf, M. Welling. Semi-Supervised Classification with Graph Convolutional Networks. *ICLR*, 2017.
- [31] W.L. Hamilton, R. Ying, J. Leskovec. Inductive Representation Learning on Large Graphs. *NeurIPS*, 2017.
- [32] P. Veličković et al. Graph Attention Networks. *ICLR*, 2018.
- [33] K. Xu, W. Hu, J. Leskovec, S. Jegelka. How Powerful are Graph Neural Networks? *ICLR*, 2019.
- [34] R. Ying et al. Hierarchical Graph Representation Learning with Differentiable Pooling. *NeurIPS*, 2018.
- [35] L. Rampášek et al. Recipe for a General, Powerful, Scalable Graph Transformer. *NeurIPS*, 2022.
- [36] V. Mnih et al. Human-Level Control Through Deep Reinforcement Learning. *Nature*, 2015.
- [37] R.J. Williams. Simple Statistical Gradient-Following Algorithms for Connectionist Reinforcement Learning. *Machine Learning*, 1992.
- [38] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, O. Klimov. Proximal Policy Optimization Algorithms. *arXiv:1707.06347*, 2017.
- [39] R.S. Sutton, A.G. Barto. *Reinforcement Learning: An Introduction*. 2nd ed., MIT Press, 2018.